

Ciencia de Datos con R

2026-08-01

Contents

1	Ciencia de Datos con R	11
2	Instalación de paquetes	17
2.1	¿Por qué esta página?	17
2.2	Instalar todos los paquetes del libro	18
2.3	Verificar qué está instalado	19
2.4	Activar los paquetes principales	20
3	Flujo de trabajo	21
3.1	Temas:	21
3.2	Creación de un proyecto:	21
3.3	Información sobre el archivo	25
3.4	Ejemplos de Gráficos	30
4	Visualización de datos	33
4.1	Temas:	33
4.2	Leer datos de un archivo	48
4.3	Vea el siguiente gráfico	51
5	Calculadora sofisticada	55
5.1	Temas:	55
6	Transformación: Estructura de un archivo y seleccionar datos	63
6.1	Evaluar el archivo el vuelo	64
6.2	Temas: Reconocer y aplicar las diferentes funciones	67
6.3	Funciones	69
6.4	Agrupar por múltiples variables	71
7	Transformación: Operaciones booleanas y lógicas	73
7.1	Agrupar por múltiples variables	74
7.2	Operaciones lógicas boolean:	74
7.3	Ejercicios	76
7.4		77
7.5	2 Ejercicios:	80

8	Transformación: Valores Faltantes	81
8.1	Valores faltantes	81
8.2	Creamos un df para aclarar estas operaciones con NA en un data frame	82
8.3	Contabilizar los NA	84
8.4	Seleccionar solamente los vuelos donde el “horario de salida” es desconocido	85
8.5	Otras funciones: NA	85
9	Transformación: Seleccionar variables	89
9.1	Funciones en este módulo	89
9.2	rename()	92
9.3	Reorganizar el orden de las columnas, usando select() y everything() en la misma función	93
10	Transformación: Mutate, transmute, lag, lead, cumsum, cmean, cumvar	97
10.1	Funciones en este módulo	97
10.2	Crear un data frame más pequeño con las variables de interés	98
10.3	Crear nuevas variables	99
10.4	transmute()	100
10.5	lag()	100
10.6	Usa “Lag” con “IncCasosSaludNuevo” en COVID-19 PR	102
10.7	lead(),	104
10.8	cumsum	106
10.9	cmean() and cvar() del paquete TidyDensity	108
10.10	Uso de varianza acumulativa en investigación:	109
10.11	Que pasa con las funciones cmean , cvar si hay NA en el archivo de datos?	110
10.12	if_else()	111
11	Transformación: Rangos y otras funciones	115
11.1	Funciones en este módulo	115
11.2	Pruebas no paramétricas	116
11.3	min_rank() y # min_rank(desc())	116
11.4	row_number()	117
11.5	dense_rank()	118
11.6	la función percent_rank()	119
11.7	la función percent_rank() sin NA	119
11.8	la función cume_dist()	120
11.9	Resúmenes con summarise() by group using group_by()	122
11.10	La aerolínea peor en atraso de salidas	124
12	Transformación: Índices paramétricos y no paramétricos	129
12.1	Funciones estadísticas básicas:	129

13 Script	131
13.1 Scripts:	131
14 Errores comunes y cómo resolverlos	133
14.1 could not find function	133
14.2 object ... not found	134
14.3 non-numeric argument to binary operator	134
14.4 unused argument	135
14.5 Confundir = con ==	135
14.6 El caso especial de NA (no es un error, pero engaña)	135
14.7 cannot open file ... No such file or directory	136
14.8 cannot open URL (descargar datos de internet)	136
14.9 Un método general para resolver errores	137
15 Análisis Exploratorio	139
15.1 Temas	139
15.2 Términos importantes	139
16 Pipes	143
16.1 Temas: El paquete “magrittr” y sus funciones	143
16.2 %>% pipes	145
16.3 Alternativas a los pipes	146
16.4 %\$%	146
16.5 %<>%	148
16.6 %in%	149
16.7 %T>%	149
17 Tibbles	151
17.1 Temas: El paquete Tibble	151
17.2 Crear un tibble	151
17.3 Crear un tribble	152
17.4 tibble vs. data.frame	153
17.5 Diferencias entre un tibble y un data frame.	156
17.6 Selección de subconjuntos de datos	157
18 Importar datos	159
18.1 Datos importados de la web.	159
18.2 Ejercicio	160
18.3 Datos importando de su computadora	160
18.4 Comparar con base R	164
18.5 Segmentar un vector	164
18.6 Números	166
18.7 Cadena de texto	166
18.8 Factores	167
18.9 Fechas, fechas-horas, horas	167
18.10 Segmentar un archivo	167

18.11Escribir un Archivo	167
18.12Otros tipos de datos y paquetes	167
19 Datos Ordenados	169
19.1 Temas: Datos Ordenados	169
20 Datos Relacionados	173
20.1 Temas: Datos Relacionados	177
20.2 Claves	177
20.3 Uniones de transformaciones	181
20.4 Entendiendo las uniones	190
20.5 Definiendo las columnas clave	190
20.6 Otras implementaciones	191
20.7 Uniones de filtro	191
20.8 Problemas con las uniones	191
21 Factores	193
21.1 ¿Qué ocurrió en el gráfico?	194
21.2 Temas:	199
21.3 A smaller figure to the right, with floating text	202
21.4 Haga un re-order por la cantidad de personas por religión	202
21.5 Modificar el orden de los factores	203
22 Cadena de Caracteres = STRINGS	211
22.1 Caracteres especiales	212
22.2 Para ver las opciones de caracteres y como trabajar cadenas de caracteres use <code>getOption("stringsAsFactors")</code> para ver	213
22.3 Lista de caracteres especiales (unicode)	213
22.4 Temas: Cadenas de Caracteres	214
22.5 Dividir cadenas basado en la posición de los caracteres en la cadena	215
22.6 You can also use <code>str_sub()</code> to modify strings:	216
22.7 Locales	217
22.8 Buscar patrones	217
22.9 Coincidencia básica	217
22.10Anclas	229
22.11Clases de caracteres y alternativas	229
22.12Repetición	229
22.13Agrupamiento y referencias previas	230
22.14Herramientas: Objetivos	230
22.15Extraer coincidencias	231
22.16Coincidencias agrupadas:	231
22.17Reemplazar coincidencias	232
22.18Divisiones	232
22.19Otro tipos de patrones	232
22.20stringi:	232
22.21El texto de Quijote	232

22.22	Cuántas palabras tiene el primer capítulo de Quijote?	233
22.23	Cuántas veces aparece la palabra “caballero” en el primer capítulo de Quijote?	233
22.24	¿Cuántas palabras tienen más de 10 letras?	234
22.25	Cual es la palabra más larga	234
22.26	Cuántas veces aparece “Sancho Panza” en el primer capítulo de Quijote?	234
22.27	Cuántas veces aparece “Dios” en el primer capítulo de Quijote?	234
22.28	Cuántas palabras tienen ñ?	235
22.29	Remover del texto todas las siguientes palabras	235
22.30	Contabilizar las palabras más comunes en el primer capítulo de Quijote	237

23 Ecuaciones Matemáticas 243

23.1	Ecuaciones matemáticas	243
23.2	Símbolos de LATEX	245
23.3	Química 101: Símbolos	245
23.4	Símbolos comunes	245
23.5	Física	246
23.6	Biología	246

24 Opciones de Knitr 249

24.1	Para ver las opciones de knitr de los chunk	249
24.2	Tablas de opciones principales	250
24.3	Emoji or Symbols	256

25 Fechas, horas y minutos 257

25.1	Temas:	257
25.2	Creando fechas/horas	257
25.3	El paquete lubridate	258
25.4	Intervalos y duraciones	260
25.5	El operador %--% simplifica el cálculo del intervalo	261
25.6	Ejercicio	262
25.7	Serie de funciones para calcular la duración de un intervalo de tiempo.	263
25.8	as.duration	264
25.9	Crear una fecha desde columnas individuales	266
25.10	%/% : integer division	268
25.11	The %% operator returns the modulus (remainder) of a division operation.	269
25.12	Extrayendo parte de las fecha-hora	273
25.13	Lapso de tiempo	274
25.14	duraciones	274
25.15	Puede agregar periodos de tiempo	274
25.16	períodos	275
25.17	Períodos	277

25.18	Intervalos	277
25.19	Resumen	277
25.20	Time zones	277
26	Funciones	279
26.1	Temas:	279
26.2	¿Cuándo escribir una función?	279
26.3	La anatomía de una función	280
26.4	Ejecución condicional: if y else	281
26.5	Argumentos y valores por defecto	282
26.6	Valores de retorno	282
26.7	Un vistazo a los vectores	283
27	Iteración: purrr	285
27.1	Temas: El paquete purrr	285
27.2	¿Qué es iterar?	285
27.3	El bucle for de base R	286
27.4	La familia map()	287
27.5	Fórmulas anónimas con ~ y .x	287
27.6	Iterar sobre dos listas con map2()	288
27.7	Un modelo por grupo	289
28	Modelos: modelr	291
28.1	Temas: Construir modelos con modelr	291
28.2	Visualizar la relación	292
28.3	Ajustar un modelo lineal con lm()	293
28.4	Predicciones con data_grid() y add_predictions()	294
28.5	Residuales con add_residuals()	294
29	Hex Stickers	297
29.1	La función y los valores “default”	297
29.2	Ejemplo de Crear un Hex con un gatito	299
30	Nubes de palabra en R	303
30.1	Ejemplo #1	303
30.2	Usando los datos en el paquete wordcloud2 que se llama demoFreq	304
30.3	Paso 1	304
30.4	Subir el texto en formato Corpus	305
30.5	Mirar el documento, para evaluar su contenido	305
30.6	Transformar el texto para reemplazar algunos caracteres espe- ciales, y reemplazarlos por espacio en blanco	305
30.7	el paquete tm es para text mining	306
30.8	Crear una matriz de las palabras de mi documento	307
30.9	Ejemplo 2	308
30.10	Como remover palabras de otra idioma	309
30.11	En español	309

30.12	Un documento para hacer un word cloud “La inteligencia artificial”	310
30.13	Convertir su documento en Corpus y identificar que es en español	310
30.14	Limpieza del documento	310
30.15	Transformar el texto en un documento de texto sencillo (Plain Text)	311
30.16	Transformar el texto para reemplazar algunos caracteres especiales	311
30.17	Crear una matriz de las palabras	311
30.18	Calcular la frecuencia de cada palabra	311
31	Temas Avanzados en Data Science con Tidyverse	313
31.1	Introducción	313
31.2	2. Modelado con tidymodels	315
31.3	3. Visualización avanzada con ggplot2	318
31.4	4. Datos anidados y listas con purrr	319
32	Leaflet: Mapas interactivos	323
32.1	Bajar los datos de la web del USGS	323
32.2	Si bajo los datos a mi computadora	325
32.3	Importación de Paquetes y Datos	326
32.4	Mapa básico con Leaflet	327
32.5	Usando la paletas de color de viridis	328
32.6	Trabajando con datos de iNaturalist	329
32.7	Utilizando Leaflet	332
32.8	Trabajando Variables Categóricas con Leaflet:	335
A	Appendix A	339
A.1	Ejercicio de Transformación	339
	Capítulo de Transformación de Datos	339
	Ejercicios con destrezas de aplicar algunas funciones de transformaciones de datos	339
1.	Busca el bateador que tuvo más carreras en cualquier año	340
2.	¿Cuál es el nombre del bateador (id_jugador) que estuvo más veces “al_bate”? Prepara una lista del bateador más frecuente al bate al menos	340
3.	¿Cuáles son las “ligas” de baseball (pelota) que están incluidas en este archivo?	340
4.	Selecciona los años (1900, 1950, 2000 y 2020) y jonrones y hacer una tabla por año que demuestra el máximo de jonrones para cada año	340
5.	Haz una tabla de la los jugadores que jugaron más años	341
6.	Selecciona solamente la liga “AL”, los años desde 2000 en adelante, y determina cuál es la suma de “carreras” anotadas por cada equipo.	341
Appendix B		343
	Ejercicios para las funciones de transformaciones 2	343
	Seleccione los siguientes datos sobre COVID en PR	343

1. Calcular el promedio de Muertes por día de nuevos muertos desde el comienzo hasta el último día del archivo	344
2. ¿Cuál es el máximo de nuevos casos en un día	344
3. ¿Calcular la cantidad acumulativa de número de camas CamasICU .	344
4. Haz una gráfica de la varianza y como cambia a través del tiempo. .	344
Appendix C	345
Calculate the age of individuals when they were diagnosed with HIV .	346
Calculate the time between the date of diagnosis of HIV and the date of death.	346
Calculate the difference in life between the date of diagnosis and death	349
Appendix D	351
A.2 COVID 19 Cumulativo	351
## [1] "2026-08-01"	

Chapter 1

Ciencia de Datos con R



¡Bienvenidos a BIOL4026! Una introducción práctica y visual a la ciencia de datos con R, RStudio, RMarkdown y el **tidyverse**. Aprenderás a importar, ordenar, transformar, **visualizar** y modelar datos reales —de Puerto Rico y de la biología— para contestar preguntas cuantitativas.

Instructor: Raymond L. Tremblay, PhD

Oficina: NL 104

Teléfono: (787) 850-0000 (dept de biología)

Correo electrónico: raymond.tremblay__at__upr_dot_edu

1.0.1 Horario de clase

Presentación de temas y discusión: Lunes y miércoles 4:30-5:50pm (80 mins) (NOTE: Necesita traer su laptop!)

Hora de consulta con Estudiantes:

- Lunes y miércoles de 1:00 a 5:30pm (NL 104).
-

1.0.2 Libros sugeridos

- Garrett Golemund, Hadley Wickham. R for Data Science En Inglés.
 - Garrett Golemund, Hadley Wickham. R para Ciencia de Datos En Español.
 - Tremblay y Hernández-Serrano, 2018.
- Artículos revisados por pares serán asignados para fomentar el método de utilizar estas herramientas en ciencias.
-

1.0.3 Programados

Instala R y RStudio (ambos gratuitos) antes de la primera clase. Luego abre el capítulo “Instalación de paquetes”, que incluye un bloque para instalar de una sola vez todo lo necesario.

- R- free statistical programming language
 - RStudio
 - MSExcel, Numbers o Google Sheet
-

1.0.4 Prerequisitos

Ninguno

1.0.5 Descripción del curso

Estudio de diferentes técnicas estadísticas con aplicación a la Biología. Se enfatizará en la estadística descriptiva, análisis de regresiones y correlaciones, pruebas de hipótesis paramétricas y no paramétricas y análisis de frecuencias y varianza. Se hará énfasis en los supuestos de las pruebas, para seleccionar cual método estadístico es adecuado para el diseño experimental y la distribución de los datos. Además, se utilizarán las computadoras como mecanismos para facilitar y agilizar el cómputo y análisis estadístico.

1.0.6 Objetivos del curso

Tema general del curso - Desarrollar destrezas de organizar y evaluar datos para contestar preguntas cuantitativas con R, RStudio, RMarkdown y Quarto y otros programados

1.0.7 Puntuación:

Este curso será evaluado con los siguientes ítems:

Item	Valor
Ejercicios práctico (10-15 total)	80%
Participación en clase	10%
Asistencia a clase	10%

NOTE: Escala de Notas:

- A (100 to 90)
 - B (89 to 80)
 - C (79 to 70)
 - D (69 to 60)
 - F (< 60)
-

1.0.8 Exámenes:

Ninguno

1.0.9 Conferencias

En la clase las notas serán basado primeramente en la participación y algunas pruebas cortas. Su participación es esencial para el aprendizaje (y para un ambiente positivo). Aprender *NO* es un proceso pasivo: los estudiantes deben participar haciendo preguntas y discutir el material con su conocimiento anterior (Su bagaje de conocimiento).

1.0.10 Ejercicios

Los ejercicios están enfocados en la aplicación de conceptos y métodos discutidos en la clase y solución de problemas. Se hará un esfuerzo de usar datos reales para demostrar cómo trabajar con los análisis, tablas, y gráficos en R, RStudio

y RMarkdown. Típicamente, tendrán solamente una semana para hacer los ejercicios y entregarlos en formato *.html*.

Los ejercicios se entregan en formato *.html*, que se genera al hacer **Knit** de tu archivo *.Rmd* en RStudio.

1.0.11 Faltar a clase y examen:

Los trabajos cortos y pruebas cortas NO se reponen. Si falta a la clase es su responsabilidad hablar con los otros estudiantes para saber lo que se discutió en la clase. Los exámenes se reponen solamente por una excusa válida.

1.0.12 Derechos de Estudiantes con Impedimentos

La UPR-Humacao cumple con las leyes ADA (Americans with Disabilities Act) y 51 (Servicios Educativos Integrales para Personas con Impedimentos) para garantizar igualdad en el acceso a la educación y servicios. Estudiantes con impedimentos: informe al (la) profesor(a) de cada curso sobre sus necesidades especiales y/o de acomodo razonable para el curso, en la tarjeta de información de la primera semana y visite la Oficina de Servicios para la Población con Impedimentos (SERPI) a la brevedad posible. Se mantendrá la confidencialidad.

1.0.13 Integridad académica

La Universidad de Puerto Rico promueve los más altos estándares de integridad académica y científica. El Artículo 6.2 del Reglamento General de Estudiantes de la Universidad de Puerto Rico (Certificación Núm. 13, 2009-2010, de la Junta de Síndicos) establece que “la deshonestidad académica incluye, pero no se limita a: acciones fraudulentas, la obtención de notas o grados académicos valiéndose de falsas o fraudulentas simulaciones, copiar total o parcialmente la labor académica de otra persona, plagiar total o parcialmente el trabajo de otra persona, copiar total o parcialmente las respuestas de otra persona a las preguntas de un examen, haciendo o consiguiendo que otro tome en su nombre cualquier prueba o examen oral o escrito, así como la ayuda o facilitación para que otra persona incurra en la referida conducta”. Cualquiera de estas acciones estará sujeta a sanciones disciplinarias en conformidad con el procedimiento disciplinario establecido en el Reglamento General de Estudiantes de la UPR vigente.

1.0.14 Comentario sobre grabar videos y/o audio de las clases

Los estudiantes no PUEDEN grabar la clase por forma de video o audio sin el permiso del profesor. Algunos estudiantes con necesidades especiales pueden hablar con el profesor para pedir el permiso. La solicitud y aprobación del permiso tiene que ser por escrito (por ejemplo por email).

1.0.15 Espacio libre de acoso sexual

La Universidad de Puerto Rico prohíbe el discrimen por razón de sexo y género en todas sus modalidades, incluyendo el hostigamiento sexual. Según la Política Institucional contra el hostigamiento sexual, en la Universidad de Puerto Rico, Cert. Núm. 130 (2014-2015) de la Junta de Gobierno, si un(a) estudiante está siendo o fue afectado por conductas relacionadas a hostigamiento sexual, puede acudir ante la Oficina del Procurador Estudiantil, el Decanato de Estudiantes o el Coordinador de Cumplimiento con Título IX para una orientación o presentar una querella.

1.0.16 Protocolo de la Clase

Los teléfonos móviles serán apagados durante la clase. Si necesita una calculadora traerla al salón. El teléfono no debería estar visible durante la clase a menos que pida permiso al instructor. Recuerda que se usarán computadoras portátiles en cada sesión.

Chapter 2

Instalación de paquetes

Fecha de la última revisión

[1] "2026-08-01"

2.1 ¿Por qué esta página?

A lo largo del libro usamos muchos paquetes de R. Un **paquete** es una colección de funciones, datos y documentación que amplía lo que R puede hacer. Antes de comenzar conviene **instalar de una sola vez** todos los paquetes que vamos a necesitar, para que ningún capítulo falle por un paquete que falta.

CRAN (*Comprehensive R Archive Network*), la red oficial de paquetes de R, funciona desde 1997 y hoy alberga más de 20,000 paquetes. El **tidyverse** — una colección de paquetes con una filosofía común para la ciencia de datos — fue creado por **Hadley Wickham** y su equipo en RStudio (hoy Posit); el nombre se popularizó alrededor de 2016.

Recuerda: solo hay que **instalar** un paquete una vez en la computadora (con `install.packages()`), pero hay que **activarlo** con `library()` en cada sesión en la que lo vayas a usar.

`install.packages("paquete")` y `library(paquete)` — instalar vs. activar.

- **`install.packages("x")`**: descarga e instala el paquete desde internet (CRAN). Se hace **una sola vez** por computadora.
- **`library(x)`**: carga el paquete en la sesión actual para poder usar sus funciones. Se hace **en cada sesión**.
- **Regla de oro**: instalar = una vez; activar con `library()` = cada vez que abres R.

Un error muy frecuente es instalar el paquete pero olvidar activarlo en la sesión. Si al usar una función aparece `could not find function`, casi siempre falta correr `library()`. Encontrarás más errores frecuentes y cómo resolverlos en el capítulo *Errores comunes* (14).

2.2 Instalar todos los paquetes del libro

Copia y ejecuta el siguiente bloque **una sola vez** en la consola de RStudio. El script revisa cuáles paquetes ya están instalados e instala **solamente los que falten**. Instala cada paquete por separado, de modo que si alguno no se puede instalar, los demás sí se instalan y al final se te informa cuáles fallaron.

Instalar paquetes descarga archivos desde CRAN, así que necesitas conexión a internet. En una red lenta el proceso puede tardar varios minutos.

```
paquetes_curso <- c(
  # --- Núcleo: tidyverse y datos en español ---
  "tidyverse", "datos", "ggversa",
  "dplyr", "tidyr", "readr", "purrr", "stringr", "forcats",
  "ggplot2", "lubridate", "magrittr",

  # --- Datos de ejemplo ---
  "nycflights13", "babynames", "nasaweather", "fueleconomy", "Lahman",

  # --- Importar / limpiar / tablas ---
  "readxl", "haven", "data.table", "hms", "zoo",
  "gt", "flextable", "knitr", "bookdown", "gridExtra",

  # --- Modelos y estadística ---
  "modelr", "broom", "tidymodels", "MASS", "DescTools",
  "GMCM", "TidyDensity",

  # --- Mapas y visualización ---
  "leaflet", "maps", "RColorBrewer",

  # --- Hex stickers e imágenes ---
  "hexSticker", "showtext", "sysfonts", "magick",

  # --- Nubes de palabras / texto ---
  "wordcloud", "wordcloud2", "tm", "SnowballC", "stopwords",

  # --- Utilidades y datos externos ---
  "pacman", "devtools", "rinat", "cranlogs", "packageRank"
)
```

```
faltantes <- setdiff(paquetes_curso, rownames(installed.packages()))

fallidos <- character(0)
for (p in faltantes) {
  ok <- tryCatch({
    install.packages(p, dependencies = TRUE)
    TRUE
  }, error = function(e) FALSE, warning = function(w) FALSE)
  if (!ok || !(p %in% rownames(installed.packages())))) fallidos <- c(fallidos, p)
}

if (length(fallidos) == 0) {
  message("¡Listo! Todos los paquetes del libro están instalados.")
} else {
  message("No se pudieron instalar automáticamente: ", paste(fallidos, collapse = ", "),
    "\nInténtalos a mano con install.packages('nombre').")
}
```

`ggversa` (Gráficas Versátiles con `ggplot2`, de Tremblay & Hernández-Serrano) está disponible en CRAN, así que se instala igual que los demás con `install.packages("ggversa")`.

2.3 Verificar qué está instalado

Este bloque **no instala nada**; solamente te dice cuáles paquetes ya tienes y cuáles te faltan.

```
paquetes_curso <- c(
  "tidyverse", "datos", "ggversa", "nycflights13", "babynames", "nasaweather",
  "fueleconomy", "Lahman", "readxl", "haven", "data.table", "hms", "zoo",
  "gt", "flextable", "knitr", "bookdown", "gridExtra", "modelr", "broom",
  "tidymodels", "MASS", "DescTools", "GMCM", "TidyDensity",
  "leaflet", "maps", "RColorBrewer", "hexSticker", "showtext", "sysfonts",
  "magick", "wordcloud", "wordcloud2", "tm", "SnowballC", "stopwords",
  "pacman", "devtools", "rinat", "cranlogs", "packageRank"
)

instalados <- paquetes_curso %in% rownames(installed.packages())
data.frame(
  paquete = paquetes_curso,
  instalado = ifelse(instalados, "sí", "NO - falta")
)
```

2.4 Activar los paquetes principales

En cada capítulo verás al inicio un bloque con `library()`. El más común es:

```
library(tidyverse)  # ggplot2, dplyr, tidyr, readr, purrr, tibble, stringr, forcats
library(datos)      # datos en español
```

Si al correr un capítulo aparece un error como `Error in library(xxx): there is no package called 'xxx'`, regresa a esta página y ejecuta el bloque de instalación de arriba.

Chapter 3

Flujo de trabajo

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/workflow-projects.html>

Español: <https://es.r4ds.hadley.nz/04-workflow-basics.html>

Fecha de la última revisión

[1] "2026-08-01"

3.1 Temas:

- Reduciendo Errores
- `getwd()`
- `setwd()`
- Su Proyecto
- Seleccionar el tab **Session** →→ **Set Working Directory** →→ **To Project Directory** ***

3.2 Creación de un proyecto:

Usa Proyectos de RStudio (menú **File** → **New Project**): mantienen juntos tus datos, scripts y resultados, y evitan depender de `setwd()`, que hace tu código difícil de compartir.

RStudio, el entorno de trabajo que usamos, fue creado por **JJ Allaire** y lanzado en 2011; la empresa se renombró **Posit** en 2022. Los *Proyectos* de RStudio nacieron para mantener cada análisis autocontenido y reproducible.

- crear un proyecto para cada curso.
- crear un proyecto para cada investigación.
- No se te olvida de añadir tus archivos de datos en el proyecto.
- Describe claramente todos tus análisis y donde conseguiste la información.
- Describe tu interpretación de los análisis o gráficos.
- Correr los “scripts” uno a la vez para asegurar que funcione.
- knit el archivo .rmd para asegurar que no falte nada.
- no mezclar proyectos de investigación en un mismo proyecto.

```
#install.packages("tidyverse")
library(tidyverse) # ggplot2, dplyr, tidyr, readr, purrr, tibble, stringr, forcats

# install.packages("datos")
library(datos)
```

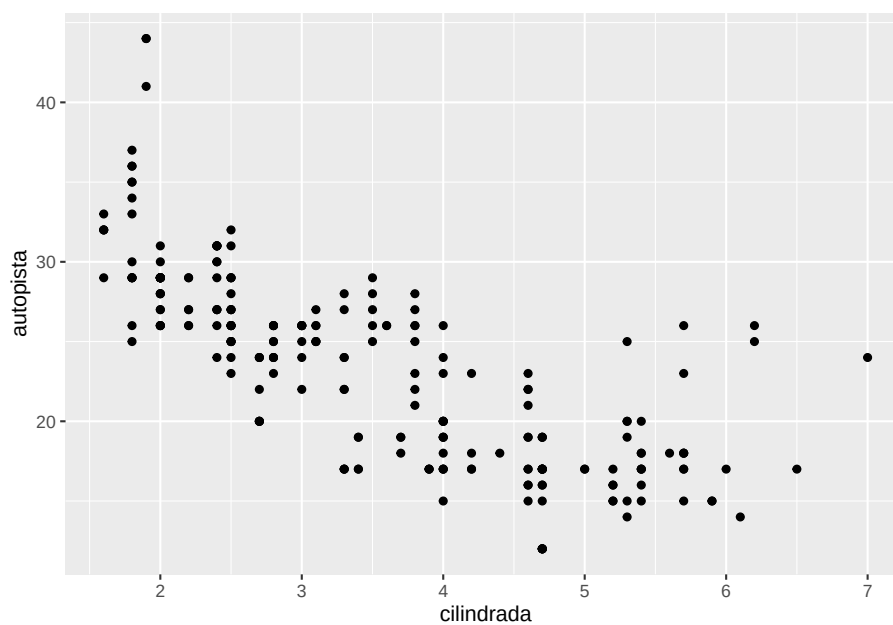
3.2.1 Un set de sobre carros en el archivo (paquete) datos

millas

```
## # A tibble: 234 x 11
##   fabricante modelo      cilindrada  anio cilindros transmision traccion ciudad
##   <chr>         <chr>          <dbl> <int>    <int> <chr>      <chr>    <int>
## 1 audi         a4              1.8  1999      4 auto(l5)    d         18
## 2 audi         a4              1.8  1999      4 manual(m5)  d         21
## 3 audi         a4              2    2008      4 manual(m6)  d         20
## 4 audi         a4              2    2008      4 auto(av)    d         21
## 5 audi         a4              2.8  1999      6 auto(l5)    d         16
## 6 audi         a4              2.8  1999      6 manual(m5)  d         18
## 7 audi         a4              3.1  2008      6 auto(av)    d         18
## 8 audi         a4 quattro      1.8  1999      4 manual(m5)  4         18
## 9 audi         a4 quattro      1.8  1999      4 auto(l5)    4         16
## 10 audi        a4 quattro      2    2008      4 manual(m6)  4         20
## # i 224 more rows
## # i 3 more variables: autopista <int>, combustible <chr>, clase <chr>
```

3.2.2 Mi primer gráfico

```
ggplot(data = millas) +
  geom_point(mapping = aes(x = cilindrada, y = autopista))
```

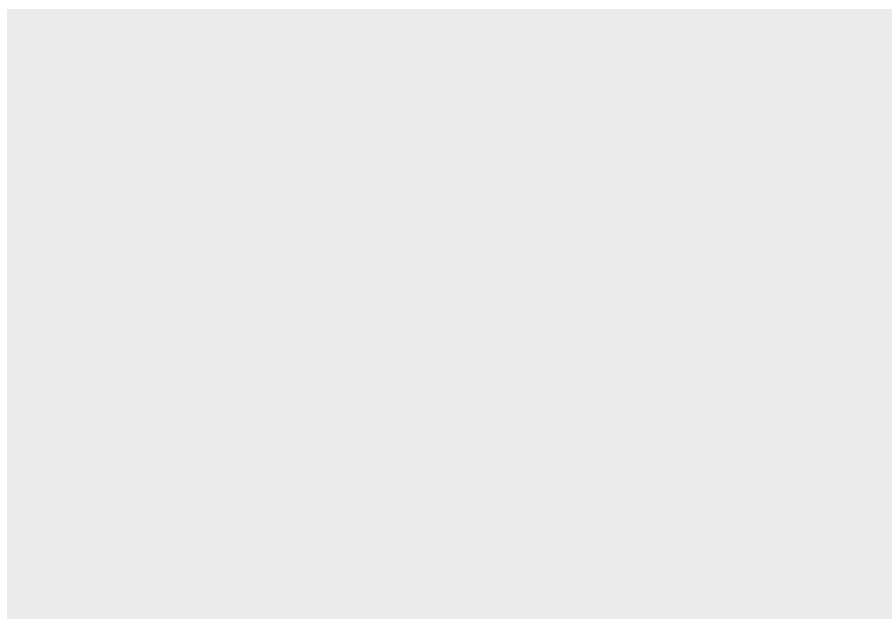


3.2.3 Las funciones head y tail

`head(x, n)` y `tail(x, n)` — primeras y últimas filas.

- **Qué hacen:** `head()` muestra las primeras `n` filas de un data frame (o elementos de un vector) y `tail()` las últimas. Por defecto `n = 6`.
- **Uso:** echar un vistazo rápido a los datos sin imprimirlos completos.

```
ggplot(data=millas)
```



```
millas
```

```
## # A tibble: 234 x 11
##   fabricante modelo      cilindrada  anio cilindros transmision traccion ciudad
##   <chr>      <chr>      <dbl> <int>    <int> <chr>      <chr>    <int>
## 1 audi      a4          1.8  1999      4 auto(l5)  d        18
## 2 audi      a4          1.8  1999      4 manual(m5) d        21
## 3 audi      a4          2    2008      4 manual(m6) d        20
## 4 audi      a4          2    2008      4 auto(av)  d        21
## 5 audi      a4          2.8  1999      6 auto(l5)  d        16
## 6 audi      a4          2.8  1999      6 manual(m5) d        18
## 7 audi      a4          3.1  2008      6 auto(av)  d        18
## 8 audi      a4 quattro    1.8  1999      4 manual(m5) 4        18
## 9 audi      a4 quattro    1.8  1999      4 auto(l5)  4        16
## 10 audi     a4 quattro    2    2008      4 manual(m6) 4        20
## # i 224 more rows
## # i 3 more variables: autopista <int>, combustible <chr>, clase <chr>
head(millas, n=3) # las primeras 6 filas
```

```
## # A tibble: 3 x 11
##   fabricante modelo cilindrada  anio cilindros transmision traccion ciudad
##   <chr>      <chr>      <dbl> <int>    <int> <chr>      <chr>    <int>
## 1 audi      a4          1.8  1999      4 auto(l5)  d        18
## 2 audi      a4          1.8  1999      4 manual(m5) d        21
## 3 audi      a4          2    2008      4 manual(m6) d        20
```



```
## # i 3 more variables: autopista <int>, combustible <chr>, clase <chr>
tail(millas, n=10) # las últimas 10 filas

## # A tibble: 10 x 11
##   fabricante modelo      cilindrada  año cilindros transmision traccion ciudad
##   <chr>          <chr>        <dbl> <int>    <int> <chr>      <chr>    <int>
## 1 volkswagen new beetle      2    1999      4 auto(l4)    d        19
## 2 volkswagen new beetle    2.5    2008      5 manual(m5)  d        20
## 3 volkswagen new beetle    2.5    2008      5 auto(s6)    d        20
## 4 volkswagen passat      1.8    1999      4 manual(m5)  d        21
## 5 volkswagen passat      1.8    1999      4 auto(l5)    d        18
## 6 volkswagen passat      2    2008      4 auto(s6)    d        19
## 7 volkswagen passat      2    2008      4 manual(m6)  d        21
## 8 volkswagen passat      2.8    1999      6 auto(l5)    d        16
## 9 volkswagen passat      2.8    1999      6 manual(m5)  d        18
## 10 volkswagen passat      3.6    2008      6 auto(s6)    d        17
## # i 3 more variables: autopista <int>, combustible <chr>, clase <chr>
```

3.3 Información sobre el archivo

Usa el signo de interrogación ? antes del nombre de la función o archivo. Re-mueve el “#” para ver el resultado

```
## #?millas
```

```
## #?head
```

3.3.1 Las dimensiones del archivo

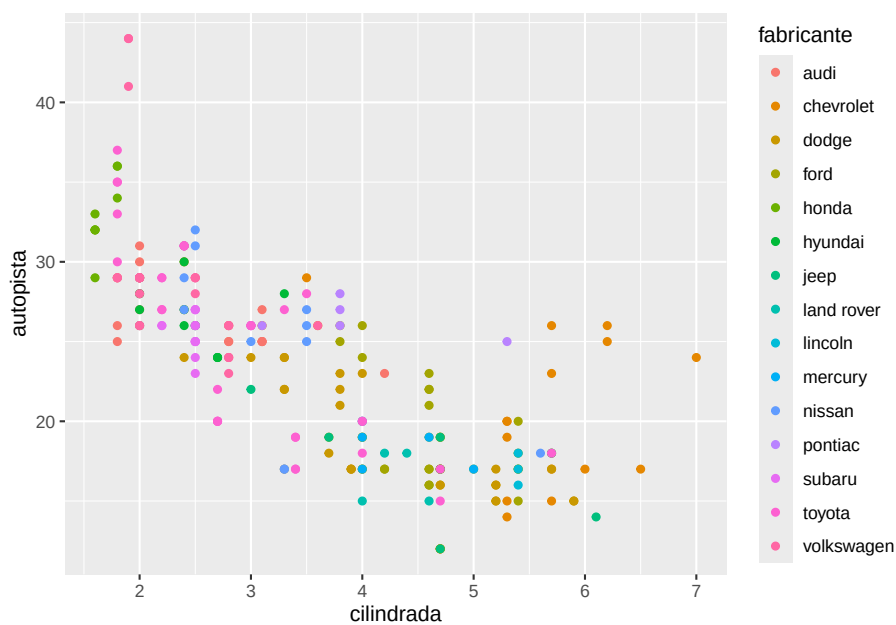
```
dim(millas) # dimension of data frame
```

```
## [1] 234 11
```

3.3.2 Construcción de mi primer gráfico

- el nombre del archivo es primero
- nombre de las variables (nombre de las columnas)
- si quiere tener un color para cada grupo

```
ggplot(data = millas) +
  geom_point(mapping = aes(x = cilindrada, y = autopista, colour=fabricante))
```

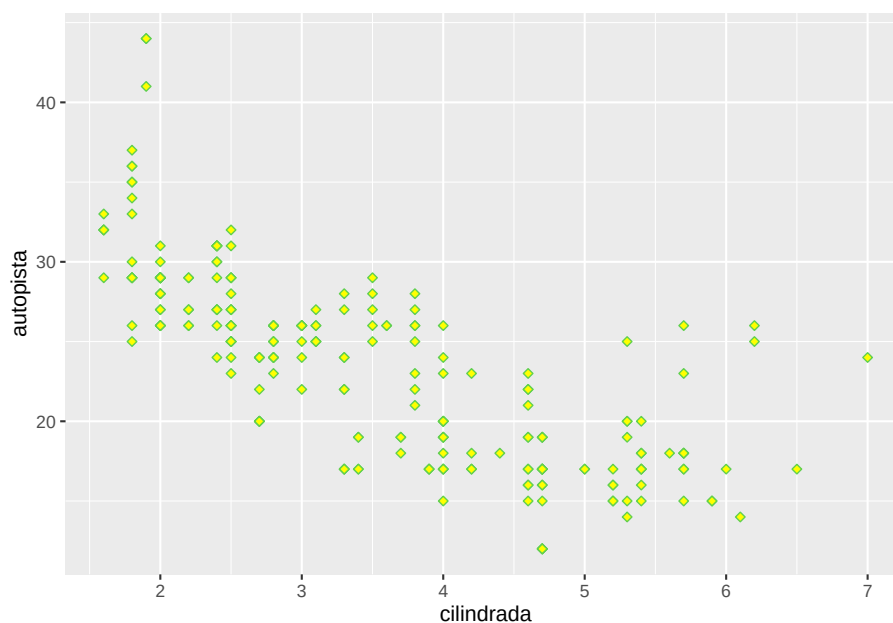


3.3.3 La función `shape` para cambiar la forma de los puntos

La función `shape` es para cambiar el estilo de los puntos

Los colores en hexadecimal deben empezar con #, por ejemplo `color = "#369787"`. Sin el #, R no reconoce el código y marcará error.

```
ggplot(data = millas) +
  geom_point(mapping = aes(x = cilindrada, y = autopista), shape=23, color = "#369787",
```

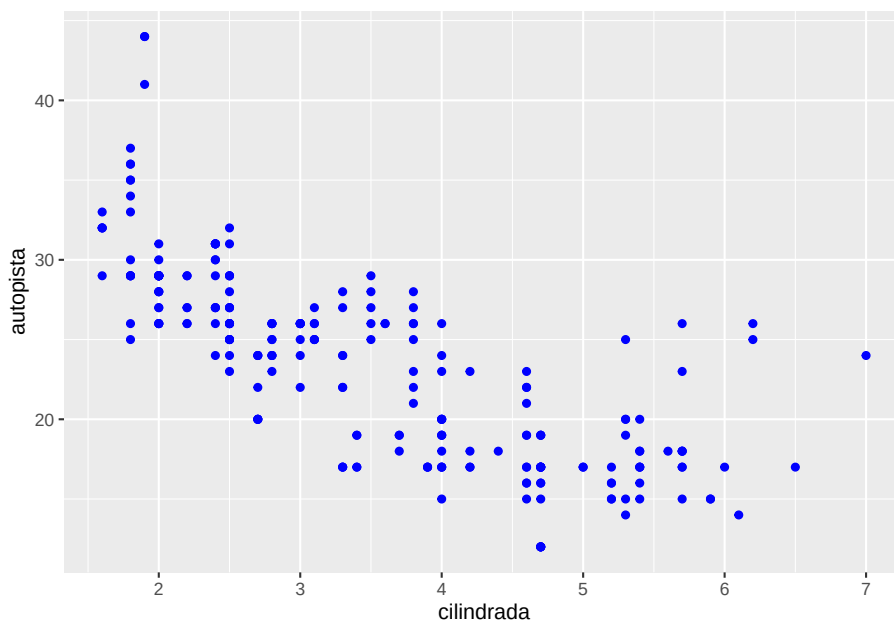


3.3.4 Salvar un gráfico en otro formato

Como salvar la figura en formato recuperable para subir en otros documentos o compartir

`ggsave()` guarda el último gráfico que creaste y lo escribe en tu directorio de trabajo. La extensión del nombre (`.png`, `.jpg`, `.tiff`) determina el formato del archivo.

```
ggplot(data = millas) +  
  geom_point(mapping = aes(x = cilindrada, y = autopista), color = "blue")
```



```
ggsave("img/cilindro_milla.jpg") #. png, .tiff
```

```
millas
```

```
## # A tibble: 234 x 11
##   fabricante modelo      cilindrada  anio cilindros transmision traccion ciudad
##   <chr>         <chr>          <dbl> <int>    <int> <chr>      <chr>    <int>
## 1 audi         a4              1.8  1999      4 auto(l5)  d        18
## 2 audi         a4              1.8  1999      4 manual(m5) d        21
## 3 audi         a4              2    2008      4 manual(m6) d        20
## 4 audi         a4              2    2008      4 auto(av)  d        21
## 5 audi         a4              2.8  1999      6 auto(l5)  d        16
## 6 audi         a4              2.8  1999      6 manual(m5) d        18
## 7 audi         a4              3.1  2008      6 auto(av)  d        18
## 8 audi         a4 quattro      1.8  1999      4 manual(m5) 4        18
## 9 audi         a4 quattro      1.8  1999      4 auto(l5)  4        16
## 10 audi        a4 quattro      2    2008      4 manual(m6) 4        20
## # i 224 more rows
## # i 3 more variables: autopista <int>, combustible <chr>, clase <chr>
```

Ejercicio para someter:

1. baja el paquete “ggversa”
2. activar el paquete “ggversa”
3. mirar las variables del archivo en este paquete que se llama “Anolis”
4. haga un gráfico que incluye lo siguiente
 - a. en el eje de x = el SVL. que es el tamaño del lagarto del hocico a la

- cloaca y en la variable de “TAIL” en el eje de y.
- b. selecciona la variable “SEX_AGE” para color
- c. salva el gráfico en .png o .jpg
- d. subir el gráfico aquí

```
library(ggversa)
head(Anolis)
```

```
## # A tibble: 6 x 15
##   STUDY      Survey_Site      LOCATION TIME DATE SEASON SPECIES SEX_AGE HEIGHT
##   <chr>      <chr>      <chr>    <tim> <chr> <chr> <chr>    <chr>    <dbl>
## 1 Mark/recap North Tower      El Verde 10:46 3/13~ dry   Anolis~ Female      0
## 2 Mark/recap Woods walkway t~ El Verde 10:15 2/20~ dry   Anolis~ Juvenil    0
## 3 Mark/recap Woods walkway t~ El Verde 11:15 2/21~ dry   Anolis~ Male        0
## 4 Mark/recap North Tower      El Verde 11:06 3/16~ dry   Anolis~ Juvenil    0.3
## 5 Mark/recap North Tower      El Verde 12:31 3/11~ dry   Anolis~ Male        0.3
## 6 Mark/recap North Tower      El Verde 01:00 3/9/~ dry   Anolis~ Female      0.4
## # i 6 more variables: DISTANCE_FROM_CENTERLINE <dbl>, PERCH_SUBSTRATE <chr>,
## #   PERCH_DIAMETER <int>, WEIGHT <dbl>, SVL <dbl>, TAIL <dbl>
```

3.3.5 Extracción de valores de un conjunto de datos

`dim()`, `nrow()`, `ncol()`, `unique()` — explorar la forma de los datos.

- `dim(x)`: filas y columnas; `nrow(x)` / `ncol(x)` dan cada una por separado.
- `unique(x$col)`: los valores distintos de una columna (útil para variables categóricas).
- `max()` / `min()`: valor máximo/mínimo de una variable numérica.

Aquí usamos el conjunto de datos de **diamantes** que se encuentra en el paquete **ggplot2**

3.3.5.1 Los primeros datos de un archivo

```
head(diamantes)
```

```
## # A tibble: 6 x 10
##   precio quilate corte      color claridad profundidad tabla      x      y      z
##   <int>    <dbl> <ord>    <ord> <ord>      <dbl> <dbl> <dbl> <dbl> <dbl>
## 1   326    0.23 Ideal      E     SI2        61.5   55  3.95  3.98  2.43
## 2   326    0.21 Premium    E     SI1        59.8   61  3.89  3.84  2.31
## 3   327    0.23 Bueno      E     VS1        56.9   65  4.05  4.07  2.31
## 4   334    0.29 Premium    I     VS2        62.4   58  4.2   4.23  2.63
## 5   335    0.31 Bueno      J     SI2        63.3   58  4.34  4.35  2.75
## 6   336    0.24 Muy bueno J     VVS2        62.8   57  3.94  3.96  2.48
```

3.3.5.2 La cantidad de fila

```
nrow(diamantes)
```

```
## [1] 53940
```

3.3.5.3 La cantidad de columnas (variables)

```
ncol(diamantes)
```

```
## [1] 10
```

3.3.5.4 Las dimensiones de un archivo

```
dim(diamantes)
```

```
## [1] 53940    10
```

3.3.5.5 El valor máximo de una variable

```
max(diamantes$precio)
```

```
## [1] 18823
```

3.3.5.6 El valor mínimo de una variable

```
min(diamantes$precio)
```

```
## [1] 326
```

3.3.5.7 Los valores discretos de una variable

```
unique(diamantes$corte)
```

```
## [1] Ideal    Premium  Bueno    Muy bueno Regular  
## Levels: Regular < Bueno < Muy bueno < Premium < Ideal
```

- `diamantes$corte`
- `diamantes$precio`

3.4 Ejemplos de Gráficos

```
library(readr)  
Vuelos_SJU_2018_Ene <- read_csv("Datos/Vuelos_SJU_2018_Ene.csv")
```

```
head(Vuelos_SJU_2018_Ene)
```

```
## # A tibble: 6 x 14
##   FL_DATE OP_UNIQUE_CARRIER ORIGIN ORIGIN_CITY_NAME ORIGIN_STATE_ABR DEST
##   <chr>   <chr>                <chr> <chr>                <chr>   <chr>
## 1 2/1/18  NK                      SJU   San Juan, PR        PR      MCO
## 2 2/1/18  AA                      MIA   Miami, FL           FL      SJU
## 3 2/1/18  AA                      SJU   San Juan, PR        PR      DFW
## 4 2/1/18  AA                      SJU   San Juan, PR        PR      MIA
## 5 2/1/18  AA                      SJU   San Juan, PR        PR      ORD
## 6 2/1/18  AA                      MIA   Miami, FL           FL      SJU
## # i 8 more variables: DEST_CITY_NAME <chr>, DEST_STATE_ABR <chr>,
## #   DEP_TIME <dbl>, DEP_DELAY <dbl>, CRS_ARR_TIME <dbl>, ARR_TIME <dbl>,
## #   ARR_DELAY <dbl>, CANCELLED <dbl>
```


Chapter 4

Visualización de datos

[1] "2026-08-01"

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/data-visualisation.html>

Español: <https://es.r4ds.hadley.nz/03-visualize.html>

4.1 Temas:

- Introducción
 - paquete “tidyverse”
 - datos de “mgp”
 - ggplot2
 - El concepto de la gramática de gráficos
-

4.1.1 Los paquetes

```
library(tidyverse)
library(ggversa)
library(datos)
```

ggplot2 implementa la *gramática de gráficos* (*Grammar of Graphics*), un marco propuesto por **Leland Wilkinson** en 1999. **Hadley Wickham** lo llevó a R en 2007, con la idea de construir cualquier gráfico por capas. Hoy es uno de los paquetes más usados del mundo R.

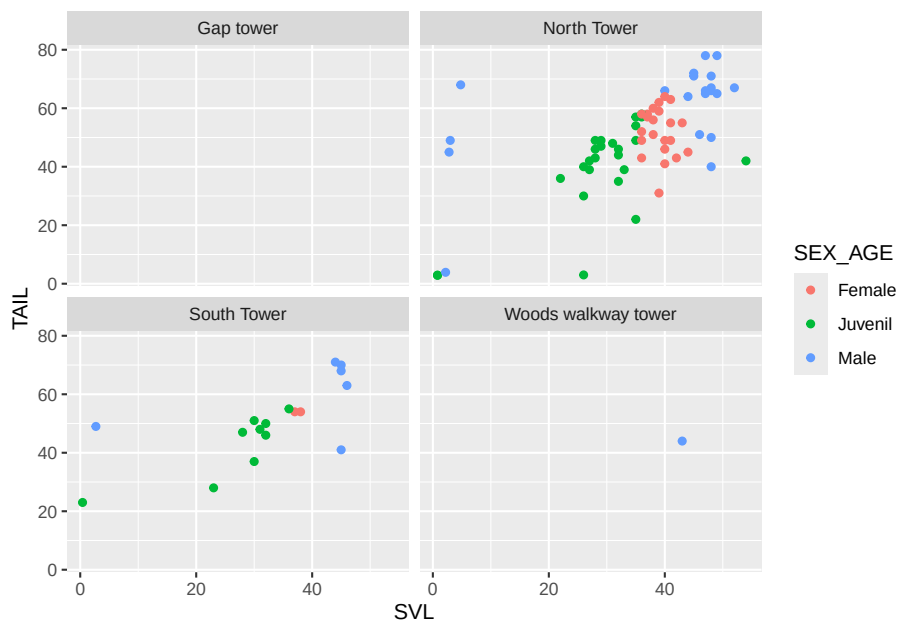
`ggplot(data) + geom_*(aes(...))` — construye gráficos por capas.

- `ggplot(data)`: define el conjunto de datos.
- `aes(x, y, color, ...)`: la *estética* — conecta variables con los ejes, el color, la forma, etc.
- `+ geom_*`: la geometría (`geom_point`, `geom_smooth`, `geom_bar`, `geom_boxplot`...). Las capas se **suman** con `+`.

4.1.2 Facet_wrap

`facet_wrap(~ variable)` divide el gráfico en un panel por cada categoría de la variable; es ideal para comparar grupos sin saturar una sola figura.

```
ggplot(data = Anolis) +
  geom_point(aes(x = SVL, y = TAIL, color=SEX_AGE)) +
  facet_wrap(~Survey_Site)
```

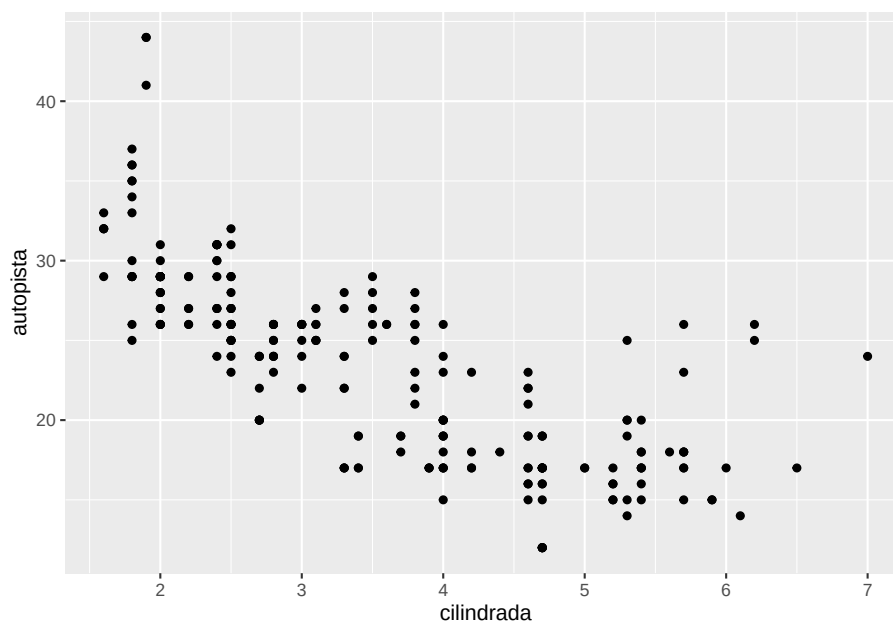


4.1.3 Regresión lineal

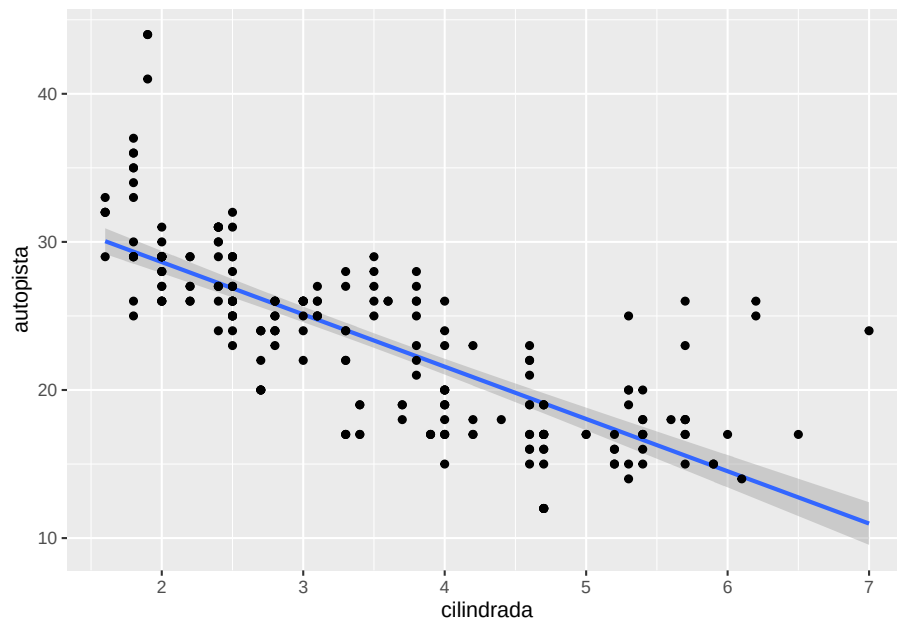
`geom_smooth(method = ...)` — añade una línea de tendencia.

- **Qué hace**: ajusta y dibuja una curva que resume la relación entre `x` e `y`, con su banda de confianza.
- `method = "lm"`: recta de regresión lineal. Sin especificar `method`, usa **LOESS** (una curva suave local) para muestras no muy grandes.
- `se = FALSE`: oculta la banda de confianza.

```
# izquierda
ggplot(data = millas) +
  geom_point(aes(x = cilindrada, y = autopista))
```



```
# derecha Linear Model
ggplot(data = millas) +
  geom_smooth(method=lm,mapping = aes(x = cilindrada, y = autopista))+ #  $y = mx+b$ 
  geom_point(aes(x = cilindrada, y = autopista))
```

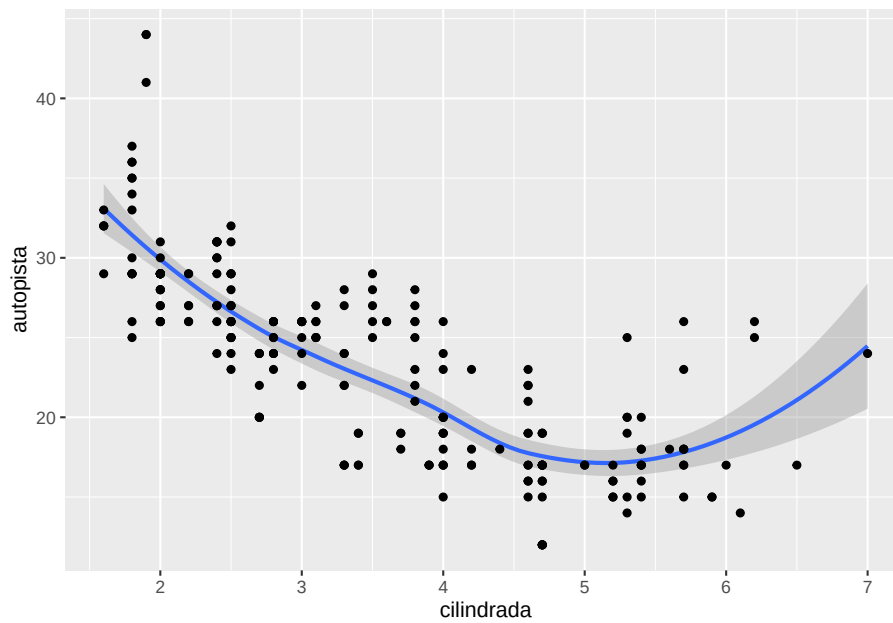


4.1.4 Regresión Loess

Vea este enlace para información sobre la regresión LOESS

<https://en.wikipedia.org/wiki/Local_regression>

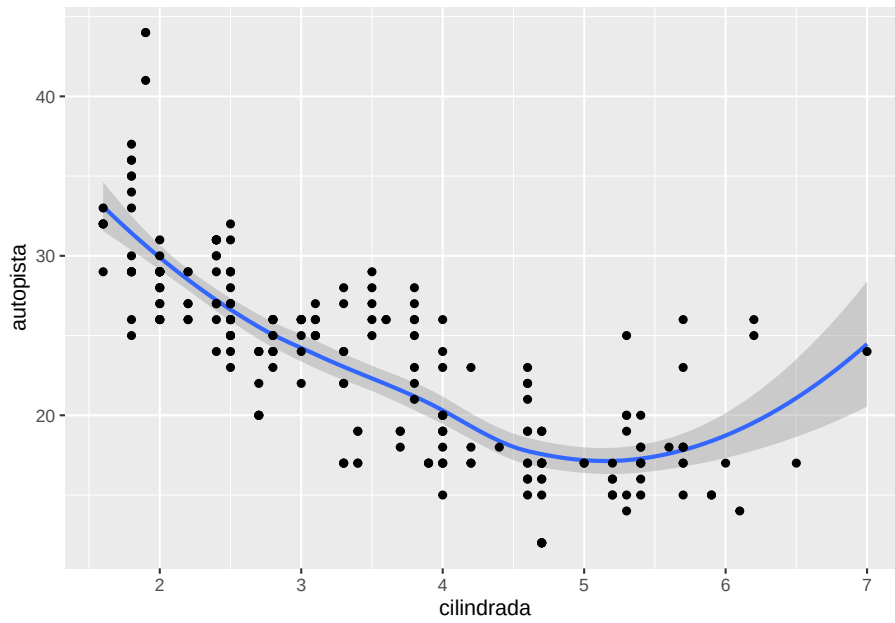
```
ggplot(data = millas) +  
  geom_smooth(mapping = aes(x = cilindrada, y = autopista)) + #  $y = mx + b$   
  geom_point(aes(x = cilindrada, y = autopista))
```



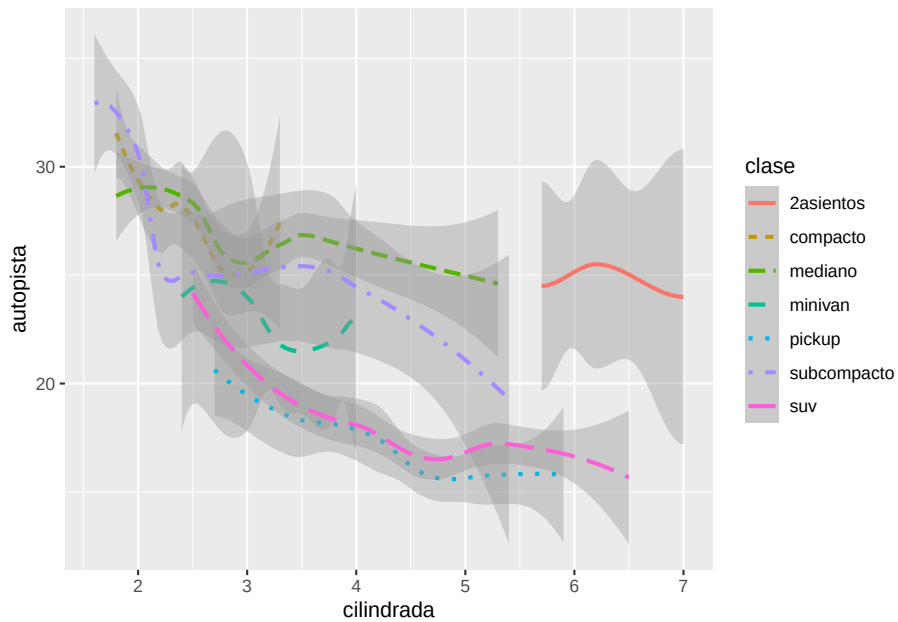
```
names(millas)
```

```
## [1] "fabricante" "modelo"      "cilindrada" "anio"      "cilindros"
## [6] "transmision" "traccion"   "ciudad"     "autopista" "combustible"
## [11] "clase"
```

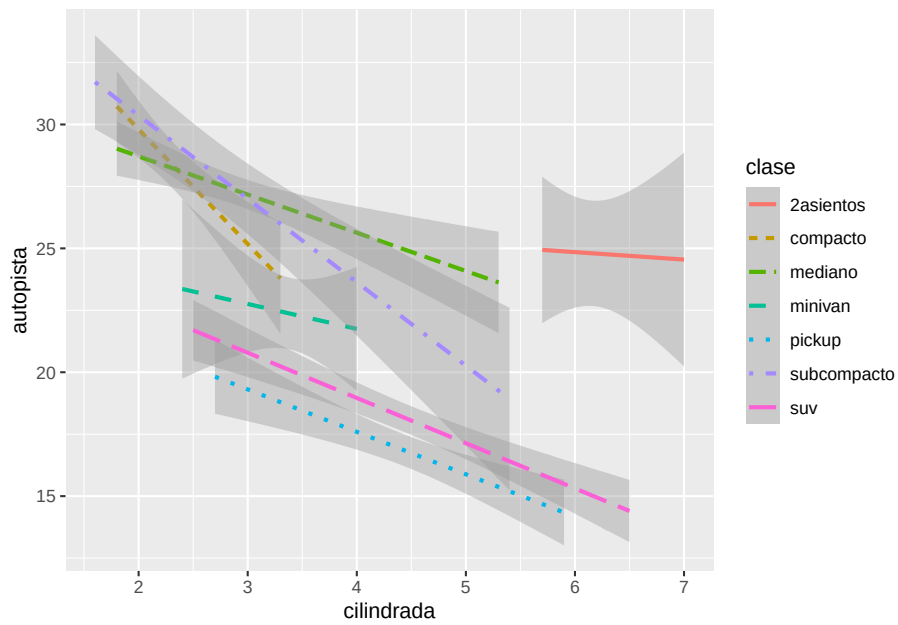
```
ggplot(data = millas) +
  geom_smooth(aes(x = cilindrada, y = autopista)) +
  geom_point(aes(x = cilindrada, y = autopista))
```



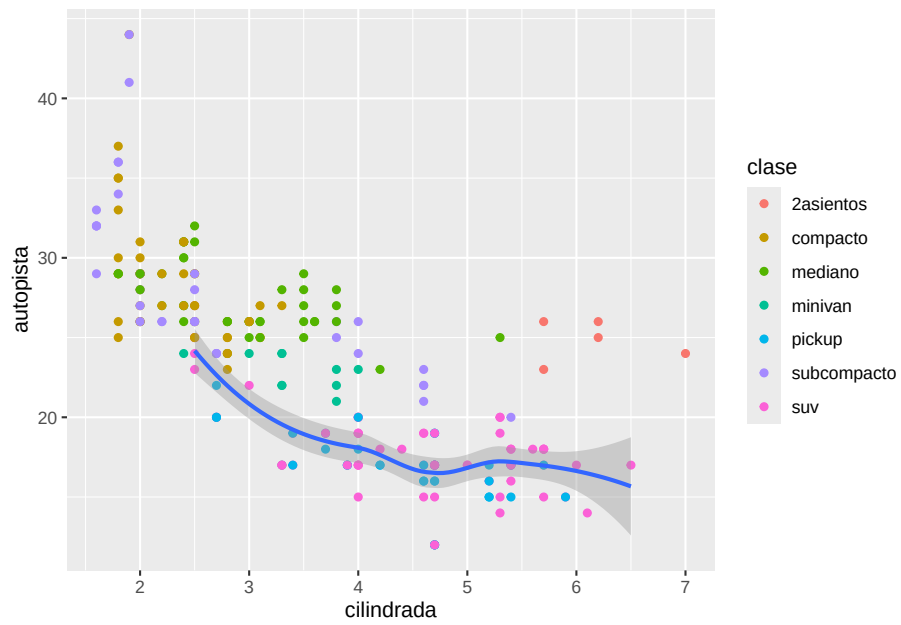
```
ggplot(data = millas) +  
  geom_smooth(aes(x = cilindrada, y = autopista, linetype = clase, colour=clase))
```



```
ggplot(data = millas) +  
  geom_smooth(method=lm, aes(x = cilindrada, y = autopista, linetype = clase, colour=clase))
```

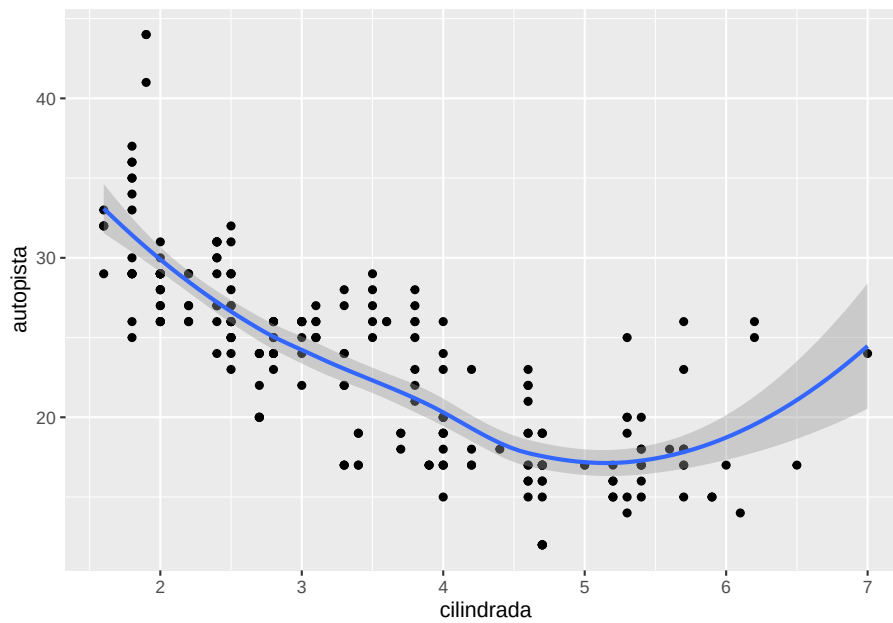


```
ggplot(millas,aes(x = cilindrada, y = autopista)) +
  geom_point(aes(color = clase)) +
  geom_smooth(data = filter(millas, clase == "suv"), se =TRUE)
```



```
ggplot(data = millas, mapping = aes(x = cilindrada, y = autopista)) +  
  geom_point() +  
  geom_smooth()
```

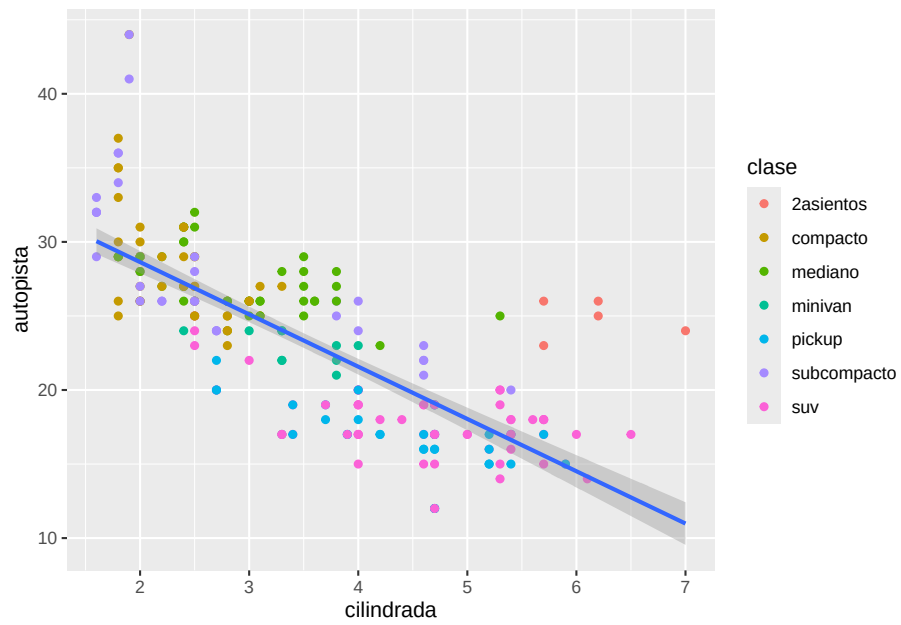
4.1.6 Regresión LOESS:



4.1.7 Regresión Lineal

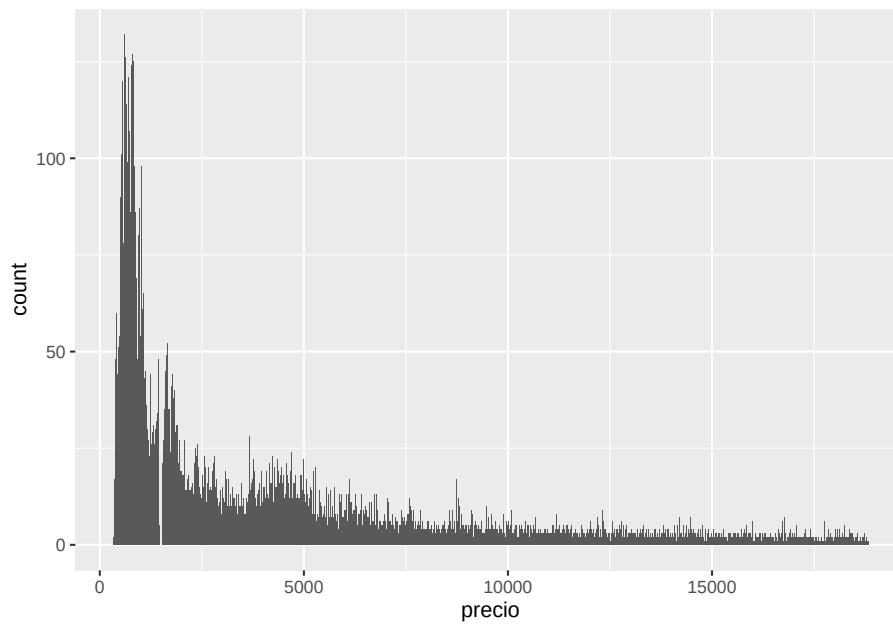
$$y = b + mx$$

```
ggplot(millas, mapping = aes(x = cilindrada, y = autopista)) +  
  geom_point(mapping = aes(color = clase)) +  
  geom_smooth(method=lm)
```

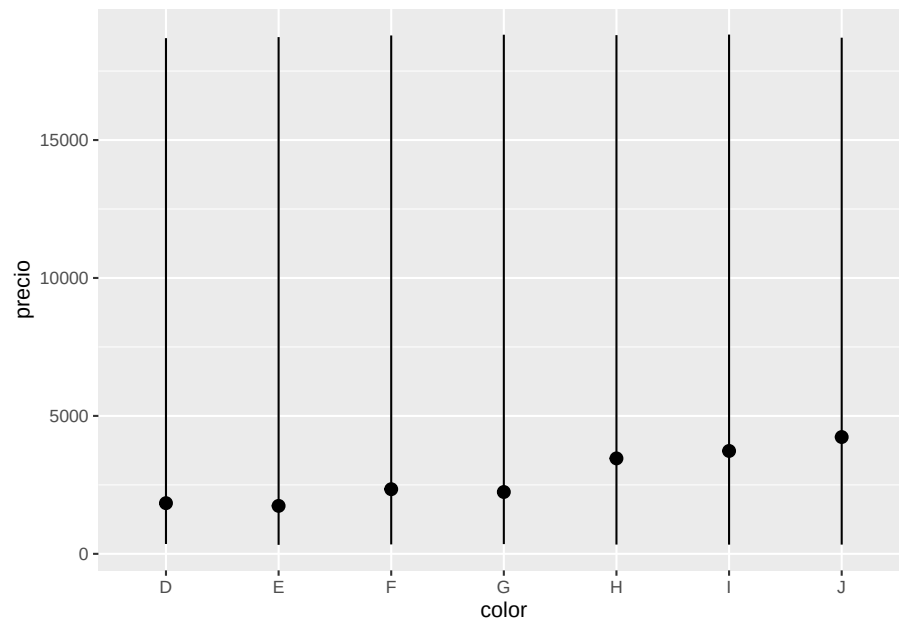


```
ggplot(diamantes) +  
  geom_bar(aes(x = precio))
```

4.1.8 geom_bar()

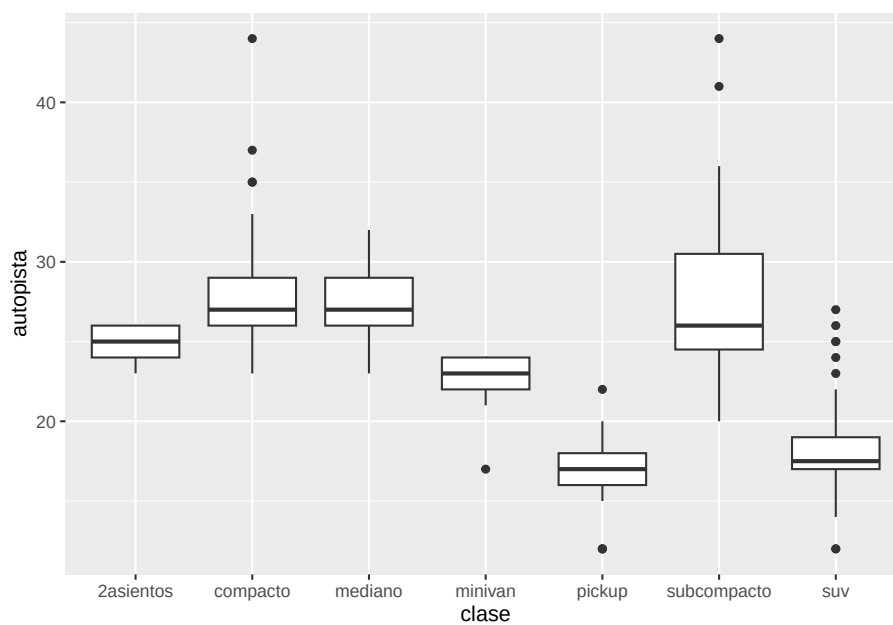


```
ggplot(diamantes) +  
  stat_summary(  
    mapping = aes(x = color, y = precio),  
    fun.min = min,  
    fun.max = max,  
    fun = median  
  )
```

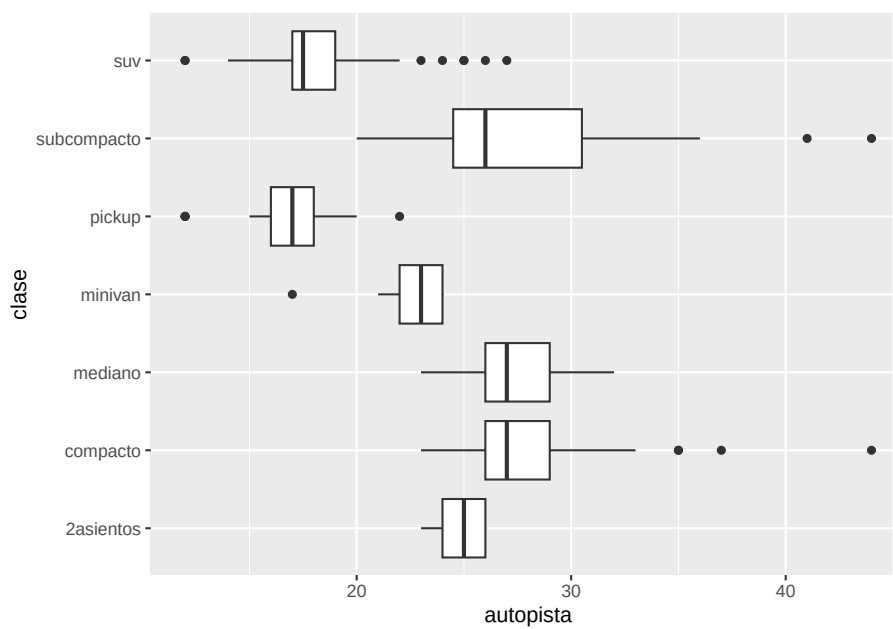


```
ggplot(millas, aes(x = clase, y = autopista)) +  
  geom_boxplot()
```

4.1.10 geom_boxplot

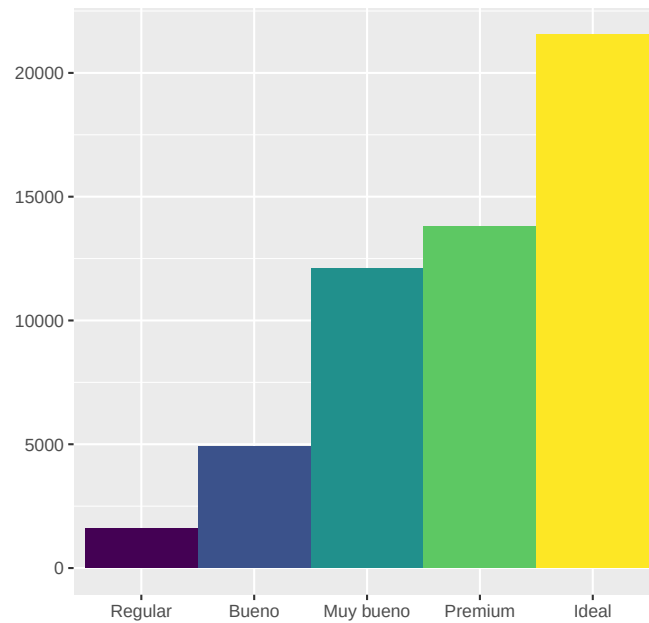


```
ggplot( millas, aes(x = clase, y = autopista)) +
  geom_boxplot() +
  coord_flip()
```

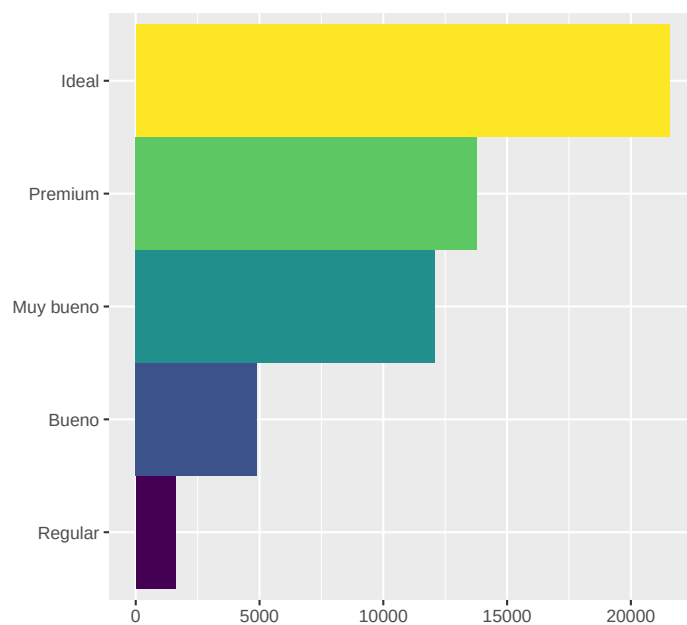


```
bar <- ggplot(data = diamantes) +  
  geom_bar(aes(x = corte, fill = corte),  
    show.legend = FALSE,  
    width = 1  
  ) +  
  theme(aspect.ratio = 1) +  
  labs(x = NULL, y = NULL)
```

bar

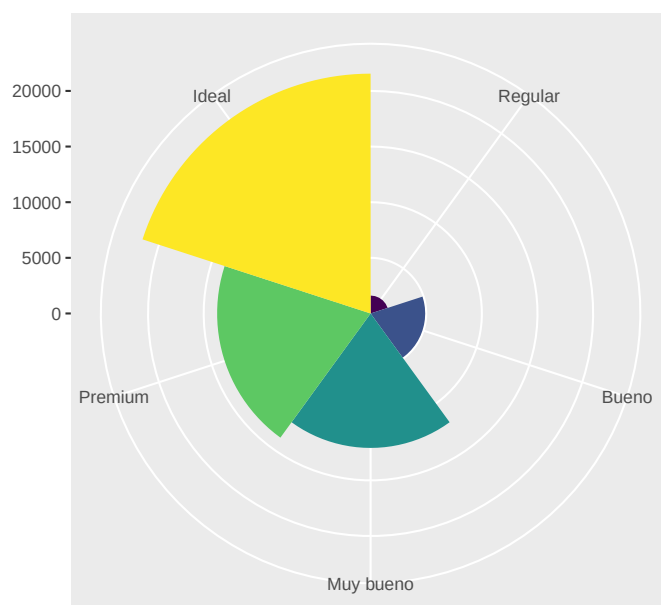


```
bar + coord_flip()
```



4.1.11.3 geom_bar & circular distribution

```
bar + coord_polar()
```



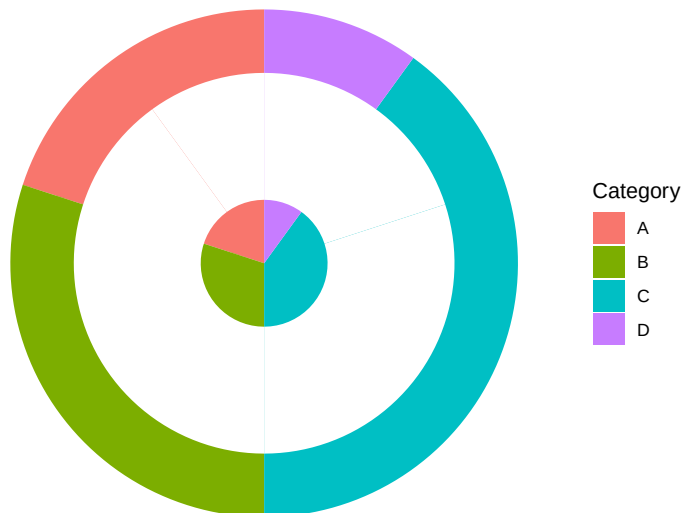
Donut Chart

```
library(ggplot2)
library(gridExtra)
data <- data.frame(
  Category = c("A", "B", "C", "D"),
  Value = c(20, 30, 40, 10)
)

pie_chart <- ggplot(data, aes(x = "", y = Value, fill = Category)) +
  geom_bar(stat = "identity", width = 1) +
  coord_polar(theta = "y") +
  theme_void() +
  labs(fill = "Category")

donut_chart <- pie_chart +
  geom_bar(data = data, aes(x = "", y = Value), stat = "identity",
    width = 0.5, fill = "white")+
  theme_void()

donut_chart
```



4.2 Leer datos de un archivo

1. Ejercicios:

Hacer los ejercicios en la sección 3.2.4 del libro en español

-
- Estética
-

2. Ejercicios:

Hacer los ejercicios en la sección 3.3.1 del libro en español

-
- Problemas comunes
 - Separar en facetas
-

3. Ejercicios:

Hacer los ejercicios en la sección 3.5.1 del libro en español

-
- Objetos geométricos ***

4. Ejercicios:

Hacer los ejercicios en la sección 3.6.1 del libro en español

-
- Transformación estadísticas
-

5. Ejercicios:

Hacer los ejercicios en la sección 3.7.1 del libro en español

-
- Ajuste de posición
-

6. Ejercicios:

Hacer los ejercicios en la sección 3.8.1 del libro en español

-
- Sistema de coordenadas
-

7. Ejercicios:

Hacer los ejercicios en la sección 3.9.1 del libro en español

Ejercicio para entregar (6 puntos)

1. Activa el paquete “ggversa”
2. Activa el paquete “tidyverse”
3. Utiliza los datos “PartosInfantes”. Leen la información sobre el archivo
Son tres graficas que tendrán que someter
4. Hacer un gráfico de puntos entre el número de muertes de infante y la cantidad de madres que mueren en el parto. (1 punto)
5. Añadir al gráfico anterior un modelo lineal (linear model). Y Demostrando todos los datos con un color por región geográfica, o sea añadir un color a los puntos por Grupo “region geográfica”. AM=America, EU= Union Europea, AF= Africa, O=Oceania, AS=Asia, Medio Oriente, (2 puntos)
6. Enseña el modelo de regresión lineal solamente para AFRICA y ASIA (en la misma gráfica) (3 puntos)

Someter las tres gráficas en formato .jpeg o .png en el portal.

```
library(ggplot2)
library(tidyverse)
library(ggversa)
```

```
head(PartosInfantes)
```

##		NMI	NMP	GSPC	Grupo	Pais
## 1	8723	400	605.1878	AM	Argentina	
## 2	60	5	1720.1595	AM	Bahamas	
## 3	42	1	1146.0417	AM	Barbados	
## 4	121	2	278.5792	AM	Belize	
## 5	7756	540	208.7842	AM	Bolivia	
## 6	45682	1400	947.4277	AM	Brazil	

```
#unique(PartosInfantes$Pais)
```

Hacer un gráfico de puntos entre el número de muertes por infante y la cantidad de madres que mueren en el parto

Añadir al gráfico anterior un modelo lineal (linear model)

Añadir un color a los puntos por grupo “region”. AM=America, EU= Union Europea, AF= Africa, O=Oceania, AS=Asia, Medio Oriente,

Demostrando todos los datos con un color por región geográfica, enseña el modelo de regresión lineal solamente para AFRICA

4.2.1 Ejercicios

Buscar el error en el código.

1. ¿Qué no va bien en este código? ¿Por qué hay puntos que no son azules en el siguiente código?

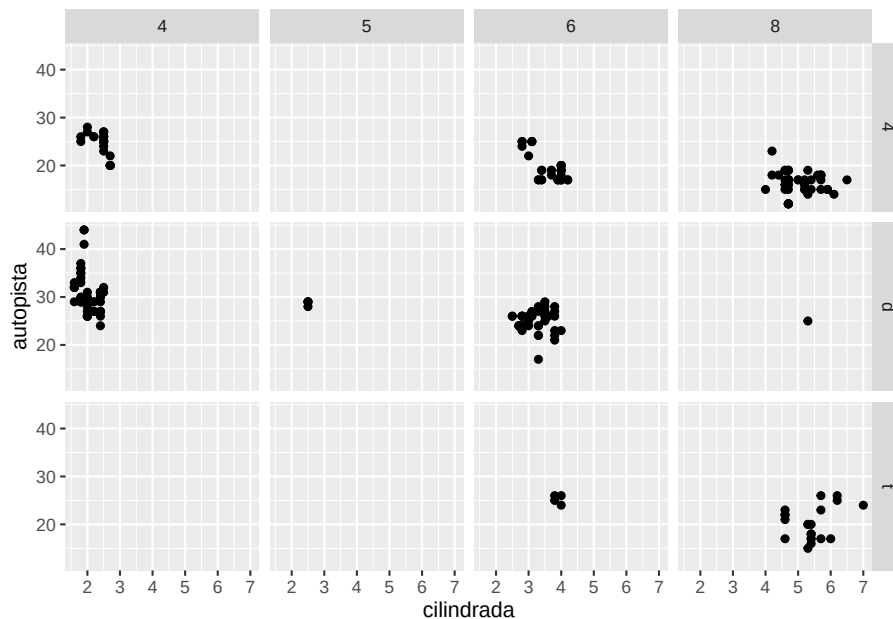
```
ggplot(data = millas) +
  geom_point(mapping = aes(x = cilindrada, y = autopista, color = "blue"))
```

2. Por que la gráfica no se produce en el siguiente código

```
ggplot(data = millas)
+ geom_point(mapping = aes(x = cilindrada, y = autopista))
```

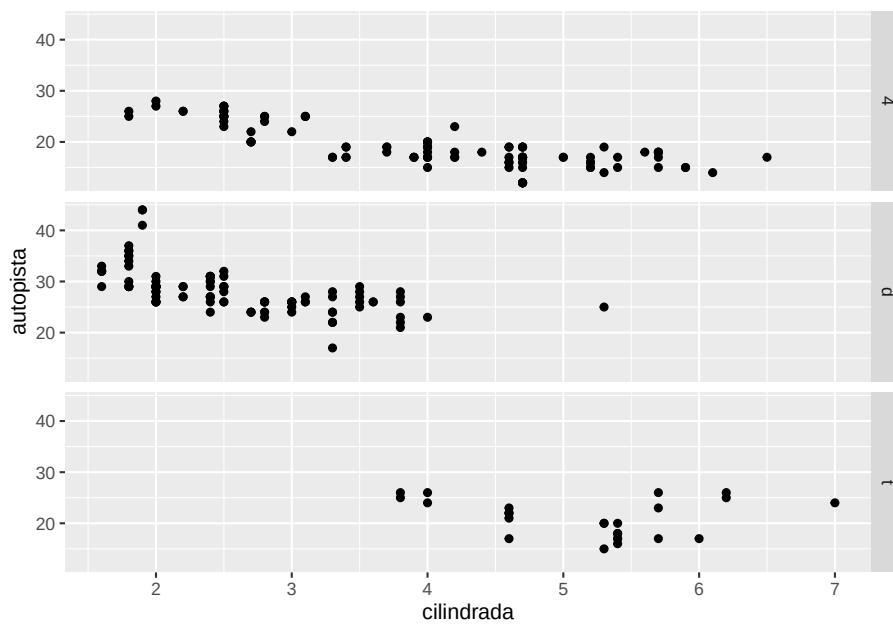
4.3 Vea el siguiente gráfico

```
ggplot(data = millas) +
  geom_point(mapping = aes(x = cilindrada, y = autopista)) +
  facet_grid(traccion ~ cilindros)
```

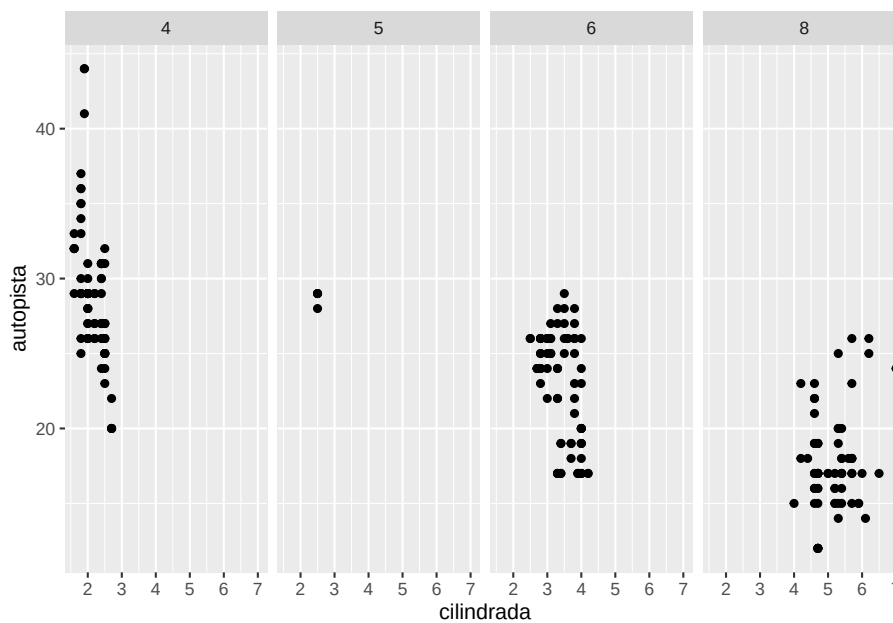


3. ¿Por qué hay facetas que son vacías?
4. Explica que hace "."? en estas funciones

```
ggplot(data = millas) +
  geom_point(mapping = aes(x = cilindrada, y = autopista)) +
  facet_grid(traccion ~ .)
```

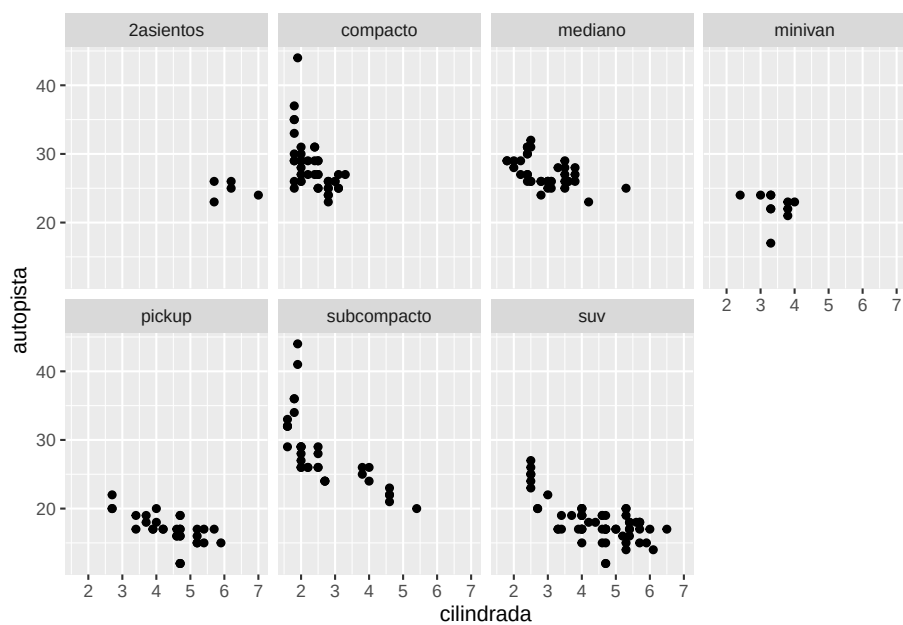


```
ggplot(data = millas) +
  geom_point(mapping = aes(x = cilindrada, y = autopista)) +
  facet_grid(. ~ cilindros)
```



5. Qué hace **nrow** en este código

```
ggplot(data = millas) +  
  geom_point(mapping = aes(x = cilindrada, y = autopista)) +  
  facet_wrap(~ clase, nrow = 2)
```



Chapter 5

Calculadora sofisticada

Fecha de la última revisión

```
## [1] "2026-08-01"
```

Activar los siguientes paquetes

```
library(tidyverse)
library(datos)
```

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/workflow-basics.html>

Español: <https://es.r4ds.hadley.nz/04-workflow-basics.html>

5.1 Temas:

- Conocimientos básicos de programación
- La importancia de los nombres
- Usando funciones

5.1.1 Sumar

```
2+2
```

```
## [1] 4
```

5.1.2 Exactamente como una calculadora

```
(23+3+3)^2+2
```

```
## [1] 843
```

5.1.3 Trigonometría

```
sin(pi/3)
```

```
## [1] 0.8660254
```

5.1.4 Log natural y a base 10

```
log(100)
```

```
## [1] 4.60517
```

```
log10(100)
```

```
## [1] 2
```

5.1.5 Asignaciones de variables

Para asignar un valor puedes usar `<-` o `=`, pero para **comparar** dos cosas se usa `==` (doble igual). Confundir `=` con `==` es uno de los errores más comunes al empezar. Encontrarás más errores frecuentes y cómo resolverlos en el capítulo *Errores comunes* (14).

```
x<- 4  
x
```

```
## [1] 4
```

```
y=3  
y
```

```
## [1] 3
```

```
x/y
```

```
## [1] 1.333333
```

5.1.6 Variables

- la función “seq” para secuencia

`seq()`, `c()`, `length()` — crear y medir vectores.

- `c(1, 2, 3)`: combina valores en un vector; `11:20` genera una secuencia consecutiva.
- `seq(101, 110)`: secuencia con más control (inicio, fin, paso).
- `length(x)`: cuántos elementos tiene un vector.

```
edad_nd=seq(101, 110)
edad_nd
```

```
## [1] 101 102 103 104 105 106 107 108 109 110
```

- variables asignada

```
edad_uprh=c(11:20)
edad_uprh
```

```
## [1] 11 12 13 14 15 16 17 18 19 20
```

5.1.7 Unir variables en un data frame

```
edad=data.frame(edad_nd, edad_uprh)
edad
```

```
##      edad_nd edad_uprh
## 1         101         11
## 2         102         12
## 3         103         13
## 4         104         14
## 5         105         15
## 6         106         16
## 7         107         17
## 8         108         18
## 9         109         19
## 10        110         20
```

```
edad$diff=edad$edad_nd-edad$edad_uprh
edad
```

```
##      edad_nd edad_uprh diff
## 1         101         11   90
## 2         102         12   90
## 3         103         13   90
## 4         104         14   90
## 5         105         15   90
## 6         106         16   90
## 7         107         17   90
## 8         108         18   90
## 9         109         19   90
## 10        110         20   90
```

```
edad$tres=edad$edad_uprh^3
edad
```

```
##      edad_nd edad_uprh diff tres
## 1         101         11   90 1331
## 2         102         12   90 1728
## 3         103         13   90 2197
## 4         104         14   90 2744
## 5         105         15   90 3375
## 6         106         16   90 4096
## 7         107         17   90 4913
## 8         108         18   90 5832
## 9         109         19   90 6859
## 10        110         20   90 8000
```

5.1.8 Errores cuando a variables no tienen la misma cantidad de entradas

```
edad_nd_1=base::seq(101, 111)
edad_nd_1
```

```
## [1] 101 102 103 104 105 106 107 108 109 110 111
```

- variables asignada

```
edad_uprh_1=c(11:20, NA)
edad_uprh_1
```

```
## [1] 11 12 13 14 15 16 17 18 19 20 NA
```

5.1.9 Unir variables en un data frame

```
edad_1=data.frame(edad_nd_1, edad_uprh_1)
edad_1
```

```
##      edad_nd_1 edad_uprh_1
## 1          101          11
## 2          102          12
## 3          103          13
## 4          104          14
## 5          105          15
## 6          106          16
## 7          107          17
## 8          108          18
## 9          109          19
## 10         110          20
```

```
## 11      111      NA
```

5.1.10 Cuantos valores hay en una variable

```
length(edad_uprh)
```

```
## [1] 10
```

```
length(edad$tres)
```

```
## [1] 10
```

```
length(edad)
```

```
## [1] 4
```

5.1.11 Seleccionar solamente parte de los datos de un archivo

- Primero visualizar los datos

```
library(datos)
```

```
millas
```

```
## # A tibble: 234 x 11
```

##	fabricante	modelo	cilindrada	anio	cilindros	transmision	traccion	ciudad
##	<chr>	<chr>	<dbl>	<int>	<int>	<chr>	<chr>	<int>
## 1	audi	a4	1.8	1999	4	auto(15)	d	18
## 2	audi	a4	1.8	1999	4	manual(m5)	d	21
## 3	audi	a4	2	2008	4	manual(m6)	d	20
## 4	audi	a4	2	2008	4	auto(av)	d	21
## 5	audi	a4	2.8	1999	6	auto(15)	d	16
## 6	audi	a4	2.8	1999	6	manual(m5)	d	18
## 7	audi	a4	3.1	2008	6	auto(av)	d	18
## 8	audi	a4 quattro	1.8	1999	4	manual(m5)	4	18
## 9	audi	a4 quattro	1.8	1999	4	auto(15)	4	16
## 10	audi	a4 quattro	2	2008	4	manual(m6)	4	20

```
## # i 224 more rows
```

```
## # i 3 more variables: autopista <int>, combustible <chr>, clase <chr>
```

5.1.11.1 Seleccionar los carros que tienen cilindros igual a 8 (solamente)

El pipe %>% lo creó Stefan Milton Bache en el paquete `magrittr` (2014); su nombre juega con el cuadro de Magritte “Ceci n’est pas une pipe”. Desde 2021, R incluye su propio pipe nativo `|>`.

La función de Pipe. %>% o |>

`select()` y `filter()` (paquete **dplyr**) — elegir columnas y filas.

- `select(columna)`: se queda con las **columnas** indicadas.
- `filter(condición)`: se queda con las **filas** que cumplen una condición (p. ej. `cilindros == 8`).
- El pipe `|>` / `%>%`: encadena pasos, pasando el resultado de uno como entrada del siguiente.

```
library(tidyverse)
millas
```

```
## # A tibble: 234 x 11
##   fabricante modelo      cilindrada  anio cilindros transmision traccion ciudad
##   <chr>      <chr>          <dbl> <int>    <int> <chr>      <chr>    <int>
## 1 audi      a4              1.8  1999      4 auto(15)    d        18
## 2 audi      a4              1.8  1999      4 manual(m5)  d        21
## 3 audi      a4              2    2008      4 manual(m6)  d        20
## 4 audi      a4              2    2008      4 auto(av)    d        21
## 5 audi      a4              2.8  1999      6 auto(15)    d        16
## 6 audi      a4              2.8  1999      6 manual(m5)  d        18
## 7 audi      a4              3.1  2008      6 auto(av)    d        18
## 8 audi      a4 quattro      1.8  1999      4 manual(m5)  4        18
## 9 audi      a4 quattro      1.8  1999      4 auto(15)    4        16
## 10 audi     a4 quattro      2    2008      4 manual(m6)  4        20
## # i 224 more rows
## # i 3 more variables: autopista <int>, combustible <chr>, clase <chr>
```

```
names(millas)
```

```
## [1] "fabricante" "modelo"      "cilindrada"  "anio"        "cilindros"
## [6] "transmision" "traccion"    "ciudad"      "autopista"   "combustible"
## [11] "clase"
```

```
millas |> dplyr::select(cilindros) |>
dplyr::filter(cilindros == 8)
```

```
## # A tibble: 70 x 1
##   cilindros
##   <int>
## 1      8
## 2      8
## 3      8
## 4      8
## 5      8
## 6      8
## 7      8
## 8      8
## 9      8
## 10     8
```

```
## # i 60 more rows
```

5.1.11.2 Seleccionar los diamantes mayor de 3 quilate

```
diamantes |> dplyr::select(quilate) |>
  filter(quilate > 3)
```

```
## # A tibble: 32 x 1
##   quilate
##   <dbl>
## 1     3.01
## 2     3.11
## 3     3.01
## 4     3.05
## 5     3.02
## 6     3.01
## 7     3.65
## 8     3.24
## 9     3.22
## 10    3.5
## # i 22 more rows
```

5.1.11.3 Seleccionar los diamantes con igual o mayor de 4000 precio

```
## # A tibble: 6 x 1
##   precio
##   <int>
## 1    4000
## 2    4001
## 3    4001
## 4    4001
## 5    4001
## 6    4002
```

5.1.11.4 Contabiliza cuantos diamantes tienen un valor de \$4000 o más

```
nrow(dia_4000)
```

```
## [1] 19380
```


Chapter 6

Transformación: Estructura de un archivo y seleccionar datos

```
## [1] "2026-08-01"
```

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/transform.html>

Español: <https://es.r4ds.hadley.nz/05-transform.html>

```
library(tidyverse)
library(datos)
head(vuelos)
```

```
## # A tibble: 6 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>             <int>         <dbl>
## 1  2013     1     1           517               515             2
## 2  2013     1     1           533               529             4
## 3  2013     1     1           542               540             2
## 4  2013     1     1           544               545            -1
## 5  2013     1     1           554               600            -6
## 6  2013     1     1           554               558            -4
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
```

```
## # hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

6.1 Evaluar el archivo el vuelo

Mirar el archivo y entender la información antes de hacer cualquier análisis/

- Cuáles son las variables
- Cuáles son los tipos de datos
 - **int** valores enteros,
 - **dbl** números reales, significa dobles
 - **chr** caracteres o vectores
 - **dtm** fecha y tiempo, minutos
 - **date** fecha
 - **S3: POSIXct** Fecha y tiempo, minutos
 - **lgl** valores lógicos “cierto” (TRUE) y “falso” (FALSE)
 - **ctr** factores categóricos.

Evalua en este archivo los tipos de variables que se encuentra.

```
vuelos
```

```
## # A tibble: 336,776 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>             <int>         <dbl>
## 1  2013     1     1           517               515             2
## 2  2013     1     1           533               529             4
## 3  2013     1     1           542               540             2
## 4  2013     1     1           544               545            -1
## 5  2013     1     1           554               600            -6
## 6  2013     1     1           554               558            -4
## 7  2013     1     1           555               600            -5
## 8  2013     1     1           557               600            -3
## 9  2013     1     1           557               600            -3
## 10 2013     1     1           558               600            -2
## # i 336,766 more rows
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

6.1.1 Cual son todos los destinos de vuelos?

- cual función usará

```
sort(unique(vuelos$destino))
```

```
## [1] "ABQ" "ACK" "ALB" "ANC" "ATL" "AUS" "AVL" "BDL" "BGR" "BHM" "BNA" "BOS"
## [13] "BQN" "BTV" "BUF" "BUR" "BWI" "BZN" "CAE" "CAK" "CHO" "CHS" "CLE" "CLT"
```



```
## [25] "CMH" "CRW" "CVG" "DAY" "DCA" "DEN" "DFW" "DSM" "DTW" "EGE" "EYW" "FLL"
## [37] "GRR" "GSO" "GSP" "HDN" "HNL" "HOU" "IAD" "IAH" "ILM" "IND" "JAC" "JAX"
## [49] "LAS" "LAX" "LEX" "LGA" "LGB" "MCI" "MCO" "MDW" "MEM" "MHT" "MIA" "MKE"
## [61] "MSN" "MSP" "MSY" "MTJ" "MVY" "MYR" "OAK" "OKC" "OMA" "ORD" "ORF" "PBI"
## [73] "PDX" "PHL" "PHX" "PIT" "PSE" "PSP" "PVD" "PWM" "RDU" "RIC" "ROC" "RSW"
## [85] "SAN" "SAT" "SAV" "SBN" "SDF" "SEA" "SFO" "SJC" "SJU" "SLC" "SMF" "SNA"
## [97] "SRQ" "STL" "STT" "SYR" "TPA" "TUL" "TVC" "TYS" "XNA"
```

6.1.2 Selecciona los vuelos donde el destino es San Juan

```
vuelos %>%
  filter(destino=="SJU")
```

```
## # A tibble: 5,819 x 19
##   anio   mes   dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>             <int>          <dbl>
## 1  2013     1     1           615               615            0
## 2  2013     1     1           628               630           -2
## 3  2013     1     1           701               700            1
## 4  2013     1     1           711               715           -4
## 5  2013     1     1           820               820            0
## 6  2013     1     1           820               820            0
## 7  2013     1     1           840               845           -5
## 8  2013     1     1           926               929           -3
## 9  2013     1     1          1202              1159            3
## 10 2013     1     1          1245              1249           -4
## # i 5,809 more rows
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

6.1.3 Selecciona los vuelos donde el destino es San Juan el día de navidad

Para comparar dentro de `filter()` se usa `==` (doble igual); un solo `=` es asignación y dará error. Para comparar contra varios valores, usa `%in% c("AA", "UA")` en vez de repetir `==`. Encontrarás más errores frecuentes y cómo resolverlos en el capítulo *Errores comunes* (14).

```
vuelos %>%
  filter(destino=="SJU") %>%
  filter(mes==12, dia==25) %>%
  filter(aerolinea == "UA")
```

```
## # A tibble: 3 x 19
```

```
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>          <int>          <int>          <dbl>
## 1 2013   12   25           736           739           -3
## 2 2013   12   25          1202          1206           -4
## 3 2013   12   25          2015          2017           -2
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

```
vuelos %>%
  filter(destino=="SJU") %>%
  filter(mes==12, dia==25) %>%
  filter(aerolinea %in% c("AA", "UA"))
```

```
## # A tibble: 6 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>          <int>          <int>          <dbl>
## 1 2013   12   25           736           739           -3
## 2 2013   12   25           759           805           -6
## 3 2013   12   25          1202          1206           -4
## 4 2013   12   25          1634          1630            4
## 5 2013   12   25          1908          1900            8
## 6 2013   12   25          2015          2017           -2
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

```
unique(vuelos$aerolinea)
```

```
## [1] "UA" "AA" "B6" "DL" "EV" "MQ" "US" "WN" "VX" "FL" "AS" "9E" "F9" "HA" "YV"
## [16] "OO"
```

```
# /> pipe Command shift M
```

6.1.4 Selecciona los vuelos que salen de San Juan en el día de tu cumpleaños y contabiliza cuanto hubo

```
df_dia_cumpl=vuelos %>%
  filter(destino=="SJU") %>%
  filter(mes== 2, dia %in% c(1,2)) |>
  nrow()

df_dia_cumpl
```

```
## [1] 28
```

6.2. TEMAS: RECONOCER Y APLICAR LAS DIFERENTES FUNCIONES67

6.1.4.1 Organizar los datos en orden de más grande a más pequeño

- Selecciona los mes, dia, destino, atraso_salida, atraso_llegada
- Selecciona solamente el mes de noviembre
- Selecciona solamente los vuelos a destino de a San Juan
- organiza los datos en orden de más atraso_salido de mayor a menor

```
names(vuelos)

## [1] "anio"          "mes"           "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
## [10] "aerolinea"      "vuelo"         "codigoCola"
## [13] "origen"         "destino"       "tiempo_vuelo"
## [16] "distancia"      "hora"          "minuto"
## [19] "fecha_hora"

vuelos %>%
  dplyr::select(mes, dia, destino, atraso_salida, atraso_llegada) %>%
  filter(mes==11) %>%
  filter(destino == "SJU") %>%
  arrange(desc(atraso_salida))

## # A tibble: 429 x 5
##   mes dia destino atraso_salida atraso_llegada
##   <int> <int> <chr>          <dbl>          <dbl>
## 1  11 7 SJU          231            190
## 2  11 28 SJU         223            235
## 3  11 23 SJU         187            154
## 4  11 24 SJU         181            188
## 5  11 23 SJU         157            165
## 6  11 12 SJU         145            127
## 7  11 10 SJU          94             88
## 8  11 24 SJU          90            112
## 9  11 30 SJU          68             59
## 10 11 21 SJU          66             42
## # i 419 more rows
```

6.2 Temas: Reconocer y aplicar las diferentes funciones

- Tipos de variables:
 - int: valores enteros
 - dbl: números reales
 - chr: caracteres
 - dttm: fecha y tiempo

- lgl: valores lógicos
- fctr: factores
- date: fecha

- Funciones de dplyr:
 - filter() : seleccionar filas
 - arrange(): ordenar filas
 - select(): seleccionar columnas
 - summarize(): resumir conjuntos de datos
 - group_by(): agrupar por variables

Los verbos de dplyr — cada uno hace una cosa y se encadenan con el pipe.

- **filter()**: selecciona **filas** por condición.
- **select()**: selecciona **columnas**.
- **arrange()**: **ordena** las filas (**desc()** para descendente).
- **group_by()** + **summarise()**: agrupa y calcula un **resumen** por grupo (media, mediana, conteo...).
- **mutate()**: crea columnas nuevas (ver capítulo de *mutate*).

dplyr fue creado por **Hadley Wickham** (2014) como una “gramática de la manipulación de datos”: un pequeño conjunto de verbos con nombres de acción. Sustituyó al paquete anterior **plyr** con una sintaxis más clara y rápida.

```
library(datos)
library(tidyverse)
por_dia <- vuelos %>% # pipe
  group_by(dia) %>% # group by day
  summarise(atraso_promedio = mean(atraso_salida, na.rm = TRUE))
por_dia
```

```
## # A tibble: 31 x 2
##       dia atraso_promedio
##   <int>         <dbl>
## 1     1           14.2
## 2     2           14.1
## 3     3           10.8
## 4     4            5.79
## 5     5            7.82
## 6     6            6.99
## 7     7           14.3
## 8     8           21.8
## 9     9           14.6
## 10    10           18.3
## # i 21 more rows
```

```
## The median of the data set by day of the month

por_dia_2 <- vuelos %>%
  group_by( dia) %>%
  summarise(atraso_promedio = mean(atraso_salida, na.rm = TRUE),
            atraso_mediana = median(atraso_salida, na.rm=TRUE))
por_dia_2
```

```
## # A tibble: 31 x 3
##       dia atraso_promedio atraso_mediana
##   <int>          <dbl>          <dbl>
## 1     1             14.2             -2
## 2     2             14.1             -1
## 3     3             10.8             -2
## 4     4              5.79            -3
## 5     5              7.82            -3
## 6     6              6.99            -2
## 7     7             14.3             -1
## 8     8             21.8             -1
## 9     9             14.6             -1
## 10    10             18.3             -1
## # i 21 more rows
```

6.3 Funciones

- summarise = resumir conjuntos de datos
- mean, # promedio
- median, # mediana
- mode, # moda
- operaciones boolean
-

```
# El valor más común en un conjunto de datos
# Crear una función: no existe en los paquetes de R

getmode <- function(v) {
  uniqv <- unique(v)
  uniqv[which.max(tabulate(match(v, uniqv)))]
}
```

```

por_dia_3=vuelos |>
  dplyr::select(dia, mes, atraso_salida) |>
  group_by(dia) %>%
  summarise(atraso_promedio = mean(atraso_salida, na.rm = TRUE),
            atraso_max=max(atraso_salida, na.rm=TRUE),
            atraso_mode = getmode(atraso_salida))
por_dia_3

```

```

## # A tibble: 31 x 4
##       dia atraso_promedio atraso_max atraso_mode
##   <int>          <dbl>      <dbl>      <dbl>
## 1     1           14.2        853         -5
## 2     2           14.1        696         -4
## 3     3           10.8        878         -4
## 4     4            5.79        545         -5
## 5     5            7.82        896         -5
## 6     6            6.99        589         -4
## 7     7           14.3        653         -5
## 8     8           21.8        520          NA
## 9     9           14.6       1301         -4
## 10    10           18.3       1126         -5
## # i 21 more rows

```

Añadir el promedio, mediana y moda al mismo data frame

```

## The mode of the value
library(DescTools)

por_dia_4 <- vuelos %>%
  group_by(dia) %>%
  summarise(atraso_promedio = mean(atraso_salida, na.rm = TRUE),
            atraso_median = median(atraso_salida, na.rm=TRUE),
            atraso_mode = getmode(atraso_salida))
por_dia_4

```

```

## # A tibble: 31 x 4
##       dia atraso_promedio atraso_median atraso_mode
##   <int>          <dbl>      <dbl>      <dbl>
## 1     1           14.2        -2         -5
## 2     2           14.1        -1         -4
## 3     3           10.8        -2         -4
## 4     4            5.79        -3         -5
## 5     5            7.82        -3         -5
## 6     6            6.99        -2         -4
## 7     7           14.3        -1         -5
## 8     8           21.8        -1          NA
## 9     9           14.6        -1         -4

```

```
## 10      10      18.3      -1      -5
## # i 21 more rows
```

6.4 Agrupar por múltiples variables

Aquí se agrupar por día y mes

```
library(tidyverse)
library(datos)
vuelos %>%
  dplyr::select(anio, mes, dia, atraso_llegada, destino, aerolinea) %>%
  filter(destino == "SJU") %>%
  filter(dia == 25) %>%
  # filter(mes == 12) %>%
  group_by(dia, aerolinea) %>%
  summarise(atraso = mean(atraso_llegada, na.rm = TRUE))

## # A tibble: 4 x 3
## # Groups:   dia [1]
##   dia aerolinea atraso
##   <int> <chr>      <dbl>
## 1    25 AA        -5.41
## 2    25 B6         4.21
## 3    25 DL        -4.19
## 4    25 UA        -4.33
```


Chapter 7

Transformación: Operaciones booleanas y lógicas

Fecha de la última revisión

```
## [1] "2026-08-01"
```

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/transform.html>

Español: <https://es.r4ds.hadley.nz/05-transform.html>

```
library(tidyverse)
library(datos)
head(vuelos)
```

```
## # A tibble: 6 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>             <int>          <dbl>
## 1  2013     1     1           517               515            2
## 2  2013     1     1           533               529            4
## 3  2013     1     1           542               540            2
## 4  2013     1     1           544               545           -1
## 5  2013     1     1           554               600           -6
## 6  2013     1     1           554               558           -4
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
```

```
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

Tema de este módulo:

- Filtrar filas con `filter()`
- Seleccionar columnas con `select()`
- Agrupar con `group_by()`
- Resumir con `summarise()`
- Ordenar con `arrange()`
- funciones Booleanas y lógicas

7.1 Agrupar por múltiples variables

Aquí se agrupa por día y mes

7.2 Operaciones lógicas boolean:

- `&` “ampersand” == “conjunción”
- `|` or = “o”
- `!`, no incluye, excluir...
- `%in%`, filtrar para múltiples valores
- `==`, es igual a... “exactamente”
- `<` “menor que”
- `>` “mayor que”
- `<=` “es menor o igual”
- `>=` “es mayor o igual”

`filter()` con operadores lógicos — combinar condiciones.

- `&` (y), `|` (o), `!` (no / excluir).
- `%in% c(...)`: ¿el valor está en esta lista? (evita repetir `==`).
- `between(x, a, b)`: equivale a `x >= a & x <= b`.
- **Comparaciones:** `==`, `!=`, `<`, `>`, `<=`, `>=`.

Boolean, que incluye múltiples opciones `%in%`

No confundas `=` con `==`. Un solo `=` asigna un valor, mientras que `==` compara si dos valores son iguales; dentro de `filter()` siempre se usa `==`. Para comparar contra varios valores usa `%in%` en lugar de encadenar muchos `==`. Encontrarás más errores frecuentes y cómo resolverlos en el capítulo *Errores comunes* (14).

```
vuelos %>%
  dplyr::select(anio, mes, dia, atraso_llegada, destino) %>%
  filter(destino == "SJU") %>%
  filter(mes %in% c(6:12) & dia == 25) %>%
  group_by(dia, mes) %>%
  summarise(atraso = mean(atraso_llegada, na.rm = TRUE))
```

```
## # A tibble: 7 x 3
## # Groups:   dia [1]
##   dia   mes atraso
##   <int> <int> <dbl>
## 1    25     6  34.3
## 2    25     7   3.58
## 3    25     8  11.1
## 4    25     9 -15.2
## 5    25    10  -3.77
## 6    25    11 -18.9
## 7    25    12   3.57
```

```
vuelos %>%
  dplyr::select(anio, mes, dia, atraso_salida, aerolinea) %>%
  group_by(mes, aerolinea) %>%
  filter(mes %in% c(1:6)) %>% # mes entre 1 y 6
  filter(aerolinea %in% c("AA", "UA")) %>%
  summarise(atraso = mean(atraso_salida, na.rm = TRUE))
```

```
## # A tibble: 12 x 3
## # Groups:   mes [6]
##   mes aerolinea atraso
##   <int> <chr>      <dbl>
## 1     1 AA         6.93
## 2     1 UA         8.33
## 3     2 AA         8.28
## 4     2 UA         7.71
## 5     3 AA         8.70
## 6     3 UA        11.7
## 7     4 AA        11.7
## 8     4 UA        13.7
## 9     5 AA         9.66
## 10    5 UA        12.3
## 11    6 AA        14.6
## 12    6 UA        20.3
```

7.2.1 Otra alternativa para filtrar

7.2.1.1 “|” or

```
vuelos %>%
  group_by(mes) %>%
  dplyr::select(anio, mes, dia, atraso_salida) %>%
  filter(mes == 1 | mes == 12) |> # mes 1 o 12
  summarise(atraso = mean(atraso_salida, na.rm = TRUE))
```

```
## # A tibble: 2 x 2
##   mes atraso
##   <int> <dbl>
## 1     1  10.0
## 2    12  16.6
```

7.2.1.2 & el “y”

```
names(vuelos)
```

```
## [1] "anio"           "mes"           "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
## [10] "aerolinea"      "vuelo"         "codigoCola"
## [13] "origen"         "destino"       "tiempo_vuelo"
## [16] "distancia"      "hora"          "minuto"
## [19] "fecha_hora"
```

```
vuelos %>%
  dplyr::select(anio, mes, dia, atraso_salida, aerolinea) %>%
  filter(mes == 12 & dia == 25 & aerolinea == "AA")
```

```
## # A tibble: 78 x 5
##   anio  mes  dia atraso_salida aerolinea
##   <int> <int> <int>         <dbl> <chr>
## 1  2013   12   25             2 AA
## 2  2013   12   25            -4 AA
## 3  2013   12   25             1 AA
## 4  2013   12   25             0 AA
## 5  2013   12   25            -3 AA
## 6  2013   12   25            -2 AA
## 7  2013   12   25            26 AA
## 8  2013   12   25            -7 AA
## 9  2013   12   25            -6 AA
## 10 2013   12   25            21 AA
## # i 68 more rows
```

7.3 Ejercicios

Como se seleccionaría los meses de diciembre pero desde la del 25 o más

7.4

!, **excluye algo**, En este caso estamos excluyendo las aerolíneas “AA” y “DL”

```
vuelos %>%
  dplyr::select(anio, mes, dia, atraso_salida, aerolinea) %>%
  filter(mes == 11 & !aerolinea %in% c("AA", "DL")) %>%
  group_by(aerolinea) %>%
  summarise(atraso = mean(atraso_salida, na.rm = TRUE))
```

```
## # A tibble: 14 x 2
##   aerolinea atraso
##   <chr>      <dbl>
## 1 9E         7.56
## 2 AS         3.08
## 3 B6         3.52
## 4 EV         9.83
## 5 F9        13.5
## 6 FL        16.9
## 7 HA        -5.44
## 8 MQ         3.28
## 9 OO         0.8
## 10 UA        6.37
## 11 US        0.576
## 12 VX        7.80
## 13 WN        11.0
## 14 YV        10.5
```

Ejercicios:

Hacer los ejercicios en la sección 5.2.4 del libro en español

5.2.4 Ejercicios

1. Encuentra todos los vuelos que:
 - a. Tuvieron un retraso de llegada de dos o más horas
 - b. Volaron a Houston (IAH o HOU)

Cuáles son estos aeropuertos, Hou == Houston, IAH == George Bush, Texas

2. Selecciona solamente los vuelos que fueron operados por United, American o Delta
3. Selecciona solamente los vuelos que partieron en invierno del hemisferio sur (julio, agosto y septiembre)
4. Selecciona solamente los vuelos excluyendo los días del 2 al 29 de cualquier mes

```
vuelos %>%
  filter(atraso_salida <=0 & atraso_llegada >=120)
```

```
## # A tibble: 29 x 19
##   anio   mes   dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>           <int>         <dbl>
## 1  2013     1    27          1419             1420          -1
## 2  2013    10     7          1350             1350           0
## 3  2013    10     7          1357             1359          -2
## 4  2013    10    16           657              700          -3
## 5  2013    11     1           658              700          -2
## 6  2013     3    18          1844             1847          -3
## 7  2013     4    17          1635             1640          -5
## 8  2013     4    18           558              600          -2
## 9  2013     4    18           655              700          -5
## 10 2013     5    22          1827             1830          -3
## # i 19 more rows
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

5. Selecciona solamente los vuelos que llegaron más de dos horas tarde, pero no salieron tarde
6. Selecciona solamente los vuelos que se retrasaron por lo menos una hora, pero repusieron más de 30 minutos en vuelo
7. Selecciona solamente los vuelos que partieron entre la medianoche y las 6a.m. (incluyente)
8. Otra función de dplyr que es útil para usar filtros es `between()`. ¿Qué hace? ¿Puedes usarla para simplificar el código necesario para responder a los desafíos anteriores?

`between(x, izq, der)` es un atajo equivalente a `x >= izq & x <= der` e incluye ambos extremos. Hace tus filtros más cortos y legibles.

9. Determinar cual “aerolinea” tiene más vuelos

7.4.1 La función `arrange()`

`arrange()`

arrange() — ordenar las filas.

- **Qué hace:** reordena el data frame según una o más columnas (ascendente por defecto).

- `desc(col)`: orden descendente (de mayor a menor).

```
vuelos %>%
  dplyr::select(horario_salida) %>%
  arrange() |>
  head(n=10)
```

```
## # A tibble: 10 x 1
##   horario_salida
##   <int>
## 1         517
## 2         533
## 3         542
## 4         544
## 5         554
## 6         554
## 7         555
## 8         557
## 9         557
## 10        558
```

```
vuelos %>%
  arrange(desc(horario_salida)) |>
  head(n=10)
```

```
## # A tibble: 10 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>             <int>         <dbl>
## 1  2013   10   30          2400             2359             1
## 2  2013   11   27          2400             2359             1
## 3  2013   12    5          2400             2359             1
## 4  2013   12    9          2400             2359             1
## 5  2013   12    9          2400             2250             70
## 6  2013   12   13          2400             2359             1
## 7  2013   12   19          2400             2359             1
## 8  2013   12   29          2400             1700            420
## 9  2013    2    7          2400             2359             1
## 10 2013    2    7          2400             2359             1
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

5.3.1 Ejercicios

Ordena vuelos para encontrar los vuelos más retrasados en salida.

Encuentra los vuelos que salieron más temprano.

Ordena vuelos para encontrar los vuelos más rápidos (que viajaron a mayor velocidad).

¿Cuáles vuelos viajaron más lejos?

¿Cuál viajó más cerca?

- Otra función
 - desc()

7.5 2 Ejercicios:

Hacer los ejercicios en la sección 5.3.1 del libro en español

Chapter 8

Transformación: Valores Faltantes

```
## [1] "2026-08-01"
```

Fecha de la última revisión

```
library(tidyverse)
library(datos)
```

8.1 Valores faltantes

- NA: Not Available

8.1.1 ¿Qué pasa con los valores faltantes y resultados de análisis sin sentido?

```
2^2
```

```
## [1] 4
```

```
1^2
```

```
## [1] 1
```

```
1^0
```

```
## [1] 1
```

```
0^1
```

```
## [1] 0
```

```
NA^0
## [1] 1
NA^1
## [1] NA
NA == 1
## [1] NA
NA | TRUE
## [1] TRUE
NA | FALSE
## [1] NA
FALSE & NA
## [1] FALSE
TRUE & NA
## [1] NA
FALSE * NA
## [1] NA
TRUE * NA
## [1] NA
NA * 0
## [1] NA
NA * 1
## [1] NA
12 * 3
## [1] 36
NA - 3
## [1] NA
```

8.2 Creamos un df para aclarar estas operaciones con NA en un data frame

- valores numéricos

8.2. CREAMOS UN DF PARA ACLARAR ESTAS OPERACIONES CON NA EN UN DATA FRAME83

- variable lógica TRUE/FALSE
- variable con NA y 0/1

```
df=tribble(~values, ~T_F, ~NA_not,  
           1, TRUE, NA,  
           0, TRUE, 1,  
           1, FALSE, NA,  
           0, FALSE, 1,  
           0, TRUE, 1,  
           0, FALSE, 0,  
           NA, NA, NA)  
  
df
```

```
## # A tibble: 7 x 3  
##   values T_F   NA_not  
##   <dbl> <lgl> <dbl>  
## 1     1 TRUE     NA  
## 2     0 TRUE     1  
## 3     1 FALSE    NA  
## 4     0 FALSE     1  
## 5     0 TRUE     1  
## 6     0 FALSE     0  
## 7    NA NA      NA
```

Ahora multiplicamos valores con la variable lógica.

```
df |>  
  mutate(values_TF = values * T_F)
```

```
## # A tibble: 7 x 4  
##   values T_F   NA_not values_TF  
##   <dbl> <lgl> <dbl>     <dbl>  
## 1     1 TRUE     NA         1  
## 2     0 TRUE     1         0  
## 3     1 FALSE    NA         0  
## 4     0 FALSE     1         0  
## 5     0 TRUE     1         0  
## 6     0 FALSE     0         0  
## 7    NA NA      NA         NA
```

Multiplicamos la variable lógica con NA/1/0

```
df |>  
  mutate(values_TF = T_F * NA_not)
```

```
## # A tibble: 7 x 4  
##   values T_F   NA_not values_TF  
##   <dbl> <lgl> <dbl>     <dbl>
```

```
## 1      1 TRUE      NA      NA
## 2      0 TRUE      1      1
## 3      1 FALSE     NA      NA
## 4      0 FALSE     1      0
## 5      0 TRUE      1      1
## 6      0 FALSE     0      0
## 7      NA NA      NA      NA
```

Multiplicamos valores con la variable NA/1/0

```
df |>
  mutate(valuesNA = values * NA_not)
```

```
## # A tibble: 7 x 4
##   values T_F  NA_not valuesNA
##   <dbl> <lgl>  <dbl>   <dbl>
## 1      1 TRUE     NA      NA
## 2      0 TRUE     1      0
## 3      1 FALSE   NA      NA
## 4      0 FALSE   1      0
## 5      0 TRUE     1      0
## 6      0 FALSE   0      0
## 7      NA NA      NA      NA
```

8.3 Contabilizar los NA

`is.na()` e `if_else()` — detectar y reemplazar valores faltantes.

- `is.na(x)`: devuelve TRUE donde hay NA; con `!is.na(x)` te quedas con lo que NO falta.
- `if_else(condición, sí, no)`: reemplaza según una condición (p. ej. `if_else(is.na(x), 0, x)`).
- **Ojo:** cualquier operación con NA da NA; por eso muchas funciones necesitan `na.rm = TRUE`.
- ¿Cuántos vuelos tienen datos faltantes en `horario_salida`?
- ¿Qué otras variables tienen valores faltantes?
- ¿Qué representan estas filas?

```
vuelos %>%
  dplyr::select(horario_salida, aerolinea) %>%
  is.na() %>%
  table()
```

```
## .
## FALSE TRUE
## 665297 8255
```

8.4 Seleccionar solamente los vuelos donde el “horario de salida” es desconocido

```
vuelos %>%
  dplyr::select(anio, mes, dia, horario_salida, aerolinea) |>
  filter(is.na(horario_salida)) %>%
  group_by(dia) %>%
  summarise(n = n())
```

```
## # A tibble: 31 x 2
##   dia      n
##   <int> <int>
## 1     1  246
## 2     2  250
## 3     3  109
## 4     4   82
## 5     5  226
## 6     6  296
## 7     7  318
## 8     8  921
## 9     9  593
## 10    10 535
## # i 21 more rows
```

8.5 Otras funciones: NA

- na.rm=TRUE
- !is.na() = is not NA, pq al frente tiene “!”

Para ignorar los NA en un cálculo usa el argumento `na.rm = TRUE` (por ejemplo `mean(x, na.rm = TRUE)`). Para eliminar filas con NA puedes usar `drop_na()`.

```
no_cancelados <- vuelos %>%
  filter(!is.na(atraso_salida), !is.na(atraso_llegada))
no_cancelados
```

```
## # A tibble: 327,346 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>          <int>          <int>          <dbl>
## 1  2013   1    1           517            515            2
## 2  2013   1    1           533            529            4
## 3  2013   1    1           542            540            2
## 4  2013   1    1           544            545           -1
## 5  2013   1    1           554            600           -6
## 6  2013   1    1           554            558           -4
## 7  2013   1    1           555            600           -5
```

```
## 8 2013      1      1          557          600          -3
## 9 2013      1      1          557          600          -3
## 10 2013     1      1          558          600          -2
## # i 327,336 more rows
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

```
#vuelos %>%
# drop_na(atraso_salida, atraso_llegada)
```

```
no_cancelados %>%
  group_by(anio, mes, dia) %>%
  summarise(mean = mean(atraso_salida))
```

```
## # A tibble: 365 x 4
## # Groups:   anio, mes [12]
##   anio  mes  dia mean
##   <int> <int> <int> <dbl>
## 1 2013     1     1 11.4
## 2 2013     1     2 13.7
## 3 2013     1     3 10.9
## 4 2013     1     4  8.97
## 5 2013     1     5  5.73
## 6 2013     1     6  7.15
## 7 2013     1     7  5.42
## 8 2013     1     8  2.56
## 9 2013     1     9  2.30
## 10 2013     1    10  2.84
## # i 355 more rows
```

```
vuelos %>%
  group_by(anio, mes, dia) %>%
  summarise(
    mean = mean(atraso_salida, na.rm = TRUE),
    max = max(atraso_salida, na.rm = TRUE),
    n = n()
  )
```

```
## # A tibble: 365 x 6
## # Groups:   anio, mes [12]
##   anio  mes  dia mean  max    n
##   <int> <int> <int> <dbl> <dbl> <int>
## 1 2013     1     1 11.5   853   842
## 2 2013     1     2 13.9   379   943
## 3 2013     1     3 11.0   291   914
```

```
## 4 2013      1      4 8.95   288   915
## 5 2013      1      5 5.73   327   720
## 6 2013      1      6 7.15   202   832
## 7 2013      1      7 5.42   366   933
## 8 2013      1      8 2.55   188   899
## 9 2013      1      9 2.28  1301   902
## 10 2013     1     10 2.84  1126   932
## # i 355 more rows
```

Reemplazar los NA por “0” en el archivo de datos

Cuál es el problema de reemplazar los NA por 0?

Reemplazar NA por 0 cambia el significado de los datos: un valor desconocido no es lo mismo que un cero real. Esto distorsiona promedios, sumas y conteos, así que piénsalo bien antes de hacerlo. Encontrarás más errores frecuentes y cómo resolverlos en el capítulo *Errores comunes* (14).

```
datos_NA = c(1, NA, 3:8, NA, 10:12)
```

```
datos_NA_df = as.data.frame(datos_NA)
datos_NA_df
```

```
##      datos_NA
## 1           1
## 2          NA
## 3           3
## 4           4
## 5           5
## 6           6
## 7           7
## 8           8
## 9          NA
## 10          10
## 11          11
## 12          12
```

```
# reemplazar NA con 0
```

```
datos_NA_df = datos_NA_df %>%
  mutate(datos_0 = if_else(is.na(datos_NA), 0, datos_NA))
```

```
datos_NA_df
```

```
##      datos_NA datos_0
## 1           1       1
## 2          NA       0
```

```
## 3      3      3
## 4      4      4
## 5      5      5
## 6      6      6
## 7      7      7
## 8      8      8
## 9      NA     0
## 10     10     10
## 11     11     11
## 12     12     12
```

AHORA

Alguien le dio un set de datos en Excel con 0 cuando no tenían datos
reemplace los valores de “zero” con NA con la función `if_else` de `dplyr`

```
library(dplyr)
Datos_Excel = c(1, 0, 3:8, 0, 10:12)

Datos_df = as.data.frame(Datos_Excel)

Datos_df = Datos_df %>%
  mutate(Datos_NA = dplyr::if_else(Datos_Excel == 0, NA, Datos_Excel))

Datos_df
```

```
##   Datos_Excel Datos_NA
## 1           1        1
## 2           0       NA
## 3           3        3
## 4           4        4
## 5           5        5
## 6           6        6
## 7           7        7
## 8           8        8
## 9           0       NA
## 10          10       10
## 11          11       11
## 12          12       12
```


Chapter 9

Transformación: Seleccionar variables

Fecha de la última revisión

[1] "2026-08-01"

9.1 Funciones en este módulo

- `starts_with()` # selecciona variables que empiezan con una palabra o letra
- `ends_with()` # selecciona variables que terminan con una palabra o letra
- `rename()` # cambiar el nombre de una variable
- `contains()` # selecciona variables que contienen una palabra o letra
- `everything()` # selecciona todas las variables
- `if_else()` # reemplaza valores en una columna basado en una condición

Es una función para seleccionar basado en una características del nombre de la columna

Los ayudantes como `starts_with()`, `ends_with()` y `contains()` te evitan escribir cada nombre de columna a mano, sobre todo en tablas con muchas variables.

```
library(datos)
names(vuelos)
```

```
## [1] "anio"           "mes"           "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
## [10] "aerolinea"      "vuelo"         "codigoCola"
## [13] "origen"         "destino"       "tiempo_vuelo"
## [16] "distancia"      "hora"          "minuto"
```

```
## [19] "fecha_hora"
```

```
library(tidyverse)
vuelos %>%
  dplyr::select(horario_salida)
```

```
## # A tibble: 336,776 x 1
##   horario_salida
##   <int>
## 1         517
## 2         533
## 3         542
## 4         544
## 5         554
## 6         554
## 7         555
## 8         557
## 9         557
## 10        558
## # i 336,766 more rows
```

```
vuelos %>%
  dplyr::select(starts_with("horario"))
```

```
## # A tibble: 336,776 x 2
##   horario_salida horario_llegada
##   <int>          <int>
## 1         517         830
## 2         533         850
## 3         542         923
## 4         544        1004
## 5         554         812
## 6         554         740
## 7         555         913
## 8         557         709
## 9         557         838
## 10        558         753
## # i 336,766 more rows
```

```
vuelos %>%
  dplyr::select(ends_with("salida"))
```

```
## # A tibble: 336,776 x 2
##   horario_salida atraso_salida
##   <int>          <dbl>
## 1         517           2
## 2         533           4
## 3         542           2
```

```
## 4          544          -1
## 5          554          -6
## 6          554          -4
## 7          555          -5
## 8          557          -3
## 9          557          -3
## 10         558          -2
## # i 336,766 more rows
```

```
vuelos %>%
  dplyr::select(contains("salida"))
```

```
## # A tibble: 336,776 x 3
##   horario_salida salida_programada atraso_salida
##   <int>          <int>          <dbl>
## 1         517         515            2
## 2         533         529            4
## 3         542         540            2
## 4         544         545           -1
## 5         554         600           -6
## 6         554         558           -4
## 7         555         600           -5
## 8         557         600           -3
## 9         557         600           -3
## 10        558         600           -2
## # i 336,766 more rows
```

```
vuelos %>%
  dplyr::select(contains("ue"))
```

```
## # A tibble: 336,776 x 2
##   vuelo tiempo_vuelo
##   <int>      <dbl>
## 1  1545        227
## 2  1714        227
## 3  1141        160
## 4   725        183
## 5   461        116
## 6  1696        150
## 7   507        158
## 8  5708         53
## 9    79        140
## 10  301        138
## # i 336,766 more rows
```

```
#vuelos %>%
# dplyr::filter(contains("AA")) # no funciona con filter
```

```
vuelos %>%
  dplyr::filter(aerolinea %in% c("AA", "DL"))
```

```
## # A tibble: 80,839 x 19
##   anio   mes   dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>         <int>         <dbl>
## 1  2013     1     1           542             540             2
## 2  2013     1     1           554             600            -6
## 3  2013     1     1           558             600            -2
## 4  2013     1     1           559             600            -1
## 5  2013     1     1           602             610            -8
## 6  2013     1     1           606             610            -4
## 7  2013     1     1           606             610            -4
## 8  2013     1     1           615             615             0
## 9  2013     1     1           623             610             13
## 10 2013     1     1           628             630            -2
## # i 80,829 more rows
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

9.2 rename()

rename(nuevo = viejo) — cambiar el nombre de columnas.

- **Qué hace:** renombra columnas; el **nombre nuevo va primero**, el original después.
- **Relacionadas:** `select(nuevo = viejo, ...)` renombra y selecciona a la vez; `everything()` reordena poniendo unas columnas al frente y el resto después.

Cambiar el nombre de la columna

```
vuelos %>%
  rename(aeropuerto_origen=origen) %>%
  rename(aeropuerto_destino=destino) %>%
  dplyr::select(contains("aeropuerto")) # nombre nuevo= nombre original
```

```
## # A tibble: 336,776 x 2
##   aeropuerto_origen aeropuerto_destino
##   <chr>             <chr>
## 1 EWR              IAH
## 2 LGA              IAH
## 3 JFK              MIA
## 4 JFK              BQN
```

9.3. REORGANIZAR EL ORDEN DE LAS COLUMNAS, USANDO SELECT() Y EVERYTHING() EN LA MISMA

```
## 5 LGA ATL
## 6 EWR ORD
## 7 EWR FLL
## 8 LGA IAD
## 9 JFK MCO
## 10 LGA ORD
## # i 336,766 more rows
```

9.3 Reorganizar el orden de las columnas, usando select() y everything() en la misma función

```
head(vuelos)
```

```
## # A tibble: 6 x 19
##   anio mes dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>         <int>         <dbl>
## 1  2013     1     1           517           515           2
## 2  2013     1     1           533           529           4
## 3  2013     1     1           542           540           2
## 4  2013     1     1           544           545          -1
## 5  2013     1     1           554           600          -6
## 6  2013     1     1           554           558          -4
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

```
vuelos %>%
  dplyr::select(distancia, aerolinea, everything())
```

```
## # A tibble: 336,776 x 19
##   distancia aerolinea anio mes dia horario_salida salida_programada
##   <dbl> <chr>    <int> <int> <int>         <int>         <int>
## 1    1400 UA      2013     1     1           517           515
## 2    1416 UA      2013     1     1           533           529
## 3    1089 AA      2013     1     1           542           540
## 4    1576 B6      2013     1     1           544           545
## 5     762 DL      2013     1     1           554           600
## 6     719 UA      2013     1     1           554           558
## 7    1065 B6      2013     1     1           555           600
## 8     229 EV      2013     1     1           557           600
## 9     944 B6      2013     1     1           557           600
## 10     733 AA      2013     1     1           558           600
## # i 336,766 more rows
```

```
## # i 12 more variables: atraso_salida <dbl>, horario_llegada <int>,
## #   llegada_programada <int>, atraso_llegada <dbl>, vuelo <int>,
## #   codigoCola <chr>, origen <chr>, destino <chr>, tiempo_vuelo <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

Seleccionar de un conjunto de variables en orden que aparece en el data.frame

```
vuelos |> dplyr::select(mes:salida_programada)
```

```
## # A tibble: 336,776 x 4
##   mes dia horario_salida salida_programada
##   <int> <int>      <int>      <int>
## 1     1     1         517          515
## 2     1     1         533          529
## 3     1     1         542          540
## 4     1     1         544          545
## 5     1     1         554          600
## 6     1     1         554          558
## 7     1     1         555          600
## 8     1     1         557          600
## 9     1     1         557          600
## 10    1     1         558          600
```

```
## # i 336,766 more rows
```

```
x=c(10:15)
```

```
x
```

```
## [1] 10 11 12 13 14 15
```

```
vuelos[,c(4:7)]
```

```
## # A tibble: 336,776 x 4
##   horario_salida salida_programada atraso_salida horario_llegada
##           <int>           <int>         <dbl>         <int>
## 1           517           515             2           830
## 2           533           529             4           850
## 3           542           540             2           923
## 4           544           545            -1          1004
## 5           554           600            -6           812
## 6           554           558            -4           740
## 7           555           600            -5           913
## 8           557           600            -3           709
## 9           557           600            -3           838
## 10          558           600            -2           753
## # i 336,766 more rows
```

9.3. REORGANIZAR EL ORDEN DE LAS COLUMNAS, USANDO SELECT() Y EVERYTHING() EN LA MISMA

3. Ejercicios:

Hacer los ejercicios en la sección 5.4.1 del libro en español

<https://es.r4ds.hadley.nz/05-transform.html#ejercicios-2>

?mean

Chapter 10

Transformación: Mutate, transmute, lag, lead, cumsum, cummean, cumvar

10.1 Funciones en este módulo

- `mutate()` # crea nuevas variables
- `transmute()` # crea nuevas variables y elimina las antiguas
- `lag()` # calcula la diferencia entre valores en la misma columna
- `lead()` # calcula el siguiente valor en un vector
- `cumsum()` # suma acumulativa
- `cummean()` # promedio acumulativo
- `cumvar()` # varianza acumulativa

`mutate()` y `transmute()` — crear columnas nuevas.

- `mutate(nueva = expresión)`: añade columnas calculadas y **conserva** las demás.
- `transmute()`: igual, pero **solo** deja las columnas nuevas.
- **Uso**: la expresión puede combinar otras columnas (p. ej. `velocidad = distancia / tiempo`).

```
library(tidyverse)
library(datos)
```

```
names(vuelos)
```

```
## [1] "anio"           "mes"            "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
```

```
## [10] "aerolinea"      "vuelo"          "codigoCola"
## [13] "origen"         "destino"        "tiempo_vuelo"
## [16] "distancia"      "hora"           "minuto"
## [19] "fecha_hora"

# /> es igual %>%
new_df=vuelos |> dplyr::select(distancia, tiempo_vuelo, atraso_salida, atraso_llegada,
  mutate( ganancia = atraso_salida - atraso_llegada,
          velocidad = distancia / tiempo_vuelo * 60))

new_df

## # A tibble: 336,776 x 6
##   distancia tiempo_vuelo atraso_salida atraso_llegada ganancia velocidad
##   <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
## 1    1400        227          2          11         -9        370.
## 2    1416        227          4          20        -16        374.
## 3    1089        160          2          33        -31        408.
## 4    1576        183         -1         -18         17        517.
## 5     762        116         -6        -25         19        394.
## 6     719        150         -4         12        -16        288.
## 7    1065        158         -5         19        -24        404.
## 8     229         53         -3        -14         11        259.
## 9     944        140         -3         -8          5        405.
## 10    733        138         -2          8        -10        319.
## # i 336,766 more rows
```

10.2 Crear un data frame más pequeño con las variables de interés

```
library(tidyverse)
library(nycflights13)
head(flights)

## # A tibble: 6 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>      <int>      <dbl>      <int>      <int>
## 1  2013     1     1     517        515         2      830        819
## 2  2013     1     1     533        529         4      850        830
## 3  2013     1     1     542        540         2      923        850
## 4  2013     1     1     544        545        -1     1004       1022
## 5  2013     1     1     554        600        -6      812        837
## 6  2013     1     1     554        558        -4      740        728
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
```

```
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
names(vuelos)

## [1] "anio"          "mes"           "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
## [10] "aerolinea"      "vuelo"         "codigoCola"
## [13] "origen"         "destino"       "tiempo_vuelo"
## [16] "distancia"      "hora"          "minuto"
## [19] "fecha_hora"

vuelos_sml <- dplyr::select(vuelos,
  anio:dia,
  starts_with("atraso"),
  distancia,
  tiempo_vuelo
)
head(vuelos_sml)

## # A tibble: 6 x 7
##   anio  mes  dia atraso_salida atraso_llegada distancia tiempo_vuelo
##   <int> <int> <int>         <dbl>         <dbl>         <dbl>         <dbl>
## 1  2013     1     1             2             11           1400           227
## 2  2013     1     1             4             20           1416           227
## 3  2013     1     1             2             33           1089           160
## 4  2013     1     1            -1            -18           1576           183
## 5  2013     1     1            -6            -25            762           116
## 6  2013     1     1            -4             12            719           150
```

10.3 Crear nuevas variables

```
mutate(vuelos_sml,
  ganado = atraso_salida - atraso_llegada,
  velocidad = distancia / tiempo_vuelo * 60
)

## # A tibble: 336,776 x 9
##   anio  mes  dia atraso_salida atraso_llegada distancia tiempo_vuelo ganado
##   <int> <int> <int>         <dbl>         <dbl>         <dbl>         <dbl> <dbl>
## 1  2013     1     1             2             11           1400           227    -9
## 2  2013     1     1             4             20           1416           227   -16
## 3  2013     1     1             2             33           1089           160   -31
## 4  2013     1     1            -1            -18           1576           183    17
## 5  2013     1     1            -6            -25            762           116    19
## 6  2013     1     1            -4             12            719           150   -16
```

```
## 7 2013      1      1          -5          19      1065          158      -24
## 8 2013      1      1          -3          -14      229           53       11
## 9 2013      1      1          -3          -8      944          140        5
## 10 2013     1      1          -2           8      733          138      -10
## # i 336,766 more rows
## # i 1 more variable: velocidad <dbl>
```

10.4 transmute()

Para guardar solamente la nueva variable usa **transmute**

Usa `mutate()` cuando quieras conservar las columnas originales y añadir nuevas; usa `transmute()` cuando solo te interesen las columnas nuevas.

```
flights |> transmute(
  gain = dep_delay - arr_delay,
  hours = air_time / 60,
  gain_per_hour = gain / hours
)
```

```
## # A tibble: 336,776 x 3
##       gain hours gain_per_hour
##   <dbl> <dbl>         <dbl>
## 1    -9  3.78          -2.38
## 2   -16  3.78          -4.23
## 3   -31  2.67         -11.6
## 4    17  3.05           5.57
## 5    19  1.93           9.83
## 6   -16  2.5           -6.4
## 7   -24  2.63          -9.11
## 8    11  0.883          12.5
## 9     5  2.33           2.14
## 10  -10  2.3           -4.35
## # i 336,766 more rows
```

10.5 lag()

`lag()` y `lead()` — valor anterior / siguiente.

- **lag(x, n):** trae el valor de **n** filas **atrás** (por defecto 1); útil para calcular cambios ($x - \text{lag}(x)$).
- **lead(x, n):** trae el valor de **n** filas **adelante**.
- **Requisito:** ordena las filas primero (`arrange()`), porque operan según el orden actual.

Para calcular diferencias entre variables en la misma columna

`lag()` y `lead()` trabajan según el orden actual de las filas. Ordena tus datos (por ejemplo por fecha con `arrange()`) antes de usarlas, o los desplazamientos no tendrán sentido.

```
set.seed(12345) # que los datos sean al azar, siempre sean los mismo, se usa el "set.seed()" para
#rnorm() DATOS CON DISTRIBUCION NORMAL
```

```
data=rpois(14, 10)
df=as_tibble(data)
df
```

```
## # A tibble: 14 x 1
##   value
##   <int>
## 1     11
## 2     12
## 3      9
## 4      8
## 5     11
## 6      4
## 7     11
## 8      7
## 9      8
## 10    11
## 11     9
## 12    10
## 13     9
## 14    11
```

```
df %>%
  dplyr::select(value) %>%
  mutate(lag1=lag(value)) %>%
  mutate(lag3=lag(value, 3)) %>%
  mutate(lag7=lag(value,5))
```

```
## # A tibble: 14 x 4
##   value lag1 lag3 lag7
##   <int> <int> <int> <int>
## 1     11    NA    NA    NA
## 2     12     11    NA    NA
## 3      9     12    NA    NA
## 4      8      9     11    NA
## 5     11      8     12    NA
## 6      4     11      9     11
## 7     11      4      8     12
## 8      7     11     11      9
```

```
## 9      8      7      4      8
## 10     11     8     11     11
## 11     9      11     7      4
## 12     10     9      8     11
## 13     9      10     11     7
## 14     11     9      9      8
```

```
## Calcular la diferencia usando lag
```

```
df %>%
  dplyr::select(value) %>%
  mutate(lag1=lag(value)) %>%
  mutate(Changes=value-lag(value, 1)) # El cambio en posición de los valores entre las
```

```
## # A tibble: 14 x 3
##   value lag1 Changes
##   <int> <int>   <int>
## 1     11    NA      NA
## 2     12    11       1
## 3      9    12     -3
## 4      8     9     -1
## 5     11     8       3
## 6      4    11     -7
## 7     11     4       7
## 8      7    11     -4
## 9      8     7       1
## 10     11     8       3
## 11      9    11     -2
## 12     10     9       1
## 13      9    10     -1
## 14     11     9       2
```

10.6 Usa “Lag” con “IncCasosSaludNuevo” en COVID-19 PR

Evalúa la diferencia en números de casos entre 7 días de la semana en números de casos nuevos de COVID, “IncCasosSaludNuevo”

```
library(readr)
library(dplyr)
#names(url_COVID_PR)
url_COVID_PR <- read_csv("Datos/url_COVID_PR.csv")
head(url_COVID_PR, 100)
```

```
## # A tibble: 100 x 18
##   ...1 Fecha    Muertes IncrementoMuertes CasosPCR_Salud IncCasosPCR_Salud
```

10.6. USA “LAG” CON “INCCASOSSALUDNUEVO” EN COVID-19 PR103

```
##      <dbl> <chr>      <dbl>          <dbl>          <dbl>          <dbl>
## 1      1 3/12/20      0              0              NA              NA
## 2      2 3/13/20      0              0              NA              NA
## 3      3 3/14/20      0              0              NA              NA
## 4      4 3/15/20      0              0              NA              NA
## 5      5 3/16/20      0              0              NA              NA
## 6      6 3/17/20      0              0              NA              NA
## 7      7 3/18/20      0              0              NA              NA
## 8      8 3/19/20      0              0              NA              NA
## 9      9 3/20/20      0              0              NA              NA
## 10     10 3/21/20      0              0              NA              NA
## # i 90 more rows
## # i 12 more variables: CasosSaludNuevo <dbl>, IncCasosSaludNuevo <dbl>,
## #   HospitCOV19 <dbl>, CamasICU <dbl>, CamasICU_disp <dbl>, Ventiladores <dbl>,
## #   MuertesSalud <dbl>, IncMueSalud <dbl>, VacDoses <dbl>, VacAdm <dbl>,
## #   N1MoreDoses <dbl>, N2Doses <dbl>

#names(url_COVID_PR)
names(url_COVID_PR)

## [1] "...1"          "Fecha"          "Muertes"
## [4] "IncrementoMuertes" "CasosPCR_Salud" "IncCasosPCR_Salud"
## [7] "CasosSaludNuevo"  "IncCasosSaludNuevo" "HospitCOV19"
## [10] "CamasICU"         "CamasICU_disp"   "Ventiladores"
## [13] "MuertesSalud"     "IncMueSalud"     "VacDoses"
## [16] "VacAdm"           "N1MoreDoses"     "N2Doses"

df2=url_COVID_PR %>%
  dplyr::select(IncCasosSaludNuevo) %>%
  mutate(Cambios_Casos=IncCasosSaludNuevo-lag(IncCasosSaludNuevo,7))

df2

## # A tibble: 587 x 2
##   IncCasosSaludNuevo Cambios_Casos
##           <dbl>         <dbl>
## 1             2             NA
## 2             3             NA
## 3             3             NA
## 4             0             NA
## 5             9             NA
## 6             7             NA
## 7             6             NA
```

```
## 8          5          3
## 9          14         11
## 10         12          9
## # i 577 more rows
```

```
df2 %>%
  dplyr::select(IncCasosSaludNuevo, Cambios_Casos) %>%
  colMeans(na.rm=TRUE)
```

```
## IncCasosSaludNuevo      Cambios_Casos
##          257.8310580          0.4507772
```

```
#df2 %>%
# select()
# slice(na.rm=TRUE) # lets you index rows by their (integer) locations. It allows you
```

10.7 lead(),

is the “next” (lead()) values in a vector/column

```
set.seed(12345)
data=rpois(15, 10)
df=as_tibble(data)
df
```

```
## # A tibble: 15 x 1
##   value
##   <int>
## 1     11
## 2     12
## 3      9
## 4      8
## 5     11
## 6      4
## 7     11
## 8      7
## 9      8
## 10    11
## 11     9
## 12    10
## 13     9
## 14    11
## 15    13
```

```
df %>%
  dplyr::select(value) %>%
  mutate(lead1=lead(value)) %>%
```



```
mutate(lead3=lead(value, 3))
```

```
## # A tibble: 15 x 3
##   value lead1 lead3
##   <int> <int> <int>
## 1    11    12     8
## 2    12     9    11
## 3     9     8     4
## 4     8    11    11
## 5    11     4     7
## 6     4    11     8
## 7    11     7    11
## 8     7     8     9
## 9     8    11    10
## 10    11     9     9
## 11     9    10    11
## 12    10     9    13
## 13     9    11    NA
## 14    11    13    NA
## 15    13    NA    NA
```

```
# Calculate the change in value from one (1) time period and four (4) time periods
df%>%
  dplyr::select(value) %>%
  mutate(lead1=value-lead(value)) %>%
  mutate(lead7=value-lead(value, 7))
```

```
## # A tibble: 15 x 3
##   value lead1 lead7
##   <int> <int> <int>
## 1    11    -1     4
## 2    12     3     4
## 3     9     1    -2
## 4     8    -3    -1
## 5    11     7     1
## 6     4    -7    -5
## 7    11     4     0
## 8     7    -1    -6
## 9     8    -3    NA
## 10    11     2    NA
## 11     9    -1    NA
## 12    10     1    NA
## 13     9    -2    NA
## 14    11    -2    NA
## 15    13    NA    NA
```

10.8 cumsum

`cumsum()`, `cmean()`, `cvar()` — acumulados.

- **`cumsum(x)`**: suma acumulada a lo largo de la columna.
- **`cmean()` / `cvar()`** (paquete **TidyDensity**): promedio y varianza acumulados.
- **Uso típico**: ver cómo evoluciona un total o un promedio a medida que se añaden datos.

Suma acumulativa: los valores se suman a lo largo del vector o columna

```
set.seed(12345) # set.seed() es para que los datos NO sean al azar y se puede replicar
```

```
x <- sample(1:15, 10, replace=TRUE) # sample es para seleccionar valores al azar de un
```

```
x
```

```
## [1] 14  3 10 12  8 10 13 11  8  2
```

```
df=as_tibble(x)
```

```
df
```

```
## # A tibble: 10 x 1
```

```
##   value
```

```
##   <int>
```

```
## 1    14
```

```
## 2     3
```

```
## 3    10
```

```
## 4    12
```

```
## 5     8
```

```
## 6    10
```

```
## 7    13
```

```
## 8    11
```

```
## 9     8
```

```
## 10    2
```

```
df %>%
```

```
  dplyr::select(value) %>%
```

```
  mutate(suma=cumsum(value))
```

```
## # A tibble: 10 x 2
```

```
##   value suma
```

```
##   <int> <int>
```

```
## 1    14    14
```

```
## 2     3    17
```

```
## 3    10    27
```

```
## 4    12    39
```

```
## 5      8      47
## 6     10     57
## 7     13     70
## 8     11     81
## 9      8     89
## 10     2     91
```

```
url_COVID_PR <- read_csv("Datos/url_COVID_PR.csv")
names(url_COVID_PR)
```

```
## [1] "...1"      "Fecha"      "Muertes"
## [4] "IncrementoMuertes" "CasosPCR_Salud" "IncCasosPCR_Salud"
## [7] "CasosSaludNuevo" "IncCasosSaludNuevo" "HospitCOV19"
## [10] "CamasICU" "CamasICU_disp" "Ventiladores"
## [13] "MuertesSalud" "IncMueSalud" "VacDoses"
## [16] "VacAdm" "N1MoreDoses" "N2Doses"
```

```
head(url_COVID_PR)
```

```
## # A tibble: 6 x 18
##   ...1 Fecha      Muertes IncrementoMuertes CasosPCR_Salud IncCasosPCR_Salud
##   <dbl> <chr>      <dbl>          <dbl>          <dbl>          <dbl>
## 1     1 1 3/12/20      0              0              NA              NA
## 2     2 2 3/13/20      0              0              NA              NA
## 3     3 3 3/14/20      0              0              NA              NA
## 4     4 4 3/15/20      0              0              NA              NA
## 5     5 5 3/16/20      0              0              NA              NA
## 6     6 6 3/17/20      0              0              NA              NA
## # i 12 more variables: CasosSaludNuevo <dbl>, IncCasosSaludNuevo <dbl>,
## #   HospitCOV19 <dbl>, CamasICU <dbl>, CamasICU_disp <dbl>, Ventiladores <dbl>,
## #   MuertesSalud <dbl>, IncMueSalud <dbl>, VacDoses <dbl>, VacAdm <dbl>,
## #   N1MoreDoses <dbl>, N2Doses <dbl>
```

```
url_COVID_PR %>%
  dplyr::select(IncCasosSaludNuevo, CasosSaludNuevo) %>%
  mutate(suma=cumsum(IncCasosSaludNuevo))
```

```
## # A tibble: 587 x 3
##   IncCasosSaludNuevo CasosSaludNuevo suma
##   <dbl>          <dbl> <dbl>
## 1           2              2      2
## 2           3              5      5
## 3           3              8      8
## 4           0              8      8
## 5           9             17     17
## 6           7             24     24
## 7           6             30     30
## 8           5             35     35
```

```
## 9          14          49    49
## 10         12          61    61
## # i 577 more rows
```

10.9 cmean() and cvar() del paquete TidyDensity

Para calcular el promedio cumulativo y la varianza

- las funciones son
- `cmean()` # del paquete TidyDensity
- `cvar()` # del paquete TidyDensity

Para calcular el promedio en periodos cumulativos

- `rollmean()` # del paquete zoo
- `rollapply()` # del paquete zoo para calcular el rolling varianza

Vea las otras funciones cumulativas en el paquete TidyDensity (`cmean`, `cvar`, `csd`, `cmedian`, `cskewness`, `ckurtosis`): ¿hay otra que podría usar en sus estudios?

```
set.seed(678)
```

```
x <- rnorm(1000, 5, .001)
```

```
head(x)
```

```
## [1] 4.999227 5.000933 5.000466 4.998915 4.997844 4.999281
```

```
df=as_tibble(x)
```

```
head(df)
```

```
## # A tibble: 6 x 1
```

```
##   value
```

```
##   <dbl>
```

```
## 1  5.00
```

```
## 2  5.00
```

```
## 3  5.00
```

```
## 4  5.00
```

```
## 5  5.00
```

```
## 6  5.00
```

```
mean(df$value)
```

```
## [1] 4.999966
```

```
# library(MASS) # innecesario aquí y enmascara dplyr::select(); cmean()/cvar() vienen
```

```
library(TidyDensity)
```

```
library(zoo)
```

```
df3=df %>%
```

```

dplyr::select(value) %>%
mutate(Prom_cum=cmean(value)) %>%
mutate(Var_cum=TidyDensity::cvar(value)) |>
mutate(promedio_corriendo= zoo::rollmean(value, k=7, fill=NA, align="left")) |>
mutate(varianza_corriendo= zoo::rollapply(value, width=7, FUN=var, fill=NA, align="left"))

```

```
df3
```

```

## # A tibble: 1,000 x 5
##   value Prom_cum      Var_cum promedio_corriendo varianza_corriendo
##   <dbl>   <dbl>      <dbl>          <dbl>          <dbl>
## 1  5.00     5.00    NaN                5.00          0.00000139
## 2  5.00     5.00  0.00000146                5.00          0.00000145
## 3  5.00     5.00  0.000000778                5.00          0.00000123
## 4  5.00     5.00  0.000000937                5.00          0.00000133
## 5  5.00     5.00  0.00000154                5.00          0.00000135
## 6  5.00     5.00  0.00000124                5.00          0.000000532
## 7  5.00     5.00  0.00000139                5.00          0.000000813
## 8  5.00     5.00  0.00000129                5.00          0.00000139
## 9  5.00     5.00  0.00000115                5.00          0.00000131
## 10 5.00     5.00  0.00000113                5.00          0.00000132
## # i 990 more rows

```

10.10 Uso de varianza cumulativa en investigación:

Un método para determinar si la cantidad de muestras recolectada es suficiente, es evaluar si la varianza cumulativa sigue cambiando cuando se añade nuevas recolección de datos.

```

df3$order=c(1:1000)
df3

```

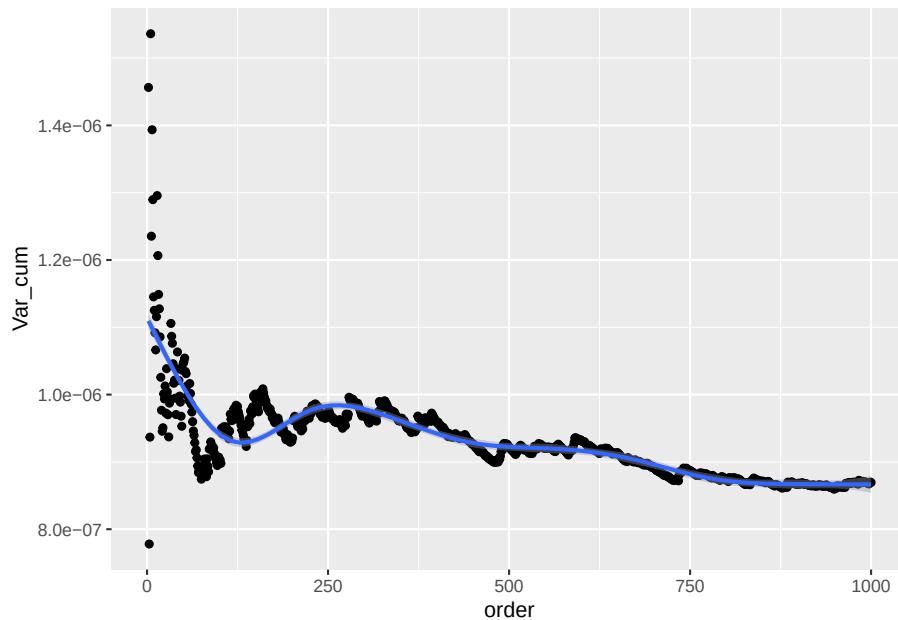
```

## # A tibble: 1,000 x 6
##   value Prom_cum      Var_cum promedio_corriendo varianza_corriendo order
##   <dbl>   <dbl>      <dbl>          <dbl>          <dbl> <int>
## 1  5.00     5.00    NaN                5.00          0.00000139     1
## 2  5.00     5.00  0.00000146                5.00          0.00000145     2
## 3  5.00     5.00  0.000000778                5.00          0.00000123     3
## 4  5.00     5.00  0.000000937                5.00          0.00000133     4
## 5  5.00     5.00  0.00000154                5.00          0.00000135     5
## 6  5.00     5.00  0.00000124                5.00          0.000000532     6
## 7  5.00     5.00  0.00000139                5.00          0.000000813     7
## 8  5.00     5.00  0.00000129                5.00          0.00000139     8

```

```
## 9 5.00      5.00  0.00000115          5.00      0.00000131      9
## 10 5.00      5.00  0.00000113         5.00      0.00000132     10
## # i 990 more rows
```

```
ggplot(df3, aes(order, Var_cum)) +
  geom_point() +
  geom_smooth()
```



10.11 Que pasa con las funciones cmean, cvar si hay NA en el archivo de datos?

10.11.1 Reemplazar los NA con 0, usando la función `replace_na()` con un valor específico `replace_na("variable", 0)`

```
datos_NA=c(1:8, NA, 10:20)
datos_NA=as.data.frame(datos_NA)

datos_NA %>%
  dplyr::select(datos_NA) %>%
  mutate(Prom_cum=cmean(datos_NA)) %>%
  mutate(Var_cum=cvar(datos_NA))
```

```
##      datos_NA Prom_cum Var_cum
## 1          1      1.0     NaN
## 2          2      1.5 0.500000
## 3          3      2.0 1.000000
## 4          4      2.5 1.666667
## 5          5      3.0 2.500000
## 6          6      3.5 3.500000
## 7          7      4.0 4.666667
## 8          8      4.5 6.000000
## 9         NA      NA      NA
## 10        10      NA      NA
## 11        11      NA      NA
## 12        12      NA      NA
## 13        13      NA      NA
## 14        14      NA      NA
## 15        15      NA      NA
## 16        16      NA      NA
## 17        17      NA      NA
## 18        18      NA      NA
## 19        19      NA      NA
## 20        20      NA      NA
```

Solución para resolver los NA. Cual problema hay con usar NA?

10.12 if_else()

La función `if_else()` es una función de reemplazo de valores en una columna basado en una condición

```
library(tidyverse)
library(dplyr)

some_data = tibble(day = 1:10)
some_data
```

```
## # A tibble: 10 x 1
##   day
##   <int>
## 1     1
## 2     2
## 3     3
## 4     4
## 5     5
## 6     6
## 7     7
## 8     8
```

```
## 9      9
## 10     10
```

```
some_data |>
  mutate(day_p = if_else(day %in% c(0:3), "pre", "normal"))
```

```
## # A tibble: 10 x 2
##       day day_p
##   <int> <chr>
## 1     1 pre
## 2     2 pre
## 3     3 pre
## 4     4 normal
## 5     5 normal
## 6     6 normal
## 7     7 normal
## 8     8 normal
## 9     9 normal
## 10    10 normal
```

```
plant_repr=tibble(num_flowers=c(0,0,0,0,1,2,0,1,2,0,2, 100, 1000))
```

```
plant_repr
```

```
## # A tibble: 13 x 1
##       num_flowers
##         <dbl>
## 1             0
## 2             0
## 3             0
## 4             0
## 5             1
## 6             2
## 7             0
## 8             1
## 9             2
## 10            0
## 11            2
## 12           100
## 13          1000
```

```
plant_repr |>
  mutate(flower_status = if_else(num_flowers == 0, "not reproductive",
                                "reproductive"))
```

```
## # A tibble: 13 x 2
##       num_flowers flower_status
##         <dbl> <chr>
```



```
## 1      0 not reproductive
## 2      0 not reproductive
## 3      0 not reproductive
## 4      0 not reproductive
## 5      1 reproductive
## 6      2 reproductive
## 7      0 not reproductive
## 8      1 reproductive
## 9      2 reproductive
## 10     0 not reproductive
## 11     2 reproductive
## 12    100 reproductive
## 13   1000 reproductive
```

```
plant_repr |>
  mutate(flower_status = if_else(num_flowers %in% c(0), "not reproductive",
                                "reproductive"))
```

```
## # A tibble: 13 x 2
##   num_flowers flower_status
##         <dbl> <chr>
## 1         0 not reproductive
## 2         0 not reproductive
## 3         0 not reproductive
## 4         0 not reproductive
## 5         1 reproductive
## 6         2 reproductive
## 7         0 not reproductive
## 8         1 reproductive
## 9         2 reproductive
## 10        0 not reproductive
## 11        2 reproductive
## 12       100 reproductive
## 13      1000 reproductive
```

```
plant_repr |>
  mutate(flower_status =
    if_else(num_flowers %in% c(0),
            "not reproductive",
    if_else(num_flowers %in% c(1:2), "reproductive",
            "super reproductive")))
```

```
## # A tibble: 13 x 2
##   num_flowers flower_status
##         <dbl> <chr>
## 1         0 not reproductive
## 2         0 not reproductive
```

##	3	0 not reproductive
##	4	0 not reproductive
##	5	1 reproductive
##	6	2 reproductive
##	7	0 not reproductive
##	8	1 reproductive
##	9	2 reproductive
##	10	0 not reproductive
##	11	2 reproductive
##	12	100 super reproductive
##	13	1000 super reproductive

Chapter 11

Transformación: Rangos y otras funciones

```
## [1] "2026-08-01"

##
## platform      aarch64-apple-darwin23
## arch          aarch64
## os            darwin23
## system        aarch64, darwin23
## status
## major         4
## minor         6.1
## year          2026
## month         06
## day           24
## svn rev       90187
## language      R
## version.string R version 4.6.1 (2026-06-24)
## nickname      Happy Hop
```

11.1 Funciones en este módulo

- `min_rank()` # asigna valores de rangos a los valores originales
- `row_number()` # asigna valores de rangos a los valores originales
- `desc()` # ordena los valores de mayor a menor
- `rank()` # asigna valores de rangos a los valores originales
- `first()`
- `dense_rank()`
- `percent_rank()`

- `cume_dist()`
- `rollmean()`

```
library(tidyverse)
library(datos)
```

11.2 Pruebas no paramétricas

La pruebas no paramétricas no son basados en distribución normal y los índices como promedio, desviación estándar no se usan.

11.3 `min_rank()` y `# min_rank(desc())`

Asignar valores de rangos a los valores originales o de más grande a más pequeño o viceversa.

`min_rank()`, `dense_rank()`, `row_number()` — asignar rangos.

- **Qué hacen:** convierten valores en su **posición** (rango) de menor a mayor; `desc()` invierte el orden.
- **Diferencia en empates:** `min_rank()` deja huecos, `dense_rank()` no, `row_number()` da un número único a cada fila.
- **Relacionadas:** `percent_rank()` y `cume_dist()` dan rangos en escala 0–1.

```
set.seed(45678)
x <- sample(1:50, 10)
head(x)
```

```
## [1] 22  1 36 46  7 39
```

```
df=as.tibble(x)
df
```

```
## # A tibble: 10 x 1
##   value
##   <int>
## 1     22
## 2      1
## 3     36
## 4     46
## 5      7
## 6     39
## 7     34
## 8     10
## 9     24
## 10    30
```

```
df %>%
  dplyr::select(value) %>%
  mutate(rango_minimo=min_rank(value)) %>%
  mutate(rango_min_desc=min_rank(desc(value)) )
```

```
## # A tibble: 10 x 3
##   value rango_minimo rango_min_desc
##   <int>         <int>         <int>
## 1     22             4             7
## 2      1             1            10
## 3     36             8             3
## 4     46            10             1
## 5      7             2             9
## 6     39             9             2
## 7     34             7             4
## 8     10             3             8
## 9     24             5             6
## 10    30             6             5
```

11.4 row_number()

Qué hace la función `row_number`?

```
set.seed(45678)
y <- c(10,21,22,NA,5,4)
head(y)
```

```
## [1] 10 21 22 NA 5 4
```

```
df=as.tibble(y)
df
```

```
## # A tibble: 6 x 1
##   value
##   <dbl>
## 1     10
## 2     21
## 3     22
## 4     NA
## 5      5
## 6      4
```

```
df %>%
  dplyr::select(value) %>%
  mutate(row=row_number(value)) # equivalente a rank,
```

```
## # A tibble: 6 x 2
```

```
##   value   row
##   <dbl> <int>
## 1    10     3
## 2    21     4
## 3    22     5
## 4     NA    NA
## 5     5     2
## 6     4     1
```

11.5 dense_rank()

`min_rank()`, `dense_rank()` y `row_number()` se diferencian en cómo tratan los empates: `min_rank()` deja huecos tras un empate, `dense_rank()` no deja huecos y `row_number()` da un valor distinto a cada fila.

```
set.seed(45678)
z <- c(10,12,12,NA,51,4)
df=as.tibble(z)
df
```

```
## # A tibble: 6 x 1
##   value
##   <dbl>
## 1    10
## 2    12
## 3    12
## 4     NA
## 5    51
## 6     4
```

```
df %>%
  dplyr::select(value) %>%
  mutate(dense=dense_rank(value)) # equivalente a rank, NOTA que los NA no son asignados
```

```
## # A tibble: 6 x 2
##   value dense
##   <dbl> <int>
## 1    10     2
## 2    12     3
## 3    12     3
## 4     NA    NA
## 5    51     4
## 6     4     1
```

11.6 la función `percent_rank()`

```
set.seed(45678)
w <- c(1,2,2,NA,5,4)
w
```

```
## [1] 1 2 2 NA 5 4
```

```
df=as.tibble(w)
df
```

```
## # A tibble: 6 x 1
##   value
##   <dbl>
## 1     1
## 2     2
## 3     2
## 4    NA
## 5     5
## 6     4
```

```
df %>%
```

```
  dplyr::select(value) %>%
```

```
  mutate(porcentaje_rank=percent_rank(value)) # equivalente a rank, Un número entre a 0 y 1 calculado
```

```
## # A tibble: 6 x 2
##   value porcentaje_rank
##   <dbl>             <dbl>
## 1     1             0
## 2     2           0.25
## 3     2           0.25
## 4    NA           NA
## 5     5             1
## 6     4           0.75
```

11.7 la función `percent_rank()` sin NA

```
set.seed(45678)
x2 <- sample(1:50, 7)
df2=as.tibble(x2)
df2
```

```
## # A tibble: 7 x 1
##   value
##   <int>
## 1    22
## 2     1
```

```
## 3    36
## 4    46
## 5     7
## 6    39
## 7    34
```

```
df2 %>%
  dplyr::select(value) %>%
  mutate(porc2=percent_rank(value))
```

```
## # A tibble: 7 x 2
##   value porc2
##   <int> <dbl>
## 1    22 0.333
## 2     1  0
## 3    36 0.667
## 4    46  1
## 5     7 0.167
## 6    39 0.833
## 7    34 0.5
```

11.8 la función cume__dist()

Es la suma cumulativa de los rangos

```
set.seed(45678)
x <- c(1,2,3,NA,5,4, 10, 8)
x
```

```
## [1]  1  2  3 NA  5  4 10  8
```

```
df=as.tibble(x)
df
```

```
## # A tibble: 8 x 1
##   value
##   <dbl>
## 1     1
## 2     2
## 3     3
## 4    NA
## 5     5
## 6     4
## 7    10
## 8     8
```

```
df %>%
  dplyr::select(value) %>%
```



```
mutate(rangos_cumulativo=cume_dist(value))
```

```
## # A tibble: 8 x 2
##   value rangos_cumulativo
##   <dbl>         <dbl>
## 1     1           0.143
## 2     2           0.286
## 3     3           0.429
## 4    NA           NA
## 5     5           0.714
## 6     4           0.571
## 7    10            1
## 8     8           0.857
```

4. Ejercicios:

Hacer los ejercicios en la sección 5.5.2 del libro en español

Encuentra los 10 vuelos más retrasados utilizando una función de ordenamiento. ¿Cómo quieres manejar los empates? Lee atentamente la documentación de `min_rank()`.

```
names(vuelos)
```

```
## [1] "anio"           "mes"            "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
## [10] "aerolinea"      "vuelo"          "codigoCola"
## [13] "origen"         "destino"        "tiempo_vuelo"
## [16] "distancia"      "hora"           "minuto"
## [19] "fecha_hora"
```

```
vuelos%>%
  dplyr::select(atraso_salida, aerolinea)%>%
  arrange(desc(atraso_salida))%>%
  mutate(mas_atrados=min_rank(desc(atraso_salida))) %>%
  head(n=10)
```

```
## # A tibble: 10 x 3
##   atraso_salida aerolinea mas_atrados
##   <dbl> <chr>         <int>
## 1      1301 HA             1
## 2      1137 MQ             2
## 3      1126 MQ             3
## 4      1014 AA             4
## 5      1005 MQ             5
## 6       960 DL             6
```

```
## 7          911 DL          7
## 8          899 DL          8
## 9          898 DL          9
## 10         896 AA         10
```

```
names(vuelos)
```

```
## [1] "anio"          "mes"          "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
## [10] "aerolinea"      "vuelo"        "codigoCola"
## [13] "origen"         "destino"      "tiempo_vuelo"
## [16] "distancia"      "hora"         "minuto"
## [19] "fecha_hora"
```

```
vuelos %>%
```

```
  arrange(atraso_salida) %>%
```

```
  mutate(rango_minimo=min_rank(atraso_salida)) %>%
```

```
  head(n=10)
```

```
## # A tibble: 10 x 20
```

```
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>          <int>          <int>          <dbl>
## 1  2013   12    7           2040           2123           -43
## 2  2013    2    3           2022           2055           -33
## 3  2013   11   10           1408           1440           -32
## 4  2013    1   11           1900           1930           -30
## 5  2013    1   29           1703           1730           -27
## 6  2013    8    9            729            755           -26
## 7  2013   10   23           1907           1932           -25
## 8  2013    3   30           2030           2055           -25
## 9  2013    3    2           1431           1455           -24
## 10 2013    5    5            934            958           -24
```

```
## # i 14 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>, rango_minimo <int>
```

11.9 Resúmenes con summarise() by group using group_by()

```
library(datos)
```

```
library(nycflights13)
```

```
summarise(flights, delay = mean(dep_delay, na.rm = TRUE))
```

11.9. RESÚMENES CON SUMMARISE() BY GROUP USING GROUP_BY()123

```
## # A tibble: 1 x 1
##   delay
##   <dbl>
## 1  12.6
```

```
flights %>%
  summarise(delay = mean(dep_delay, na.rm = TRUE))
```

```
## # A tibble: 1 x 1
##   delay
##   <dbl>
## 1  12.6
```

```
by_day <- group_by(flights, year, month, day)

summarise(by_day, delay = mean(dep_delay, na.rm = TRUE))
```

```
## # A tibble: 365 x 4
## # Groups:   year, month [12]
##   year month   day delay
##   <int> <int> <int> <dbl>
## 1  2013     1     1  11.5
## 2  2013     1     2  13.9
## 3  2013     1     3  11.0
## 4  2013     1     4   8.95
## 5  2013     1     5   5.73
## 6  2013     1     6   7.15
## 7  2013     1     7   5.42
## 8  2013     1     8   2.55
## 9  2013     1     9   2.28
## 10 2013     1    10   2.84
## # i 355 more rows
```

```
flights %>%
  group_by(year, month, day) %>%
  summarise(delay = mean(dep_delay, na.rm = TRUE))
```

```
## # A tibble: 365 x 4
## # Groups:   year, month [12]
##   year month   day delay
##   <int> <int> <int> <dbl>
## 1  2013     1     1  11.5
## 2  2013     1     2  13.9
## 3  2013     1     3  11.0
## 4  2013     1     4   8.95
## 5  2013     1     5   5.73
## 6  2013     1     6   7.15
## 7  2013     1     7   5.42
```

```
## 8 2013      1      8 2.55
## 9 2013      1      9 2.28
## 10 2013     1     10 2.84
## # i 355 more rows
```

11.10 La aerolínea peor en atraso de salidas

```
flights %>%
  group_by(carrier) %>%
  summarise(delay = mean(dep_delay, na.rm = TRUE)) %>%
  arrange(desc(delay))
```

```
## # A tibble: 16 x 2
##   carrier delay
##   <chr>   <dbl>
## 1 F9      20.2
## 2 EV      20.0
## 3 YV      19.0
## 4 FL      18.7
## 5 WN      17.7
## 6 9E      16.7
## 7 B6      13.0
## 8 VX      12.9
## 9 OO      12.6
## 10 UA     12.1
## 11 MQ     10.6
## 12 DL       9.26
## 13 AA       8.59
## 14 AS       5.80
## 15 HA       4.90
## 16 US       3.78
```

```
by_dest <- group_by(flights, dest)
by_dest
```

```
## # A tibble: 336,776 x 19
## # Groups:   dest [105]
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
## 1 2013     1     1     517           515           2     830           819
## 2 2013     1     1     533           529           4     850           830
## 3 2013     1     1     542           540           2     923           850
## 4 2013     1     1     544           545          -1    1004          1022
## 5 2013     1     1     554           600          -6     812           837
## 6 2013     1     1     554           558          -4     740           728
```

```
## 7 2013      1      1      555      600      -5      913      854
## 8 2013      1      1      557      600      -3      709      723
## 9 2013      1      1      557      600      -3      838      846
## 10 2013     1      1      558      600      -2      753      745
## # i 336,766 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
delay <- summarise(by_dest,
  count = n(),
  dist = mean(distance, na.rm = TRUE),
  delay = mean(arr_delay, na.rm = TRUE))
```

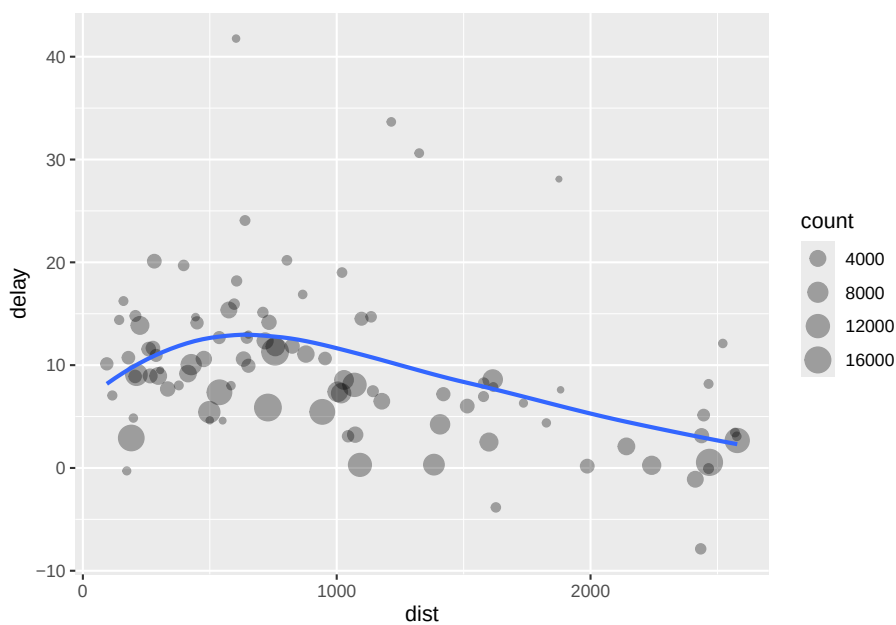
```
delay
```

```
## # A tibble: 105 x 4
##   dest    count    dist delay
##   <chr> <int> <dbl> <dbl>
## 1 ABQ      254  1826   4.38
## 2 ACK      265   199   4.85
## 3 ALB      439   143  14.4
## 4 ANC         8  3370  -2.5
## 5 ATL    17215   757.  11.3
## 6 AUS     2439  1514.   6.02
## 7 AVL       275   584.   8.00
## 8 BDL       443   116   7.05
## 9 BGR       375   378   8.03
## 10 BHM       297   866.  16.9
## # i 95 more rows
```

```
#> `summarise()` ungrouping output (override with `.groups` argument)
delay <- filter(delay, count > 20, dest != "HNL")
```

```
# It looks like delays increase with distance up to ~750 miles
# and then decrease. Maybe as flights get longer there's more
# ability to make up delays in the air?
```

```
ggplot(data = delay, aes(x = dist, y = delay)) +
  geom_point(aes(size = count), alpha = 1/3) +
  geom_smooth(se = FALSE)
```



```
#> `geom_smooth()` using method = 'loess' and formula 'y ~ x'
```

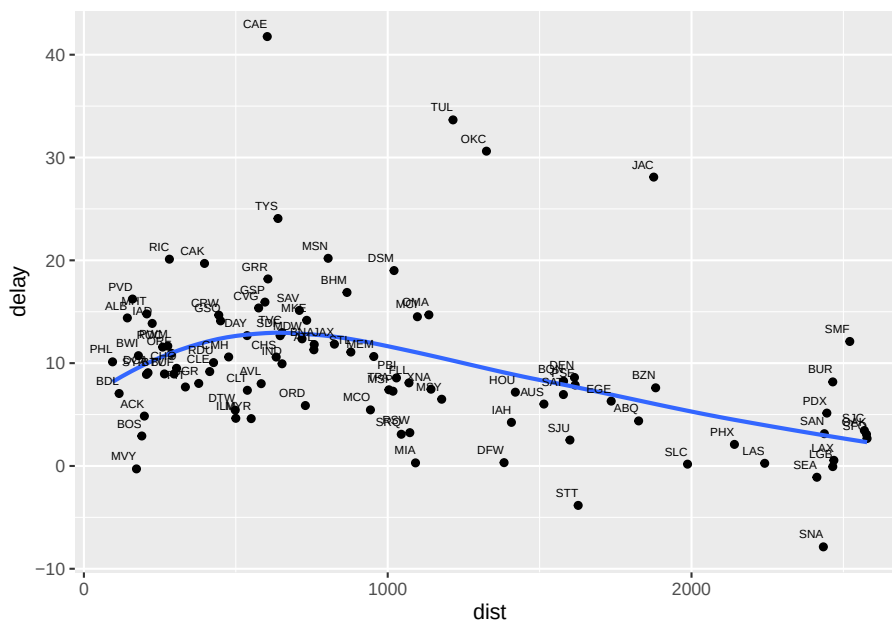
```
names(delay)
```

```
## [1] "dest" "count" "dist" "delay"
```

```
head(delay)
```

```
## # A tibble: 6 x 4
##   dest  count  dist delay
##   <chr> <int> <dbl> <dbl>
## 1 ABQ     254 1826  4.38
## 2 ACK     265  199  4.85
## 3 ALB     439  143 14.4
## 4 ATL   17215  757. 11.3
## 5 AUS    2439 1514.  6.02
## 6 AVL     275  584.  8.00
```

```
ggplot(data = delay, aes(x = dist, y = delay, label=dest)) +
  geom_point() +
  geom_smooth(se = FALSE)+
  geom_text(size=2,aes(label=dest), hjust=1, vjust=-1)
```



Rolling average:

<https://stackoverflow.com/questions/743812/calculating-moving-average>

```
library(tidyverse)
library(zoo)

some_data = tibble(day = 1:10)
some_data
```

```
## # A tibble: 10 x 1
##   day
##   <int>
## 1     1
## 2     2
## 3     3
## 4     4
## 5     5
## 6     6
## 7     7
## 8     8
## 9     9
## 10    10
```

```
# cma = centered moving average
# tma = trailing moving average
some_data %>%
  mutate(cma = rollmean(day, k = 3, fill = NA)) %>%
  mutate(tma = rollmean(day, k = 3, fill = NA, align = "right")) %>%
  mutate(lma = rollmean(day, k = 3, fill = NA, align = "left")) %>%
  mutate(cmax=rollmax(day, k=30, fill=NA))
```

```
## # A tibble: 10 x 5
##   day    cma    tma    lma cmax
##   <int> <dbl> <dbl> <dbl> <lgl>
## 1     1    NA    NA     2  NA
## 2     2     2    NA     3  NA
## 3     3     3     2     4  NA
## 4     4     4     3     5  NA
## 5     5     5     4     6  NA
## 6     6     6     5     7  NA
## 7     7     7     6     8  NA
## 8     8     8     7     9  NA
## 9     9     9     8    NA  NA
## 10    10    NA     9    NA  NA
```

```
some_data
```

```
## # A tibble: 10 x 1
##   day
##   <int>
## 1     1
## 2     2
## 3     3
## 4     4
## 5     5
## 6     6
## 7     7
## 8     8
## 9     9
## 10    10
```


Chapter 12

Transformación: Índices paramétricos y no paramétricos

```
## [1] "2026-08-01"
```

12.1 Funciones estadísticas básicas:

Casi todas estas funciones (`mean()`, `sd()`, `min()`, `max()`, `IQR()` ...) devuelven NA si hay valores faltantes. Añade `na.rm = TRUE` para calcularlas ignorando los NA.

Funciones estadísticas de resumen — cada una devuelve un solo número.

- **Centro:** `mean()` (promedio), `median()` (mediana).
- **Dispersión:** `sd()` (desviación estándar), `IQR()` (rango intercuartil), `mad()` (desviación absoluta mediana).
- **Extremos y posición:** `min()`, `max()`, `quantile()`, `first()`, `last()`.
- **Recuerda:** añade `na.rm = TRUE` si hay valores faltantes.

Tarea de grupo.

Preparar un `.rmd` con las explicaciones de cómo utilizar la función mencionada abajo. Debe incluir

1. La definición de la función en palabra y matemática.
2. un script sencillo (con pocos datos) (uno o más ejemplos) para explicar la función
3. un script con los datos de “vuelos” o de “Covid-19 de PR”.
4. a las 2:20pm cada grupo presentará su trabajo.

5. Después de la clase cada grupo, mejorará su .rmd con los comentarios recibidos.
6. Domingo se subirá el .rmd y el .html en MSteam (cada estudiante lo subirá: tendrá el nombre de cada estudiante en el trabajo)
7. El profesor revisará los trabajos y subsiguiente se distribuirá los .rmd y html a los estudiantes.
 - IQR() ## G1
 - mad() ## G2
 - first() ## G3
 - last() ## G3
 - quantile() ## G4
 - signif() ## G5
 - min()
 - max()
 - mean()
 - sd() ***

1. **Ejercicios:**

Hacer los ejercicios en la sección 5.7.1 del libro en español

Chapter 13

Script

Fecha de la última revisión

```
## [1] "2026-08-01"
```

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/workflow-scripts.html>

Español: <https://es.r4ds.hadley.nz/06-workflow-scripts.html>

Temas super importante:

13.1 Scripts:

- Ejecutando códigos
- Diagnóstico de Rstudio

No copies y pegues en la consola: coloca el cursor en la línea (o selecciona varias) y ejecútala con Cmd/Ctrl+Enter directamente desde el script.

1. Ejercicios:

Hacer los ejercicios en la sección 6.3 del libro en español

```
library(dplyr)
library(datos)
library(nycflights13)

names(vuelos)
```

```
## [1] "anio"          "mes"          "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
## [10] "aerolinea"      "vuelo"        "codigoCola"
## [13] "origen"         "destino"      "tiempo_vuelo"
## [16] "distancia"      "hora"         "minuto"
## [19] "fecha_hora"

no_cancelado <- vuelos %>%
  filter(!is.na(atraso_salida), !is.na(atraso_llegada))
head(no_cancelado)

## # A tibble: 6 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>          <int>          <int>          <dbl>
## 1  2013     1     1           517            515            2
## 2  2013     1     1           533            529            4
## 3  2013     1     1           542            540            2
## 4  2013     1     1           544            545           -1
## 5  2013     1     1           554            600           -6
## 6  2013     1     1           554            558           -4
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>

no_cancelado %>%
  group_by(anio, mes, dia) %>%
  summarise(media = mean(atraso_salida))

## # A tibble: 365 x 4
## # Groups:   anio, mes [12]
##   anio  mes  dia media
##   <int> <int> <int> <dbl>
## 1  2013     1     1  11.4
## 2  2013     1     2  13.7
## 3  2013     1     3  10.9
## 4  2013     1     4   8.97
## 5  2013     1     5   5.73
## 6  2013     1     6   7.15
## 7  2013     1     7   5.42
## 8  2013     1     8   2.56
## 9  2013     1     9   2.30
## 10 2013     1    10   2.84
## # i 355 more rows

x <- 10
```

Chapter 14

Errores comunes y cómo resolverlos

Fecha de la última revisión

```
## [1] "2026-08-01"
```

Equivocarse **es parte normal de programar**. Hasta quienes llevan años usando R ven errores todos los días. La destreza que de verdad importa no es *evitar* los errores, sino **aprender a leerlos y resolverlos**, porque cuando hagas tu propio análisis nadie va a haber “arreglado” el código de antemano.

Un mensaje de error casi siempre te dice **dos cosas**: *qué* salió mal y, muchas veces, *dónde* (la función o la línea). Léelo despacio de arriba hacia abajo; la primera línea suele ser la más importante.

En este capítulo provocamos **a propósito** los errores más frecuentes para que los reconozcas cuando aparezcan. En cada ejemplo verás el mensaje real de R y luego cómo resolverlo.

```
library(dplyr)
```

14.1 could not find function

```
promedio(c(2, 4, 6))
```

```
## Error in `promedio()`:  
## ! could not find function "promedio"
```

could not find function "promedio" significa que R no conoce esa función. Las tres causas más comunes son: (1) escribiste mal el nombre, (2) **olvidaste activar el paquete** con `library()`, o (3) la función es de otro paquete que no está instalado. Solución: revisa la ortografía y corre el `library()` correspondiente.

R **distingue mayúsculas de minúsculas**, así que `Sqrt` no es lo mismo que `sqrt`:

```
Sqrt(9)
```

```
## Error in `Sqrt()`:
## ! could not find function "Sqrt"
```

Si una función “no existe” pero estás seguro de que sí, revisa las mayúsculas: `sqrt()`, no `Sqrt()`; `TRUE`, no `True`.

14.2 object ... not found

```
resultado + 1
```

```
## Error:
## ! object 'resultado' not found
```

`object 'resultado' not found` quiere decir que usaste una variable que **todavía no existe**. Suele pasar cuando hay una errata en el nombre o cuando no corriste antes el bloque que crea esa variable. Solución: crea la variable primero (`resultado <- ...`) y ejecuta los bloques **en orden**.

14.3 non-numeric argument to binary operator

```
"3" * 2
```

```
## Error in `"3" * 2`:
## ! non-numeric argument to binary operator
```

R no puede multiplicar **texto** por un número: `"3"` (con comillas) es una cadena de caracteres, no el número 3. Conviértelo con `as.numeric("3")` antes de operar.

14.4 unused argument

```
mean(c(1, 2, 3), n = 2)
```

```
## [1] 2
```

unused argument (n = 2) significa que le pasaste a la función un argumento que **no reconoce**. Casi siempre es un nombre de argumento equivocado. Revisa la ayuda con `?mean` para ver los nombres correctos (aquí no existe `n`; quizás querías `trim`).

14.5 Confundir = con ==

```
filter(mtcars, cyl = 6)
```

```
## Error in `filter()`:  
## ! We detected a named input.  
## i This usually means that you've used `=` instead of `==`.  
## i Did you mean `cyl == 6`?
```

Dentro de `filter()` (y para comparar en general) se usa `==`, no `=`. Un solo `=` **asigna**; el doble `==` **compara**. Por suerte, `dplyr` hasta te sugiere la corrección: `cyl == 6`.

14.6 El caso especial de NA (no es un error, pero engaña)

Esto **no** produce un error, pero el resultado sorprende a muchos:

```
mean(c(1, 2, NA))
```

```
## [1] NA
```

Casi cualquier cálculo con un valor faltante (NA) devuelve NA. No es un error: R te avisa que no puede calcular la media sin saber ese dato. La solución es pedir que ignore los faltantes con `na.rm = TRUE`:

```
mean(c(1, 2, NA), na.rm = TRUE)
```

```
## [1] 1.5
```

14.7 cannot open file ... No such file or directory

```
read.csv("mis_datos_que_no_existen.csv")
```

```
## Error in `file()`:  
## ! cannot open the connection
```

Este error aparece cuando R **no encuentra el archivo**. Las causas típicas son: el nombre está mal escrito, el archivo está en otra carpeta, o tu **directorio de trabajo** no es el que crees. La mejor prevención es trabajar siempre dentro de un **Proyecto de RStudio** y usar rutas relativas como "Datos/mi_archivo.csv".

14.8 cannot open URL (descargar datos de internet)

Cuando un capítulo baja datos en vivo, por ejemplo con `fread("https://...")` o `download.file()`, y no hay conexión (o el servidor no responde), verás algo como esto:

```
Error in download.file(...) :  
cannot open URL 'https://ejemplo.com/datos.csv'
```

(No ejecutamos ese código aquí a propósito, para no depender de la red.)

cannot open URL no es un error de tu código: significa que no hay internet o que el servidor está caído o lento. Verifica tu conexión y vuelve a intentar.

Una técnica profesional para que tu análisis no se caiga por completo cuando esto pasa es **tryCatch()**: intenta la descarga en vivo y, si falla, usa una copia local que guardaste antes. Este es exactamente el patrón que usamos en el capítulo de mapas:

```
datos <- tryCatch(  
  fread("https://ejemplo.com/datos.csv"),           # intenta en vivo  
  error = function(e) readr::read_csv("Datos/copia_local.csv") # si falla, usa la copia local  
)
```

Buena práctica: descarga los datos **una vez**, guárdalos con `write_csv()` en la carpeta **Datos/**, y en adelante trabaja con la copia local. Así tu análisis es reproducible aunque no tengas internet.

14.9 Un método general para resolver errores

1. **Lee el mensaje completo**, sobre todo la primera línea.
2. Identifica la **función** o la **línea** que menciona.
3. Pregúntate: ¿escribí bien el nombre?, ¿activé el paquete?, ¿existe la variable?, ¿el archivo está donde digo?
4. Si te atorras, **copia el mensaje de error y búscalo en internet**: casi siempre alguien ya tuvo el mismo problema.
5. Reduce el problema a un **ejemplo mínimo** que lo reproduzca; muchas veces al simplificarlo descubres la causa.

Guarda la calma: un error no significa que “no sirves para esto”. Significa que R necesita que le aclares algo. Cada error que resuelves te hace mejor analista.

Chapter 15

Análisis Exploratorio

Fecha de la última revisión

[1] "2026-08-01"

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/exploratory-data-analysis.html>

Español: <https://es.r4ds.hadley.nz/07-eda.html>

15.1 Temas

- Variación
 - Valores faltantes
 - Covariación
 - Patrones y modelos
 - Argumentos en ggplot2
-

15.2 Términos importantes

- variable
 - valor
 - observación
 - datos tabulados
-

15.2.1 Variación

El **análisis exploratorio de datos** (EDA) fue impulsado por **John W. Tukey**, quien publicó el libro *Exploratory Data Analysis* en 1977. Su idea: antes de probar hipótesis, hay que **mirar** los datos con gráficos sencillos para descubrir patrones, valores típicos y atípicos.

count() y los **geom_* de exploración** — resumir y visualizar.

- **count(variable):** cuenta cuántas observaciones hay por categoría.
- **Una variable:** `geom_histogram()` / `geom_bar()` para ver su distribución.
- **Dos variables:** `geom_point()`, `geom_boxplot()`, `geom_count()`, `geom_tile()`, `geom_hex()` según los tipos.
- visualización de las distribuciones
- `count()`
- `cut_width()`
- valores típicos
- valores inusuales o atípicos

1. Ejercicios:

Hacer los ejercicios en la sección 7.3.4 del libro en español

15.2.2 Valores faltantes

- `between()`
- `ifelse()`

2. Ejercicios:

Hacer los ejercicios en la sección 7.4.1 del libro en español

En R los valores faltantes se representan con **NA**. Funciones como `mean()` o `sum()` los propagan y devuelven **NA**, por eso conviene usar el argumento `na.rm = TRUE`.

15.2.3 Covariación

- `geom_boxplot`
 - `IQR()`
-

2. Ejercicios:

Hacer los ejercicios en la sección 7.5.1.1 del libro en español

15.2.4 Dos variables categóricas

- `geom_count()`
 - `geom_tile()`
-

2. Ejercicios:

Hacer los ejercicios en la sección 7.5.2.1 del libro en español

15.2.5 Dos variables continuas

- `geom_point()`
 - `geom_hex()`
 - `geom_boxplot()`
-

3. Ejercicios:

Hacer los ejercicios en la sección 7.5.3.1 del libro en español

15.2.6 Patrones y modelos

- construir un modelo estadístico

Chapter 16

Pipes

Fecha de la última revisión

[1] "2026-08-01"

```
library(tidyverse)
library(magrittr) # El paquete de Pipe, se activa usando tidyverse
```

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/pipes.html>

Español: <https://es.r4ds.hadley.nz/>

16.1 Temas: El paquete “magrittr” y sus funciones

El operador pipe `%>%` fue creado por **Stefan Milton Bache** en el paquete **magrittr** (2014); el nombre juega con el cuadro de Magritte «*Ceci n'est pas une pipe*». Fue tan popular que en 2021 R incorporó su propio pipe nativo `|>`. La idea: leer el código de izquierda a derecha, como una receta de pasos.

Los pipes que vamos a evaluar son los siguientes:

- `%>%` “Pipe”
- `%!>%` “Eager Pipe”
- `%%` “Exposition pipe”

- %<>% “Assignment pipe”
- %T>% “Tee Pipe”
- %in% “In Pipe”

Los pipes de **magrittr** — encadenar operaciones.

- %>% / |>: pasa el resultado de la izquierda como primer argumento de la derecha.
- %\$%: expone las columnas para usarlas por nombre (p. ej. `cor(x, y)`).
- %<>%: aplica y **reassigna** al objeto original.
- %T>%: ejecuta un paso “lateral” (como `plot`) y sigue pasando el objeto original.

Función básica para calcular el precio básico de diamantes por quilates

```
head(diamonds, n=3)
```

```
## # A tibble: 3 x 10
##   carat cut      color clarity depth table price      x      y      z
##   <dbl> <ord>   <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
## 1  0.23 Ideal    E      SI2     61.5   55   326  3.95  3.98  2.43
## 2  0.21 Premium E      SI1     59.8   61   326  3.89  3.84  2.31
## 3  0.23 Good    E      VS1     56.9   65   327  4.05  4.07  2.31
```

```
diamonds$Precio_q = diamonds$price/diamonds$carat # crear una nueva variable
```

```
head(diamonds, n=3)
```

```
## # A tibble: 3 x 11
##   carat cut      color clarity depth table price      x      y      z Precio_q
##   <dbl> <ord>   <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl> <dbl>
## 1  0.23 Ideal    E      SI2     61.5   55   326  3.95  3.98  2.43  1417.
## 2  0.21 Premium E      SI1     59.8   61   326  3.89  3.84  2.31  1552.
## 3  0.23 Good    E      VS1     56.9   65   327  4.05  4.07  2.31  1422.
```

```
diamonds2 <- diamonds %>%
  dplyr::mutate(price_per_carat = price / carat)
```

Calcular el promedio de precio por la calidad de los diamantes usando pipes

Desde la versión 4.1, R trae un pipe nativo `|>` que no requiere paquetes. El `%>%` proviene de **magrittr** y añade funciones extra como el marcador de posición `..`

```
diamonds %>%
  group_by(cut) %>%
  summarize(Precio_color= mean(price))
```

```
## # A tibble: 5 x 2
```



```
##      cut      Precio_color
##   <ord>      <dbl>
## 1 Fair      4359.
## 2 Good      3929.
## 3 Very Good 3982.
## 4 Premium   4584.
## 5 Ideal     3458.
```

Calcular el promedio de precio por color de los diamantes usando pipes

16.2 %>% pipes

Un ejemplo básico del pipe

```
car_data <-
  mtcars %>%
  subset(hp > 100) %>%
  aggregate(. ~ cyl, data = ., FUN = . %>% mean %>% round(2)) %>%
  transform(kpl = mpg %>% multiply_by(0.4251)) %>%
  print
```

```
##   cyl  mpg  disp    hp drat   wt  qsec    vs  am gear carb    kpl
## 1   4 25.90 108.05 111.00 3.94  2.15 17.75  1.00 1.00  4.50  2.00 11.010090
## 2   6 19.74 183.31 122.29 3.59  3.12 17.98  0.57 0.43  3.86  3.43  8.391474
## 3   8 15.10 353.10 209.21 3.23  4.00 16.77  0.00 0.14  3.29  3.50  6.419010
```

Un ejemplo sin usar el pipe para calcular la misma información

```
car_data <-
  transform(aggregate(. ~ cyl,
                      data = subset(mtcars, hp > 100),
                      FUN = function(x) round(mean(x), 2)),
            kpl = mpg*0.4251)
```

```
car_data
```

```
##   cyl  mpg  disp    hp drat   wt  qsec    vs  am gear carb    kpl
## 1   4 25.90 108.05 111.00 3.94  2.15 17.75  1.00 1.00  4.50  2.00 11.010090
## 2   6 19.74 183.31 122.29 3.59  3.12 17.98  0.57 0.43  3.86  3.43  8.391474
## 3   8 15.10 353.10 209.21 3.23  4.00 16.77  0.00 0.14  3.29  3.50  6.419010
```

```
car_data <-
  mtcars %>%
  subset(hp > 100) %>%
  group_by(cyl) %>%
  summarise(across(everything(), list(mean))) %>%
```

```

transform(kpl = mpg_1 %>% multiply_by(0.4251)) %>%
print

##   cyl   mpg_1  disp_1    hp_1  drat_1    wt_1  qsec_1    vs_1    am_1
## 1    4 25.90000 108.0500 111.0000 3.940000 2.146500 17.75000 1.0000000 1.0000000
## 2    6 19.74286 183.3143 122.2857 3.585714 3.117143 17.97714 0.5714286 0.4285714
## 3    8 15.10000 353.1000 209.2143 3.229286 3.999214 16.77214 0.0000000 0.1428571
##   gear_1  carb_1    kpl
## 1 4.500000 2.000000 11.010090
## 2 3.857143 3.428571  8.392689
## 3 3.285714 3.500000  6.419010

```

16.3 Alternativas a los pipes

- Pasos intermedios
 - Sobre escribir el original
-

16.4 %\$%

```

library(magrittr)

#iris %>%
#  subset(Sepal.Length > mean(Sepal.Length)) |>
#  cor(Sepal.Length, Sepal.Width)

iris %>%
  subset(Sepal.Length > mean(Sepal.Length)) %$%
  cor(Sepal.Length, Sepal.Width)

## [1] 0.3361992

iris %>%
  subset(Sepal.Length > mean(Sepal.Length)) |>
  summarize(corr= cor(Sepal.Length, Sepal.Width))

##           corr
## 1 0.3361992

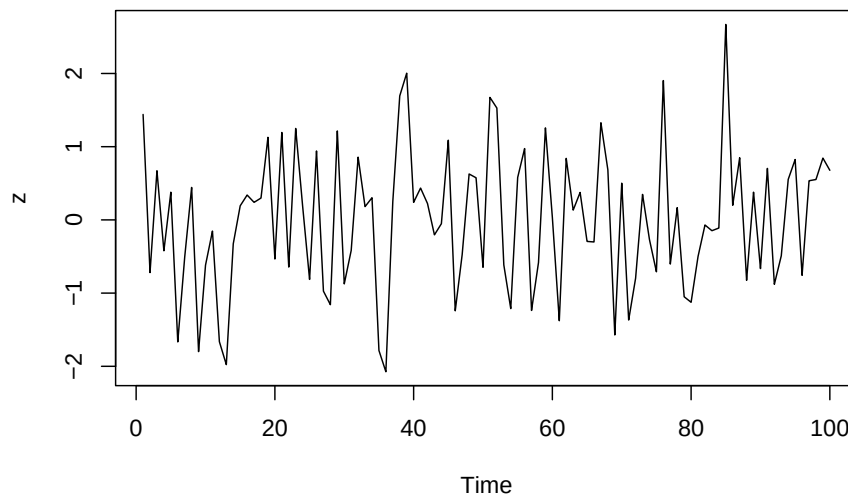
```

```
data.frame(z = rnorm(100)) %$%  
  ts.plot(z)
```

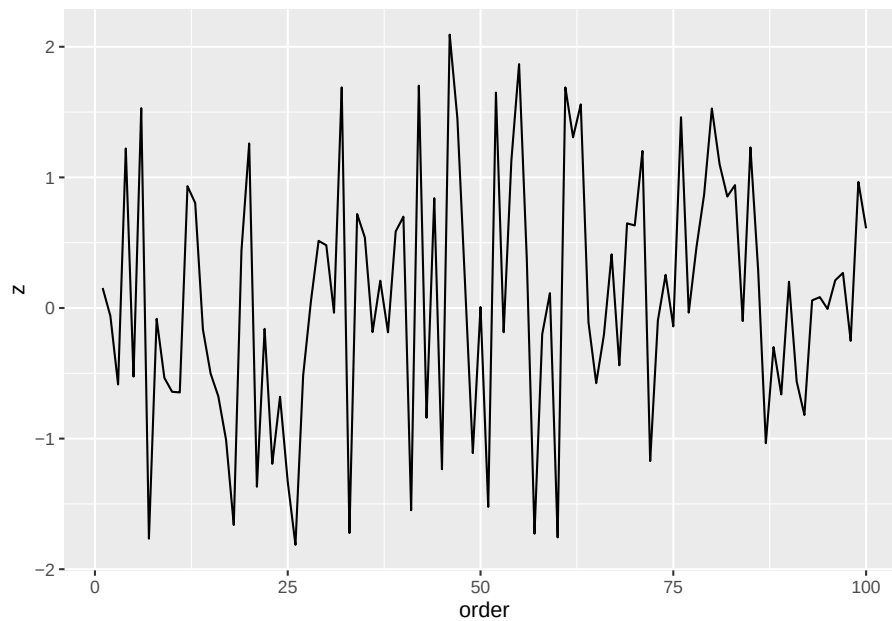
16.4. %\$%

147

16.4.1 Otro ejemplo de Pipe: %\$%



```
#data.frame(z = rnorm(100)) |>  
#  ts.plot(z)  
  
#Para hacerlo la misma grafico con Tidyverse  
z1=data.frame(z = rnorm(100))  
library(data.table)  
z1=data.table(z1)  
z1$order= c(1:100)  
z1|>  
  ggplot(aes(order, z))+  
  geom_line()
```



16.5 %<>%

Pipe an object forward into a function or call expression and update the lhs object with the resulting value.

lhs %<>% rhs

- lhs : An object which serves both as the initial value and as target.
- rhs : a function call using the magrittr semantics.

Example 1

```
head(iris)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1         5.1         3.5          1.4          0.2   setosa
## 2         4.9         3.0          1.4          0.2   setosa
## 3         4.7         3.2          1.3          0.2   setosa
## 4         4.6         3.1          1.5          0.2   setosa
## 5         5.0         3.6          1.4          0.2   setosa
## 6         5.4         3.9          1.7          0.4   setosa
```

```
head(iris$Sepal.Length %<>% sqrt)
```

```
## [1] 2.258318 2.213594 2.167948 2.144761 2.236068 2.323790
```

```
# Example 2
x <- rnorm(100)
head(x, n=10)

## [1]  1.361612477 -0.189292059  0.462969745 -0.007385678  2.689108440
## [6]  1.459546809 -1.653300235 -0.203078277  0.618497216  0.597135379

x %<>%
  abs %>%
  sort

head(x, 20)

## [1] 0.007385678 0.029096523 0.030999873 0.035387591 0.040816992 0.054441440
## [7] 0.069795203 0.081581732 0.087776321 0.110958305 0.119075263 0.120358116
## [13] 0.120542457 0.131861167 0.133266112 0.145019780 0.145731942 0.150293091
## [19] 0.155538095 0.164714712
```

16.6 %in%

- Filter for values specific values within a variable

```
# Example 1
names(iris)

## [1] "Sepal.Length" "Sepal.Width" "Petal.Length" "Petal.Width" "Species"

iris |>
  filter(Species %in% c("setosa", "virginica")) |>
  head()

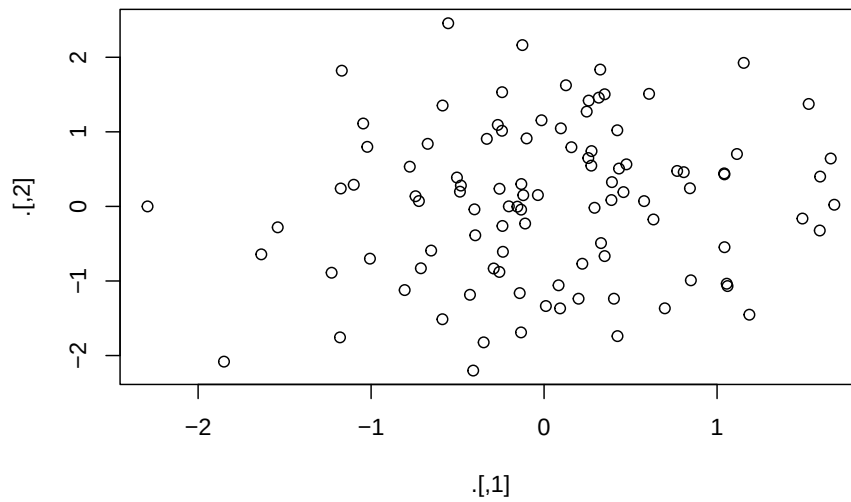
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1      2.258318         3.5          1.4          0.2   setosa
## 2      2.213594         3.0          1.4          0.2   setosa
## 3      2.167948         3.2          1.3          0.2   setosa
## 4      2.144761         3.1          1.5          0.2   setosa
## 5      2.236068         3.6          1.4          0.2   setosa
## 6      2.323790         3.9          1.7          0.4   setosa
```

16.7 %T>%

The tee pipe, %T>%, is useful when a series of operations have a function that does not return any value.

<https://stackoverflow.com/questions/61196304/magrittr-tee-pipe-t-equivalent>

```
rnorm(200) %>%  
matrix(ncol = 2) %T>%  
plot %>% # plot usually does not return anything.  
colSums
```



```
## [1] 1.395464 5.606604
```

Chapter 17

Tibbles

Fecha de la última revisión

[1] "2026-08-01"

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/tibbles.html>

Español: <https://es.r4ds.hadley.nz/10-tibble.html>

17.1 Temas: El paquete Tibble

Los `tibble` son la versión moderna del `data.frame` dentro del tidyverse: hacen menos «magia» automática (no cambian nombres ni tipos de columna) y ofrecen una impresión más legible en la consola.

- `as_tibble()`
-

17.2 Crear un tibble

`tibble()`, `tribble()`, `as_tibble()` — crear tablas modernas.

- `tibble(col = valores, ...)`: crea un tibble por columnas.
- `tribble(~col1, ~col2, v, v, ...)`: lo crea **fila por fila** (cómodo para tablas pequeñas escritas a mano).
- `as_tibble(df)`: convierte un `data.frame` en tibble.

- `tibble()`

Crear un tibble con la siguiente información. Una columna que tiene los valores del 1 al 8 que se llama “secuencia”, una segunda columna que tiene los siguientes valores 23,26, 24,26,27, 11,20,21 que se llama edad y una tercera columna con la siguiente información, Jose, Maria, Carol, Moncho, Liz, Maria, Jorge, Miguel, que se llama Nombre

```
library(tidyverse)
#library(datos)

tibble(
  secuencia = c(1:6),
  edad = c(23,26, 24,26,27, NA),
  nombre = c("Jose", "Maria", "Carol", "Moncho", "Liz", "Maria")
)
```

```
## # A tibble: 6 x 3
##   secuencia edad nombre
##       <int> <dbl> <chr>
## 1         1    23 Jose
## 2         2    26 Maria
## 3         3    24 Carol
## 4         4    26 Moncho
## 5         5    27 Liz
## 6         6    NA Maria
```

17.3 Crear un tribble

- `tribble()`

Haga un tribble con las primeras 3 líneas del tibble anterior

```
tribble(
  ~secuencia, ~edad, ~nombre,
  1, 23, "Jose",
  2, 26, "Maria",
  3, 23, "Carol"
)
```

```
## # A tibble: 3 x 3
##   secuencia edad nombre
##       <dbl> <dbl> <chr>
## 1         1    23 Jose
```



```
## 2      2      26 Maria
## 3      3      23 Carol
```

17.4 tibble vs. data.frame

```
tb = tibble(
  a = lubridate::now() + runif(1e2) * 86400,
  b = lubridate::today() + runif(1e2) * 30,
  c = 1:100,
  d = runif(1e2),
  e = sample(letters, 1e2, replace = TRUE)
)
tb
```

```
## # A tibble: 100 x 5
##       a                b          c      d e
##   <dtm>          <date>    <int>  <dbl> <chr>
## 1 2026-08-02 00:19:22 2026-08-19     1 0.559 o
## 2 2026-08-02 06:48:37 2026-08-01     2 0.250 o
## 3 2026-08-02 01:42:13 2026-08-03     3 0.0653 d
## 4 2026-08-02 23:28:28 2026-08-02     4 0.573 v
## 5 2026-08-02 07:00:43 2026-08-23     5 0.651 k
## 6 2026-08-02 15:12:32 2026-08-12     6 0.581 p
## 7 2026-08-02 19:50:37 2026-08-25     7 0.0175 x
## 8 2026-08-02 18:50:36 2026-08-12     8 0.0133 f
## 9 2026-08-02 13:05:21 2026-08-15     9 0.661 c
## 10 2026-08-02 19:13:33 2026-08-24    10 0.380 y
## # i 90 more rows
```

```
df_tb=as.data.frame(tb)
df_tb
```

```
##           a                b          c      d e
## 1 2026-08-02 00:19:22 2026-08-19     1 0.55895788 o
## 2 2026-08-02 06:48:37 2026-08-01     2 0.24952694 o
## 3 2026-08-02 01:42:13 2026-08-03     3 0.06531608 d
## 4 2026-08-02 23:28:28 2026-08-02     4 0.57306761 v
## 5 2026-08-02 07:00:43 2026-08-23     5 0.65098699 k
## 6 2026-08-02 15:12:32 2026-08-12     6 0.58124937 p
## 7 2026-08-02 19:50:37 2026-08-25     7 0.01748210 x
## 8 2026-08-02 18:50:36 2026-08-12     8 0.01331984 f
## 9 2026-08-02 13:05:21 2026-08-15     9 0.66064715 c
## 10 2026-08-02 19:13:33 2026-08-24    10 0.38021846 y
## 11 2026-08-02 23:22:50 2026-08-23    11 0.55374044 o
```

```

## 12 2026-08-02 20:50:23 2026-08-12 12 0.57734132 r
## 13 2026-08-02 21:15:03 2026-08-03 13 0.89874595 s
## 14 2026-08-02 15:35:04 2026-08-14 14 0.02335350 s
## 15 2026-08-02 10:14:12 2026-08-24 15 0.36233644 a
## 16 2026-08-02 11:59:19 2026-08-08 16 0.79219053 c
## 17 2026-08-02 13:17:10 2026-08-08 17 0.79451792 q
## 18 2026-08-02 01:10:39 2026-08-11 18 0.03461087 h
## 19 2026-08-02 05:48:51 2026-08-02 19 0.83758986 y
## 20 2026-08-02 12:04:10 2026-08-12 20 0.08880666 h
## 21 2026-08-02 04:24:22 2026-08-14 21 0.81779321 m
## 22 2026-08-02 02:48:44 2026-08-28 22 0.88725936 k
## 23 2026-08-02 15:43:05 2026-08-02 23 0.26002925 a
## 24 2026-08-02 02:53:48 2026-08-12 24 0.31542900 t
## 25 2026-08-02 09:30:52 2026-08-06 25 0.91124396 m
## 26 2026-08-02 04:50:41 2026-08-15 26 0.69990520 u
## 27 2026-08-02 14:01:49 2026-08-30 27 0.83810166 o
## 28 2026-08-02 11:31:46 2026-08-04 28 0.39910988 q
## 29 2026-08-02 21:26:30 2026-08-24 29 0.99993174 t
## 30 2026-08-02 13:09:31 2026-08-10 30 0.66218230 y
## 31 2026-08-02 08:07:19 2026-08-05 31 0.98046836 c
## 32 2026-08-02 12:17:10 2026-08-13 32 0.13905381 o
## 33 2026-08-02 16:19:05 2026-08-27 33 0.34755612 m
## 34 2026-08-02 05:13:43 2026-08-20 34 0.99072892 t
## 35 2026-08-02 20:27:05 2026-08-23 35 0.01158322 g
## 36 2026-08-02 10:40:53 2026-08-21 36 0.59905566 w
## 37 2026-08-02 08:42:06 2026-08-21 37 0.66253655 m
## 38 2026-08-02 20:01:26 2026-08-09 38 0.81108752 p
## 39 2026-08-02 03:40:48 2026-08-28 39 0.86941914 m
## 40 2026-08-02 13:26:25 2026-08-25 40 0.18531162 v
## 41 2026-08-02 21:14:53 2026-08-05 41 0.93845374 p
## 42 2026-08-02 01:45:02 2026-08-11 42 0.98626153 o
## 43 2026-08-02 01:41:59 2026-08-24 43 0.36978729 g
## 44 2026-08-02 01:29:55 2026-08-12 44 0.40597546 h
## 45 2026-08-01 23:54:30 2026-08-03 45 0.45127090 p
## 46 2026-08-02 16:30:29 2026-08-05 46 0.83761479 d
## 47 2026-08-02 13:22:41 2026-08-30 47 0.20941751 c
## 48 2026-08-02 09:14:22 2026-08-26 48 0.99204004 y
## 49 2026-08-02 09:31:57 2026-08-02 49 0.50421065 r
## 50 2026-08-02 03:06:39 2026-08-12 50 0.33143023 g
## 51 2026-08-02 22:30:14 2026-08-11 51 0.73076078 w
## 52 2026-08-02 13:26:05 2026-08-07 52 0.46601006 v
## 53 2026-08-02 09:55:15 2026-08-21 53 0.93914242 z
## 54 2026-08-02 14:49:22 2026-08-27 54 0.88173091 s
## 55 2026-08-02 07:48:06 2026-08-25 55 0.34532379 i
## 56 2026-08-02 01:51:01 2026-08-19 56 0.58456342 l
## 57 2026-08-01 23:53:06 2026-08-27 57 0.88058255 v

```

```
## 58 2026-08-02 01:45:51 2026-08-02 58 0.22127013 v
## 59 2026-08-02 15:26:28 2026-08-19 59 0.89288290 o
## 60 2026-08-02 07:34:01 2026-08-07 60 0.02081485 y
## 61 2026-08-02 04:11:47 2026-08-17 61 0.27143203 n
## 62 2026-08-02 09:09:14 2026-08-08 62 0.95400736 c
## 63 2026-08-02 07:16:01 2026-08-09 63 0.86419811 j
## 64 2026-08-02 08:10:29 2026-08-10 64 0.25909042 i
## 65 2026-08-02 16:08:31 2026-08-11 65 0.71488890 t
## 66 2026-08-02 16:40:24 2026-08-10 66 0.14559841 m
## 67 2026-08-02 13:47:18 2026-08-15 67 0.33884110 a
## 68 2026-08-02 17:13:06 2026-08-14 68 0.85914380 c
## 69 2026-08-02 14:16:36 2026-08-22 69 0.78041364 o
## 70 2026-08-02 08:53:42 2026-08-21 70 0.83340269 v
## 71 2026-08-02 19:43:31 2026-08-22 71 0.46138001 k
## 72 2026-08-02 19:00:41 2026-08-23 72 0.95366063 x
## 73 2026-08-02 05:38:29 2026-08-28 73 0.58594067 i
## 74 2026-08-02 07:48:18 2026-08-17 74 0.07427914 i
## 75 2026-08-02 00:07:39 2026-08-29 75 0.41196391 p
## 76 2026-08-02 21:15:41 2026-08-11 76 0.64245513 l
## 77 2026-08-02 09:22:20 2026-08-14 77 0.88572561 a
## 78 2026-08-02 03:53:46 2026-08-30 78 0.27894255 z
## 79 2026-08-02 09:47:54 2026-08-21 79 0.11545121 p
## 80 2026-08-02 00:51:33 2026-08-27 80 0.27069348 w
## 81 2026-08-02 14:37:17 2026-08-02 81 0.09039557 m
## 82 2026-08-02 13:49:41 2026-08-30 82 0.84375956 u
## 83 2026-08-02 05:16:56 2026-08-13 83 0.44452948 t
## 84 2026-08-02 03:37:06 2026-08-09 84 0.17054338 a
## 85 2026-08-02 09:38:03 2026-08-30 85 0.94064577 n
## 86 2026-08-02 12:05:42 2026-08-06 86 0.55504992 s
## 87 2026-08-02 03:18:54 2026-08-29 87 0.74138904 o
## 88 2026-08-02 03:01:24 2026-08-12 88 0.69146950 m
## 89 2026-08-02 15:09:21 2026-08-23 89 0.95055655 v
## 90 2026-08-02 19:05:00 2026-08-27 90 0.50241356 h
## 91 2026-08-02 19:24:38 2026-08-26 91 0.94041471 l
## 92 2026-08-02 13:28:33 2026-08-12 92 0.66040567 e
## 93 2026-08-02 07:20:39 2026-08-17 93 0.74075930 s
## 94 2026-08-02 18:55:50 2026-08-17 94 0.49824345 q
## 95 2026-08-01 23:34:58 2026-08-29 95 0.10436335 j
## 96 2026-08-02 08:33:31 2026-08-01 96 0.06179632 g
## 97 2026-08-02 11:36:07 2026-08-10 97 0.21776393 s
## 98 2026-08-02 22:55:54 2026-08-01 98 0.15662152 x
## 99 2026-08-02 09:39:23 2026-08-19 99 0.62424786 s
## 100 2026-08-02 14:36:39 2026-08-03 100 0.39037936 c
```

```
tb %>%
  print(n = 4, width = Inf)
```

```
## # A tibble: 100 x 5
##   a                b                c      d e
##   <dtm>          <date>        <int>  <dbl> <chr>
## 1 2026-08-02 00:19:22 2026-08-19      1 0.559 o
## 2 2026-08-02 06:48:37 2026-08-01      2 0.250 o
## 3 2026-08-02 01:42:13 2026-08-03      3 0.0653 d
## 4 2026-08-02 23:28:28 2026-08-02      4 0.573 v
## # i 96 more rows
```

```
lubridate::now()
```

```
## [1] "2026-08-01 23:31:40 AST"
```

```
lubridate::today()
```

```
## [1] "2026-08-01"
```

- To convert a data frame to a tibble use `as_tibble()`
- To convert a tibble to a data frame use `as.data.frame`

17.5 Diferencias entre un tibble y un data frame.

- Tibbles tiene un buen método de impresión que muestra solo las primeras 10 filas y todas las columnas que caben en la pantalla.
- Tibbles no admite nombres de filas. Se eliminan al convertir a tibble o al crear subconjuntos:

Operaciones aritméticas en tibbles

A diferencia de los data frame, los tibbles NO admiten operaciones aritméticas en todas las columnas. El resultado fuerza silenciosamente a un data frame.

```
tbl <- tibble(a = 1:3, b = 4:6)
result=tbl * 2
```

```
is_tibble(result)
```

```
## [1] FALSE
```

```
is.data.frame(result)
```

```
## [1] TRUE
```

```
result2=tbl |> mutate(c=b*2)
result2
```

```
## # A tibble: 3 x 3
##       a     b     c
##   <int> <int> <dbl>
```

```
## 1      1      4      8
## 2      2      5     10
## 3      3      6     12
```

```
is_tibble(result2)
```

```
## [1] TRUE
```

- Los tibbles también son más estrictos con \$. Tibbles nunca realiza coincidencias parciales y generará una advertencia y devolverá NULL si la columna no existe:

```
df <- data.frame(abc = 1)
df$a
```

```
## [1] 1
```

```
#> [1] 1
```

```
df2 <- tibble(abc = 1)
df2$abc # nota el error
```

```
## [1] 1
```

- Para extraer una columna de un tibble, use [\$. Esto es más seguro que \$ porque nunca realiza coincidencias parciales:

17.6 Selección de subconjuntos de datos

```
library(datos)
```

```
head(vuelos$salida_programada, n=20)
```

```
## [1] 515 529 540 545 600 558 600 600 600 600 600 600 600 600 600 559 600 600 600
## [20] 600
```

```
vuelos[["salida_programada"]] |> head(n=20)
```

```
## [1] 515 529 540 545 600 558 600 600 600 600 600 600 600 600 600 559 600 600 600
## [20] 600
```

```
#Alternativas usar esta función dentro de pipe
```

```
vuelos %>% .$salida_programada |> head(n=20)
```

```
## [1] 515 529 540 545 600 558 600 600 600 600 600 600 600 600 600 559 600 600 600
## [20] 600
```

```
vuelos %>% .[["salida_programada"]] |> head(n=20)
```

```
## [1] 515 529 540 545 600 558 600 600 600 600 600 600 600 600 559 600 600 600
## [20] 600
```

```
vuelos %>% .[["atraso_salida"]] |>
  head(n=20)
```

```
## [1] 2 4 2 -1 -6 -4 -5 -3 -3 -2 -2 -2 -2 -2 -1 0 -1 0 0 1
```

1. Ejercicios:

Hacer los ejercicios en la sección 10.5 del libro en español

Chapter 18

Importar datos

Fecha de la última revisión

[1] "2026-08-01"

18.1 Datos importados de la web.

Los ejemplos de esta sección **descargan datos en vivo desde internet** con `fread("https://...")`, así que necesitas conexión para ejecutarlos. Si ves `Error in download.file(...): cannot open URL` o un `HTTP error`, revisa tu conexión o inténtalo más tarde: el servidor puede estar caído o tardar en responder. Una buena práctica es descargar el archivo una vez y guardarlo con `write_csv()` en `Datos/` para no depender de la red. Encontrarás más errores frecuentes y cómo resolverlos en el capítulo *Errores comunes* (14).

18.1.1 Usar datos que provienen de una base de datos del web.

Aquí un website que recopila la cantidad de personas vacunadas en los EEUU.

```
library(tidyverse)
library(data.table)
```

Seleccionar solamente los datos de Puerto Rico

18.1.2 COVID world

Otro website de datos de COVID, estos representan los datos de todos los países del mundo

Para encontrar datos de PR vea el siguiente enlace: Este sitio del gobierno de Puerto Rico almacena datos muchos diferentes tipos que están relacionados a la isla.

https://datos.estadisticas.pr/dataset?res_format=CSV

Nota que para leer los datos de un archivo CSV, se usa la función “fread()” del paquete “data.table”.

- Cuando entre a la página, selecciona uno de los archivos, por ejemplo **Facultad -Conservatorio de Musica**
- Presione el botón “Explore” y selecciona “Preview”
- Arriba de la página verá el enlace de **https://.....** para poner en un script.
- Copie y pegue en el script.

Otra alternativa es bajar los datos en formato .csv o Excel y subirlo a su proyecto en RStudio.

18.2 Ejercicio

- Selecciona solamente los datos de los bebés que nacieron primer mes del año, pero agrupado por el día de la semana y contabilizado

COVID_PR

- Calcula la cantidad de muertes por día en PR, y el máximo de muerto en un día por COVID
-

18.3 Datos importando de su computadora

18.3.1 Temas: Funciones para importar datos con el paquete readr

- `read_csv()` # separado por coma
- `read_csv2()` # separado por punto y coma

- `read_tsv()` # separado por tab
- `read_delim()` # separado por delimitador
- `read_fwf()`
- `read_width()`
- `read_positions()`
- `read_table()`

`read_csv()` (readr) y `fread()` (data.table) — importar datos.

- `read_csv("archivo.csv")`: lee un CSV a un tibble; variantes: `read_csv2()` (separado por ;), `read_tsv()` (tabs), `read_delim()` (delimitador a elección).
- `fread("https://...")`: rápido y práctico para leer directamente desde una URL.
- `write_csv(objeto, "archivo.csv")`: guarda tus datos para no depender de la red.

```
library(tidyverse)
library(readr)
Newborn_Karn_Penrose <- read_csv("Datos/Newborn_Karn_Penrose.csv")

#library(readxl)
#Newborn_Karn_Penrose1 <- read_excel("~/Google Drive/GitHub_Google_Drive/GitHub/Vintage_DATA/DATAS")

head(Newborn_Karn_Penrose)
```

```
## # A tibble: 6 x 4
##   Mother_age Count_Non_survivors Count_Total_Birth Birth_weight
##   <dbl>         <dbl>         <dbl>         <dbl>
## 1      16             0             1             9.5
## 2      16             0             2             9
## 3      16             0             1             8.5
## 4      16             0             3             8
## 5      16             0             7             7.5
## 6      16             0             2             7
```

Hacer una columna de la proporción de los niños que no sobrevivieron.

```
Newborn_Karn_Penrose %>% dplyr::select(Count_Non_survivors, Count_Total_Birth)
%>% arrange(Count_Total_Birth, Count_Non_survivors) %>% mu-
tate(Perc_sobrevivieron = Count_Non_survivors/Count_Total_Birth)
```

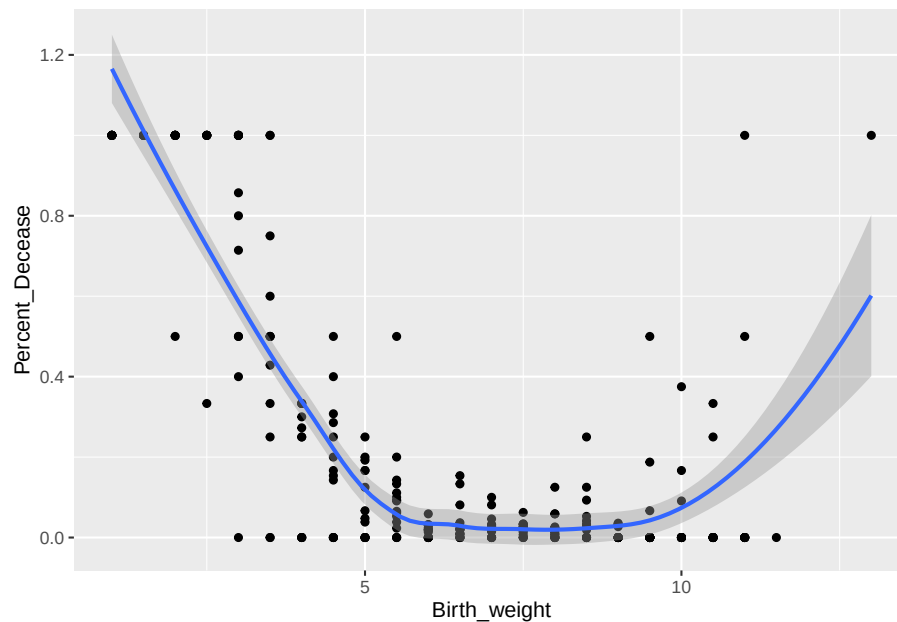
```
Newborn_Karn_Penrose %>%
  mutate(Percent_Decease = Count_Non_survivors/Count_Total_Birth)
```

```
## # A tibble: 261 x 5
##   Mother_age Count_Non_survivors Count_Total_Birth Birth_weight Percent_Decease
##   <dbl>         <dbl>         <dbl>         <dbl>         <dbl>
```

```
## 1      16      0      1      9.5      0
## 2      16      0      2      9      0
## 3      16      0      1      8.5      0
## 4      16      0      3      8      0
## 5      16      0      7      7.5      0
## 6      16      0      2      7      0
## 7      16      0      4      6.5      0
## 8      16      0      2      6      0
## 9      16      0      2      5.5      0
## 10     16      0      1      5      0
## # i 251 more rows
```

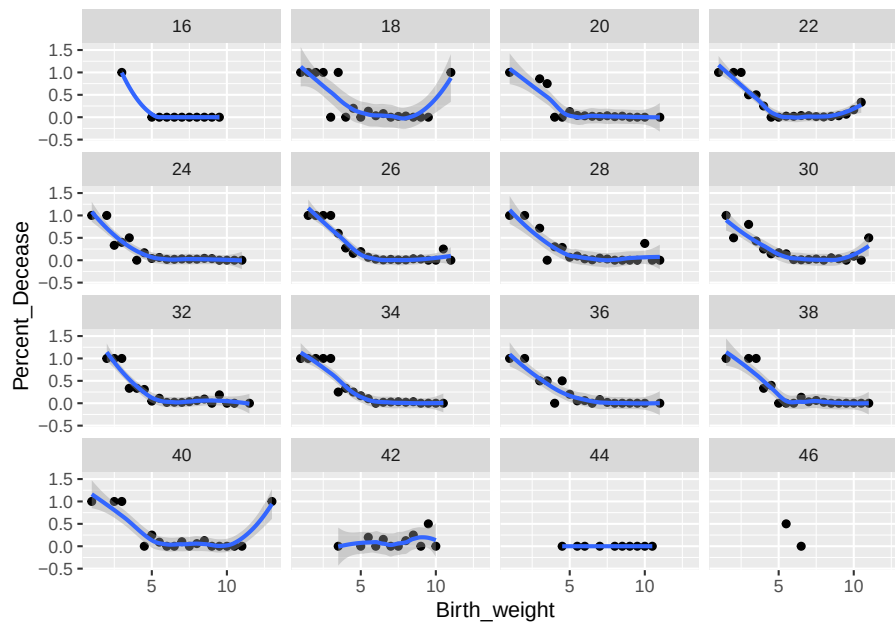
```
Newborn_Karn_Penrose %>% dplyr::select(Count_Non_survivors, Count_Total_Birth,
Birth_weight) %>% arrange(Count_Total_Birth, Count_Non_survivors)
%>% mutate(porc_sobrevivieron = Count_Non_survivors / Count_Total_Birth) %>%
ggplot(aes(x= Birth_weight, y=porc_sobrevivieron))+ geom_point()+
geom_smooth()
```

```
Newborn_Karn_Penrose %>%
  mutate(Percent_Decease = Count_Non_survivors/Count_Total_Birth) %>%
  ggplot(aes(Birth_weight, Percent_Decease)) +
  geom_point() +
  geom_smooth()
```

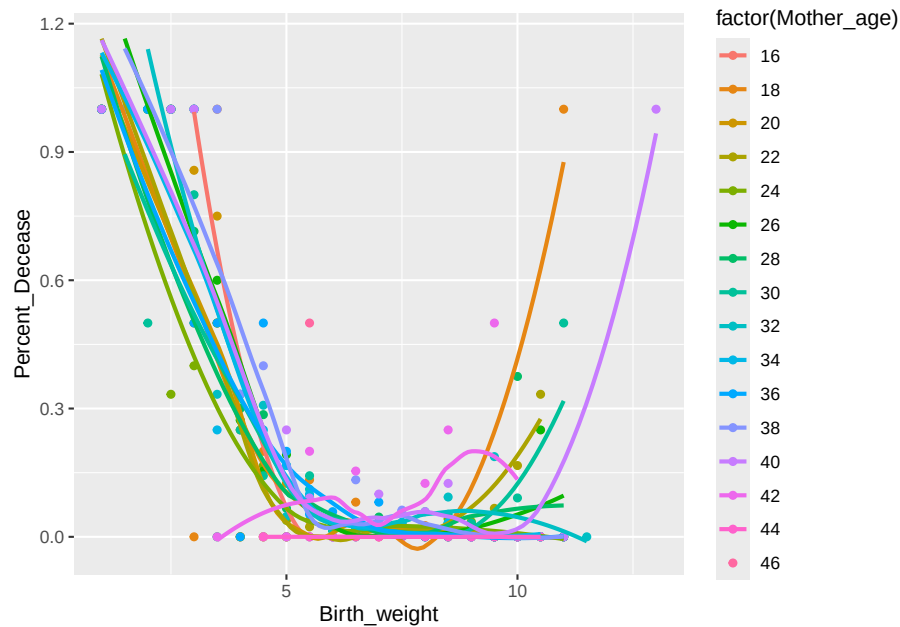


```
Newborn_Karn_Penrose %>%
  mutate(Percent_Decease = Count_Non_survivors/Count_Total_Birth) %>%
```

```
ggplot(aes(Birth_weight, Percent_Decease)) +
  geom_point() +
  geom_smooth()+
  facet_wrap(~Mother_age)
```



```
Newborn_Karn_Penrose %>%
  mutate(Percent_Decease = Count_Non_survivors/Count_Total_Birth) %>%
  ggplot(aes(Birth_weight, Percent_Decease, color=Mother_age)) +
  geom_point(aes(color=factor(Mother_age))) +
  geom_smooth(se=FALSE, aes(color=factor(Mother_age)))
```



18.4 Comparar con base R

1. Ejercicios:

Hacer los ejercicios en la sección 11.2.2 del libro en español

18.5 Segmentar un vector

`parse_*()` (readr) — convertir texto al tipo correcto.

- **Qué hacen:** `parse_integer()`, `parse_double()`, `parse_number()`, `parse_logical()`, `parse_date()`, `parse_factor()` convierten cadenas de texto a números, fechas, etc.
- `parse_number("$123.45")`: extrae el número ignorando símbolos como \$ o %.
- `locale = locale(...)`: controla la codificación, el separador de miles y el formato de fecha.
- `str(parse_logical())`

```

x= c("TRUE", "FALSE", "NA")
x

## [1] "TRUE" "FALSE" "NA"
str(parse_logical(x))

## logi [1:3] TRUE FALSE NA
y= c("Carlos", "Kelvin")
y

## [1] "Carlos" "Kelvin"
str(parse_logical(y))

## logi [1:2] NA NA
## - attr(*, "problems")= tibble [2 x 4] (S3: tbl_df/tbl/data.frame)
## ..$ row      : int [1:2] 1 2
## ..$ col      : int [1:2] NA NA
## ..$ expected: chr [1:2] "1/O/T/F/TRUE/FALSE" "1/O/T/F/TRUE/FALSE"
## ..$ actual   : chr [1:2] "Carlos" "Kelvin"
str(parse_logical(c("TRUE", "FALSE", "NA"))))

## logi [1:3] TRUE FALSE NA
z=c("1", "2", "3")
z

## [1] "1" "2" "3"
mean(z)

## [1] NA
z1=str(parse_integer(z))

## int [1:3] 1 2 3
str(parse_integer(c("1", "2", "3"))))

## int [1:3] 1 2 3
str(parse_date(c("2010-01-01", "1979-10-14"))))

## Date[1:2], format: "2010-01-01" "1979-10-14"
m=c("$4000", "$500" )
parse_number(m)

## [1] 4000 500

```

```

parse_number("$100")

## [1] 100
#> [1] 100
parse_number("20%")

## [1] 20
#> [1] 20
parse_number("It cost $123.45")

## [1] 123.45
#> [1] 123.45

```

- `str(parse_integer())`
 - `str(parse_date())`
-

18.6 Números

- `parse_double()`
 - `parse_number()`
 - `parse_number(locale=locale(grouping_mark = "."))`
-

18.7 Cadena de texto

- `parse_character()`
- `charToRaw()`

```

charToRaw("Ramn")

## [1] 52 61 6d 6e

x1 <- "El Ni\xf1o was particularly bad this year"
x2 <- "\x82\xb1\x82\xf1\x82\xc9\x82\xbf\x82\xcd"
x3 <- "The boy was particularly bad this year"

library(readr)
parse_character(x1, locale = locale(encoding = "Latin1"))

## [1] "El Niño was particularly bad this year"
#> [1] "El Niño was particularly bad this year"
parse_character(x2, locale = locale(encoding = "Shift-JIS"))

```

```
## [1] " "
```

```
parse_character(x2, locale = locale(encoding = "Shift-JIS"))
```

```
## [1] " "
```

```
#> [1] " "
```

- `guess_encoding(charToRaw())`

```
guess_encoding(charToRaw(x3))
```

```
## # A tibble: 1 x 2
##   encoding confidence
##   <chr>           <dbl>
## 1 ASCII           1
```

18.8 Factores

- `parse_factor()`

18.9 Fechas, fechas-horas, horas

- `parse_datetime()`
- `library(hms)`
- `parse_time()`

2. Ejercicios:

Hacer los ejercicios en la sección 11.3.5 del libro en español

18.10 Segmentar un archivo

- `guess_parser`

18.11 Escribir un Archivo

Después de descargar datos de internet, guárdalos con `write_csv()` para trabajar sin depender de la conexión en la próxima sesión.

- `write_csv()`
- `write_tsv()`

18.12 Otros tipos de datos y paquetes

- `library(haven)` # SPSS, Stata y SAS

- `readxl()` lee archivo Excel en formato .xls, .xlsx
- `BDI` lee archivo RMySQL y otros

Chapter 19

Datos Ordenados

[1] "2026-08-01"

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/tidy-data.html>

Español: <https://es.r4ds.hadley.nz/12-tidy.html>

19.1 Temas: Datos Ordenados

Datos ordenados (*tidy data*): cada variable es una columna, cada observación es una fila y cada valor ocupa su propia celda. Con este formato el tidyverse trabaja de forma más natural.

El concepto de *tidy data* (datos ordenados) fue formalizado por **Hadley Wickham** en un influyente artículo de 2014. La idea —una variable por columna, una observación por fila— unificó la forma de preparar datos para el análisis.

19.1.1 El paquete tidyr

Las funciones `pivot_longer()` y `pivot_wider()` de **tidyr** son las herramientas principales para pasar los datos entre el formato ancho y el formato largo.

`pivot_longer()` y `pivot_wider()` (tidyr) — cambiar la forma de la tabla.

- **`pivot_longer()`**: pasa de **ancho a largo** (junta varias columnas en pares nombre-valor).
- **`pivot_wider()`**: pasa de **largo a ancho** (reparte una columna en varias).
- **Uso**: dejar los datos *ordenados* para poder graficar y resumir con facilidad.

Ver vignettes

```
library(tidyverse)
library(datos)
```

tabla1

```
## # A tibble: 6 x 4
##   pais      anio  casos  poblacion
##   <chr>    <dbl> <dbl>    <dbl>
## 1 Afganistán 1999    745  19987071
## 2 Afganistán 2000   2666  20595360
## 3 Brasil    1999  37737  172006362
## 4 Brasil    2000  80488  174504898
## 5 China     1999 212258 1272915272
## 6 China     2000 213766 1280428583
```

tabla2

```
## # A tibble: 12 x 4
##   pais      anio tipo      cuenta
##   <chr>    <dbl> <chr>    <dbl>
## 1 Afganistán 1999 casos      745
## 2 Afganistán 1999 población 19987071
## 3 Afganistán 2000 casos      2666
## 4 Afganistán 2000 población 20595360
## 5 Brasil    1999 casos      37737
## 6 Brasil    1999 población 172006362
## 7 Brasil    2000 casos      80488
## 8 Brasil    2000 población 174504898
## 9 China     1999 casos      212258
## 10 China    1999 población 1272915272
## 11 China    2000 casos      213766
## 12 China    2000 población 1280428583
```

tabla3

```
## # A tibble: 6 x 3
##   pais      anio tasa
##   <chr>    <dbl> <chr>
## 1 Afganistán 1999 745/19987071
## 2 Afganistán 2000 2666/20595360
## 3 Brasil    1999 37737/172006362
## 4 Brasil    2000 80488/174504898
## 5 China     1999 212258/1272915272
## 6 China     2000 213766/1280428583
```

tabla4a

```
## # A tibble: 3 x 3
```

```
##   pais      `1999` `2000`  
##   <chr>      <dbl> <dbl>  
## 1 Afganistán    745   2666  
## 2 Brasil        37737 80488  
## 3 China         212258 213766
```

tabla4b

```
## # A tibble: 3 x 3  
##   pais      `1999`      `2000`  
##   <chr>      <dbl>      <dbl>  
## 1 Afganistán 19987071 20595360  
## 2 Brasil    172006362 174504898  
## 3 China     1272915272 1280428583
```


Chapter 20

Datos Relacionados

```
## [1] "2026-08-01"
```

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/relational-data.html>

Español: <https://es.r4ds.hadley.nz/13-relational-data.html>

```
library(tidyverse)
library(datos)
```

```
vuelos # data set #1
```

```
## # A tibble: 336,776 x 19
##   anio  mes  dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>         <int>         <dbl>
## 1  2013     1     1           517           515           2
## 2  2013     1     1           533           529           4
## 3  2013     1     1           542           540           2
## 4  2013     1     1           544           545          -1
## 5  2013     1     1           554           600          -6
## 6  2013     1     1           554           558          -4
## 7  2013     1     1           555           600          -5
## 8  2013     1     1           557           600          -3
## 9  2013     1     1           557           600          -3
## 10 2013     1     1           558           600          -2
## # i 336,766 more rows
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
```

```
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

```
names(vuelos)
```

```
## [1] "anio"           "mes"            "dia"
## [4] "horario_salida" "salida_programada" "atraso_salida"
## [7] "horario_llegada" "llegada_programada" "atraso_llegada"
## [10] "aerolinea"      "vuelo"          "codigoCola"
## [13] "origen"         "destino"        "tiempo_vuelo"
## [16] "distancia"      "hora"           "minuto"
## [19] "fecha_hora"
```

```
aerolineas # data set #2
```

```
## # A tibble: 16 x 2
##   aerolinea nombre
##   <chr>      <chr>
## 1 9E        Endeavor Air Inc.
## 2 AA        American Airlines Inc.
## 3 AS        Alaska Airlines Inc.
## 4 B6        JetBlue Airways
## 5 DL        Delta Air Lines Inc.
## 6 EV        ExpressJet Airlines Inc.
## 7 F9        Frontier Airlines Inc.
## 8 FL        AirTran Airways Corporation
## 9 HA        Hawaiian Airlines Inc.
## 10 MQ       Envoy Air
## 11 OO       SkyWest Airlines Inc.
## 12 UA       United Air Lines Inc.
## 13 US       US Airways Inc.
## 14 VX       Virgin America
## 15 WN       Southwest Airlines Co.
## 16 YV       Mesa Airlines Inc.
```

```
names(aerolineas)
```

```
## [1] "aerolinea" "nombre"
```

```
aeropuertos
```

```
## # A tibble: 1,458 x 8
##   codigo_aeropuerto nombre  latitud longitud altura zona_horaria horario_verano
##   <chr>              <chr>    <dbl>    <dbl>    <dbl>    <dbl> <chr>
## 1 04G                Lansdo~  41.1    -80.6    1044      -5 A
## 2 06A                Moton ~  32.5    -85.7    264      -6 A
## 3 06C                Schaum~  42.0    -88.1    801      -6 A
## 4 06N                Randal~  41.4    -74.4    523      -5 A
## 5 09J                Jekyll~  31.1    -81.4     11      -5 A
```

```
## 6 OA9          Elizab~    36.4   -82.2   1593          -5 A
## 7 OG6          Willia~    41.5   -84.5    730          -5 A
## 8 OG7          Finger~    42.9   -76.8    492          -5 A
## 9 OP2          Shoest~    39.8   -76.6   1000          -5 U
## 10 OS9         Jeffer~    48.1  -123.    108          -8 A
## # i 1,448 more rows
## # i 1 more variable: zona_horaria_iana <chr>
```

```
names(aeropuertos)
```

```
## [1] "codigo_aeropuerto" "nombre"          "latitud"
## [4] "longitud"          "altura"          "zona_horaria"
## [7] "horario_verano"    "zona_horaria_iana"
```

```
aviones
```

```
## # A tibble: 3,322 x 9
##   codigoCola anio tipo fabricante modelo motores asientos velocidad
##   <chr>      <int> <chr>      <chr>      <chr>      <int>      <int>      <int>
## 1 N10156     2004 Ala fija mult~ EMBRAER    EMB-1~      2         55         NA
## 2 N102UW     1998 Ala fija mult~ AIRBUS IN~ A320--      2        182         NA
## 3 N103US     1999 Ala fija mult~ AIRBUS IN~ A320--      2        182         NA
## 4 N104UW     1999 Ala fija mult~ AIRBUS IN~ A320--      2        182         NA
## 5 N10575     2002 Ala fija mult~ EMBRAER    EMB-1~      2         55         NA
## 6 N105UW     1999 Ala fija mult~ AIRBUS IN~ A320--      2        182         NA
## 7 N107US     1999 Ala fija mult~ AIRBUS IN~ A320--      2        182         NA
## 8 N108UW     1999 Ala fija mult~ AIRBUS IN~ A320--      2        182         NA
## 9 N109UW     1999 Ala fija mult~ AIRBUS IN~ A320--      2        182         NA
## 10 N110UW    1999 Ala fija mult~ AIRBUS IN~ A320--      2        182         NA
## # i 3,312 more rows
## # i 1 more variable: tipo_motor <chr>
```

```
clima
```

```
## # A tibble: 26,115 x 15
##   origen anio mes dia hora temperatura punto_rocio humedad
##   <chr>  <int> <int> <int> <int>      <dbl>      <dbl>      <dbl>
## 1 EWR    2013     1     1     1      39.0      26.1      59.4
## 2 EWR    2013     1     1     2      39.0      27.0      61.6
## 3 EWR    2013     1     1     3      39.0      28.0      64.4
## 4 EWR    2013     1     1     4      39.9      28.0      62.2
## 5 EWR    2013     1     1     5      39.0      28.0      64.4
## 6 EWR    2013     1     1     6      37.9      28.0      67.2
## 7 EWR    2013     1     1     7      39.0      28.0      64.4
## 8 EWR    2013     1     1     8      39.9      28.0      62.2
## 9 EWR    2013     1     1     9      39.9      28.0      62.2
## 10 EWR    2013     1     1    10      41        28.0      59.6
## # i 26,105 more rows
```

```
## # i 7 more variables: direccion_viento <dbl>, velocidad_viento <dbl>,
## #   velocidad_rafaga <dbl>, precipitacion <dbl>, presion <dbl>,
## #   visibilidad <dbl>, fecha_hora <dtm>
```

```
vuelos
```

```
## # A tibble: 336,776 x 19
##   anio   mes   dia horario_salida salida_programada atraso_salida
##   <int> <int> <int>         <int>           <int>           <dbl>
## 1  2013     1     1           517             515             2
## 2  2013     1     1           533             529             4
## 3  2013     1     1           542             540             2
## 4  2013     1     1           544             545            -1
## 5  2013     1     1           554             600            -6
## 6  2013     1     1           554             558            -4
## 7  2013     1     1           555             600            -5
## 8  2013     1     1           557             600            -3
## 9  2013     1     1           557             600            -3
## 10 2013     1     1           558             600            -2
## # i 336,766 more rows
## # i 13 more variables: horario_llegada <int>, llegada_programada <int>,
## #   atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
## #   origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
## #   hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

```
aviones
```

```
## # A tibble: 3,322 x 9
##   codigoCola anio tipo fabricante modelo motores asientos velocidad
##   <chr>      <int> <chr>      <chr>      <chr>      <int>      <int>      <int>
## 1 N10156    2004 Ala fija mult~ EMBRAER    EMB-1~      2         55        NA
## 2 N102UW    1998 Ala fija mult~ AIRBUS    IN~ A320~      2        182        NA
## 3 N103US    1999 Ala fija mult~ AIRBUS    IN~ A320~      2        182        NA
## 4 N104UW    1999 Ala fija mult~ AIRBUS    IN~ A320~      2        182        NA
## 5 N10575    2002 Ala fija mult~ EMBRAER    EMB-1~      2         55        NA
## 6 N105UW    1999 Ala fija mult~ AIRBUS    IN~ A320~      2        182        NA
## 7 N107US    1999 Ala fija mult~ AIRBUS    IN~ A320~      2        182        NA
## 8 N108UW    1999 Ala fija mult~ AIRBUS    IN~ A320~      2        182        NA
## 9 N109UW    1999 Ala fija mult~ AIRBUS    IN~ A320~      2        182        NA
## 10 N110UW   1999 Ala fija mult~ AIRBUS    IN~ A320~      2        182        NA
## # i 3,312 more rows
## # i 1 more variable: tipo_motor <chr>
```

```
aviones %>%
  count(codigoCola) %>%
  filter(n > 1)
```

```
## # A tibble: 0 x 2
```



```
## # i 2 variables: codigoCola <chr>, n <int>
```

20.1 Temas: Datos Relacionados

- mutating joins
- filtering joins
- operations

20.1.1 Los datos de vuelos

1. Ejercicios:

Hacer los ejercicios en la sección 13.2.1 del libro en español

20.2 Claves

2. Ejercicios:

Hacer los ejercicios en la sección 13.3.1 del libro en español

Identify the keys in the following datasets

Lahman::Batting, babynames::babynames nasaweather::atmos fueleconomy::vehicles ggplot2::diamonds

```
head(Lahman::Batting)
```

```
##      playerID yearID stint teamID lgID  G AB R H X2B X3B HR RBI SB CS BB SO IBB
## 1 aardsda01   2004     1   SFN   NL 11  0 0 0  0  0  0  0  0  0  0  0  0
## 2 aardsda01   2006     1   CHN   NL 45  2 0 0  0  0  0  0  0  0  0  0  0
## 3 aardsda01   2007     1   CHA   AL 25  0 0 0  0  0  0  0  0  0  0  0  0
## 4 aardsda01   2008     1   BOS   AL 47  1 0 0  0  0  0  0  0  0  0  1  0
## 5 aardsda01   2009     1   SEA   AL 73  0 0 0  0  0  0  0  0  0  0  0  0
## 6 aardsda01   2010     1   SEA   AL 53  0 0 0  0  0  0  0  0  0  0  0  0
##      HBP SH SF GIDP
## 1     0  0  0    0
## 2     0  1  0    0
## 3     0  0  0    0
## 4     0  0  0    0
## 5     0  0  0    0
## 6     0  0  0    0
```

```
head(datos::bateadores)
```

```
##      id_jugador id_anio orden_equipos id_equipo id_liga juegos al_bate carreras
## 1 aardsda01   2004           1      SFN      NL     11      0      0
## 2 aardsda01   2006           1      CHN      NL     45      2      0
```

```
## 3 aardsda01 2007 1 CHA AL 25 0 0
## 4 aardsda01 2008 1 BOS AL 47 1 0
## 5 aardsda01 2009 1 SEA AL 73 0 0
## 6 aardsda01 2010 1 SEA AL 53 0 0
## golpes dobles triples cuadrangulares carreras_empujadas bases_robadas
## 1 0 0 0 0 0 0
## 2 0 0 0 0 0 0
## 3 0 0 0 0 0 0
## 4 0 0 0 0 0 0
## 5 0 0 0 0 0 0
## 6 0 0 0 0 0 0
## atrapado_robando base_bolas ponches base_intencional golpeado
## 1 0 0 0 0 0
## 2 0 0 0 0 0
## 3 0 0 0 0 0
## 4 0 0 1 0 0
## 5 0 0 0 0 0
## 6 0 0 0 0 0
## toque_sacrificio elavado_sacrificio doble_matanza
## 1 0 0 0
## 2 1 0 0
## 3 0 0 0
## 4 0 0 0
## 5 0 0 0
## 6 0 0 0
```

```
Lahman::Batting %>%
  count(playerID) %>%
  filter(n>1) |>
  head()
```

```
## playerID n
## 1 aardsda01 9
## 2 aaronha01 23
## 3 aaronto01 7
## 4 aasedo01 13
## 5 abadan01 3
## 6 abadfe01 12
```

```
bateadores %>%
  count(id_jugador,id_anio,orden_equipos) %>%
  filter(n > 1) |>
  head()
```

```
## [1] id_jugador id_anio orden_equipos n
## <0 rows> (or 0-length row.names)
```

```

Lahman::Batting %>%
  count(playerID, yearID, stint) %>%
  filter(n>1) |>
  head()

## [1] playerID yearID  stint    n
## <0 rows> (or 0-length row.names)

#babynames::babynames

babynames::babynames %>%
  count(year, name, sex) %>%
  filter(n>1)

## # A tibble: 0 x 4
## # i 4 variables: year <dbl>, name <chr>, sex <chr>, n <int>

¿Cuál fue el primer año que su nombre aparece en la base de datos?

babynames::babynames %>%
  filter(name == "Abimelys")

## # A tibble: 0 x 5
## # i 5 variables: year <dbl>, sex <chr>, name <chr>, n <int>, prop <dbl>

max(babynames::babynames$year)

## [1] 2017

nasaweather::atmos

## # A tibble: 41,472 x 11
##   lat long year month surf temp pressure ozone cloudlow cloudmid
##   <dbl> <dbl> <int> <int>   <dbl> <dbl>   <dbl> <dbl>   <dbl>   <dbl>
## 1 36.2 -114. 1995     1    273. 272.    835  304     7.5    34.5
## 2 33.7 -114. 1995     1    280. 282.    940  304    11.5    32.5
## 3 31.2 -114. 1995     1    285. 285.    960  298    16.5     26
## 4 28.7 -114. 1995     1    289. 291.    990  276    20.5    14.5
## 5 26.2 -114. 1995     1    292. 293.   1000  274     26    10.5
## 6 23.7 -114. 1995     1    294. 294.   1000  264     30     9.5
## 7 21.2 -114. 1995     1    295. 295.   1000  258    29.5     11
## 8 18.7 -114. 1995     1    298. 297.   1000  252    26.5    17.5
## 9 16.2 -114. 1995     1    300. 298.   1000  250    27.5    18.5
## 10 13.7 -114. 1995     1    300. 299.   1000  250     26    16.5
## # i 41,462 more rows
## # i 1 more variable: cloudhigh <dbl>

nasaweather::atmos %>%
  count(lat, long, year, month) %>%
  filter(n>1)

```

```
## # A tibble: 0 x 5
## # i 5 variables: lat <dbl>, long <dbl>, year <int>, month <int>, n <int>
fueleconomy::vehicles

## # A tibble: 33,442 x 12
##       id make  model      year class trans drive  cyl displ fuel   hwy   cty
##   <dbl> <chr> <chr>    <dbl> <chr> <chr> <chr> <dbl> <dbl> <chr> <dbl> <dbl>
## 1 13309 Acura 2.2CL/3.0CL 1997 Subc~ Auto~ Fron~    4   2.2 Regu~   26   20
## 2 13310 Acura 2.2CL/3.0CL 1997 Subc~ Manu~ Fron~    4   2.2 Regu~   28   22
## 3 13311 Acura 2.2CL/3.0CL 1997 Subc~ Auto~ Fron~    6   3   Regu~   26   18
## 4 14038 Acura 2.3CL/3.0CL 1998 Subc~ Auto~ Fron~    4   2.3 Regu~   27   19
## 5 14039 Acura 2.3CL/3.0CL 1998 Subc~ Manu~ Fron~    4   2.3 Regu~   29   21
## 6 14040 Acura 2.3CL/3.0CL 1998 Subc~ Auto~ Fron~    6   3   Regu~   26   17
## 7 14834 Acura 2.3CL/3.0CL 1999 Subc~ Auto~ Fron~    4   2.3 Regu~   27   20
## 8 14835 Acura 2.3CL/3.0CL 1999 Subc~ Manu~ Fron~    4   2.3 Regu~   29   21
## 9 14836 Acura 2.3CL/3.0CL 1999 Subc~ Auto~ Fron~    6   3   Regu~   26   17
## 10 11789 Acura 2.5TL      1995 Comp~ Auto~ Fron~    5   2.5 Prem~   23   18
## # i 33,432 more rows

fueleconomy::vehicles %>%
  count(id) %>%
  filter(n>1)

## # A tibble: 0 x 2
## # i 2 variables: id <dbl>, n <int>
ggplot2::diamonds

## # A tibble: 53,940 x 10
##   carat cut      color clarity depth table price    x    y    z
##   <dbl> <ord>    <ord> <ord>    <dbl> <dbl> <int> <dbl> <dbl> <dbl>
## 1 0.23 Ideal    E      SI2     61.5    55   326  3.95  3.98  2.43
## 2 0.21 Premium  E      SI1     59.8    61   326  3.89  3.84  2.31
## 3 0.23 Good     E      VS1     56.9    65   327  4.05  4.07  2.31
## 4 0.29 Premium  I      VS2     62.4    58   334  4.2   4.23  2.63
## 5 0.31 Good     J      SI2     63.3    58   335  4.34  4.35  2.75
## 6 0.24 Very Good J      VVS2     62.8    57   336  3.94  3.96  2.48
## 7 0.24 Very Good I      VVS1     62.3    57   336  3.95  3.98  2.47
## 8 0.26 Very Good H      SI1     61.9    55   337  4.07  4.11  2.53
## 9 0.22 Fair     E      VS2     65.1    61   337  3.87  3.78  2.49
## 10 0.23 Very Good H      VS1     59.4    61   338  4     4.05  2.39
## # i 53,930 more rows

ggplot2::diamonds %>%
  count(carat, cut, color, clarity, depth, price, x, y, z) %>%
  filter(n>1)

## # A tibble: 283 x 10
```

```
##      carat cut      color clarity depth price      x      y      z      n
##      <dbl> <ord>      <ord> <ord>  <dbl> <int> <dbl> <dbl> <dbl> <int>
##  1  0.23 Very Good E      VVS2    60.9   530  3.96  3.99  2.42    2
##  2  0.24 Very Good E      VVS1    61.7   485  3.95  3.99  2.45    2
##  3  0.24 Very Good G      VVS2    62     449  4     4.03  2.49    2
##  4  0.26 Ideal      G      SI1    62     394  4.08  4.11  2.54    2
##  5  0.3   Good      J      VS1    63.4   394  4.23  4.26  2.69    2
##  6  0.3   Very Good D      SI1    62.5   552  4.26  4.28  2.67    2
##  7  0.3   Very Good E      SI1    62.9   526  4.25  4.3   2.69    2
##  8  0.3   Very Good E      VVS2    61.4   789  4.25  4.28  2.62    2
##  9  0.3   Very Good G      VS2    63     526  4.29  4.31  2.71    2
## 10  0.3   Very Good H      SI1    62.9   421  4.28  4.31  2.7     2
## # i 273 more rows
```

Draw a diagram illustrating the connections between the Batting, People, and Salaries tables in the Lahman package. Draw another diagram that shows the relationship between People, Managers, AwardsManagers.

Dibuja un diagrama que ilustre las conexiones entre las tablas bateadores, personas y salarios incluidas en el paquete datos. Dibuja otro diagrama que muestre la relación entre personas, dirigentes y premios_dirigentes.

```
names(Lahman::Batting)
```

```
## [1] "playerID" "yearID" "stint" "teamID" "lgID" "G"
## [7] "AB" "R" "H" "X2B" "X3B" "HR"
## [13] "RBI" "SB" "CS" "BB" "SO" "IBB"
## [19] "HBP" "SH" "SF" "GIDP"
```

```
names(Lahman::Salaries)
```

```
## [1] "yearID" "teamID" "lgID" "playerID" "salary"
```

```
names(Lahman::People)
```

```
## [1] "playerID" "birthYear" "birthMonth" "birthDay" "birthCity"
## [6] "birthCountry" "birthState" "deathYear" "deathMonth" "deathDay"
## [11] "deathCountry" "deathState" "deathCity" "nameFirst" "nameLast"
## [16] "nameGiven" "weight" "height" "bats" "throws"
## [21] "debut" "bbrefID" "finalGame" "retroID" "deathDate"
## [26] "birthDate"
```

20.3 Uniones de transformaciones

- mutating join

`left_join()`, `inner_join()`, `full_join()`, `right_join()` (dplyr) — combinar tablas.

- **Qué hacen:** unen dos tablas emparejando filas por una columna **clave** (by =).
- **left_join(x, y):** conserva todas las filas de x; **inner_join** solo las coincidencias; **full_join** todas las de ambas.
- **De filtro:** `semi_join()` y `anti_join()` no añaden columnas, solo filtran filas de x.

```
vuelos2 <- vuelos %>%
  dplyr::select(anio, dia, hora, origen, destino, codigoCola, aerolinea)
vuelos2
```

```
## # A tibble: 336,776 x 7
##   anio   dia hora origen destino codigoCola aerolinea
##   <int> <int> <dbl> <chr>  <chr>    <chr>      <chr>
## 1 2013     1     5 EWR   IAH     N14228    UA
## 2 2013     1     5 LGA   IAH     N24211    UA
## 3 2013     1     5 JFK   MIA     N619AA    AA
## 4 2013     1     5 JFK   BQN     N804JB    B6
## 5 2013     1     6 LGA   ATL     N668DN    DL
## 6 2013     1     5 EWR   ORD     N39463    UA
## 7 2013     1     6 EWR   FLL     N516JB    B6
## 8 2013     1     6 LGA   IAD     N829AS    EV
## 9 2013     1     6 JFK   MCO     N593JB    B6
## 10 2013     1     6 LGA   ORD     N3ALAA    AA
## # i 336,766 more rows
```

```
names(aerolineas)
```

```
## [1] "aerolinea" "nombre"
```

```
aerolineas
```

```
## # A tibble: 16 x 2
##   aerolinea nombre
##   <chr>      <chr>
## 1 9E        Endeavor Air Inc.
## 2 AA        American Airlines Inc.
## 3 AS        Alaska Airlines Inc.
## 4 B6        JetBlue Airways
## 5 DL        Delta Air Lines Inc.
## 6 EV        ExpressJet Airlines Inc.
## 7 F9        Frontier Airlines Inc.
## 8 FL        AirTran Airways Corporation
## 9 HA        Hawaiian Airlines Inc.
## 10 MQ       Envoy Air
## 11 00       SkyWest Airlines Inc.
```

```
## 12 UA      United Air Lines Inc.
## 13 US      US Airways Inc.
## 14 VX      Virgin America
## 15 WN      Southwest Airlines Co.
## 16 YV      Mesa Airlines Inc.
```

```
vuelos2 %>%
  dplyr::select(-origen, -destino) %>%
  left_join(aerolineas, by = "aerolinea")
```

```
## # A tibble: 336,776 x 6
##   anio dia hora codigoCola aerolinea nombre
##   <int> <int> <dbl> <chr>      <chr>      <chr>
## 1 2013 1 5 N14228 UA United Air Lines Inc.
## 2 2013 1 5 N24211 UA United Air Lines Inc.
## 3 2013 1 5 N619AA AA American Airlines Inc.
## 4 2013 1 5 N804JB B6 JetBlue Airways
## 5 2013 1 6 N668DN DL Delta Air Lines Inc.
## 6 2013 1 5 N39463 UA United Air Lines Inc.
## 7 2013 1 6 N516JB B6 JetBlue Airways
## 8 2013 1 6 N829AS EV ExpressJet Airlines Inc.
## 9 2013 1 6 N593JB B6 JetBlue Airways
## 10 2013 1 6 N3ALAA AA American Airlines Inc.
## # i 336,766 more rows
```

```
x <- tribble(
  ~key, ~val_x,
  1, "x1",
  2, "x2",
  3, "x3"
)
y <- tribble(
  ~key, ~val_y,
  1, "y1",
  2, "y2",
  4, "y3"
)
x
```

```
## # A tibble: 3 x 2
##   key val_x
##   <dbl> <chr>
## 1 1 x1
## 2 2 x2
## 3 3 x3
```

y

```
## # A tibble: 3 x 2
##   key val_y
##   <dbl> <chr>
## 1     1 y1
## 2     2 y2
## 3     4 y3
```

```
x %>%
  inner_join(y, by="key")
```

```
## # A tibble: 2 x 3
##   key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
```

#left_join

```
x %>%
  left_join(y, by="key")
```

```
## # A tibble: 3 x 3
##   key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     3 x3    <NA>
```

#right_join

```
x %>%
  right_join(y, by="key")
```

```
## # A tibble: 3 x 3
##   key val_x val_y
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     4 <NA> y3
```

#full_join

```
x %>%
  full_join(y, by="key")
```

```
## # A tibble: 4 x 3
##   key val_x val_y
```



```
##      <dbl> <chr> <chr>
## 1      1 x1     y1
## 2      2 x2     y2
## 3      3 x3     <NA>
## 4      4 <NA>  y3
```

```
left_join
```

```
x <- tribble(
  ~key, ~nombre_x,
  1, "Juan",
  2, "Maria",
  3, "Orlando"
)
y <- tribble(
  ~key, ~edad_y,
  1, 20,
  2, 21,
  4, 22
)
x
```

```
## # A tibble: 3 x 2
##   key nombre_x
##   <dbl> <chr>
## 1     1 Juan
## 2     2 Maria
## 3     3 Orlando
y
```

```
## # A tibble: 3 x 2
##   key edad_y
##   <dbl> <dbl>
## 1     1    20
## 2     2    21
## 3     4    22
```

```
x %>%
  full_join(y, by="key")
```

```
## # A tibble: 4 x 3
##   key nombre_x edad_y
##   <dbl> <chr>    <dbl>
## 1     1 Juan      20
## 2     2 Maria      21
## 3     3 Orlando    NA
## 4     4 <NA>      22
```

```
x %>%
  inner_join(y, by="key")
```

```
## # A tibble: 2 x 3
##   key nombre_x edad_y
##   <dbl> <chr>    <dbl>
## 1     1 Juan      20
## 2     2 Maria     21
```

```
x %>%
  left_join(y, by="key")
```

```
## # A tibble: 3 x 3
##   key nombre_x edad_y
##   <dbl> <chr>    <dbl>
## 1     1 Juan      20
## 2     2 Maria     21
## 3     3 Orlando   NA
```

```
x %>%
  right_join(y, by="key")
```

```
## # A tibble: 3 x 3
##   key nombre_x edad_y
##   <dbl> <chr>    <dbl>
## 1     1 Juan      20
## 2     2 Maria     21
## 3     4 <NA>      22
```

```
x %>%
  full_join(y, by="key")
```

```
## # A tibble: 4 x 3
##   key nombre_x edad_y
##   <dbl> <chr>    <dbl>
## 1     1 Juan      20
## 2     2 Maria     21
## 3     3 Orlando   NA
## 4     4 <NA>      22
```

```
x %>%
  inner_join(y, by="key")
```

```
## # A tibble: 2 x 3
##   key nombre_x edad_y
##   <dbl> <chr>    <dbl>
## 1     1 Juan      20
## 2     2 Maria     21
```

START HERE!!!!

```
vuelos2 %>%
  dplyr::select(-origen, -destino) %>%
  left_join(aerolineas, by = "aerolinea")
```

```
## # A tibble: 336,776 x 6
##   anio   dia hora codigoCola aerolinea nombre
##   <int> <int> <dbl> <chr>      <chr>      <chr>
## 1 2013     1     5 N14228      UA      United Air Lines Inc.
## 2 2013     1     5 N24211      UA      United Air Lines Inc.
## 3 2013     1     5 N619AA      AA      American Airlines Inc.
## 4 2013     1     5 N804JB      B6      JetBlue Airways
## 5 2013     1     6 N668DN      DL      Delta Air Lines Inc.
## 6 2013     1     5 N39463      UA      United Air Lines Inc.
## 7 2013     1     6 N516JB      B6      JetBlue Airways
## 8 2013     1     6 N829AS      EV      ExpressJet Airlines Inc.
## 9 2013     1     6 N593JB      B6      JetBlue Airways
## 10 2013     1     6 N3ALAA      AA      American Airlines Inc.
## # i 336,766 more rows
```

```
names(vuelos2)
```

```
## [1] "anio"          "dia"           "hora"          "origen"        "destino"
## [6] "codigoCola"   "aerolinea"
```

```
names(clima)
```

```
## [1] "origen"          "anio"          "mes"           "dia"
## [5] "hora"           "temperatura"   "puntoRocio"    "humedad"
## [9] "direccionViento" "velocidadViento" "velocidadRafaga" "precipitacion"
## [13] "presion"        "visibilidad"   "fecha_hora"
```

```
vuelos2 %>%
  left_join(clima)
```

```
## # A tibble: 3,974,448 x 18
##   anio   dia hora origen destino codigoCola aerolinea   mes temperatura
##   <int> <int> <dbl> <chr>  <chr>  <chr>      <chr>    <int>    <dbl>
## 1 2013     1     5 EWR    IAH    N14228      UA         1      39.0
## 2 2013     1     5 EWR    IAH    N14228      UA         2      28.0
## 3 2013     1     5 EWR    IAH    N14228      UA         3      35.1
## 4 2013     1     5 EWR    IAH    N14228      UA         4      45.0
## 5 2013     1     5 EWR    IAH    N14228      UA         5      44.1
## 6 2013     1     5 EWR    IAH    N14228      UA         6      72.0
## 7 2013     1     5 EWR    IAH    N14228      UA         7      75.0
## 8 2013     1     5 EWR    IAH    N14228      UA         8      72.0
## 9 2013     1     5 EWR    IAH    N14228      UA         9      73.9
## 10 2013     1     5 EWR    IAH    N14228      UA        10      53.1
```

```
## # i 3,974,438 more rows
## # i 9 more variables: punto_rocio <dbl>, humedad <dbl>, direccion_viento <dbl>,
## #   velocidad_viento <dbl>, velocidad_rafaga <dbl>, precipitacion <dbl>,
## #   presion <dbl>, visibilidad <dbl>, fecha_hora <dtm>

#> Joining, by = c("anio", "mes", "dia", "hora", "origen")
vuelos2 %>%
left_join(clima, by = c("anio", "dia", "hora", "origen"))
```

```
## # A tibble: 3,974,448 x 18
##   anio dia hora origen destino codigoCola aerolinea mes temperatura
##   <int> <int> <dbl> <chr> <chr> <chr> <chr> <int> <dbl>
## 1 2013 1 5 EWR IAH N14228 UA 1 39.0
## 2 2013 1 5 EWR IAH N14228 UA 2 28.0
## 3 2013 1 5 EWR IAH N14228 UA 3 35.1
## 4 2013 1 5 EWR IAH N14228 UA 4 45.0
## 5 2013 1 5 EWR IAH N14228 UA 5 44.1
## 6 2013 1 5 EWR IAH N14228 UA 6 72.0
## 7 2013 1 5 EWR IAH N14228 UA 7 75.0
## 8 2013 1 5 EWR IAH N14228 UA 8 72.0
## 9 2013 1 5 EWR IAH N14228 UA 9 73.9
## 10 2013 1 5 EWR IAH N14228 UA 10 53.1
## # i 3,974,438 more rows
## # i 9 more variables: punto_rocio <dbl>, humedad <dbl>, direccion_viento <dbl>,
## #   velocidad_viento <dbl>, velocidad_rafaga <dbl>, precipitacion <dbl>,
## #   presion <dbl>, visibilidad <dbl>, fecha_hora <dtm>
```

```
names(vuelos2)
```

```
## [1] "anio" "dia" "hora" "origen" "destino"
## [6] "codigoCola" "aerolinea"
```

```
names(aeropuertos)
```

```
## [1] "codigoAeropuerto" "nombre" "latitud"
## [4] "longitud" "altura" "zonaHoraria"
## [7] "horarioVerano" "zonaHorariaIANA"
```

```
head(vuelos2, n=3)
```

```
## # A tibble: 3 x 7
##   anio dia hora origen destino codigoCola aerolinea
##   <int> <int> <dbl> <chr> <chr> <chr> <chr>
## 1 2013 1 5 EWR IAH N14228 UA
## 2 2013 1 5 LGA IAH N24211 UA
## 3 2013 1 5 JFK MIA N619AA AA
```

```
aeropuertos
```

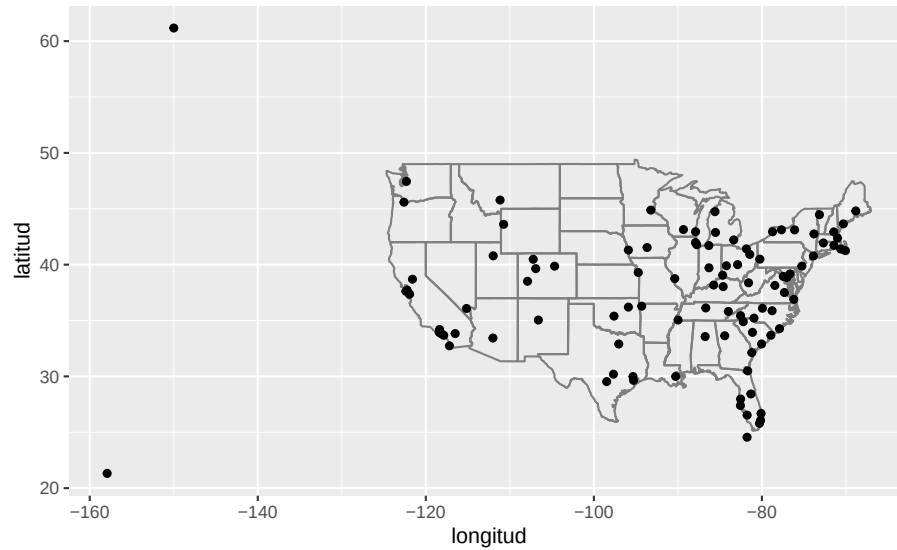
```
## # A tibble: 1,458 x 8
##   codigo_aeropuerto nombre  latitud longitud altura zona_horaria horario_verano
##   <chr>                <chr>    <dbl>    <dbl>    <dbl>    <dbl> <chr>
## 1 04G                 Lansdo~  41.1    -80.6    1044      -5 A
## 2 06A                 Moton ~  32.5    -85.7    264      -6 A
## 3 06C                 Schaum~  42.0    -88.1    801      -6 A
## 4 06N                 Randal~  41.4    -74.4    523      -5 A
## 5 09J                 Jekyll~  31.1    -81.4     11      -5 A
## 6 0A9                 Elizab~  36.4    -82.2   1593      -5 A
## 7 0G6                 Willia~  41.5    -84.5    730      -5 A
## 8 0G7                 Finger~  42.9    -76.8    492      -5 A
## 9 OP2                 Shoest~  39.8    -76.6   1000      -5 U
## 10 OS9                Jeffer~  48.1   -123.    108      -8 A
## # i 1,448 more rows
## # i 1 more variable: zona_horaria_iana <chr>

vuelos2 %>%
  right_join(aeropuertos, c("origen" = "codigo_aeropuerto"))
```

```
## # A tibble: 338,231 x 14
##   anio dia hora origen destino codigoCola aerolinea nombre latitud
##   <int> <int> <dbl> <chr> <chr> <chr> <chr> <chr> <dbl>
## 1 2013 1 5 EWR IAH N14228 UA Newark Libert~ 40.7
## 2 2013 1 5 LGA IAH N24211 UA La Guardia 40.8
## 3 2013 1 5 JFK MIA N619AA AA John F Kenned~ 40.6
## 4 2013 1 5 JFK BQN N804JB B6 John F Kenned~ 40.6
## 5 2013 1 6 LGA ATL N668DN DL La Guardia 40.8
## 6 2013 1 5 EWR ORD N39463 UA Newark Libert~ 40.7
## 7 2013 1 6 EWR FLL N516JB B6 Newark Libert~ 40.7
## 8 2013 1 6 LGA IAD N829AS EV La Guardia 40.8
## 9 2013 1 6 JFK MCO N593JB B6 John F Kenned~ 40.6
## 10 2013 1 6 LGA ORD N3ALAA AA La Guardia 40.8
## # i 338,221 more rows
## # i 5 more variables: longitud <dbl>, altura <dbl>, zona_horaria <dbl>,
## # horario_verano <chr>, zona_horaria_iana <chr>

library(maps)

aeropuertos %>%
  semi_join(vuelos, c("codigo_aeropuerto" = "destino")) %>%
  ggplot(aes(longitud, latitud)) +
  borders("state") +
  geom_point() +
  coord_quickmap()
```



- `mutate()`
 - `select()`
 - `left_join()`
 - `match()`
-

20.4 Entendiendo las uniones

20.4.1 Unión interiores

20.4.2 Unión exteriores

20.4.3 Claves duplicadas

Cuando la clave está duplicada en ambas tablas, la unión genera todas las combinaciones posibles (producto cartesiano) y el resultado tendrá más filas de las esperadas. Verifica con `count(clave) %>% filter(n > 1)` antes de unir.

20.5 Definiendo las columnas clave

3. Ejercicios:

Hacer los ejercicios en la sección 13.4.6 del libro en español

20.6 Otras implementaciones

- `merge()`
-

20.7 Uniones de filtro

Las uniones de filtro (`semi_join()` y `anti_join()`) nunca añaden columnas nuevas: solo conservan o descartan filas de la tabla de la izquierda según coincidan con la de la derecha.

- `semi_join()`
- `anti_join()`

4. Ejercicios:

Hacer los ejercicios en la sección 13.5.1 del libro en español

20.8 Problemas con las uniones

Chapter 21

Factores

Fecha de la última revisión

```
## [1] "2026-08-01"
```

El tema proviene de los siguientes sitios. Aunque los ejemplos son distintos

English: <https://r4ds.had.co.nz/factors.html>

Español: <https://es.r4ds.hadley.nz/15-factors.html>

```
library(tidyverse)
library(datos)
library(forcats)
```

```
month <-c("December" ,"January", "March", "April")
```

```
month
```

```
## [1] "December" "January"  "March"    "April"
```

```
x=c(1:4)
str(month)
```

```
## chr [1:4] "December" "January" "March" "April"
```

```
str(x)
```

```
## int [1:4] 1 2 3 4
```

```
valores <- c(7, 3, 2, 10)
```

Unir las dos listas

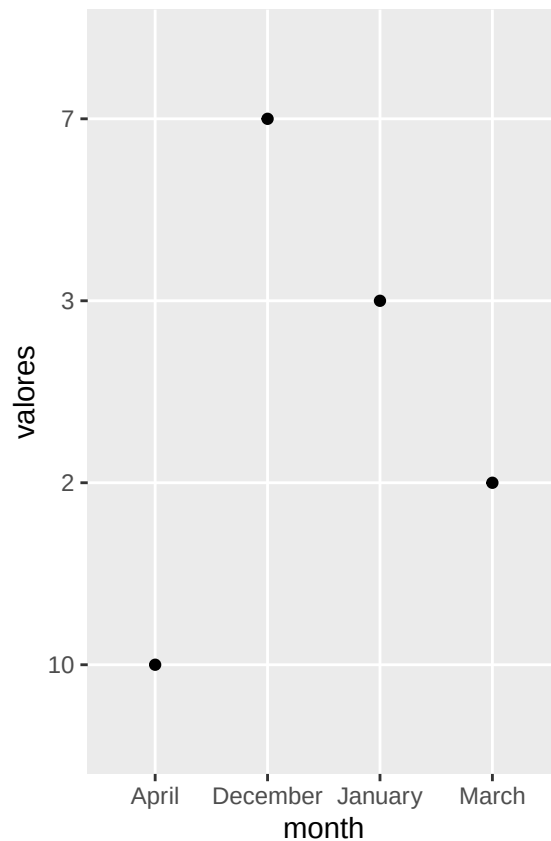
```
df=as.data.frame(cbind(month, valores, x))
df
```

```
##      month valores x
## 1 December      7  1
## 2  January      3  2
## 3   March      2  3
## 4   April     10  4
```

21.1 ¿Qué ocurrió en el gráfico?

```
library(tidyverse)

ggplot(df, aes(month, valores))+
  geom_point()
```



Order months in correct order

use **month.name**

Hay variables ya descrita en R, para facilitar el uso de constantes

- LETTERS: Mayúscula
- letters: minúscula
- month.abb: abreviación de meses en Inglés
- month.name: nombre de meses en Inglés
- pi = el valor de pi

See this website for more details

<https://datacornering.com/how-to-get-the-month-name-from-the-number-in-r/>

LETTERS

```
## [1] "A" "B" "C" "D" "E" "F" "G" "H" "I" "J" "K" "L" "M" "N" "O" "P" "Q" "R" "S"
## [20] "T" "U" "V" "W" "X" "Y" "Z"
```

letters

```
## [1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s"
## [20] "t" "u" "v" "w" "x" "y" "z"
```

month.abb

```
## [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "Dec"
```

month.name

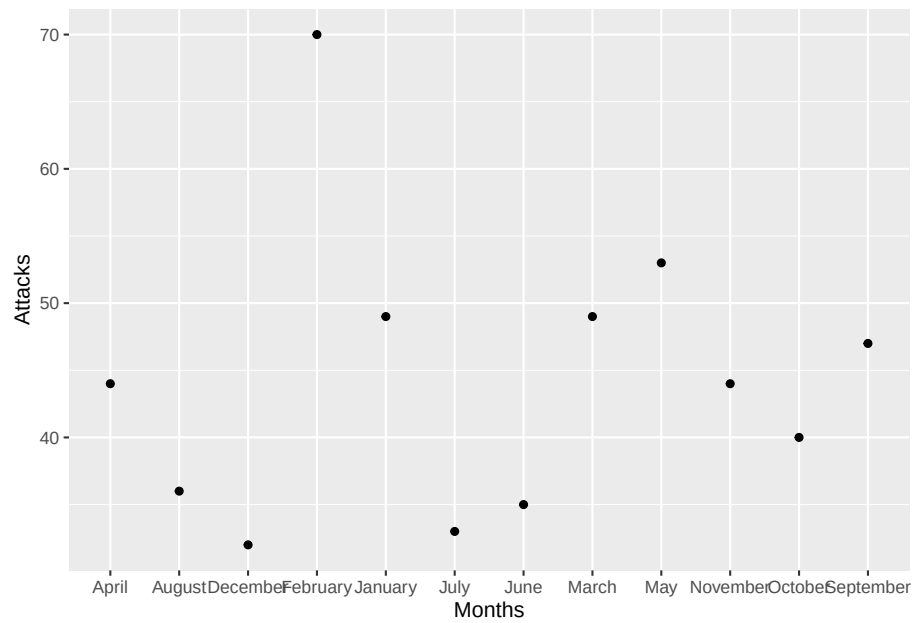
```
## [1] "January" "February" "March" "April" "May" "June"
## [7] "July" "August" "September" "October" "November" "December"
```

pi

```
## [1] 3.141593
```

```
df <- data.frame(Months=c("January", "February", "March", "April", "May", "June", "July", "August",
                          "September", "October", "November", "December"),
                 Attacks=c(49, 70, 49, 44, 53, 35, 33, 36, 47, 40, 44, 32))
```

```
ggplot(df, aes(Months, Attacks))+
  geom_point()
```

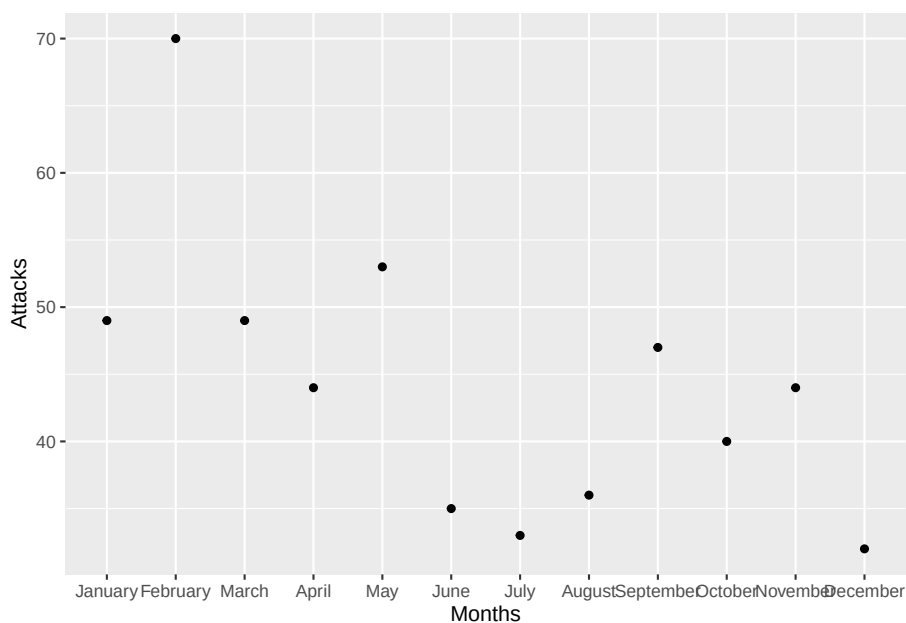


```
### validar el supuesto que tengo información escrita correctamente para cada mes de a
all(df$Months %in% month.name)
```

```
## [1] TRUE
```

```
### convert to a factor, order defined by `month.name`.
df$Months <- factor(df$Months, levels=month.name)

ggplot(data=df, aes(x=Months, y=Attacks)) +
  geom_point()
```



21.1.1 Usando las funciones `factor` y `levels`, estamos asignando los factores a un nivel específico en el orden de la lista

21.1.2 `parse_factor()` is similar a `factor()`, but generates a warning if levels have been specified and some elements of `x` are not found in those levels.

```
x1= c("Dic", "Ene", "Mar", "Dii") # Nota que esta lista tengo un mes que se llama "Dii"
niveles_meses=c("Ene", "Mar", "Dic")

y1 <-factor(x1, levels = niveles_meses)
y1

## [1] Dic  Ene  Mar  <NA>
## Levels: Ene Mar Dic

y1 <- parse_factor(x1, levels = niveles_meses)
y1

## [1] Dic  Ene  Mar  <NA>
## attr("problems")
## # A tibble: 1 x 4
##   row   col expected      actual
```

```
##   <int> <int> <chr>           <chr>
## 1     4     NA value in level set Dii
## Levels: Ene Mar Dic
```

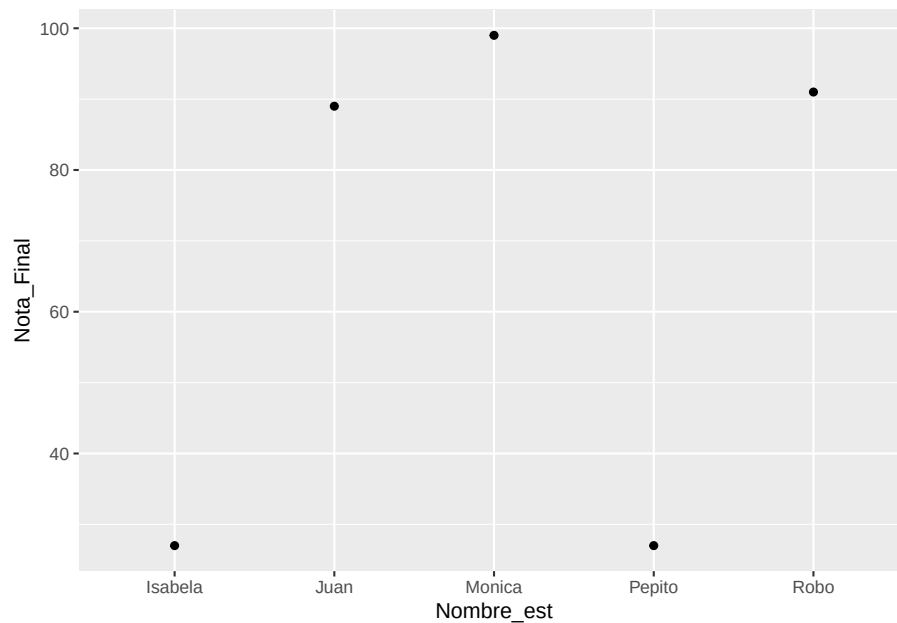
Si quiere que los valores aparezcan en el orden que aparecen en la base de datos use `unique(variable)`

```
df_est=data.frame(Nombre_est=c("Monica", "Isabela", "Juan", "Robo", "Pepito"),
                  Nota_Final=c(99, 27, 89, 91, 27))
```

```
df_est
```

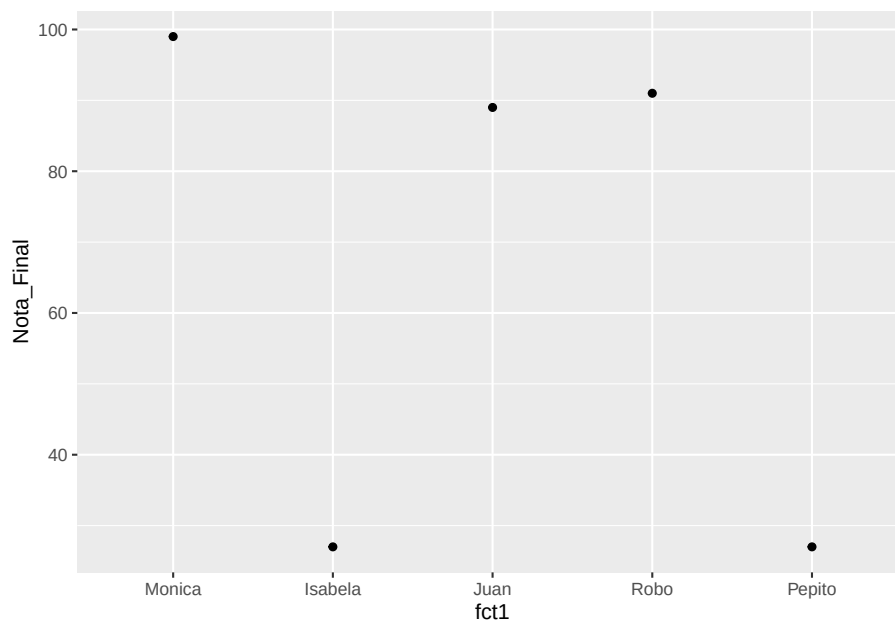
```
##   Nombre_est Nota_Final
## 1    Monica         99
## 2   Isabela         27
## 3     Juan         89
## 4     Robo         91
## 5   Pepito         27
```

```
ggplot(df_est, aes(Nombre_est, Nota_Final))+
  geom_point()
```



```
df_est$fct1=factor(df_est$Nombre_est, levels = unique(df_est$Nombre_est))
```

```
ggplot(df_est, aes(fct1, Nota_Final))+
  geom_point()
```



```
f2 <- x1 %>% factor() %>% fct_inorder()
f2
```

```
## [1] Dic Ene Mar Dii
## Levels: Dic Ene Mar Dii
```

```
levels(f2)
```

```
## [1] "Dic" "Ene" "Mar" "Dii"
```

21.2 Temas:

- library(forcats)

factor() y las funciones **fct_*** (**forcats**) — manejar variables categóricas.

- **factor(x, levels = ...)**: define una variable categórica y el **orden** de sus niveles.
- **fct_reorder(f, y)**: reordena los niveles según otra variable (ideal para gráficos).
- **Otras:** **fct_infreq()** (por frecuencia), **fct_rev()** (invertir), **fct_relevel()** (mover niveles a mano).
- sort()
- factor()
- parse_factor()
- unique()

- `fact_inorder()`
- `levels()`

21.2.1 Datos : Encuesta Social General

- `count()`

1. Ejercicios:

```
library(datos)
encuesta
```

```
## # A tibble: 21,483 x 9
##   anio estado_civil  edad raza  ingreso partido religion denominacion horas_tv
##   <int> <fct>      <int> <fct> <fct> <fct> <fct> <fct>      <int>
## 1 2000 Nunca se ha~ 26 Blan~ 8000 -- Ind, p~ Protest~ Bautistas d~ 12
## 2 2000 Divorciado 48 Blan~ 8000 -- No fue~ Protest~ Bautista, n~ NA
## 3 2000 Viudo 67 Blan~ No apl~ Indepe~ Protest~ No denomina~ 2
## 4 2000 Nunca se ha~ 39 Blan~ No apl~ Ind, p~ Cristia~ No aplica 4
## 5 2000 Divorciado 25 Blan~ No apl~ No fue~ Ninguna No aplica 1
## 6 2000 Casado 25 Blan~ 20000 ~ Fuerte~ Protest~ Bautistas d~ NA
## 7 2000 Nunca se ha~ 36 Blan~ 25000 ~ No fue~ Cristia~ No aplica 3
## 8 2000 Divorciado 44 Blan~ 7000 -- Ind, p~ Protest~ Sínodo lute~ NA
## 9 2000 Casado 44 Blan~ 25000 ~ No fue~ Protest~ Otra 0
## 10 2000 Casado 47 Blan~ 25000 ~ Fuerte~ Protest~ Bautistas d~ 3
## # i 21,473 more rows
```

Contabilizar la cantidad de personas por “estado civil”

```
df=encuesta %>%
  count(estado_civil)
df
```

```
## # A tibble: 6 x 2
##   estado_civil      n
##   <fct>      <int>
## 1 Sin respuesta    17
## 2 Nunca se ha casado 5416
## 3 Separado        743
## 4 Divorciado      3383
## 5 Viudo           1807
## 6 Casado          10117
```

knitr

```
unique(df$estado_civil)
```

```
## [1] Sin respuesta      Nunca se ha casado Separado      Divorciado
## [5] Viudo              Casado
## 6 Levels: Sin respuesta Nunca se ha casado Separado Divorciado ... Casado
```



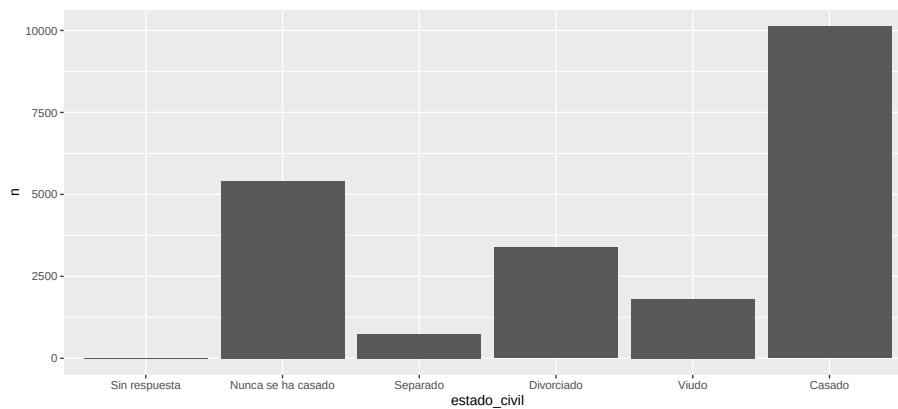
```
df$estado_est_civil <- factor(df$estado_civil, levels = df$estado_civil)

df$niveles_est_civil <- fct_reorder(df$estado_civil, df$n )

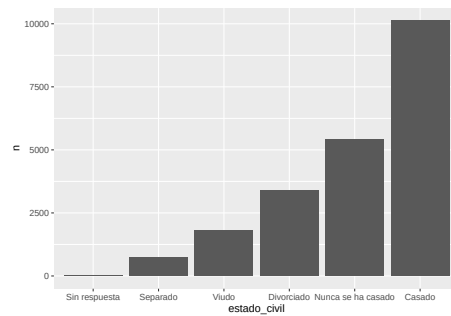
df
```

```
## # A tibble: 6 x 4
##   estado_civil      n estado_est_civil  niveles_est_civil
##   <fct>          <int> <fct>          <fct>
## 1 Sin respuesta      17 Sin respuesta    Sin respuesta
## 2 Nunca se ha casado 5416 Nunca se ha casado Nunca se ha casado
## 3 Separado          743 Separado        Separado
## 4 Divorciado        3383 Divorciado      Divorciado
## 5 Viudo             1807 Viudo           Viudo
## 6 Casado            10117 Casado          Casado
```

```
ggplot(df,
  aes(estado_civil,n )) +
  geom_col()
```



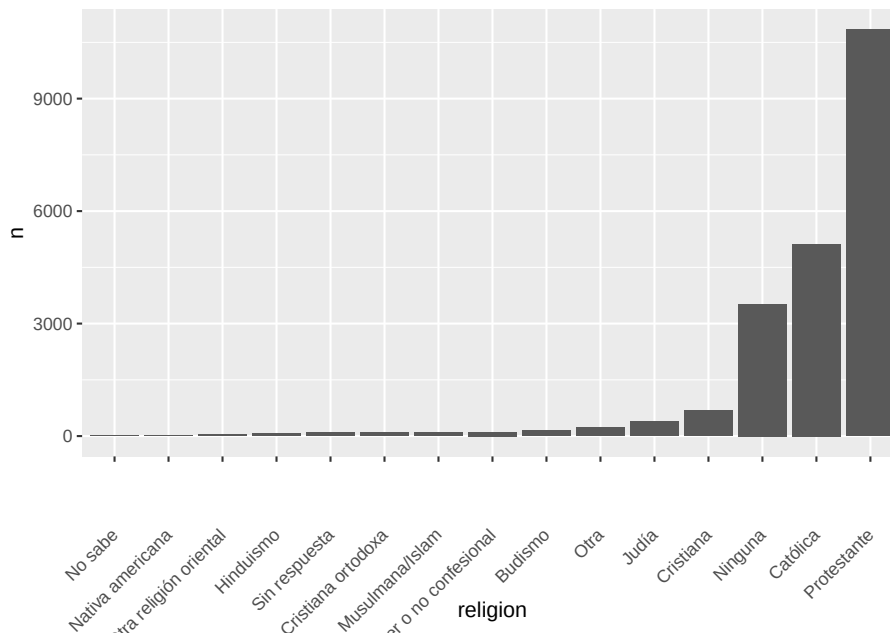
21.3 A smaller figure to the right, with floating text



LOTS of text here to wrap around the figure. dfg hjkljhgdxchvbvcxevbnnbvbb-
vbbvvcvxcv. fgffhgevcvcbhbvbvcvbnv bcbnbbnmmmbnmmnb vbcbnbvvnbbb vbb
v b b cgjhkggbcv

```
df2=encuesta %>%
  count(religion)

df2 %>%
  mutate(religion = fct_reorder(religion, n)) %>%
  ggplot(aes(x = religion, y = n)) +
  geom_col() +
  theme(axis.text.x = element_text(angle = 45, vjust = 0.5, hjust=1))
```



Hacer los ejercicios en la sección 15.3.1 del libro en español

21.5 Modificar el orden de los factores

Usa `fct_reorder()` para ordenar los niveles de un factor según otra variable (por ejemplo la frecuencia o la media). Los gráficos quedan mucho más fáciles de leer que con el orden alfabético por defecto.

- `fct_reorder()`
- `!is.na( )` remover los “NA”
- `fct_infreq()`
- `fct_rev()`
- `fct_recode()`
- `fct_collapse()`
- `fct_lump()`

```
#model=lm(y~x, df)
#summary(model)

resumen_religion <- encuesta %>%
  group_by(religion) %>%
  summarise(
    edad = mean(edad, na.rm = TRUE),
```

```

    horas_tv = mean(horas_tv, na.rm = TRUE),
    sd_edad = sd(edad, na.rm = TRUE),
    n = n()
  )
resumen_religion

```

```

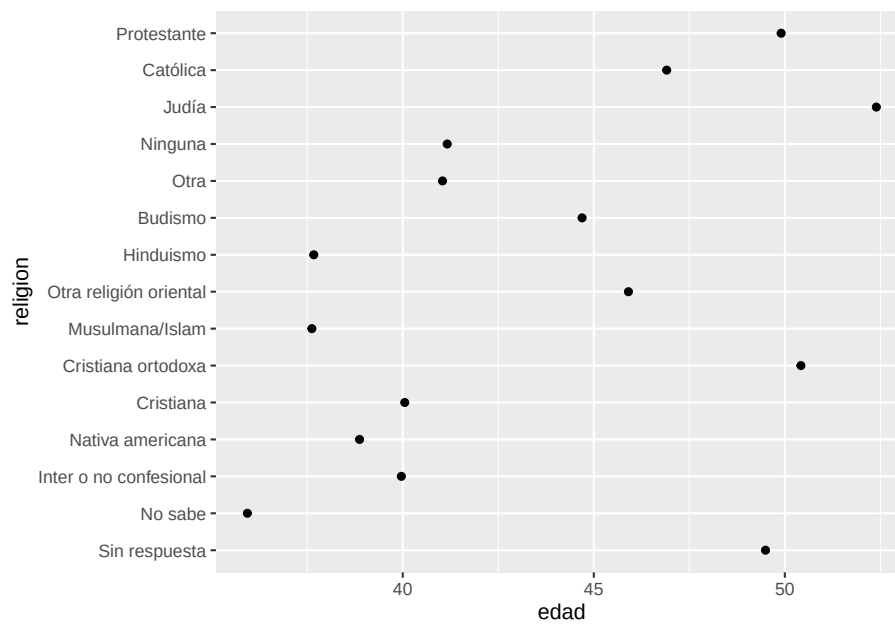
## # A tibble: 15 x 5
##   religion      edad horas_tv sd_edad      n
##   <fct>      <dbl>   <dbl>   <dbl> <int>
## 1 Sin respuesta  49.5     2.72    NA     93
## 2 No sabe       35.9     4.62    NA     15
## 3 Inter o no confesional 40.0     2.87    NA    109
## 4 Nativa americana 38.9     3.46    NA     23
## 5 Cristiana     40.1     2.79    NA    689
## 6 Cristiana ortodoxa 50.4     2.42    NA     95
## 7 Musulmana/Islam  37.6     2.44    NA    104
## 8 Otra religión oriental 45.9     1.67    NA     32
## 9 Hinduismo      37.7     1.89    NA     71
## 10 Budismo       44.7     2.38    NA    147
## 11 Otra          41.0     2.73    NA    224
## 12 Ninguna       41.2     2.71    NA   3523
## 13 Judía         52.4     2.52    NA    388
## 14 Católica      46.9     2.96    NA   5124
## 15 Protestante   49.9     3.15    NA  10846

```

```

ggplot(resumen_religion, aes(edad, religion)) +
  geom_point()

```

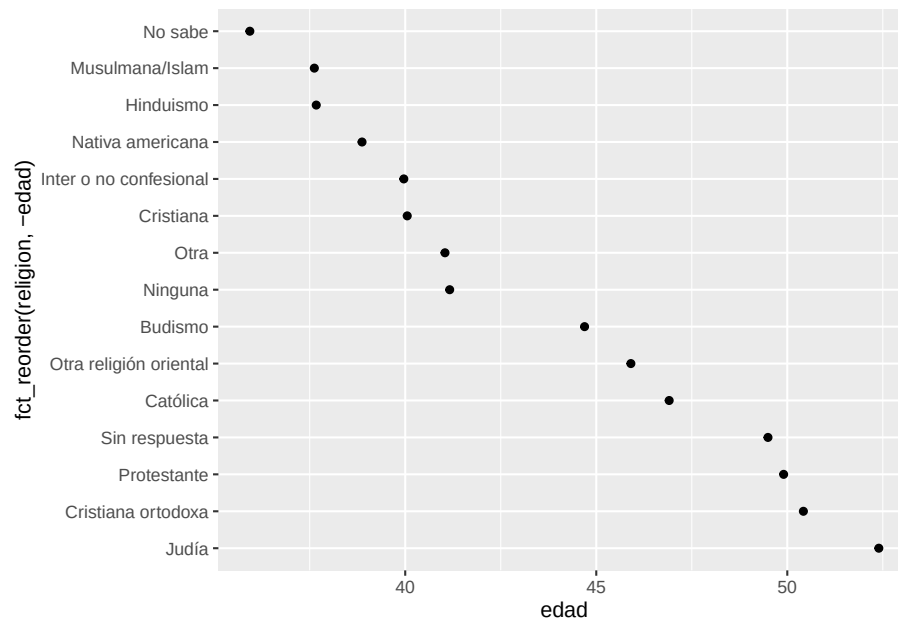


```
min(encuesta$horas_tv, na.rm = TRUE)
```

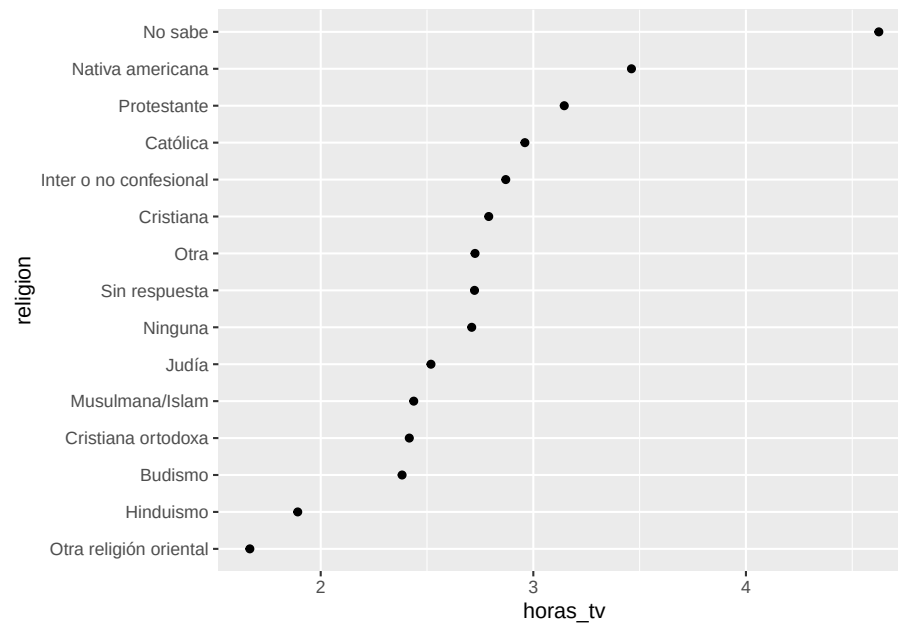
```
## [1] 0
```

```
ggplot(resumen_religion, aes(edad, fct_reorder(religion, -edad))) +  
  geom_point()
```

21.5.1 NOTA el efecto de añadir un "--" antes de edad



```
resumen_religion %>%
  mutate(religion = fct_reorder(religion, horas_tv)) %>%
  ggplot(aes(horas_tv, religion)) +
  geom_point()
```



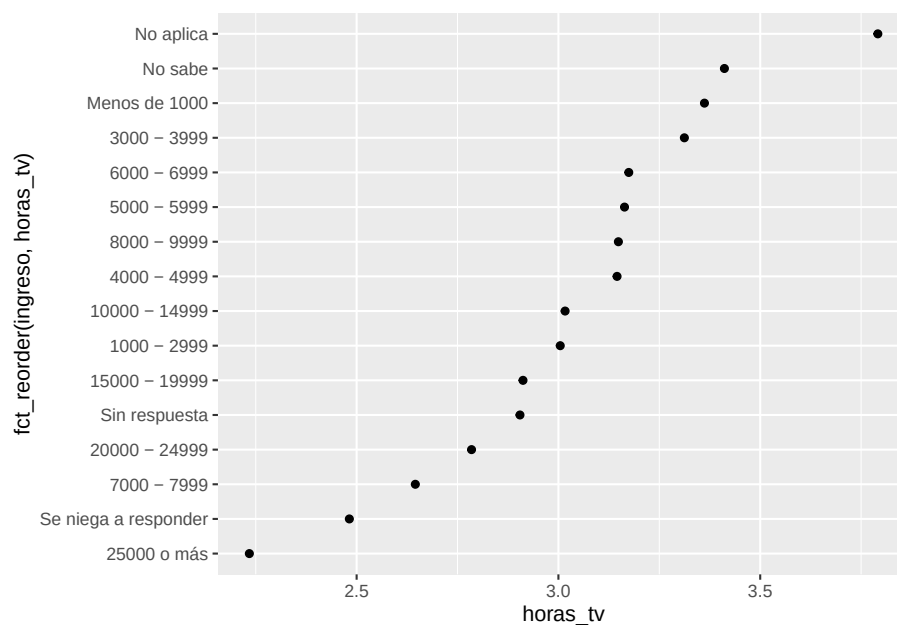
Reorder factor levels by sorting along another variable Description `fct_reorder()` is useful for 1d displays where the factor is mapped to position; `fct_reorder2()` for 2d displays where the factor is mapped to a non-position aesthetic. `last2()` and `first2()` are helpers for `fct_reorder2()`; `last2()` finds the last value of y when sorted by x; `first2()` finds the first value.

```
resumen_ingreso <- encuesta %>%
  group_by(ingreso) %>%
  summarise(
    edad = mean(edad, na.rm = TRUE),
    horas_tv = mean(horas_tv, na.rm = TRUE),
    n = n()
  )

resumen_ingreso
```

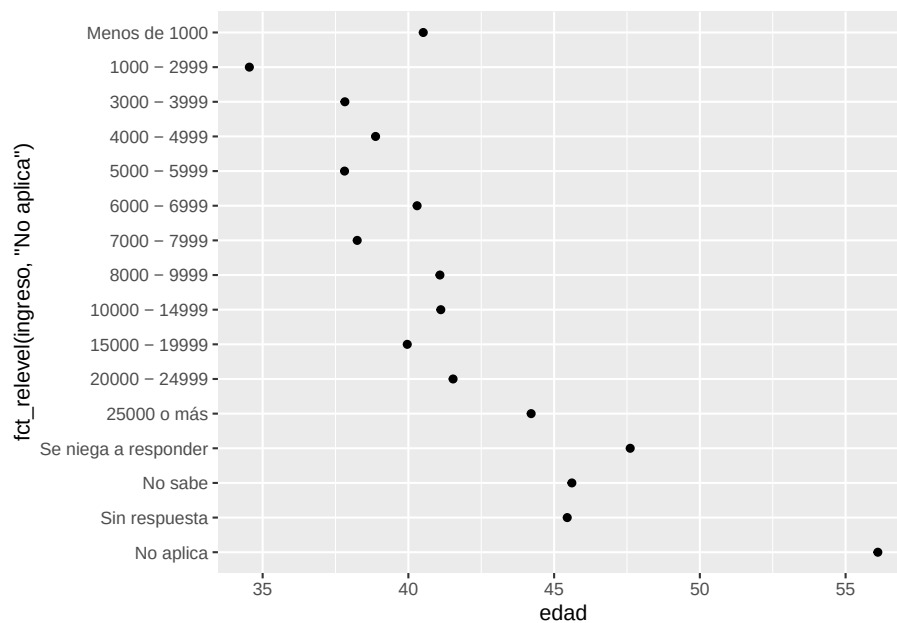
```
## # A tibble: 16 x 4
##   ingreso          edad horas_tv      n
##   <fct>          <dbl>   <dbl> <int>
## 1 Sin respuesta  45.5     2.90   183
## 2 No sabe       45.6     3.41   267
## 3 Se niega a responder 47.6     2.48   975
## 4 25000 o más   44.2     2.23  7363
## 5 20000 - 24999 41.5     2.78  1283
## 6 15000 - 19999 40.0     2.91  1048
## 7 10000 - 14999 41.1     3.02  1168
## 8 8000 - 9999   41.1     3.15   340
## 9 7000 - 7999   38.2     2.65   188
## 10 6000 - 6999  40.3     3.17   215
## 11 5000 - 5999  37.8     3.16   227
## 12 4000 - 4999  38.9     3.15   226
## 13 3000 - 3999  37.8     3.31   276
## 14 1000 - 2999  34.5     3.00   395
## 15 Menos de 1000 40.5     3.36   286
## 16 No aplica    56.1     3.79  7043

ggplot(resumen_ingreso, aes(horas_tv, fct_reorder(ingreso, horas_tv))) +
  geom_point()
```



Reorder factor levels by hand Description This is a generalisation of `stats::relevel()` that allows you to move any number of levels to any location.

```
ggplot(resumen_ingreso, aes(edad, fct_relevel(ingreso, "No aplica")) +  
  geom_point())
```



2. Ejercicios:

Hacer los ejercicios en la sección 15.5.1 del libro en español

Chapter 22

Cadena de Caracteres = STRINGS

```
## [1] "2026-08-01"
```

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/strings.html>

Español: <https://es.r4ds.hadley.nz/14-strings.html>

```
library(tidyverse)
library(datos)
library(stringr)
```

```
string1 <- "El Pequeño Principe; Saint Exupery"
string2 <- "El segundo sexo: Simone de Beauvoir"
string3 <- "El segundo sexo \\ Simone de Beauvoir"
```

```
string1
```

```
## [1] "El Pequeño Principe; Saint Exupery"
```

```
string2
```

```
## [1] "El segundo sexo: Simone de Beauvoir"
```

```
string3
```

```
## [1] "El segundo sexo \\ Simone de Beauvoir"
```

Caracteres especiales en cadenas de caracteres

```
comilla_doble <- '\"' # ó '''
comilla_simple <- '\'' # ó '''

comilla_ampersand <- "&"

comilla_doble
```

```
## [1] "\"
comilla_simple
```

```
## [1] ""
comilla_ampersand
```

```
## [1] "&"

&
```

Backslash

```
x <- c("\\", "\\")
x
```

```
## [1] "\" "\\ "
writeLines(x)
```

```
## "
## \
```

22.1 Caracteres especiales

```
"\n"
```

```
## [1] "\n"
```

```
"\t"
```

```
## [1] "\t"
```

22.2 Para ver las opciones de caracteres y como trabajar cadenas de caracteres use ;“” para ver

```
? " ' "

'\n'  newline (aka 'line feed')
'\r'  carriage return
'\t'  tab
'\b'  backspace
'\a'  alert (bell)
'\f'  form feed
'\v'  vertical tab
'\\'  backslash '\'
'\''  ASCII apostrophe ''
'\''  ASCII quotation mark '"'
'\'`' ASCII grave accent (backtick) '`'
```

22.3 Lista de caracteres especiales (unicode)

Vea el siguiente enlace para los unicodes

https://en.wikipedia.org/wiki/List_of_Unicode_characters

```
x <- "\u00b5" # mu
x
```

```
## [1] "µ"
```

```
y <- "\u00b6" # pilcrow
y
```

```
## [1] "¶"
```

```
z <- "\u00A1" # ¡
z
```

```
## [1] "¡"
```

```
w <- "\u01FC\u00b6" # Ě¶
w
```

```
## [1] "Ě¶"
```

Escribir su nombre en el alfabeto cirílico

- Vea este enlace para conocer el alfabeto cirílico

<<https://www.google.com/search?client=safari&rls=en&q=cyrillic+alphabet&ie=UTF-8&oe=UTF-8#vhid=cMLOx1fN0Zg5UM&vssid=1>>

```
Raymond = "\u048E\u0400\u042B\u043C\u043E\u043D\u0434, Asi se escribe mi nombre en ruso"
Raymond
```

```
## [1] "      , Asi se escribe mi nombre en ruso"
```

necesito poner una letra extraña en mi frase, "\u048E".

22.4 Temas: Cadenas de Caracteres

- `library(stringr)`

Solamente trabajaremos con el paquete `stringr`, donde las funciones escritas de forma que sean más intuitivas para recordar. Todas las funciones comienzan con `str_`

Las funciones `str_*` (`stringr`) — trabajar con texto.

- `str_length(x)`: número de caracteres.
- `str_c(...)`: une cadenas (concatena).
- `str_sub(x, a, b)`: extrae o reemplaza una porción por posición.
- `str_to_lower()` / `str_to_upper()`: cambian mayúsculas/minúsculas.
- `str_detect()`, `str_replace()`, `str_view()`: buscan y reemplazan patrones (regex).

22.4.1 El largo de una cadena de caracteres con `str_length`

```
library(stringr)
str_length("Yo quiero un Lamborghini")
```

```
## [1] 24
```

22.4.2 Combinar cadenas con `str_c`

Nota aquí que une la primera oración con la segunda oración

```
str_c("Es una oración con muchas letras y palabras",
      "Mi película favorita es Cinema Paraíso")
```

22.5. DIVIDIR CADENAS BASADO EN LA POSICIÓN DE LOS CARACTERES EN LA CADENA215

```
## [1] "Es una oración con muchas letras y palabrasMi película favorita es Cinema Paraiso"
```

Usa el argumento `sep = "xxx"` para controlar cómo separar las oraciones

```
str_c("Es una oración con muchas letras y palabras",  
      "Mi película favorita es Cinema Paraiso", sep = ", ")
```

```
## [1] "Es una oración con muchas letras y palabras, Mi película favorita es Cinema Paraiso"
```

- `str_replace_na()`

```
x <- c("Mi película favorita es Cinema Paraiso", NA)  
x
```

```
## [1] "Mi película favorita es Cinema Paraiso"  
## [2] NA
```

```
str_c("|-", x, "-|")
```

```
## [1] "|-Mi película favorita es Cinema Paraiso-|"  
## [2] NA
```

22.5 Dividir cadenas basado en la posición de los caracteres en la cadena

- `str_sub()`

```
x= c( "Piña", "Manzana", "Toronja")  
str_sub(x, 1,1)
```

```
## [1] "P" "M" "T"
```

22.5.1 Seleccionar los caracteres al último de las palabras

```
x= c( "Piña", "Manzana", "Toronja", "Hello")  
str_sub(x, -2,-1)
```

```
## [1] "ña" "na" "ja" "lo"
```

22.6 You can also use `str_sub()` to modify strings:

```
x <- "BBCDEF"
str_sub(x, 1, 1) <- "A"; x

## [1] "ABCDEF"

x <- "BBCDEF"
str_sub(x, -1, -1) <- "K"; x

## [1] "BBCDEK"

x <- "BBCDEF"
str_sub(x, -2, -2) <- "HIJ"; x

## [1] "BBCDGHIJF"

x <- "BBCDEF"
str_sub(x, -3, -3) <- "BC"; x

## [1] "BBCBCEF"

x <- "BBCDEF"
str_sub(x, 2, -2) <- ""; x

## [1] "BF"
```

22.6.1 Cambiar a todo minúscula

```
x= c( "Piña", "Manzana", "Toronja", "HAPPY")
str_to_lower(x)

## [1] "piña"      "manzana"  "toronja"  "happy"
```

22.6.2 Cambiar a mayúscula

```
x= c( "Piña", "Manzana", "Toronja")
str_to_upper(x)

## [1] "PIÑA"      "MANZANA"  "TORONJA"
```

22.7 Locales

- ¿Qué es un locale?
- `str_to_lower()`
- `str_to_upper()`
- `str_order()` # que hace esta función?
- `str_sort()` # que hace esta función?

```
x
## [1] "Piña"      "Manzana" "Toronja"
str_order(x) # regresa el orden de las palabras

## [1] 2 1 3
str_sort(x) # reorganiza las palabras en orden alfabético

## [1] "Manzana" "Piña"     "Toronja"
```

1. Ejercicios:

Hacer los ejercicios en la sección 14.2.5 del libro en español

22.8 Buscar patrones

Las expresiones regulares surgieron en los años 1950 del trabajo del matemático Stephen Kleene sobre lenguajes formales, y hoy son el estándar para describir patrones de texto en casi todos los lenguajes de programación.

22.9 Coincidencia básica

Para visualizar patrones de letras

- `str_view()`
- `str_view( , .x.)`

```
library(stringr)
x <- c("manzana", "banana", "pera", "extraer")
str_view(x, "er")

## [3] | p<er>a
## [4] | extra<er>
```

Ejemplo de cómo uno puede utilizar esas funciones

```

str_detect()
x= c( "Piña", "Manzana", "Toronja", "Jugo", "Monique", "Raymond", "mami")
str_detect(x, "a") # detectar la a

## [1] TRUE TRUE TRUE FALSE FALSE TRUE TRUE
str_detect(x, "i") # detectar la i

## [1] TRUE FALSE FALSE FALSE TRUE FALSE TRUE
str_detect(x, "ñ")

## [1] TRUE FALSE FALSE FALSE FALSE FALSE FALSE

```

22.9.1 Detectar las palabras que comienzan con una letra específica con “^”. Nota el uso de mayúsculas y minúsculas y sumamos cuantas palabras cumplen con la condición.

```

# ¿Cuántas palabras comunes empiezan con m o M?
# str_detect(x, "^M")
x

## [1] "Piña" "Manzana" "Toronja" "Jugo" "Monique" "Raymond" "mami"
sum(str_detect(x, "^m"))

## [1] 1
sum(str_detect(x, "^M"))

## [1] 2
sum(str_detect(x, "[mM]")) # ambas mayusculas y minusculas

## [1] 3

```

22.9.2 Detectar las palabras que terminan con una letra específica con “\$”

```

# ¿Qué proporción de palabras comunes terminan con una vocal?

library(datos)
mean(str_detect(palabras, "[aáéííoóuú]$"))

## [1] 0.561

# ¿Qué proporción de palabras comunes terminan con una vocal específica?
mean(str_detect(palabras, "[ao]$"))

```

```
## [1] 0.428
mean(str_detect(x, "[d]$")) # Raymond
```

```
## [1] 0.1428571
```

Dataset: palabras

- Comienza para mirar el archivo palabras
- Cuentas palabras tiene ese archivo?

```
library(stringr)
library(datos)
palabras
```

```
##      [1] "a"           "abril"       "acción"     "acciones"
##      [5] "acerca"     "actitud"    "actividad"  "actividades"
##      [9] "acto"      "actual"     "acuerdo"    "adelante"
##     [13] "además"    "administración" "afirmó"     "agua"
##     [17] "ahí"       "ahora"      "aire"       "al"
##     [21] "algo"      "alguien"    "algún"      "alguna"
##     [25] "algunas"   "algunos"    "allá"       "allí"
##     [29] "alrededor" "alta"       "alto"       "ambiente"
##     [33] "ambos"     "américa"    "amigo"      "amigos"
##     [37] "amor"      "análisis"   "animales"   "ante"
##     [41] "anterior"  "antes"      "antonio"    "año"
##     [45] "años"      "aparece"    "apenas"     "apoyo"
##     [49] "aquel"     "aquella"    "aquellas"   "aquellos"
##     [53] "aquí"      "área"       "argentina"  "armas"
##     [57] "arriba"    "arte"       "artículo"   "así"
##     [61] "asimismo"  "asociación" "aspecto"    "aspectos"
##     [65] "asunto"    "atención"   "atrás"      "aumento"
##     [69] "aun"       "aún"        "aunque"     "autor"
##     [73] "autoridades" "ayer"      "ayuda"      "b"
##     [77] "baja"      "bajo"       "banco"      "barcelona"
##     [81] "base"      "bastante"   "bien"       "blanca"
##     [85] "blanco"    "boca"       "buen"       "buena"
##     [89] "bueno"     "buenos"     "busca"      "buscar"
##     [93] "c"         "cabeza"     "cabo"       "cada"
##     [97] "calidad"   "calle"      "cama"       "cámara"
##    [101] "cambio"    "cambios"    "camino"     "campaña"
##    [105] "campo"     "cantidad"   "capacidad"  "capaz"
##    [109] "capital"   "cara"       "carácter"   "características"
##    [113] "cargo"     "carlos"     "carne"      "carrera"
##    [117] "carta"     "casa"       "casi"       "caso"
##    [121] "casos"     "causa"      "central"    "centro"
```

##	[125]	"centros"	"cerca"	"chile"	"ciencia"
##	[129]	"ciento"	"cierta"	"cierto"	"cinco"
##	[133]	"cine"	"ciudad"	"civil"	"claro"
##	[137]	"clase"	"club"	"color"	"comenzó"
##	[141]	"comercio"	"comisión"	"como"	"cómo"
##	[145]	"compañía"	"común"	"comunicación"	"comunidad"
##	[149]	"con"	"concepto"	"conciencia"	"condiciones"
##	[153]	"congreso"	"conjunto"	"conocer"	"conocimiento"
##	[157]	"consecuencia"	"conseguir"	"consejo"	"considera"
##	[161]	"constitución"	"construcción"	"consumo"	"contenido"
##	[165]	"contra"	"contrario"	"control"	"corazón"
##	[169]	"corte"	"cosa"	"cosas"	"costa"
##	[173]	"creación"	"crecimiento"	"creo"	"crisis"
##	[177]	"cuadro"	"cual"	"cuales"	"cualquier"
##	[181]	"cuando"	"cuanto"	"cuarto"	"cuatro"
##	[185]	"cuba"	"cuenta"	"cuerpo"	"cuestión"
##	[189]	"cultura"	"cultural"	"curso"	"cuya"
##	[193]	"cuyo"	"d"	"da"	"dado"
##	[197]	"dan"	"dar"	"datos"	"de"
##	[201]	"debe"	"deben"	"debía"	"debido"
##	[205]	"decía"	"decir"	"decisión"	"defensa"
##	[209]	"deja"	"dejar"	"dejó"	"del"
##	[213]	"demás"	"demasiado"	"democracia"	"dentro"
##	[217]	"derecha"	"derecho"	"derechos"	"desarrollo"
##	[221]	"desde"	"deseo"	"después"	"destino"
##	[225]	"día"	"diario"	"días"	"dice"
##	[229]	"dicen"	"dicho"	"diciembre"	"diez"
##	[233]	"diferencia"	"diferentes"	"difícil"	"dijo"
##	[237]	"dinero"	"dio"	"dios"	"dirección"
##	[241]	"director"	"distintas"	"distintos"	"diversas"
##	[245]	"diversos"	"doctor"	"dólares"	"dolor"
##	[249]	"domingo"	"don"	"donde"	"dónde"
##	[253]	"dos"	"duda"	"durante"	"e"
##	[257]	"economía"	"económica"	"económico"	"edad"
##	[261]	"educación"	"efecto"	"efectos"	"ejemplo"
##	[265]	"ejército"	"el"	"él"	"elecciones"
##	[269]	"electoral"	"elementos"	"ella"	"ellas"
##	[273]	"ello"	"ellos"	"embargo"	"empresa"
##	[277]	"empresas"	"en"	"encima"	"encontrar"
##	[281]	"encuentra"	"encuentran"	"encuentro"	"energía"
##	[285]	"enero"	"enfermedad"	"entonces"	"entrada"
##	[289]	"entrar"	"entre"	"época"	"equipo"
##	[293]	"era"	"eran"	"es"	"esa"
##	[297]	"esas"	"escuela"	"ese"	"esfuerzo"
##	[301]	"eso"	"esos"	"espacio"	"españa"
##	[305]	"español"	"española"	"españoles"	"especial"

##	[309]	"especialmente"	"especie"	"espera"	"esta"
##	[313]	"está"	"ésta"	"estaba"	"estaban"
##	[317]	"estado"	"estados"	"estamos"	"están"
##	[321]	"estar"	"estas"	"este"	"éste"
##	[325]	"estilo"	"esto"	"estos"	"estoy"
##	[329]	"estructura"	"estudio"	"estudios"	"estuvo"
##	[333]	"etapa"	"etc"	"europa"	"europea"
##	[337]	"evitar"	"ex"	"existe"	"existen"
##	[341]	"existencia"	"éxito"	"experiencia"	"explicó"
##	[345]	"expresión"	"exterior"	"fácil"	"falta"
##	[349]	"familia"	"favor"	"fecha"	"felipe"
##	[353]	"fernando"	"figura"	"fin"	"final"
##	[357]	"finalmente"	"fiscal"	"flores"	"fondo"
##	[361]	"forma"	"formación"	"formas"	"francia"
##	[365]	"francisco"	"frente"	"fue"	"fuego"
##	[369]	"fuentes"	"fuera"	"fueron"	"fuerte"
##	[373]	"fuerza"	"fuerzas"	"función"	"fútbol"
##	[377]	"futuro"	"g"	"garcía"	"general"
##	[381]	"generales"	"gente"	"gobierno"	"gonzález"
##	[385]	"gracias"	"grado"	"gran"	"grande"
##	[389]	"grandes"	"grupo"	"grupos"	"guerra"
##	[393]	"ha"	"haber"	"había"	"habían"
##	[397]	"habla"	"hablar"	"habrá"	"habría"
##	[401]	"hace"	"hacen"	"hacer"	"hacerlo"
##	[405]	"hacia"	"hacia"	"haciendo"	"han"
##	[409]	"has"	"hasta"	"hay"	"haya"
##	[413]	"he"	"hecho"	"hechos"	"hemos"
##	[417]	"hermano"	"hicieron"	"hija"	"hijo"
##	[421]	"hijos"	"historia"	"hizo"	"hombre"
##	[425]	"hombres"	"hora"	"horas"	"hospital"
##	[429]	"hoy"	"hubiera"	"hubo"	"humana"
##	[433]	"humano"	"humanos"	"i"	"iba"
##	[437]	"idea"	"ideas"	"iglesia"	"igual"
##	[441]	"ii"	"imagen"	"imágenes"	"importancia"
##	[445]	"importante"	"importantes"	"imposible"	"incluso"
##	[449]	"industria"	"información"	"informe"	"instituciones"
##	[453]	"instituto"	"interés"	"intereses"	"interior"
##	[457]	"internacional"	"investigación"	"ir"	"izquierda"
##	[461]	"j"	"jamás"	"jefe"	"jorge"
##	[465]	"josé"	"joven"	"jóvenes"	"juan"
##	[469]	"juego"	"juez"	"juicio"	"julio"
##	[473]	"junio"	"junto"	"justicia"	"la"
##	[477]	"lado"	"larga"	"largo"	"las"
##	[481]	"le"	"lejos"	"lenguaje"	"les"
##	[485]	"ley"	"libertad"	"libre"	"libro"
##	[489]	"libros"	"líder"	"línea"	"llama"

##	[493]	"llamado"	"llega"	"llegado"	"llegar"
##	[497]	"llegó"	"lleva"	"llevar"	"lo"
##	[501]	"local"	"lópez"	"los"	"lucha"
##	[505]	"luego"	"lugar"	"luis"	"luz"
##	[509]	"m"	"madre"	"madrid"	"mal"
##	[513]	"manera"	"mano"	"manos"	"mantener"
##	[517]	"manuel"	"mañana"	"mar"	"marcha"
##	[521]	"marco"	"maría"	"martín"	"marzo"
##	[525]	"más"	"materia"	"material"	"máximo"
##	[529]	"mayo"	"mayor"	"mayores"	"mayoría"
##	[533]	"me"	"media"	"mediante"	"médico"
##	[537]	"medida"	"medidas"	"medio"	"medios"
##	[541]	"mejor"	"mejores"	"memoria"	"menor"
##	[545]	"menos"	"mercado"	"mes"	"mesa"
##	[549]	"meses"	"metros"	"méxico"	"mi"
##	[553]	"mí"	"miedo"	"miembros"	"mientras"
##	[557]	"miguel"	"mil"	"militar"	"militares"
##	[561]	"millones"	"ministerio"	"ministro"	"minutos"
##	[565]	"mira"	"mirada"	"mis"	"misma"
##	[569]	"mismo"	"mismos"	"mitad"	"modelo"
##	[573]	"modo"	"momento"	"momentos"	"movimiento"
##	[577]	"mucho"	"muchas"	"mucho"	"muchos"
##	[581]	"muerte"	"muerto"	"muestra"	"mujer"
##	[585]	"mujeres"	"mundial"	"mundo"	"música"
##	[589]	"muy"	"n"	"nacional"	"nada"
##	[593]	"nadie"	"natural"	"naturaleza"	"necesario"
##	[597]	"necesidad"	"negro"	"ni"	"ningún"
##	[601]	"ninguna"	"niño"	"niños"	"nivel"
##	[605]	"niveles"	"no"	"noche"	"nombre"
##	[609]	"norte"	"nos"	"nosotros"	"noviembre"
##	[613]	"nuestra"	"nuestras"	"nuestro"	"nuestros"
##	[617]	"nueva"	"nuevas"	"nuevo"	"nuevos"
##	[621]	"número"	"nunca"	"o"	"objetivo"
##	[625]	"objeto"	"obra"	"obras"	"obstante"
##	[629]	"ocasión"	"ocasiones"	"ocho"	"octubre"
##	[633]	"oficial"	"ojos"	"operación"	"opinión"
##	[637]	"oposición"	"orden"	"organización"	"origen"
##	[641]	"oro"	"otra"	"otras"	"otro"
##	[645]	"otros"	"p"	"pablo"	"paciente"
##	[649]	"pacientes"	"padre"	"padres"	"país"
##	[653]	"países"	"palabra"	"palabras"	"papel"
##	[657]	"para"	"parece"	"parecía"	"parís"
##	[661]	"parte"	"partes"	"participación"	"particular"
##	[665]	"partido"	"partidos"	"partir"	"pasa"
##	[669]	"pasado"	"pasar"	"paso"	"pasó"
##	[673]	"paz"	"pedro"	"película"	"pensar"

##	[677]	"peor"	"pequeña"	"pequeño"	"perdido"
##	[681]	"período"	"permite"	"pero"	"persona"
##	[685]	"personal"	"personas"	"pesar"	"pese"
##	[689]	"pesetas"	"peso"	"pie"	"piel"
##	[693]	"plan"	"plaza"	"plazo"	"población"
##	[697]	"poco"	"pocos"	"podemos"	"poder"
##	[701]	"podía"	"podrá"	"podría"	"polición"
##	[705]	"política"	"políticas"	"político"	"políticos"
##	[709]	"pone"	"poner"	"popular"	"por"
##	[713]	"porque"	"posibilidad"	"posibilidades"	"posible"
##	[717]	"posición"	"pp"	"práctica"	"precio"
##	[721]	"precios"	"precisamente"	"pregunta"	"premio"
##	[725]	"prensa"	"presencia"	"presenta"	"presente"
##	[729]	"presidente"	"primer"	"primera"	"primeras"
##	[733]	"primero"	"primeros"	"principal"	"principales"
##	[737]	"principio"	"principios"	"problema"	"problemas"
##	[741]	"proceso"	"producción"	"produce"	"producto"
##	[745]	"productos"	"profesional"	"programa"	"programas"
##	[749]	"pronto"	"propia"	"propio"	"propios"
##	[753]	"propuesta"	"próximo"	"proyecto"	"proyectos"
##	[757]	"prueba"	"psoe"	"pública"	"público"
##	[761]	"pudo"	"pueblo"	"pueda"	"puede"
##	[765]	"pueden"	"puedo"	"puerta"	"puerto"
##	[769]	"pues"	"puesto"	"punto"	"puntos"
##	[773]	"puso"	"que"	"qué"	"queda"
##	[777]	"quedó"	"quería"	"quien"	"quién"
##	[781]	"quienes"	"quiere"	"quiero"	"quizá"
##	[785]	"r"	"radio"	"rafael"	"razón"
##	[789]	"razones"	"real"	"realidad"	"realizar"
##	[793]	"recuerdo"	"recursos"	"reforma"	"régimen"
##	[797]	"región"	"relación"	"relaciones"	"república"
##	[801]	"respetto"	"respuesta"	"resto"	"resulta"
##	[805]	"resultado"	"resultados"	"reunión"	"revolución"
##	[809]	"rey"	"riesgo"	"río"	"rodríguez"
##	[813]	"rosa"	"s"	"sabe"	"saber"
##	[817]	"sabía"	"sala"	"salida"	"salir"
##	[821]	"salud"	"san"	"sangre"	"santa"
##	[825]	"santiago"	"se"	"sé"	"sea"
##	[829]	"sean"	"secretario"	"sector"	"sectores"
##	[833]	"seguir"	"según"	"segunda"	"segundo"
##	[837]	"seguridad"	"seguro"	"seis"	"semana"
##	[841]	"semanas"	"sentido"	"señaló"	"señor"
##	[845]	"señora"	"septiembre"	"ser"	"será"
##	[849]	"serán"	"sería"	"serie"	"servicio"
##	[853]	"servicios"	"si"	"sí"	"sido"
##	[857]	"siempre"	"siendo"	"siete"	"siglo"

##	[861]	"sigue"	"siguiente"	"siguientes"	"silencio"
##	[865]	"sin"	"sino"	"siquiera"	"sistema"
##	[869]	"sistemas"	"situación"	"sobre"	"social"
##	[873]	"sociales"	"socialista"	"sociedad"	"sol"
##	[877]	"sola"	"solamente"	"solo"	"sólo"
##	[881]	"solución"	"somos"	"son"	"soy"
##	[885]	"su"	"suelo"	"sueño"	"suerte"
##	[889]	"suficiente"	"superior"	"supuesto"	"sur"
##	[893]	"sus"	"tal"	"tales"	"también"
##	[897]	"tampoco"	"tan"	"tanto"	"tarde"
##	[901]	"te"	"teatro"	"técnica"	"televisión"
##	[905]	"tema"	"temas"	"tendrá"	"tenemos"
##	[909]	"tener"	"tengo"	"tenía"	"tenían"
##	[913]	"tenido"	"teoría"	"tercera"	"términos"
##	[917]	"texto"	"tiempo"	"tiempos"	"tiene"
##	[921]	"tienen"	"tierra"	"tipo"	"tipos"
##	[925]	"título"	"toda"	"todas"	"todavía"
##	[929]	"todo"	"todos"	"toma"	"tomar"
##	[933]	"torno"	"total"	"trabajadores"	"trabajar"
##	[937]	"trabajo"	"tras"	"trata"	"tratamiento"
##	[941]	"través"	"tres"	"tribunal"	"tu"
##	[945]	"tú"	"tuvo"	"u"	"última"
##	[949]	"último"	"últimos"	"un"	"una"
##	[953]	"unas"	"única"	"único"	"unidad"
##	[957]	"unidos"	"unión"	"universidad"	"uno"
##	[961]	"unos"	"uso"	"usted"	"va"
##	[965]	"valor"	"valores"	"vamos"	"van"
##	[969]	"varias"	"varios"	"ve"	"veces"
##	[973]	"ver"	"verdad"	"vez"	"vía"
##	[977]	"viaje"	"victoria"	"vida"	"viejo"
##	[981]	"viene"	"vino"	"vio"	"violencia"
##	[985]	"visita"	"vista"	"visto"	"vivir"
##	[989]	"voluntad"	"volver"	"volvió"	"voy"
##	[993]	"voz"	"vuelta"	"vuelve"	"y"
##	[997]	"ya"	"yo"	"zona"	"zonas"

```
# Encuentra todas las palabras que contengan al menos una vocal, y luego niégalo
sin_vocales_1 <- !str_detect(palabras, "[aáeéííoóuúüü]")
```

```
sin_vocales_1
```

```
##      [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
##     [13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
##     [25] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```


[illegible]

```
## [589] FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [601] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [613] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [625] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [637] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE
## [649] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [661] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [673] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [685] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [697] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [709] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE
## [721] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [733] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [745] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [757] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [769] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [781] FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [793] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [805] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE
## [817] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [829] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [841] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [853] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [865] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [877] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [889] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [901] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [913] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [925] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [937] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [949] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [961] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [973] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [985] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE
## [997] FALSE FALSE FALSE FALSE
```

```
# Encuentra todas las palabras consistentes solo en consonantes (no vocales)
sin_vocales_2 <- str_detect(palabras, "[^aáeéííoóuúü]+")

head(identical(sin_vocales_1, sin_vocales_2), 100)
```

```
## [1] TRUE
```

Suma la cantidad de palabras que no tienen vocales?

```

library(tidyverse)
df <- tibble(
  palabra = palabras,
  i = seq_along(palabra)
) ## poner los datos en un dataframe

df %>%
  filter(str_detect(palabras, "x$")) # filtrar todas la palabras para solamente las terminan en x

## # A tibble: 1 x 2
##   palabra      i
##   <chr>    <int>
## 1 ex      338

df %>%
  filter(str_detect(palabras, "z$")) # filtrar todas la palabras para solamente las terminan en z

## # A tibble: 10 x 2
##   palabra      i
##   <chr>    <int>
## 1 capaz      108
## 2 diez       232
## 3 gonzález   384
## 4 juez       470
## 5 lópez      502
## 6 luz        508
## 7 paz        673
## 8 rodríguez  812
## 9 vez        975
## 10 voz       993

library(readr)
fragmentos_neruda_sep <- read_csv("Datos/fragmentos_neruda_sep.csv")

fragmentos_neruda_sep

## # A tibble: 10 x 2
##       id texto
##   <dbl> <chr>
## 1     1 "\"Puedo\" \"escribir\" \"los\" \"versos\" \"más\" \"tristes\" \"esta\" ~
## 2     2 "\"Es\" \"tan\" \"corto\" \"el\" \"amor,\" \"y\" \"es\" \"tan\" \"larg~
## 3     3 "\"Me\" \"gustas\" \"cuando\" \"callas\" \"porque\" \"estás\" \"como\" ~
## 4     4 "\"Para\" \"que\" \"nada\" \"nos\" \"separe\" \"que\" \"nada\" \"nos\" ~
## 5     5 "\"En\" \"un\" \"beso\" \"sabrás\" \"todo\" \"lo\" \"que\" \"he\" \"ca~
## 6     6 "\"Te\" \"amo\" \"como\" \"se\" \"aman\" \"ciertas\" \"cosas\" \"oscur~
## 7     7 "\"Quiero\" \"hacer\" \"contigo\" \"lo\" \"que\" \"la\" \"primavera\" ~

```

```
## 8      8 "\"Algún\" \"día\" \"en\" \"cualquier\" \"parte,\" \"en\" \"cualquier\"~
## 9      9 "\"Si\" \"nada\" \"nos\" \"salva\" \"de\" \"la\" \"muerte,\" \"al\" \"~
## 10     10 "\"Podrán\" \"cortar\" \"todas\" \"las\" \"flores,\" \"pero\" \"no\" \"~
```

Contar cuantas “a” hay en cada palabra con `str_count()`

```
#x= c( "Piña", "Manzana", "Toronja", "Jugo")
```

```
str_count(fragmentos_neruda_sep, "e")
```

```
##      id texto
##      0      57
```

```
# En promedio, ¿cuántas vocales hay por palabra?
mean(str_count(palabras, "[aáeéííoóuüü]"))
```

```
## [1] 2.72
```

```
mean(str_count(palabras, "[x]"))
```

```
## [1] 0.013
```

Contar cuantas vocales y consonantes hay en cada palabra

```
df %>%
  mutate(
    vocales = str_count(palabra, "[aáeéííoóuüü]"), # Cuenta cuantas vocales
    consonantes = str_count(palabra, "[^aáeéííoóuüü]") # cuenta cuantos consonantes
  ) |> head(n=30)
```

```
## # A tibble: 30 x 4
##   palabra      i vocales consonantes
##   <chr>      <int>  <int>      <int>
## 1 a          1      1          0
## 2 abril      2      2          3
## 3 acción      3      3          3
## 4 acciones    4      4          4
## 5 acerca      5      3          3
## 6 actitud     6      3          4
## 7 actividad   7      4          5
## 8 actividades 8      5          6
## 9 acto        9      2          2
## 10 actual    10      3          3
## # i 20 more rows
```

- los puntos y caracteres especiales

2. Ejercicios:

Hacer los ejercicios en la sección 14.2.7.1 del libro en español

22.10 Anclas

- `^` al principio
- `$` al final

3. Ejercicios:

Hacer los ejercicios en la sección 14.2.8.1 del libro en español

22.11 Clases de caracteres y alternativas

- `\d`
- `\s`
- `[abc]`
- `[\^abc]`
- `$.|?*+( )[\{`

```
str_view(c("cómo", "como", "Raymond"), "c(ó|o)mo")
```

```
## [1] | <cómo>
```

```
## [2] | <como>
```

4. Ejercicios:

Hacer los ejercicios en la sección 14.2.9.1 del libro en español

22.12 Repetición

- `?`, 0 o 1, presencia o ausencia
- `+`, 1 o más
- `*`, 0 o más
- `str_view()`
- `{n}`: exactamente n
- `{n, }`: n o más
- `{ , m}`: no más de m
- `{n, m}`: entre n y m
- `{n, m}?`: la cadena más larga

- `{n, m}+?`: la cadena más corta

5. Ejercicios:

Hacer los ejercicios en la sección 14.2.10.1 del libro en español

22.13 Agrupamiento y referencias previas

- `str_view(frutas, "(.)\\1", match=TRUE)`

6. Ejercicios:

Hacer los ejercicios en la sección 14.2.11.1 del libro en español

22.14 Herramientas: Objetivos

- Determinar qué cadenas coinciden con un patrón.
- Encontrar la posición de una coincidencia.
- Extraer el contenido de las coincidencias.
- Remplazar coincidencias con nuevos valores.
- Dividir una cadena de acuerdo a una coincidencia.

22.14.1 Detectar coincidencias

- `str_detect(x, "e")`

```
library(datos)
#fruit
head(palabras)

## [1] "a"          "abril"      "acción"     "acciones"  "acerca"    "actitud"
# Encuentra todas las palabras que contengan al menos una vocal, y luego niégalo
sin_vocales_1 <- !str_detect(palabras, "[aáeéiíoóuúüü]")

#sin_vocales_1
# Encuentra todas las palabras consistentes solo en consonantes (no vocales)
sin_vocales_2 <- str_detect(palabras, "^^[aáeéiíoóuúüü]+$")
#sin_vocales_1
head(identical(sin_vocales_1, sin_vocales_2), 30)

## [1] TRUE
#> [1] TRUE
```

- `str_subset()`

- seq_along()
- str_count()

7. Ejercicios:

Hacer los ejercicios en la sección 14.3.2 del libro en español

Ejercicio

22.15 Extraer coincidencias

- str_extract()
- str_extract_all()

8. Ejercicios:

Hacer los ejercicios en la sección 14.3.3.1 del libro en español

22.16 Coincidencias agrupadas:

- str_match*

```
sustantivo <- "(el|la|los|las|lo|un|una|unos|unas) ([^ ]+)"

tiene_sustantivo <- oraciones %>%
  str_subset(sustantivo)

#tiene_sustantivo
tiene_sustantivo %>%
  str_extract(sustantivo)
```

```
## [1] "los de"      "el camión"   "la mejor"    "la cuenta"   "las ruinas."
## [6] "la hoja"     "la cocina." "la taza"     "el tanque."  "el calor"
## [11] "el aire"     "lo en"      "el papel"    "la parte"    "el sofá"
## [16] "lo de"      "lo curó"    "los actores." "la azul"     "el banco"
## [21] "una llama"   "la cabeza." "la ventana." "la hierba"   "el sol"
## [26] "el lugar"   "el sofá"    "un suéter"   "el gráfico"  "los encajes"
## [31] "el verde"   "la caja"    "la pistola"  "los niños"   "una delgada"
## [36] "lo en"      "la hierba"  "la primavera" "la es"

#> [1] "los de"      "el camión"   "la mejor"    "la cuenta"   "las ruinas."
#> [6] "la hoja"     "la cocina."  "la taza"     "el tanque."  "el calor"
```

8. Ejercicios:

Hacer los ejercicios en la sección 14.3.4.1 del libro en español

22.17 Reemplazar coincidencias

- `str_replace()`
- `str_replace_all()`

9. Ejercicios:

Hacer los ejercicios en la sección 14.3.5.1 del libro en español

22.18 Divisiones

- `str_split()`
- `str_split(" ", simplify=TRUE)`
- `str_split(x, boundary("word"))[[1]]`

10. Ejercicios:

Hacer los ejercicios en la sección 14.3.6.1 del libro en español

22.19 Otro tipos de patrones

- `regex("x", ignore_case=TRUE)`
- separar números telefónicos

22.20 stringi:

10. Ejercicios:

Hacer los ejercicios en la sección 14.3.6.1 del libro en español Buscar 3 funciones en stringi y aplica y explica las funciones con un set de datos.

22.21 El texto de Quijote

22.21.1 how to import a .txt file in R

22.22. CUANTAS PALABRAS TIENE EL PRIMER CAPÍTULO DE QUIJOTE?233

```
quijote <- read_lines("Datos/el_quijote.txt")

head(quijote, n=10)

## [1] "DON QUIJOTE DE LA MANCHA"
## [2] "Miguel de Cervantes Saavedra"
## [3] ""
## [4] "PRIMERA PARTE"
## [5] "CAPÍTULO 1: Que trata de la condición y ejercicio del famoso hidalgo D. Quijote de la Ma
## [6] "En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que viv
## [7] "Tuvo muchas veces competencia con el cura de su lugar (que era hombre docto graduado en
## [8] "En resolución, él se enfrascó tanto en su lectura, que se le pasaban las noches leyendo
## [9] "historia más cierta en el mundo."
## [10] "Decía él, que el Cid Ruy Díaz había sido muy buen caballero; pero que no tenía que ver c
```

22.21.2 Cuantas oraciones tiene el text primer capitulo de Quijote?

```
length(quijote)
```

```
## [1] 2186
```

22.22 Cuantas palabras tiene el primer capítulo de Quijote?

```
quijote_palabras <- str_split(quijote, " ")

quijote_palabras_vector <- unlist(quijote_palabras)

head(quijote_palabras_vector) # observar las primeras palabras

## [1] "DON" "QUIJOTE" "DE" "LA" "MANCHA" "Miguel"
length(quijote_palabras_vector) # cuantas palabras hay en total

## [1] 187019
```

22.23 Cuantas veces aparece la palabra “caballero” en el primer capitulo de Quijote?

```
sum(str_detect(quijote_palabras_vector, "caballero"))

## [1] 475
```

22.24 ¿Cuántas palabras tienen más de 10 letras?

```
sum(str_length(quijsote_palabras_vector) > 10)
```

```
## [1] 5326
```

22.25 Cual es la palabra más larga

```
quijsote_palabras_vector %>%  
  str_length() %>%  
  which.max() %>%  
  quijsote_palabras_vector[.]
```

```
## [1] "procuremos.Levántate,"
```

22.26 Cuántas veces aparece “Sancho Panza” en el primer capítulo de Quijote?

```
sum(str_detect(quijsote, "Sancho Panza"))
```

```
## [1] 95
```

22.27 Cuántas veces aparece “Dios” en el primer capítulo de Quijote?

```
sum(str_detect(quijsote, "Dios"))
```

```
## [1] 166
```

```
# Cuántas palabras tiene “ñ” en cualquier posición de la palabra?  
!str_detect(palabras, "[aáeéííoóuúüü]")
```

```
str_detect(quijsote_palabras_vector, "e") |> head() # cuántas veces aparece la letra e en to
```

```
## [1] FALSE FALSE FALSE FALSE FALSE TRUE
```

```
sum(str_detect(quijsote_palabras_vector, "e")) # cuántas veces aparece la letra e en to
```

```
## [1] 96484
```

22.28 Cuantas palabras tienen ñ?

```
sum(str_detect(quijote_palabras_vector, "[aáeéííoóuúü]")) # cuantas veces aparece la letra ñ en
## [1] 177633
sum(str_detect(quijote_palabras_vector, "[ñ]")) # cuantas veces aparece la letra ñ en total
## [1] 49227
```

22.29 Remove del texto todas las siguientes palabras

las palabras como “que”, “de”, “y”, “la”, “el”, “en”, “a”, “los”, “del”, “se”, “las”, “un”, “por”, “con”, “no”, “una”, “su”, “para”, “es”, “al”, “lo”, “como”, “más”, “pero”, “sus”, “le”, “ya”, “o”, “este”, “sí”, “porque”, “esta”, “entre”, “cuando”, “muy”, “sin”, “sobre”, “también”, “me”, “hasta”, “hay”, “donde”, “quien”, etc.

```
stop_words <- c("que", "de", "y", "la", "el", "en", "a", "los", "del", "se",
               "las", "un", "por", "con", "no", "una", "su", "para", "es",
               "al", "lo", "como", "más", "pero", "sus", "le", "ya", "o",
               "este", "sí", "porque", "esta", "entre", "cuando", "muy",
               "sin", "sobre", "también", "me", "hasta", "hay", "donde",
               "quien")

quijote_palabras_vector_limpio <- quijote_palabras_vector[!str_detect(quijote_palabras_vector,
                              str_c("^", stop_words, "$", collapse = "|"))]

library(cranlogs)
cranlogs_badge("ggverssa", summary = "grand-total")

## [1] "[! [CRAN RStudio mirror downloads] (https://cranlogs.r-pkg.org/badges/grand-total/ggversa?
cran_downloads(packages = "ggverssa", when = "last-month")

##           date count package
## 1 2026-07-02      8 ggverssa
## 2 2026-07-03     13 ggverssa
## 3 2026-07-04     15 ggverssa
## 4 2026-07-05      3 ggverssa
## 5 2026-07-06     10 ggverssa
## 6 2026-07-07     13 ggverssa
## 7 2026-07-08      4 ggverssa
## 8 2026-07-09      6 ggverssa
## 9 2026-07-10      4 ggverssa
## 10 2026-07-11     2 ggverssa
## 11 2026-07-12      6 ggverssa
```

```
## 12 2026-07-13      9 ggversa
## 13 2026-07-14      9 ggversa
## 14 2026-07-15      6 ggversa
## 15 2026-07-16     14 ggversa
## 16 2026-07-17      4 ggversa
## 17 2026-07-18      5 ggversa
## 18 2026-07-19      5 ggversa
## 19 2026-07-20      3 ggversa
## 20 2026-07-21      3 ggversa
## 21 2026-07-22      5 ggversa
## 22 2026-07-23      3 ggversa
## 23 2026-07-24      5 ggversa
## 24 2026-07-25      2 ggversa
## 25 2026-07-26      1 ggversa
## 26 2026-07-27      3 ggversa
## 27 2026-07-28      2 ggversa
## 28 2026-07-29      6 ggversa
## 29 2026-07-30      1 ggversa
## 30 2026-07-31      7 ggversa

# Replace start date with the earliest date ggversa is available on CRAN
df <- cran_downloads("ggversa", from = "2017-08-05", to = Sys.Date())

# Sum all daily downloads
total_downloads <- sum(df$count, na.rm = TRUE)
print(total_downloads)

## [1] 36576

# Desactivado para el build: install_github requiere credenciales de GitHub.
#install.packages("devtools")
devtools::install_github("arunsrinivasan/cran.stats")
library(cran.stats)

# Get logs for a specific day
dt <- read_logs(start = as.Date("2025-11-09"),
                 end   = as.Date("2025-11-09"))

# Filter for ggversa and summarize downloads by country
library(data.table)
dt <- as.data.table(dt)
dt_ggversa <- dt[package == "ggversa", .N, by = country]
print(dt_ggversa)

#install.packages("packageRank")
library(packageRank)
```

```

# Define target countries
target_countries <- c(
  "United States", "Puerto Rico", "Spain", "Mexico", "Argentina", "Colombia", "Chile", "Peru",
  "Venezuela", "Ecuador", "Guatemala", "Cuba", "Bolivia", "Honduras", "Paraguay", "El Salvador",
  "Nicaragua", "Costa Rica", "Panama", "Uruguay", "Dominican Republic", "Equatorial Guinea"
)

# Downloads for ggversa by country (latest day)
dates <- seq(as.Date("2025-10-01"), as.Date("2025-11-09"), by = "day")
results <- lapply(dates, function(d) {
  tryCatch({
    df <- packageCountry(packages = "ggversa", date = d)
    df$date <- d
    df
  }, error = function(e) NULL)
})

# Combine and filter
country_data <- bind_rows(results)
filtered_data <- country_data %>% filter(country %in% target_countries)

# Summarize
summary <- filtered_data %>%
  group_by(country) %>%
  summarise(total_downloads = sum(downloads)) %>%
  arrange(desc(total_downloads))

print(summary)

```

22.30 Contabilizar las palabras más comunes en el primer capítulo de Quijote

```

table(quijote_palabras_vector_limpio) %>%
  sort(decreasing = TRUE) %>%
  head(40)

```

```
## quijote_palabras_vector_limpio
##      si      más      mi      yo      tan      don      había      él
##      899      882      851      814      750      713      653      627
##      ni      todo      ser      ha      era      bien      vuestra      Y
##      617      509      466      460      452      451      445      407
##      todos      dijo      Don      fue      te      cual      así      sino

```

```
##      372      348      345      343      340      326      321      313
##      esto Sancho      que, Quijote      aquel merced      dos      hacer
##      312      312      310      310      293      286      282      279
##      señor      ella aunque Quijote,      otra      he      nos      cosa
##      275      269      265      265      262      246      241      240
```

```
clean=str_replace_all(quijote_palabras_vector_limpio, "[,\\.\\!\\:\\'\\\"\\?]", "")
```

```
table(clean) %>%
  sort(decreasing = TRUE) %>%
  head(40)
```

```
## clean
##      más      si      yo      mi      tan      él      don      había
##      958      917      886      876      750      737      718      668
##      Quijote      ni      dijo      Sancho      todo      bien      ser      así
##      668      620      579      560      560      507      503      489
##      era      esto      ha      vuestra      Y      todos      cual      merced
##      475      462      460      453      422      416      399      395
##      señor      fue      ella      Don      pues      te      sino      hacer
##      363      358      347      345      344      340      316      312
##      que caballero      dos      aquel      otra      cosa      aunque      decir
##      310      309      299      296      280      274      265      264
```

```
\\b[^\s]*;[^\s]*\\b matches:
```

```
\\b: word boundary
[^\s]*: any number of non-space characters
;: the semicolon
[^\s]*: more non-space characters
\\b: word boundary
```

```
words_with_semicolon <- str_extract_all(clean, "\\b[^\s]*;[^\s]*\\b")[[1]]
```

```
words_with_semicolon
```

```
## character(0)
```

Contabilizar la cantidad de veces los numeros aparecen en el texto

```
str(clean)
```

```
## chr [1:111341] "DON" "QUIJOTE" "DE" "LA" "MANCHA" "Miguel" "Cervantes" ...
```

```
library(purrr)
```

```
numeros <- c("uno", "dos", "tres", "cuatro", "cinco", "seis", "siete", "ocho", "nueve"
```

```
clean <- str_replace_all(clean, "[[:punct:]]", "")
```

```
clean_text <- paste(clean, collapse = " ")
counts <- sapply(numeros, function(n) str_count(tolower(clean_text), fixed(n)))
names(counts) <- numeros
print(counts)
```

```
##      uno      dos      tres cuatro cinco      seis siete      ocho nueve      diez
##      445     1563      117      72      13      31      9      17      5      18
```

```
## [1] "*** START OF THE PROJECT GUTENBERG EBOOK 11 ***"
## [2] ""
## [3] "[Illustration]"
## [4] ""
## [5] ""
## [6] ""
## [7] ""
## [8] "Alice's Adventures in Wonderland"
## [9] ""
## [10] "by Lewis Carroll"

## [1] "childhood: and how she would gather about her other little children,"
## [2] "and make _their_ eyes bright and eager with many a strange tale,"
## [3] "perhaps even with the dream of Wonderland of long ago: and how she"
## [4] "would feel with all their simple sorrows, and find a pleasure in all"
## [5] "their simple joys, remembering her own child-life, and the happy summer"
## [6] "days."
## [7] ""
## [8] "THE END"
## [9] ""
## [10] "*** END OF THE PROJECT GUTENBERG EBOOK 11 ***"
```

22.30.1 Cuántas oraciones tiene el text de Alice?

```
## [1] 3384
```

22.30.2 Cuántas palabras tiene el texto de Alice?

```
## [1] "***"      "START"      "OF"      "THE"      "PROJECT"      "GUTENBERG"
## [1] 28017
```

22.30.3 Convertir todo a minúsculas

Determinar cuántas veces aparece la palabra “alice” en el texto

```
## [1] 399
```

22.30.4 ¿Cuántas palabras tienen más de 10 letras?

```
## [1] 391
```

22.30.5 Remover todos las puntuaciones en el texto, usando “[[:punct:]]”

```
## [1] "" "start" "of" "the" "project" "gutenberg"
```

22.30.6 Haz una lista de las 20 palabras más comunes en el texto de Alice

```
## Alice_palabras_vector_limpio
## the and to a she it of said i alice in you
## 1640 1538 846 721 632 537 526 511 462 401 385 367 360
## was that as her at on with
## 357 276 262 248 209 193 181
```

Remover los espacios vacíos

Haga una lista de las 20 palabras más comunes en el texto de Alice sin los espacios vacíos, para averguar que removiste los espacios vacíos

```
## Alice_palabras_vector_limpio
## the and to a she it of said i alice in you was
## 1640 846 721 632 537 526 511 462 401 385 367 360 357
## that as her at on with all
## 276 262 248 209 193 181 179
```

Ahora remueve todos los stopwords en ingles del texto de Alice usando la libreria stopwords

Ahora enseñe las 20 palabras más comunes en el texto de Alice sin los stopwords

```
## Alice_palabras_vector_limpio2
## said alice little one know like went thought queen time
## 462 385 129 101 85 85 83 74 68 68
## see king dont well began im mock now turtle gryphon
## 66 61 60 60 58 57 57 57 56 55
```

Using the following list of main characters in the book, count how many times each character is mentioned in the text of Alice in Wonderland

- Alice
- White Rabbit
- Mad Hatter
- Hatter
- Queen of Hearts
- Queen
- Cheshire Cat

22.30. CONTABILIZAR LAS PALABRAS MÁS COMUNES EN EL PRIMER CAPITULO DE QUIJOTE241

- March Hare
- Caterpillar
- Dormouse
- King of Hearts
- Gryphon
- Mock Turtle
- Dodo
- Bill the Lizard
- Knave of Hearts

```
characters <- c("Alice", "White Rabbit", "Rabbit", "Mad Hatter", "Hatter", "Queen of Hearts", "Qu  
               "Cheshire Cat", "Cat", "March Hare", "Caterpillar", "Dormouse",  
               "King of Hearts", "King", "Gryphon", "Mock Turtle", "Dodo", "Knave of Hearts")
```

##	Alice	White Rabbit	Rabbit	Mad Hatter	Hatter
##	399	22	54	0	57
##	Queen of Hearts	Queen	Cheshire Cat	Cat	March Hare
##	4	77	6	90	31
##	Caterpillar	Dormouse	King of Hearts	King	Gryphon
##	29	40	0	179	55
##	Mock Turtle	Dodo	Knave of Hearts		
##	57	13	3		

Chapter 23

Ecuaciones Matemáticas

[1] "2026-08-01"

23.1 Ecuaciones matemáticas

TeX es la mejor manera de escribir funciones matemáticas en R. Donald Knuth diseñó el paquete *T_EX* cuando se sintió frustrado por el tiempo que les tomaba a los tipógrafos y terminar su libro, que contenía muchas funciones matemáticas. Una característica interesante de *R Markdown* es su capacidad para leer código LaTeX directamente.

Si está haciendo una tesis, artículo científico, presentaciones o quiere aclarar en su documento las funciones que estás usando, típicamente esto involucrará muchas ecuaciones matemáticas. Por consecuencia querrá leer la siguiente sección que ha sido comentada.

El sistema **TeX** fue creado por Donald Knuth en 1978 porque no le convenía la calidad tipográfica de su libro *The Art of Computer Programming*. **LaTeX**, la variante que usamos hoy, fue construida sobre **TeX** por Leslie Lamport.

23.1.1 Matemática y Física

El promedio $\bar{x} = \frac{\sum x_i}{n}$ se escribe como `$\bar{x} = \frac{\sum x_i}{n}$` . Nota que para poner una ecuación en una línea se usa solamente una `$` antes y después, si queremos que sea en una línea aparte se usa dos `$$` antes y después.

$$\bar{x} = \frac{\sum x_i}{n}$$

$$\sum_{j=1}^n (\delta\theta_j)^2 \leq \frac{\beta_i^2}{\delta_i^2 + \rho_i^2} \left[2\rho_i^2 + \frac{\delta_i^2 \beta_i^2}{\delta_i^2 + \rho_i^2} \right] \equiv \omega_i^2$$

De Informational Dynamics, tenemos lo siguiente (Dave Braden):

Después de n tales encuentros, la densidad posterior de θ es

$$\pi(\theta|X_1 < y_1, \dots, X_n < y_n) \propto \pi(\theta) \prod_{i=1}^n \int_{-\infty}^{y_i} \exp\left(-\frac{(x-\theta)^2}{2\sigma^2}\right) dx$$

Otra Ecuaciones

$$\det \begin{vmatrix} c_0 & c_1 & c_2 & \cdots & c_n \\ c_1 & c_2 & c_3 & \cdots & c_{n+1} \\ c_2 & c_3 & c_4 & \cdots & c_{n+2} \\ \vdots & \vdots & \vdots & & \vdots \\ c_n & c_{n+1} & c_{n+2} & \cdots & c_{2n} \end{vmatrix} > 0$$

Lapidus y Píndaro, Solución numérica de ecuaciones diferenciales parciales en ciencia y Ingeniería. Página 54

$$\int_t \left\{ \sum_{j=1}^3 T_j \left(\frac{d\phi_j}{dt} + k\phi_j \right) - kT_e \right\} w_i(t) dt = 0, \quad i = 1, 2, 3.$$

Funciones de ponderación del método L&P Galerkin. Página 55

$$\sum_{j=1}^3 T_j \int_0^1 \left\{ \frac{d\phi_j}{dt} + k\phi_j \right\} \phi_i dt = \int_0^1 k T_e \phi_i dt, \quad i = 1, 2, 3$$

Otra de L&P (p145)

$$\int_{-1}^1 \int_{-1}^1 \int_{-1}^1 f(\xi, \eta, \zeta) = \sum_{k=1}^n \sum_{j=1}^n \sum_{i=1}^n w_i w_j w_k f(\xi, \eta, \zeta).$$

Otra de L&P (p126)

$$\int_{A_e} (\cdot) dx dy = \int_{-1}^1 \int_{-1}^1 (\cdot) \det[J] d\xi d\eta.$$

23.2 Símbolos de LATEX

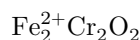
Pueden encontrar los símbolos de Latex matemáticos en la siguiente pagina web:
https://oeis.org/wiki/List_of_LaTeX_mathematical_symbols

23.3 Química 101: Símbolos

Las fórmulas químicas se verán mejor si no están en cursiva. Evite la cursiva automática del modo matemático en LaTeX usando el argumento `$\mathrm{fórmula\ aquí}$` , con su fórmula dentro de las llaves. (Observe el uso de comillas invertidas aquí que encierran texto y que actúa como código para ver el código original).

Entonces, $\text{Fe}_2^{2+}\text{Cr}_2\text{O}_4$ es escrito como `$\mathrm{Fe_2^{2+}Cr_2O_4}$` .

Esta fórmula es parte de la frase



- Exponente or Sobrescrito: O^- y se escribe como `$\mathrm{O^{-}}$`
- Subíndice: CH_4 y se escribe como `$\mathrm{CH_4}$`
- Para apilar números o letras como en Fe_2^{2+} , primero se define el subíndice y luego el superíndice y se escribe como `$\mathrm{Fe_2^{2+}}$` .
- punto se usa la palabra **bullet**:



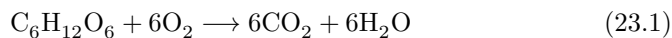
donde se escribe como `$\text{CuCl} \bullet 7\text{H}_2\text{O}$`

23.4 Símbolos comunes

- Delta: Δ y se escribe como `$\Delta$`
- Gamma mayúscula Γ
- Gamma minúscula γ
- Aquí está mi súper fórmula de ϕ que voy a usar en el examen de matemáticas.
- Flecha de Reacción: $\longrightarrow$ or $\xrightarrow{\text{fabilous}}$ y se escribe como `$\longrightarrow$` or `$\xrightarrow{\text{solution}}$`
- Flecha de Resonancia: $\leftrightarrow$ y se escribe como `$\leftrightarrow$`
- Flechas de reacción reversibles: $\rightleftharpoons$ y se escribe como `$\rightleftharpoons$`

23.4.1 Tipografía de reacciones

Es posible que desees poner tu reacción en un entorno de ecuaciones, lo que significa que LaTeX colocará la reacción donde encaje y numerará las ecuaciones por ti.



y se escribe como

```
\begin{equation}
  \mathrm{C_6H_{12}O_6 + 6O_2} \longrightarrow \mathrm{6CO_2 + 6H_2O}
  (\#eq:reaction)
\end{equation}
```

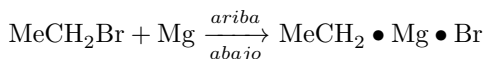
Podemos hacer referencia a esta reacción de combustión de glucosa mediante la ecuación (23.1).

23.4.2 Otros ejemplos de reacciones



y se escribe como

```
$\mathrm{NH_4Cl_{(s)}}$ $\rightleftharpoons$ $\mathrm{NH_{3(g)}+HCl_{(g)}}$
```



y se escribe como

```
$\mathrm{MeCH_2Br + Mg}$ $\xrightarrow[abajo]{arriba}$ $\mathrm{MeCH_2\bullet}$
Mg $\bullet$ Br$
```

23.5 Física

Muchos de los símbolos que necesitará se pueden encontrar en la página de matemáticas.

<https://web.reed.edu/cis/help/latex/math.html> y la guía completa de símbolos LaTeX (<https://mirror.utexas.edu/ctan/info/symbols/comprehensive/symbols-letter.pdf>).

23.6 Biología

Probablemente encontrará los recursos en <https://www.lecb.ncifcrf.gov/~toms/latex.html> útil, particularmente los enlaces a bsts de varias revistas. Quizás también le interese TeXShade para composición tipográfica de nucleótidos (<https://www.lecb.ncifcrf.gov/~toms/latex.html>).

//homepages.uni-tuebingen.de/beitz/txe.html). Asegúrese de leer el capítulo anterior sobre gráficos y tablas.

La fórmula del promedio

$\mu = \frac{\sum x_i}{n}$ se escribe así `\mu=\frac{\sum{x}_{i}}{n}`

NOTE la diferencia en tipo de letra.....entre las dos fórmulas

Nota aquí que la formula se escribe en la oración $\mu = \frac{\sum x_i}{n}$ se escribe así `\mathrm{\mu=\frac{\sum{x}_{i}}{n}}`

Pero si quiere que la formula este en una linea aparte lo escribe así dentro de `\begin{equation}`, la formula aquí y termina con `\end{equation}`

$$= \frac{\sum x_i}{n} \quad (23.2)$$

$$s^2 = \frac{\sum (x_i - \bar{x})^2}{n-1}$$

Sitios en el web para preparar las ecuaciones Latex

1. <https://latex.codecogs.com/eqneditor/editor.php>
2. <https://latexeditor.lagrida.com>
3. <https://www.hostmath.com>
4. https://www.overleaf.com/learn/latex/Mathematical_expressions

Chapter 24

Opciones de Knitr

```
## [1] "2026-08-01"
```

La mayoría de los ejemplos y ideas de esta sección proviene de **yihui** en el siguiente website <<https://yihui.org/knitr/>>

el paquete **knitr** fue diseñado para ser transparente para generar reporte de R, añadiendo componente de animación y Latex (Ecuaciones) y otras funciones.

Hay muchas opciones para determinar lo que hace un **chunk**

- Code evaluation: evaluación de código
- Text output: Formateo de texto
- Code decoration: Decoración del código
- Cache: Cache
- Plots: Gráficos
- Animation: Animaciones
- Code chunk: Código del chunk
- Child Document: Documento asociados
- Language engines: Asociación de lenguaje
- Option Template:
- Extracting source code:
- Other chunk options:
- Package Options:
- Global R Options:

24.1 Para ver las opciones de knitr de los chunk

knitr fue creado por **Yihui Xie** en 2012; es el motor que ejecuta el código de R dentro de los documentos R Markdown y “teje” (*knit*) el resultado junto con el texto. Sustituyó a Sweave con una sintaxis más flexible.

Opciones de los chunks — controlan qué se muestra.

- **echo=FALSE**: oculta el código (muestra solo el resultado).
- **eval=FALSE**: muestra el código pero no lo ejecuta.
- **include=FALSE**: ejecuta el código pero no muestra nada.
- **warning=FALSE** / **message=FALSE**: ocultan avisos y mensajes.
- **fig.width** / **fig.height** / **out.width**: tamaño de las figuras.

Este **chunk** es para que la información de los chunk en la parte de **knitr** se vea.

```
library(knitr)
hook_chunk = knitr_hooks$get('chunk')
knitr_hooks$set(chunk = function(x, options) {
  if (!is.null(options$echo_opts)) {
    return(paste0("~~~~~ {r ", options$params.src, "} ~~~~~", x, "~~~~~"))
  } else {
    return(hook_chunk(x, options)) # pass to default hook
  }
})

opts_knit$set(eval.after = 'fig.cap')
```

Los ejemplos aquí serán limitados a las alternativas para los documentos **.Rmd**

- Las opciones están escrita de forma de **tag=selección de la alternativa**.

Opciones de bloques

Se personaliza los bloques con “opciones” **options** de los chunks.

En la rueda de a la derecha de cada **chunk** hay opciones para seleccionar opciones.

24.2 Tablas de opciones principales

Opción	Ejecuta	Muestra	Output	Gráficos	Mensajes	Advertencias
eval=FALSE	-		-	-	-	-
include=FALSE		-	-	-	-	-
echo=FALSE		-				
results=“hide”			-			
fig.show =“hide”				-		
message=FALSE					-	
warning=FALSE						-
error=FALSE						-

- Comenzando con dándole un nombre al chunk “mi-chunk”

Ponerle nombre a cada chunk (como `mi-chunk`) facilita localizar errores, navegar por el documento y reutilizar el caché de los cálculos costosos.

```
library(tidyverse)
```

```
algun_nombre, echo=TRUE
```

Eso es lo que va a ver cuando hace el knit

```
library(tidyverse)
```

Nota aquí tengo `include=FALSE`, por consecuencia cuando se hace un knit, no se ve lo que hay en el chunk.

- Ahora cambialo a `TRUE`

```
head(cars)
```

```
##    speed dist
## 1      4     2
## 2      4    10
## 3      7     4
## 4      7    22
## 5      8    16
## 6      9    10
```{r c22-7, include=TRUE, echo_opts=TRUE}
```

```
head(cars)
```

```
speed dist
1 4 2
2 4 10
3 7 4
4 7 22
5 8 16
6 9 10
```
```

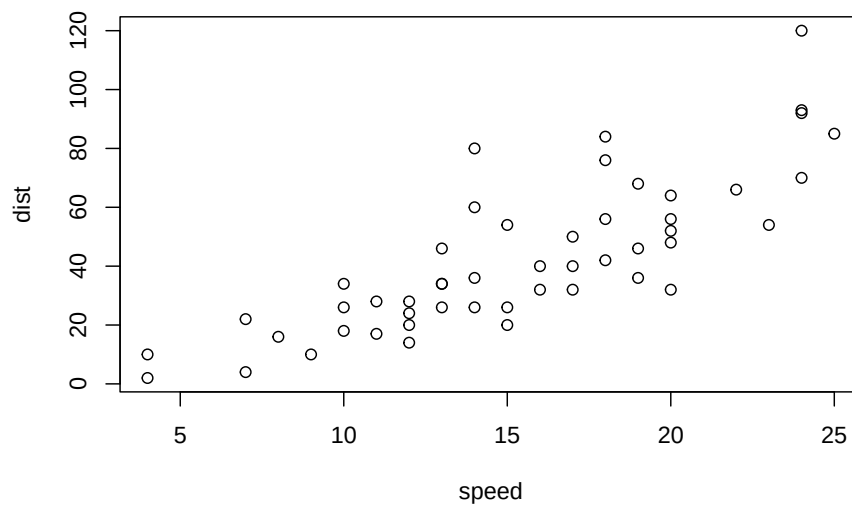
Añade una figura

```
data=c(1:100)
```

```
mean(data)
```

```
## [1] 9.6
```

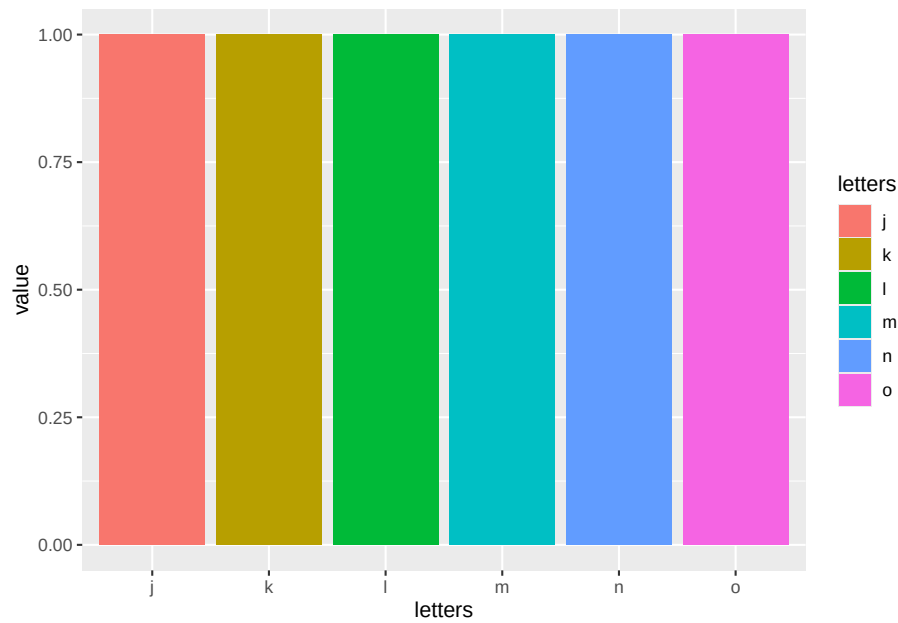
```
```{r c22-11, include=TRUE, fig.cap='El nombre de mi figura.',
fig.topcaption=TRUE, echo_opts=TRUE}
plot(cars)
```



```
...
```

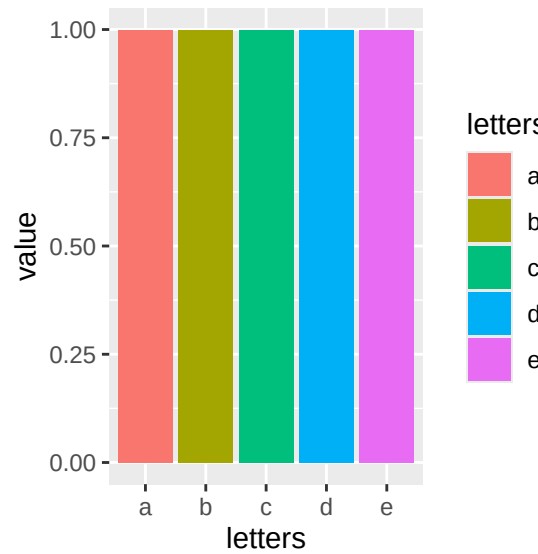
```
```{r c22-12, include=TRUE, fig.cap= "An incredible figure",
echo_opts=TRUE}
library(ggplot2)
df <- data.frame(letters = letters[10:15], value = 1)

ggplot(df, aes(letters, value, fill = letters)) +
  geom_bar(stat = "identity")
```



...

```
```{r c22-13, include=TRUE, fig.cap= "An incredible figure",
```



```
fig.width = 3, fig.height = 3, echo_opts=TRUE, echo=FALSE}
```
```

Figure alignment: default, left, right, and center

fig.align

```
```{r c22-14, include=TRUE, fig.cap= "caption", fig.width = 3,
fig.height = 3, fig.align='right', echo_opts=TRUE}
```

```
library(ggplot2)
df <- data.frame(letters = letters[1:5], value = 1)

ggplot(df, aes(letters, value, fill = letters)) +
 geom_bar(stat = "identity")
```

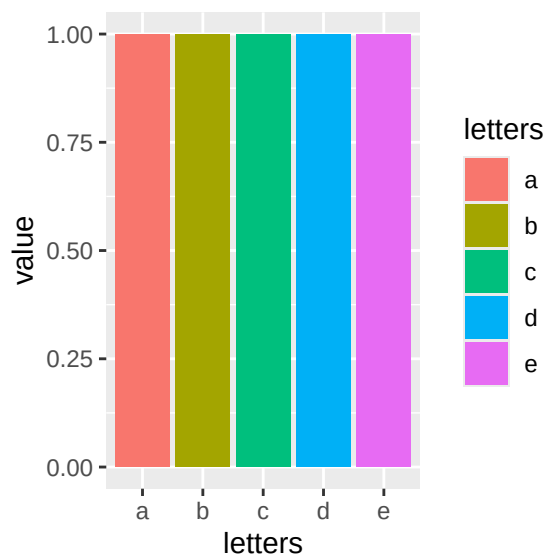


Figure 24.1: caption

...

Wrap text around figure

Use ese código para que el texto “wraps” alrededor de las figura (ese código es de otro idioma de computadora de se llama **css**)

AQUI el CHUNK

#### 24.2.0.0.1 R Markdown wrapping código css

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

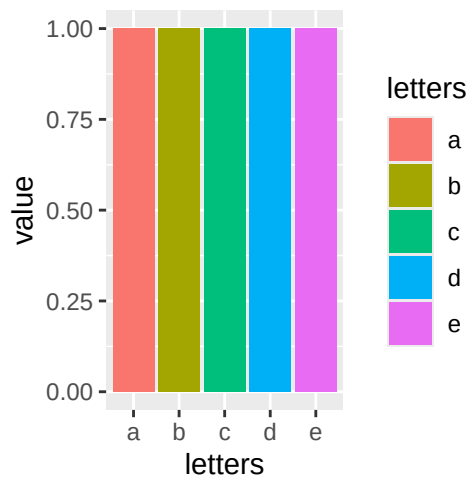


Figure 24.2: caption

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Note that the `echo = TRUE` parameter was added to the code chunk to have printing of the R code that generated the plot.

### 24.2.1 OTRAS funciones de knitr

- `eval=` to evaluate the code FALSE or TRUE
- `echo =` TO show the code FALSE or TRUE
- `message =` FALSE or TRUE
- `warning =` FALSE or TRUE
- `error =`FALSE or TRUE
- `results = 'asis'` To show text exactly as printed/written

```
[1] 1 2 3 4 5 6 7 8 9 10
```

```
library(MASS)
```

```
library(dplyr)
```

```
library(MASS)
```

```
cat("I'm raw **Markdown** content.\n",
 "\n",
```

```
" My poetry of Puerto Rico\n")
```

I'm raw **Markdown** content.

My poetry of Puerto Rico

---



---

## 24.3 Emoji or Symbols

### 24.3.1 Select Under the Edit Emoji & Symbols.

**24.3.1.0.1 Remember you can add emoji's to your document** Sometimes you want to around a bit and add some to your document. Well we have a gift for you if you add interesting emoji's to your document, above all if you are a .



## Chapter 25

# Fechas, horas y minutas

```
[1] "2026-08-01"
```

---

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/dates-and-times.html>

Español: <https://es.r4ds.hadley.nz/16-datetimes.html>

---

### 25.1 Temas:

- `library(lubridate)`
- `library(datos)`
- `library(tidyverse)`

### 25.2 Creando fechas/horas

En R y en los sistemas tipo Unix el tiempo se cuenta como el número de segundos (o días) transcurridos desde la medianoche del 1 de enero de 1970, conocida como la época Unix. Por eso `as.numeric()` de una fecha devuelve los días desde ese punto.

- `date`,
- `time`,
- `date-time`,

<https://github.com/edgararuiz/guias-rapidas/blob/master/fechas.pdf>

```
library(tidyverse)
library(lubridate) # paquete para convertir fechas

tidyverse_packages() # lista completa de los paquetes activado con tidyverse

[1] "broom" "conflicted" "cli" "dbplyr"
[5] "dplyr" "dtplyr" "forcats" "ggplot2"
[9] "googledrive" "googlesheets4" "haven" "hms"
[13] "httr" "jsonlite" "lubridate" "magrittr"
[17] "modelr" "pillar" "purrr" "ragg"
[21] "readr" "readxl" "reprex" "rlang"
[25] "rstudioapi" "rvest" "stringr" "tibble"
[29] "tidyr" "xml2" "tidyverse"
```

### 25.3 El paquete lubridate

`ymd()`, `mdy()`, `dmy()` (**lubridate**) — convertir texto a fechas.

- **Qué hacen:** interpretan una cadena según el **orden** de sus partes:  
`ymd("2024-03-01")`, `mdy("3/1/2024")`, `dmy("1/3/2024")`.
- **Con hora:** añade `_hms`, `_hm` o `_h` (p. ej. `dmy_hms()`).
- **Extraer partes:** `year()`, `month()`, `day()`, `wday()` (día de la semana).

La función `today()` regresa la fecha de hoy, similar a `Sys.date()`

```
today()
```

```
[1] "2026-08-01"
```

Use la función de `tribble()`, del paquete `tibble`, para crear una tabla manualmente

```
fechas <- tribble(
 ~ codigo, ~ fecha,
 "001", "01/01/2019 00:00:00",
 "002", "31/03/2019 01:05:00",
 "003", "14/06/2019 20:00:00",
 "004", "01/12/1859 11:32:13",
 "005", "01/01/0501 05:04:33"
)
```

```
fechas
```

```
A tibble: 5 x 2
codigo fecha
<chr> <chr>
1 001 01/01/2019 00:00:00
2 002 31/03/2019 01:05:00
3 003 14/06/2019 20:00:00
```

```
4 004 01/12/1859 11:32:13
5 005 01/01/0501 05:04:33
```

Hay varias opciones para convertir texto a fechas, el más usado es `mdy_hms()`, pero en este caso el formato utilizado no funciona bien.

Un error muy común es usar `mdy()` cuando las fechas están en formato día/mes/año. Como no existe un mes 31, obtendrás `NA` o fechas equivocadas; elige la función (`dmy()`, `mdy()`, `ymd()`) según el orden real de la fecha.

### 25.3.1 Un error típico

```
fechas %>%
 mutate(fecha = mdy_hms(fecha))
```

```
A tibble: 5 x 2
codigo fecha
<chr> <dtm>
1 001 2019-01-01 00:00:00
2 002 NA
3 003 NA
4 004 1859-01-12 11:32:13
5 005 0501-01-01 05:04:33
```

Ya que el día es el primero, y no el mes, usamos `dmy_hms()`

```
nueva_fechas <- fechas %>%
 mutate(fecha = dmy_hms(fecha))
```

```
nueva_fechas
```

```
A tibble: 5 x 2
codigo fecha
<chr> <dtm>
1 001 2019-01-01 00:00:00
2 002 2019-03-31 01:05:00
3 003 2019-06-14 20:00:00
4 004 1859-12-01 11:32:13
5 005 0501-01-01 05:04:33
```

lubridate tiene varias funciones para extraer partes de la fecha, por ejemplo: año, mes, día, hora, minuto, y cuatrimestre.

```
nueva_fechas %>%
 mutate(a = year(fecha),
 m = month(fecha),
 d = day(fecha),
 h = hour(fecha),
 mn = minute(fecha),
```

```

 q = quarter(fecha)
)

A tibble: 5 x 8
codigo fecha a m d h mn q
<chr> <dtm> <int> <int> <int> <int> <int> <int>
1 001 2019-01-01 00:00:00 2019 1 1 0 0 1
2 002 2019-03-31 01:05:00 2019 3 31 1 5 1
3 003 2019-06-14 20:00:00 2019 6 14 20 0 2
4 004 1859-12-01 11:32:13 1859 12 1 11 32 4
5 005 0501-01-01 05:04:33 501 1 1 5 4 1

```

Las funciones `round_date()`, `ceiling_date()` y `floor_date()` permiten redondear la fecha al número más cercano de la unidad especificada

```

nueva_fechas %>%
 mutate(
 redondear = round_date(fecha, unit = "month"), # round_date es redondea al mes más cercano
 techo = ceiling_date(fecha, unit = "day"), # ceiling_date redondea al día más cercano
 suelo = floor_date(fecha, unit = "month"), # floor_date redondea al mes más cercano
 suelo_hour = floor_date(fecha, unit = "hour") # redondea a la hora más cercana
)

```

```

A tibble: 5 x 6
codigo fecha redondear techo
<chr> <dtm> <dtm> <dtm>
1 001 2019-01-01 00:00:00 2019-01-01 00:00:00 2019-01-01 00:00:00
2 002 2019-03-31 01:05:00 2019-04-01 00:00:00 2019-04-01 00:00:00
3 003 2019-06-14 20:00:00 2019-06-01 00:00:00 2019-06-15 00:00:00
4 004 1859-12-01 11:32:13 1859-12-01 00:00:00 1859-12-02 00:00:00
5 005 0501-01-01 05:04:33 0501-01-01 00:00:00 0501-01-02 00:00:00
i 2 more variables: suelo <dtm>, suelo_hour <dtm>

```

## 25.4 Intervalos y duraciones

La función `interval()` crea un objeto R de intervalo de tiempo. En este caso, el intervalo entre la fecha en la tabla, y el día de hoy. Nota que esto puede ser muy útil si quiere calcular la cantidad de tiempo entre dos fechas.

```

nueva_fechas %>%
 mutate(intervalo = interval(fecha, today())) # intervalo entre la fecha y hoy

A tibble: 5 x 3
codigo fecha intervalo
<chr> <dtm> <Interval>
1 001 2019-01-01 00:00:00 2019-01-01 00:00:00 UTC--2026-08-01 UTC
2 002 2019-03-31 01:05:00 2019-03-31 01:05:00 UTC--2026-08-01 UTC

```

## 25.5. EL OPERADOR %--% SIMPLIFICA EL CÁLCULO DEL INTERVALO 261

```
3 003 2019-06-14 20:00:00 2019-06-14 20:00:00 UTC--2026-08-01 UTC
4 004 1859-12-01 11:32:13 1859-12-01 11:32:13 UTC--2026-08-01 UTC
5 005 0501-01-01 05:04:33 0501-01-01 05:04:33 UTC--2026-08-01 UTC
```

int\_length() regresa el número de segundos dentro del intervalo

```
nueva_fechas %>%
 mutate(
 intervalo = interval(fecha, today()), #
 segundos = int_length(intervalo) # número de segundos en el intervalo
)
```

```
A tibble: 5 x 4
codigo fecha intervalo segundos
<chr> <dtm> <Interval> <dbl>
1 001 2019-01-01 00:00:00 2019-01-01 00:00:00 UTC--2026-08-01 UTC 239241600
2 002 2019-03-31 01:05:00 2019-03-31 01:05:00 UTC--2026-08-01 UTC 231548100
3 003 2019-06-14 20:00:00 2019-06-14 20:00:00 UTC--2026-08-01 UTC 225000000
4 004 1859-12-01 11:32:13 1859-12-01 11:32:13 UTC--2026-08-01 UTC 5259472067
5 005 0501-01-01 05:04:33 0501-01-01 05:04:33 UTC--2026-08-01 UTC 48142666527
```

## 25.5 El operador %--% simplifica el cálculo del intervalo

Compara con el script anterior

```
nueva_fechas %>%
 mutate(intervalo = fecha %--% today())
```

```
A tibble: 5 x 3
codigo fecha intervalo
<chr> <dtm> <Interval>
1 001 2019-01-01 00:00:00 2019-01-01 00:00:00 UTC--2026-08-01 UTC
2 002 2019-03-31 01:05:00 2019-03-31 01:05:00 UTC--2026-08-01 UTC
3 003 2019-06-14 20:00:00 2019-06-14 20:00:00 UTC--2026-08-01 UTC
4 004 1859-12-01 11:32:13 1859-12-01 11:32:13 UTC--2026-08-01 UTC
5 005 0501-01-01 05:04:33 0501-01-01 05:04:33 UTC--2026-08-01 UTC
```

Para saber el número de días en el intervalo, divida el intervalo por la función que corresponde a días, days()

```
nueva_fechas %>%
 mutate(dias = fecha %--% today() / days(),
 anio = fecha %--% today() / years()) # número de días en el intervalo
```

```
A tibble: 5 x 4
codigo fecha dias anio
<chr> <dtm> <dbl> <dbl>
```

```
1 001 2019-01-01 00:00:00 2769 7.58
2 002 2019-03-31 01:05:00 2680. 7.34
3 003 2019-06-14 20:00:00 2604. 7.13
4 004 1859-12-01 11:32:13 60874. 167.
5 005 0501-01-01 05:04:33 557207. 1526.
```

*#Cuántos años de diferencia en el intervalo?*

Intervalo entre una lista de fecha y otra fecha y hora específica

```
nueva_fechas %>%
 mutate(dias = fecha %--% "2020-01-01 00:00:00" / days())
```

```
A tibble: 5 x 3
codigo fecha dias
<chr> <dtm> <dbl>
1 001 2019-01-01 00:00:00 365
2 002 2019-03-31 01:05:00 276.
3 003 2019-06-14 20:00:00 200.
4 004 1859-12-01 11:32:13 58470.
5 005 0501-01-01 05:04:33 554803.
```

## 25.6 Ejercicio

Usa esta función y calcula el número de día que ha transcurrido entre el día de su nacimiento y el día de hoy

```
segundos = "2018-08-14 00:00:00" %--% today() / seconds()
segundos
```

```
[1] 251337600
```

```
dias = "2018-08-14 00:00:00" %--% today() / days()
dias
```

```
[1] 2909
```

```
mes= "2018-08-14 00:00:00" %--% today() / months(1)
mes
```

```
[1] 95.58065
```

```
anio = "2018-08-14 00:00:00" %--% today() / years()
anio
```

```
[1] 7.964384
```

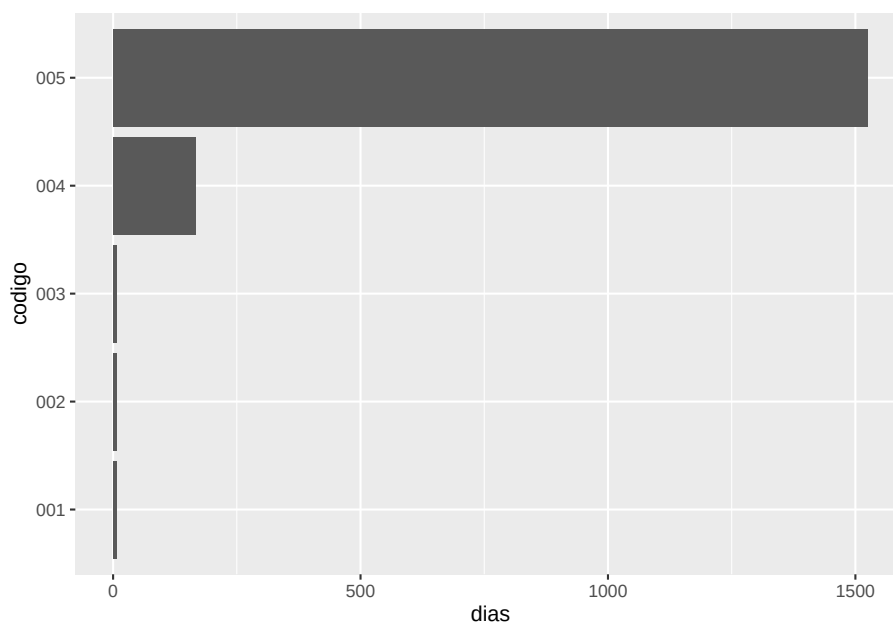
```
millisecond = "2018-08-14 00:00:00" %--% today() / milliseconds()
millisecond
```

## 25.7. SERIE DE FUNCIONES PARA CALCULAR LA DURACIÓN DE UN INTERVALO DE TIEMPO.263

```
[1] 251337600000
```

Los resultados se pueden visualizar usando `ggplot2`

```
nueva_fechas %>%
 mutate(dias = fecha %--% today() / years()) %>%
 ggplot() +
 geom_col(aes(codigo, dias)) +
 coord_flip()
```



## 25.7 Serie de funciones para calcular la duración de un intervalo de tiempo.

- `dyears()` # años
- `dmonths()` # meses
- `dweeks()` # semanas
- `ddays()` # días
- `dhours()` # horas
- `dminutes()` # minutos
- `dseconds()` # segundos
- `dmilliseconds()` # milisegundos
- `dmicroseconds()` # microsegundos
- `dnanoseconds()` # nanosegundos
- `dpicoseconds()` # picosegundos

## 25.8 as.duration

`as.duration()` crea un objeto en R que contiene la duración del intervalo de tiempo.

```
nueva_fechas %>%
 mutate(desde_hoy = as.duration(fecha %--% today())) # duración desde la fecha hasta i
```

```
A tibble: 5 x 3
codigo fecha desde_hoy
<chr> <dtm> <Duration>
1 001 2019-01-01 00:00:00 239241600s (~7.58 years)
2 002 2019-03-31 01:05:00 231548100s (~7.34 years)
3 003 2019-06-14 20:00:00 2.25e+08s (~7.13 years)
4 004 1859-12-01 11:32:13 5259472067s (~166.66 years)
5 005 0501-01-01 05:04:33 48142666527s (~1525.55 years)
```

El objeto de duración de tiempo se puede filtrar fácilmente basado en una variedad de tipo de tiempos. En este caso, `dweeks()` crea un objeto de duración de la largura especificada

```
nueva_fechas %>%
 mutate(desde_hoy = as.duration(fecha %--% today())) %>%
 filter(desde_hoy > dyears(7))
```

```
A tibble: 5 x 3
codigo fecha desde_hoy
<chr> <dtm> <Duration>
1 001 2019-01-01 00:00:00 239241600s (~7.58 years)
2 002 2019-03-31 01:05:00 231548100s (~7.34 years)
3 003 2019-06-14 20:00:00 2.25e+08s (~7.13 years)
4 004 1859-12-01 11:32:13 5259472067s (~166.66 years)
5 005 0501-01-01 05:04:33 48142666527s (~1525.55 years)
```

Otra opción es `ddays()`.

```
nueva_fechas %>%
 mutate(desde_hoy = as.duration(fecha %--% today())) %>%
 filter(desde_hoy < ddays(2500)) # Nota que se puede filtrar por el número de días q
```

```
A tibble: 0 x 3
i 3 variables: codigo <chr>, fecha <dtm>, desde_hoy <Duration>
```

### 25.8.1 Funciones

- `today()`
- `now()`



```
today()
```

```
[1] "2026-08-01"
```

```
now()
```

```
[1] "2026-08-01 23:32:04 AST"
```

### 25.8.2 Fechas con años, meses y días

- `ymd()`
- `ydm()`
- `mdy()`
- `myd()`
- `dmy()`
- `dym()`

### 25.8.3 Fechas con horas, minutos y segundos

- `ymd_hms()`
- `ymd_hm()`
- `ymd_h()`
- `ydm_hms()`
- `ydm_hm()`
- `etc`

### 25.8.4 Fechas

- `yq()` Year quarter (quarters are 1, 2, 3, 4, Jan-March, April-Jun, Jul-Sept, Oct-Dec)

```
x =c("2012.1", "1970.4")
```

```
yq(x)
```

```
[1] "2012-01-01" "1970-10-01"
```

```
yq("2012.1")
```

```
[1] "2012-01-01"
```

- **Cuidado** con el paquete **hms** que tiene funciones igual como **lubridate** (`hms`, `hm`, y `ms`) , pero no necesariamente hace lo mismo.
-

## 25.9 Crear una fecha desde columnas individuales

- `make_date()`
- `make_datetime()`

Unir el año, mes, día, hora y minutos que estén en diferentes columnas en uno

- sumamente practico cuando se tiene una base de datos con las fechas en diferentes columnas

```
library(datos)
head(vuelos)
```

```
A tibble: 6 x 19
anio mes dia horario_salida salida_programada atraso_salida
<int> <int> <int> <int> <int> <dbl>
1 2013 1 1 517 515 2
2 2013 1 1 533 529 4
3 2013 1 1 542 540 2
4 2013 1 1 544 545 -1
5 2013 1 1 554 600 -6
6 2013 1 1 554 558 -4
i 13 more variables: horario_llegada <int>, llegada_programada <int>,
atraso_llegada <dbl>, aerolinea <chr>, vuelo <int>, codigoCola <chr>,
origen <chr>, destino <chr>, tiempo_vuelo <dbl>, distancia <dbl>,
hora <dbl>, minuto <dbl>, fecha_hora <dtm>
```

```
vuelos %>%
 dplyr::select(anio, mes, dia, hora, minuto) %>%
 mutate(salida_dia=make_date(anio, mes, dia),
 salida_dia_minutos = make_datetime(anio, mes, dia, hora, minuto))
```

```
A tibble: 336,776 x 7
anio mes dia hora minuto salida_dia salida_dia_minutos
<int> <int> <int> <dbl> <dbl> <date> <dtm>
1 2013 1 1 5 15 2013-01-01 2013-01-01 05:15:00
2 2013 1 1 5 29 2013-01-01 2013-01-01 05:29:00
3 2013 1 1 5 40 2013-01-01 2013-01-01 05:40:00
4 2013 1 1 5 45 2013-01-01 2013-01-01 05:45:00
5 2013 1 1 6 0 2013-01-01 2013-01-01 06:00:00
6 2013 1 1 5 58 2013-01-01 2013-01-01 05:58:00
7 2013 1 1 6 0 2013-01-01 2013-01-01 06:00:00
8 2013 1 1 6 0 2013-01-01 2013-01-01 06:00:00
9 2013 1 1 6 0 2013-01-01 2013-01-01 06:00:00
10 2013 1 1 6 0 2013-01-01 2013-01-01 06:00:00
i 336,766 more rows
```

Unir el año, mes, día, hora que estén en diferentes columnas en uno

```
vuelos %>%
 dplyr::select(anio, mes, dia, hora, minuto) %>%
 mutate(salida2 = make_date(anio, mes, dia))
```

```
A tibble: 336,776 x 6
anio mes dia hora minuto salida2
<int> <int> <int> <dbl> <dbl> <date>
1 2013 1 1 5 15 2013-01-01
2 2013 1 1 5 29 2013-01-01
3 2013 1 1 5 40 2013-01-01
4 2013 1 1 5 45 2013-01-01
5 2013 1 1 6 0 2013-01-01
6 2013 1 1 5 58 2013-01-01
7 2013 1 1 6 0 2013-01-01
8 2013 1 1 6 0 2013-01-01
9 2013 1 1 6 0 2013-01-01
10 2013 1 1 6 0 2013-01-01
i 336,766 more rows
```

## Conversión de fechas que no son estándar

```
Datos_raros=tribble(~Ind, ~Persona, ~fecha_de_nacimiento,
 1, "Abuela", "1/1/30",
 2, "Abuelo", "1/30/01",
 3, "Tio", "1/30/09")
```

Datos\_raros

```
A tibble: 3 x 3
Ind Persona fecha_de_nacimiento
<dbl> <chr> <chr>
1 1 Abuela 1/1/30
2 2 Abuelo 1/30/01
3 3 Tio 1/30/09
```

```
Datos_raros %>%
 mutate(nueva_fecha = mdy(fecha_de_nacimiento))
```

```
A tibble: 3 x 4
Ind Persona fecha_de_nacimiento nueva_fecha
<dbl> <chr> <chr> <date>
1 1 Abuela 1/1/30 2030-01-01
2 2 Abuelo 1/30/01 2001-01-30
3 3 Tio 1/30/09 2009-01-30
```

```
as.Date(0, origin = "1970-01-01")
```

```
[1] "1970-01-01"
```

```
Returns "1970-01-01"
```

```
df <- tibble(
 Persona = c("Abuela", "Abuelo", "Tio"),
 fecha_de_nacimiento = c("1/1/1930", "1/30/1901", "1/30/1909")
)
```

```
df <- df %>%
 mutate(
 nueva_fecha = mdy(fecha_de_nacimiento),
 nueva_fecha = if_else(year(nueva_fecha) > year(today()),
 nueva_fecha - years(100),
 nueva_fecha)
)
df
```

```
A tibble: 3 x 3
Persona fecha_de_nacimiento nueva_fecha
<chr> <chr> <date>
1 Abuela 1/1/1930 1930-01-01
2 Abuelo 1/30/1901 1901-01-30
3 Tio 1/30/1909 1909-01-30
```

```
#Datos_raros$fecha2=as.Date(as.Date(format(as.Date(Datos_raros$fecha_de_nacimiento, fo
```

```
#Datos_raros
```

```
*** HERE ***
```

## 25.10 %/%: integer division

```
vuelos |> dplyr::select(tiempo_vuelo) |>
 mutate(t=tiempo_vuelo %/% 100) |> # Division
 mutate(m= tiempo_vuelo %% 100) # modulus... Cambia el 100 para 1, 10, 1000... y l
```

```
A tibble: 336,776 x 3
tiempo_vuelo t m
<dbl> <dbl> <dbl>
1 227 2 27
2 227 2 27
3 160 1 60
4 183 1 83
5 116 1 16
6 150 1 50
```

```
7 158 1 58
8 53 0 53
9 140 1 40
10 138 1 38
i 336,766 more rows
```

## 25.11 The %% operator returns the modulus (remainder) of a division operation.

- For instance, `5 %% 2` would return 1, as the remainder of 5 divided by 2 is 1.

```
names(vuelos)
```

```
[1] "anio" "mes" "dia"
[4] "horario_salida" "salida_programada" "atraso_salida"
[7] "horario_llegada" "llegada_programada" "atraso_llegada"
[10] "aerolinea" "vuelo" "codigoCola"
[13] "origen" "destino" "tiempo_vuelo"
[16] "distancia" "hora" "minuto"
[19] "fecha_hora"
```

```
hacer_fechahora_100 <- function(anio, mes, dia, tiempo) {
 make_datetime(anio, mes, dia, tiempo %/% 100, tiempo %% 100)
}
```

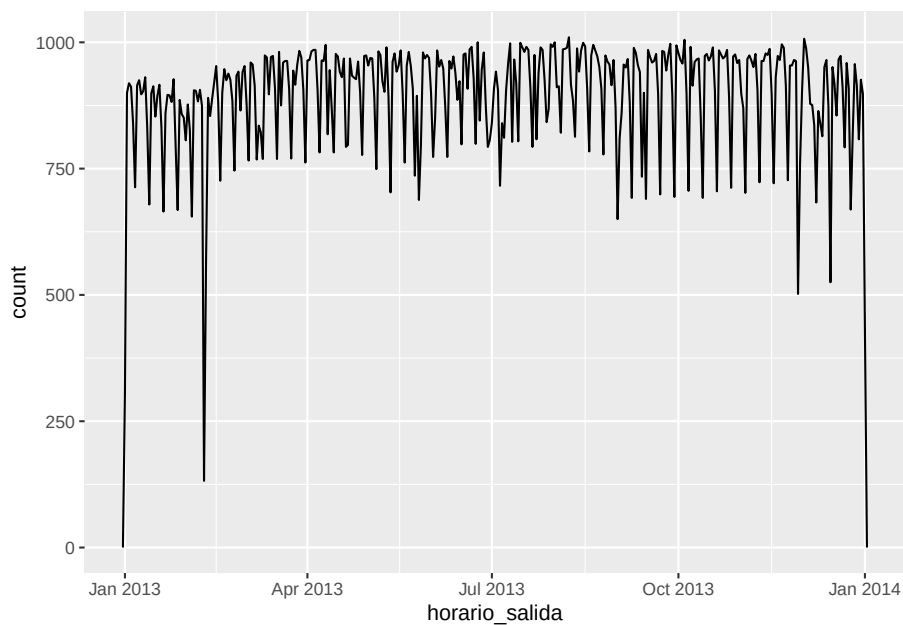
```
vuelos_dt <- vuelos %>%
 filter(!is.na(horario_salida), !is.na(horario_llegada)) %>%
 mutate(
 horario_salida = hacer_fechahora_100(anio, mes, dia, horario_salida),
 horario_llegada = hacer_fechahora_100(anio, mes, dia, horario_llegada),
 salida_programada = hacer_fechahora_100(anio, mes, dia, salida_programada),
 llegada_programada = hacer_fechahora_100(anio, mes, dia, llegada_programada)
) %>%
 dplyr::select(origen, destino, starts_with("atraso"), starts_with("horario"), ends_with("programa"))
```

```
head(vuelos_dt)
```

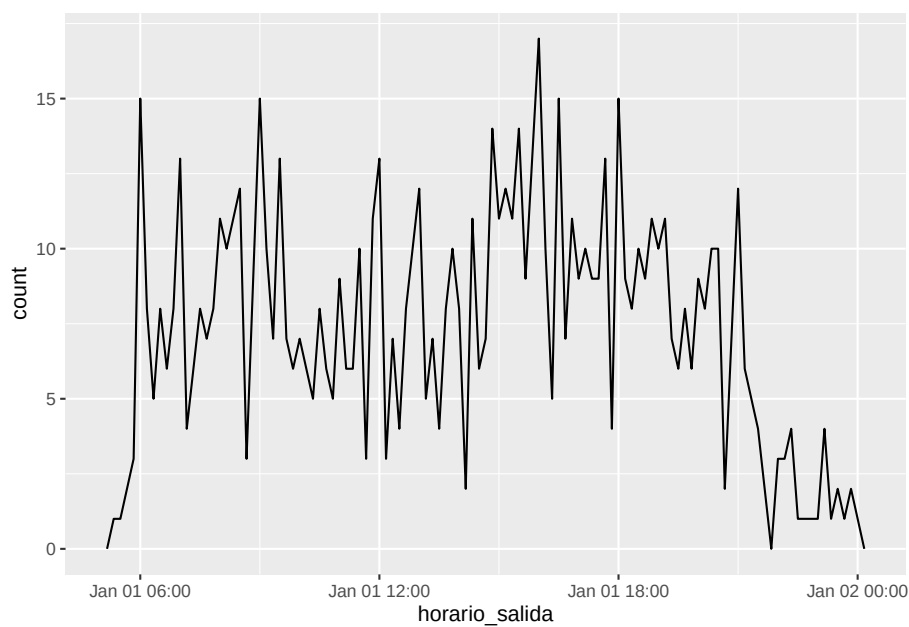
```
A tibble: 6 x 9
origen destino atraso_salida atraso_llegada horario_salida
<chr> <chr> <dbl> <dbl> <dtm>
1 EWR IAH 2 11 2013-01-01 05:17:00
2 LGA IAH 4 20 2013-01-01 05:33:00
3 JFK MIA 2 33 2013-01-01 05:42:00
4 JFK BQN -1 -18 2013-01-01 05:44:00
5 LGA ATL -6 -25 2013-01-01 05:54:00
```

```
6 EWR ORD -4 12 2013-01-01 05:54:00
i 4 more variables: horario_llegada <dtm>, salida_programada <dtm>,
llegada_programada <dtm>, tiempo_vuelo <dbl>
```

```
vuelos_dt %>%
 ggplot(aes(horario_salida)) +
 geom_freqpoly(binwidth = 86400) # 86400 segundos = 1 día
```



```
vuelos_dt %>%
 filter(horario_salida < ymd(20130102)) %>%
 ggplot(aes(horario_salida)) +
 geom_freqpoly(binwidth = 600) # 600 segundos = 10 minutos
```



Nota variables que no funciona

```
ymd(c("2010-10-10", "bananas"))
```

```
[1] "2010-10-10" NA
```

```
d1 <- "Jan 1, 2010"
d1
```

```
[1] "Jan 1, 2010"
```

```
mdy(d1)
```

```
[1] "2010-01-01"
```

```
d2 <- "2015-Mar-07"
d2
```

```
[1] "2015-Mar-07"
```

```
ymd(d2)
```

```
[1] "2015-03-07"
```

```
d3 <- "06-Jun-2017"
d3
```

```
[1] "06-Jun-2017"
```

```
dmy(d3)
```

```
[1] "2017-06-06"
```

```
d4 <- c("Aug 19 (2015)", "Jul 1 (2015)")
d4
```

```
[1] "Aug 19 (2015)" "Jul 1 (2015)"
```

```
mdy(d4)
```

```
[1] "2015-08-19" "2015-07-01"
```

```
d5 <- "12/30/14" # Diciembre 30, 2014
d5
```

```
[1] "12/30/14"
```

```
mdy(d5)
```

```
[1] "2014-12-30"
```

```
d6 <- "ene 1, 2010"
d6
```

```
[1] "ene 1, 2010"
```

```
mdy(d6)
```

```
[1] NA
```

```
d7 <- c("Agosto 19 (2015)", "Julio 1 (2015)")
d7
```

```
[1] "Agosto 19 (2015)" "Julio 1 (2015)"
```

```
mdy(d7)
```

```
[1] NA NA
```

```
mdy(d1)
```

```
[1] "2010-01-01"
```

```
ymd(d2)
```

```
[1] "2015-03-07"
```

```
dmy(d3)
```

```
[1] "2017-06-06"
```

```
mdy(d4)
```

```
[1] "2015-08-19" "2015-07-01"
```



```
mdy(d5)
```

```
[1] "2014-12-30"
```

```
mdy(d6)
```

```
[1] NA
```

```
mdy(d7)
```

```
[1] NA NA
```

### 1. Ejercicios:

Hacer los ejercicios en la sección 16.2.4 del libro en español

---

## 25.12 Extrayendo parte de las fecha-hora

- `year()`
- `month()`
- `mday()`
- `yday()`
- `wday()`
- `ceiling_date()`
- `floor_date()`
- `round_date()`

### 2. Ejercicios:

Hacer estos ejercicios en la sección 16.3.4 del libro en español. Entregar por MSTeams.

¿Cómo cambia la demora promedio durante el curso de un día? ¿Deberías usar `horario_salida` o `salida_programada`? ¿Por qué?

¿En qué día de la semana deberías salir si quieres minimizar las posibilidades de una demora?

Confirma nuestra hipótesis de que las salidas programadas en los minutos 20-30 y 50-60 están causadas por los vuelos programados que salen más temprano. Pista: crea una variable binaria que te diga si un vuelo tuvo o no demora.

---

## 25.13 Lapso de tiempo

### 25.14 duraciones

```
¿Qué edad tiene Charles Darwin? fecha de nacimiento February 12, 1809
edad_h <- today() - ymd("1809-02-12")
edad_h <- today() - ymd(18090212)
edad_h
```

```
Time difference of 79428 days
as.duration(edad_h)
```

```
[1] "6862579200s (~217.46 years)"
x=as.duration(edad_h)
```

Otras funciones de duración

```
dseconds(15)
```

```
[1] "15s"
```

```
dminutes(10)
```

```
[1] "600s (~10 minutes)"
```

```
dhours(c(12, 24))
```

```
[1] "43200s (~12 hours)" "86400s (~1 days)"
```

```
ddays(0:5)
```

```
[1] "0s" "86400s (~1 days)" "172800s (~2 days)"
```

```
[4] "259200s (~3 days)" "345600s (~4 days)" "432000s (~5 days)"
```

```
dweeks(3)
```

```
[1] "1814400s (~3 weeks)"
```

```
dyears(1)
```

```
[1] "31557600s (~1 years)"
```

## 25.15 Puede agregar periodos de tiempo

```
2 * dyears(1)
```

```
[1] "63115200s (~2 years)"
```

```
dyears(1) + dweeks(12) + dhours(15)
```

```
[1] "38869200s (~1.23 years)"
```

```
ayer <- today() - ddays(1)
```

```
anio_pasado <- today() - dyears(1)
```

```
ayer
```

```
[1] "2026-07-31"
```

```
anio_pasado
```

```
[1] "2025-07-31 18:00:00 UTC"
```

## 25.16 períodos

```
una_pm <- ymd_hms("2016-03-12 13:00:00", tz = "America/New_York")
```

```
una_pm
```

```
[1] "2016-03-12 13:00:00 EST"
```

```
#> [1] "2016-03-12 13:00:00 EST"
```

```
una_pm + ddays(1)
```

```
[1] "2016-03-13 14:00:00 EDT"
```

```
#> [1] "2016-03-13 14:00:00 EDT"
```

```
seconds(15)
```

```
[1] "15S"
```

```
#> [1] "15S"
```

```
minutes(10)
```

```
[1] "10M 0S"
```

```
#> [1] "10M 0S"
```

```
hours(c(12, 24))
```

```
[1] "12H 0M 0S" "24H 0M 0S"
```

```
#> [1] "12H 0M 0S" "24H 0M 0S"
```

```
days(7)
```

```
[1] "7d 0H 0M 0S"
```

```
#> [1] "7d 0H 0M 0S"
```

```
months(1:6)
```

```
[1] "1m 0d 0H 0M 0S" "2m 0d 0H 0M 0S" "3m 0d 0H 0M 0S" "4m 0d 0H 0M 0S"
[5] "5m 0d 0H 0M 0S" "6m 0d 0H 0M 0S"
#> [1] "1m 0d 0H 0M 0S" "2m 0d 0H 0M 0S" "3m 0d 0H 0M 0S" "4m 0d 0H 0M 0S"
#> [5] "5m 0d 0H 0M 0S" "6m 0d 0H 0M 0S"
weeks(3)
```

```
[1] "21d 0H 0M 0S"
#> [1] "21d 0H 0M 0S"
years(1)
```

```
[1] "1y 0m 0d 0H 0M 0S"
#> [1] "1y 0m 0d 0H 0M 0S"
10 * (months(6) + days(1))
```

```
[1] "60m 10d 0H 0M 0S"
#> [1] "60m 10d 0H 0M 0S"
days(50) + hours(25) + minutes(2)
```

```
[1] "50d 25H 2M 0S"
#> [1] "50d 25H 2M 0S"
Un año bisiesto
ymd("2016-01-01") + dyears(1)
```

```
[1] "2016-12-31 06:00:00 UTC"
#> [1] "2016-12-31 06:00:00 UTC"
ymd("2016-01-01") + years(1)
```

```
[1] "2017-01-01"
#> [1] "2017-01-01"
Horarios de verano
una_pm + ddays(1)
```

```
[1] "2016-03-13 14:00:00 EDT"
#> [1] "2016-03-13 14:00:00 EDT"
una_pm + days(1)
```

```
[1] "2016-03-13 13:00:00 EDT"
#> [1] "2016-03-13 13:00:00 EDT"
10 * (months(6) + days(1))
```

```
[1] "60m 10d 0H 0M 0S"
#> [1] "60m 10d 0H 0M 0S"
days(50) + hours(25) + minutes(2)
```

```
[1] "50d 25H 2M 0S"
#> [1] "50d 25H 2M 0S"
```

- intervalos

### 25.16.1 Multiplicaciones y sumas de fechas

## 25.17 Períodos

- seconds()
- minutes()
- hours()
- days()
- months()
- weeks()
- years()

### 25.17.1 Multiplicaciones y sumas de periodos

## 25.18 Intervalos

- %-% start end

## 25.19 Resumen

### 2. Ejercicios:

Hacer los ejercicios en la sección 16.4.5 del libro en español

---

## 25.20 Time zones

- Sys.timezone()
- head(OlsonNames())
- tz= “ ”

### 3. Ejercicios:

Seleccionar 3 archivos de los vuelos que salen o llegan a PR, (el código del aeropuerto es “SJU”) de la base de datos de <https://www.transtats.bts.gov/>

DL\_SelectFields.asp?Table\_ID=236 Pueden ser el mismo mes en 3 diferentes años o 3 diferentes meses en el mismo año.

- Repite la mayoría de los análisis enseñado arriba (como práctica).
  - Evaluar el tiempo de retraso de los vuelos que salen de SJU en cada periodo seleccionado, y haz *una* gráfica para visualizar el patrón
  - Cuál es el día preferible para no tener retraso
  - Cuál es la mejor hora de salida para no tener retraso
  - Compara por lo menos 3 diferentes líneas saliendo de SJU y el periodo de retraso.
-

## Chapter 26

# Funciones

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/functions.html>

Español: <https://es.r4ds.hadley.nz/funciones.html>

---

Fecha de la última revisión

## [1] "2026-08-01"

### 26.1 Temas:

- ¿Cuándo escribir una función?
- La anatomía de una función: nombre, argumentos y cuerpo
- Ejecución condicional: `if` y `else`
- Argumentos y valores por defecto
- Valores de retorno
- Un vistazo a los vectores

En el capítulo de *tidyverse avanzado* verás funciones más sofisticadas que usan *tidy evaluation* (el operador `{{ }}`). Aquí construimos la base.

---

### 26.2 ¿Cuándo escribir una función?

Una **función** te permite darle un nombre a un bloque de código para poder reutilizarlo. La regla práctica es: **si copiaste y pegaste el mismo código más de dos veces, escribe una función**. Copiar y pegar es una de las

fuentes de errores más comunes, porque es fácil olvidar cambiar el nombre de una variable en una de las copias.

Regla de tres: si copias y pegas el mismo bloque de código más de dos veces, conviértelo en una función. Así reduces errores y solo corriges el código en un lugar.

Observa este código que reescala tres columnas para que queden entre 0 y 1:

```
df <- data.frame(
 a = c(2, 5, 8, 1),
 b = c(10, 20, 30, 40),
 c = c(-5, 0, 5, 10)
)

df$a <- (df$a - min(df$a)) / (max(df$a) - min(df$a))
df$b <- (df$b - min(df$b)) / (max(df$b) - min(df$b))
df$c <- (df$c - min(df$c)) / (max(df$c) - min(df$c))
df

a b c
1 0.1428571 0.0000000 0.0000000
2 0.5714286 0.3333333 0.3333333
3 1.0000000 0.6666667 0.6666667
4 0.0000000 1.0000000 1.0000000
```

¿Notaste el patrón repetido? Eso es una señal clara de que conviene una función.

## 26.3 La anatomía de una función

**R** es un lenguaje de *programación funcional*: las funciones son “ciudadanos de primera clase” (se pueden guardar en variables y pasar a otras funciones). Heredó esta filosofía del lenguaje **S** y de **Scheme**, que inspiraron a sus creadores.

`function(argumentos) { cuerpo }` — crear tu propia función.

- **Estructura:** `nombre <- function(x) { ... }`; se llama luego con `nombre(valor)`.
- **Argumentos:** pueden tener valores por defecto (`function(x, recorte = 0.1)`).
- **Retorno:** devuelve el último valor evaluado, o el que indiques con `return()`.

Toda función tiene tres partes:

1. Un **nombre** (aquí, `reescalar01`).
2. Los **argumentos** que van dentro de `function( )` (aquí, `x`).
3. El **cuerpo**, el código dentro de las llaves `{ }`.



```
reescalar01 <- function(x) {
 rango <- range(x, na.rm = TRUE)
 (x - rango[1]) / (rango[2] - rango[1])
}

Probar la función con un ejemplo cuya respuesta ya conocemos
reescalar01(c(0, 5, 10))
```

```
[1] 0.0 0.5 1.0
```

Ahora el código repetido se reduce a esto, mucho más fácil de leer y de corregir:

```
df <- data.frame(
 a = c(2, 5, 8, 1),
 b = c(10, 20, 30, 40),
 c = c(-5, 0, 5, 10)
)

df$a <- reescalar01(df$a)
df$b <- reescalar01(df$b)
df$c <- reescalar01(df$c)
df
```

```
a b c
1 0.1428571 0.0000000 0.0000000
2 0.5714286 0.3333333 0.3333333
3 1.0000000 0.6666667 0.6666667
4 0.0000000 1.0000000 1.0000000
```

Una función siempre se debe **probar con valores cuya respuesta ya conoces** antes de confiar en ella.

## 26.4 Ejecución condicional: if y else

Dentro de una función puedes tomar decisiones con `if` y `else`.

```
signo <- function(x) {
 if (x > 0) {
 "positivo"
 } else if (x < 0) {
 "negativo"
 } else {
 "cero"
 }
}
```

```
signo(7)

[1] "positivo"
signo(-3)

[1] "negativo"
signo(0)

[1] "cero"
```

---

## 26.5 Argumentos y valores por defecto

Los argumentos pueden tener un **valor por defecto**, que se usa cuando quien llama la función no especifica ese argumento.

```
Calcular la media recortada; por defecto recorta un 10%
media_recortada <- function(x, recorte = 0.1) {
 mean(x, trim = recorte, na.rm = TRUE)
}

valores <- c(1, 2, 3, 4, 5, 100) # el 100 es un valor atípico
media_recortada(valores) # usa recorte = 0.1

[1] 19.16667

media_recortada(valores, recorte = 0.2)

[1] 3.5

mean(valores) # comparación: la media normal

[1] 19.16667
```

---

## 26.6 Valores de retorno

Una función en R devuelve el valor de la **última línea** de su cuerpo. También puedes usar `return()` para devolver un valor antes del final, por ejemplo para salir temprano en un caso especial.

```
coef_variacion <- function(x) {
 if (mean(x, na.rm = TRUE) == 0) {
 return(NA) # evitar división por cero
 }
```

```
sd(x, na.rm = TRUE) / mean(x, na.rm = TRUE)
}

coef_variacion(c(2, 4, 4, 4, 5, 5, 7, 9))

[1] 0.427618
coef_variacion(c(-5, 0, 5)) # la media es 0 -> NA

[1] NA
```

---

## 26.7 Un vistazo a los vectores

Las funciones trabajan sobre **vectores**. Conocer los tipos básicos de vectores atómicos explica muchos “comportamientos raros” de R.

```
lgl <- c(TRUE, FALSE, NA) # lógico
int <- c(1L, 5L, 10L) # entero
dbl <- c(1.5, 2.7, 3.14) # doble (números reales)
chr <- c("a", "b", "c") # cadena de caracteres

typeof(lgl)

[1] "logical"
typeof(int)

[1] "integer"
typeof(dbl)

[1] "double"
typeof(chr)

[1] "character"
```

**Coerción:** cuando mezclas tipos, R convierte todo al tipo “más flexible”.

```
c(1, 2, "tres") # todo se convierte en texto

[1] "1" "2" "tres"
c(TRUE, 1L, 2.5) # lógico -> entero -> doble

[1] 1.0 1.0 2.5
```

**Valores faltantes (NA):** casi cualquier operación con NA da NA. Por eso muchas funciones tienen el argumento `na.rm = TRUE`.

Si olvidas `na.rm = TRUE`, basta con un solo NA en los datos para que `mean()`, `sd()` y muchas otras funciones devuelvan NA sin avisarte. Encontrarás más errores frecuentes y cómo resolverlos en el capítulo *Errores comunes* (14).

```
x <- c(1, 2, NA, 4)
mean(x) # NA

[1] NA
mean(x, na.rm = TRUE) # 2.333...

[1] 2.333333
is.na(x) # ¿cuáles son NA?

[1] FALSE FALSE TRUE FALSE
```

---

### 1. Ejercicios:

Hacer los ejercicios en la sección 19.2 del libro en español.

---

### 2. Ejercicios:

Escribe una función `fahrenheit_a_celsius()` que reciba una temperatura en grados Fahrenheit y devuelva grados Celsius. La fórmula es  $C = (F - 32) \times 5/9$ . Pruébala con 32 (debe dar 0) y con 212 (debe dar 100).

---

### 3. Ejercicios:

Hacer los ejercicios en la sección 19.4 del libro en español.

## Chapter 27

# Iteración: purrr

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/iteration.html>

Español: <https://es.r4ds.hadley.nz/iteración.html>

---

Fecha de la última revisión

## [1] "2026-08-01"

### 27.1 Temas: El paquete purrr

- La idea de iterar (repetir una acción)
- El bucle `for` de base R
- La familia `map()` de `purrr`
- Fórmulas anónimas con `~` y `.x`
- Iterar sobre dos listas con `map2()`
- Un modelo por grupo

`purrr` es parte de `tidyverse`, así que se activa con `library(tidyverse)`.

```
library(tidyverse)
library(datos)
```

---

### 27.2 ¿Qué es iterar?

**Iterar** es repetir la misma acción sobre cada elemento de una lista, vector o columna. Igual que con las funciones, la meta es **no repetir código**. Si las

funciones evitan copiar y pegar código *dentro* de una tarea, la iteración evita copiar y pegar la *misma tarea* muchas veces.

Supón que queremos la mediana de cada columna de este data frame:

```
df <- tibble(
 a = rnorm(10),
 b = rnorm(10),
 c = rnorm(10),
 d = rnorm(10)
)

Repetir a mano es tedioso y propenso a errores:
median(df$a)

[1] -0.2364174

median(df$b)

[1] 0.2332814

median(df$c)

[1] 1.136037

median(df$d)

[1] 0.4082237
```

---

## 27.3 El bucle for de base R

La forma clásica de iterar es con un bucle for:

```
resultado <- vector("double", ncol(df)) # espacio para guardar la salida
for (i in seq_along(df)) { # para cada columna
 resultado[[i]] <- median(df[[i]]) # calcular y guardar
}

resultado

[1] -0.2364174 0.2332814 1.1360375 0.4082237
```

Funciona, pero hay que escribir bastante “andamiaje” (crear el vector de salida, manejar el índice *i*). **purrr** simplifica esto.

Crea el vector de salida con su tamaño final *antes* del bucle. Hacerlo crecer dentro del for (por ejemplo con `c()` en cada vuelta) obliga a R a copiarlo una y otra vez y vuelve el código muy lento.

---

## 27.4 La familia map()

La idea de **map** (aplicar una función a cada elemento) viene de la programación funcional y del lenguaje **Lisp** (años 1960). El paquete **purrr**, de **Hadley Wickham** y **Lionel Henry**, la trajo al tidyverse con una familia de funciones de tipos predecibles.

La familia **map()** (**purrr**) — aplicar una función a cada elemento.

- **map(x, f)**: aplica **f** a cada elemento y devuelve una **lista**.
- **Variantes por tipo**: **map\_dbl()**, **map\_int()**, **map\_chr()**, **map\_lgl()** devuelven un vector del tipo indicado.
- **Fórmula anónima**: **~ f(.x)** define una función corta donde **.x** es cada elemento; **map2()** itera sobre dos listas a la vez.

Las funciones **map()** toman un vector o lista, aplican una función a **cada elemento**, y devuelven un resultado. Hay una versión por tipo de salida:

- **map()** devuelve una **lista**
- **map\_dbl()** devuelve un vector de **dobles**
- **map\_int()** devuelve un vector de **enteros**
- **map\_chr()** devuelve un vector de **texto**
- **map\_lgl()** devuelve un vector **lógico**

```
map_dbl(df, median) # la mediana de cada columna, en una sola línea
```

```
a b c d
-0.2364174 0.2332814 1.1360375 0.4082237
```

```
map_dbl(df, mean)
```

```
a b c d
-0.3534311 0.1467468 1.0998756 0.2888905
```

```
map_dbl(df, sd)
```

```
a b c d
0.9753734 0.8503055 0.5548967 1.0877976
```

Compara: el bucle **for** completo de arriba se reduce a **map\_dbl(df, median)**.

## 27.5 Fórmulas anónimas con ~ y .x

A veces necesitas una pequeña operación que no tiene nombre de función. Puedes escribirla como una **fórmula** empezando con **~**, y usar **.x** para representar “el elemento actual”.

```
El rango (máximo - mínimo) de cada columna
map_dbl(df, ~ max(.x) - min(.x))
```

```
a b c d
3.042304 2.203092 1.832097 3.338961
¿Cuántos valores positivos hay en cada columna?
map_int(df, ~ sum(.x > 0))
```

```
a b c d
3 5 10 6
```

Esto es muy útil con datos reales. Por ejemplo, el promedio de atraso de salida por mes en los vuelos:

```
vuelos %>%
 split(.$mes) %>% # una lista: un data frame por me
 map_dbl(~ mean(.$atraso_salida, na.rm = TRUE)) # promedio de atraso por mes
```

```
1 2 3 4 5 6 7 8
10.036665 10.816843 13.227076 13.938038 12.986859 20.846332 21.727787 12.611040
9 10 11 12
6.722476 6.243988 5.435362 16.576688
```

## 27.6 Iterar sobre dos listas con map2()

Cuando necesitas recorrer **dos** listas en paralelo, usa `map2()`. Los elementos se representan con `.x` (de la primera) y `.y` (de la segunda).

```
medias <- c(0, 10, 100)
desv <- c(1, 2, 3)

Generar 5 números al azar de cada combinación de media y desviación
map2(medias, desv, ~ rnorm(5, mean = .x, sd = .y))
```

```
[[1]]
[1] 1.01720062 -0.45401384 1.11807753 0.02410736 -0.30014527
##
[[2]]
[1] 13.09959 10.12764 10.11323 9.00243 10.06829
##
[[3]]
[1] 99.31260 99.88415 98.67224 100.17103 103.52231
```



## 27.7 Un modelo por grupo

Un uso poderoso es ajustar **un modelo por grupo**. Aquí ajustamos una regresión del atraso de llegada contra el de salida, por separado para cada aerolínea.

Como el paquete `maps` también se carga en este libro, escribimos `purrr::map()` de forma explícita para no confundirla con `maps::map()`.

```
modelos <- vuelos %>%
 split(.$aerolinea) %>%
 purrr::map(~ lm(atraso_llegada ~ atraso_salida, data = .x))

Ver el R2 de cada modelo
modelos %>%
 purrr::map(summary) %>%
 map_dbl("r.squared") %>%
 head()
```

```
9E AA AS B6 DL EV
0.8622935 0.7952061 0.7012039 0.8369837 0.8192728 0.9080101
```

Este patrón —dividir, aplicar un modelo a cada parte, y resumir— es la base del tema de *muchos modelos* con `purrr` y `broom`.

---

### 1. Ejercicios:

Hacer los ejercicios en la sección 21.5 del libro en español.

---

### 2. Ejercicios:

Usa `map_dbl()` para calcular el número de valores únicos (`n_distinct()`) en cada columna del data frame `diamantes`.

---

### 3. Ejercicios:

Hacer los ejercicios en la sección 21.9 del libro en español.



## Chapter 28

# Modelos: modelr

El tema proviene de los siguientes sitios.

English: <https://r4ds.had.co.nz/model-basics.html>

Español: <https://es.r4ds.hadley.nz/conceptos-b%C3%A1sicos-de-modelos.html>

---

Fecha de la última revisión

## [1] "2026-08-01"

### 28.1 Temas: Construir modelos con modelr

- Visualizar la relación entre dos variables
- Ajustar un modelo lineal con `lm()`
- Predicciones con `data_grid()` y `add_predictions()`
- Residuales con `add_residuals()`

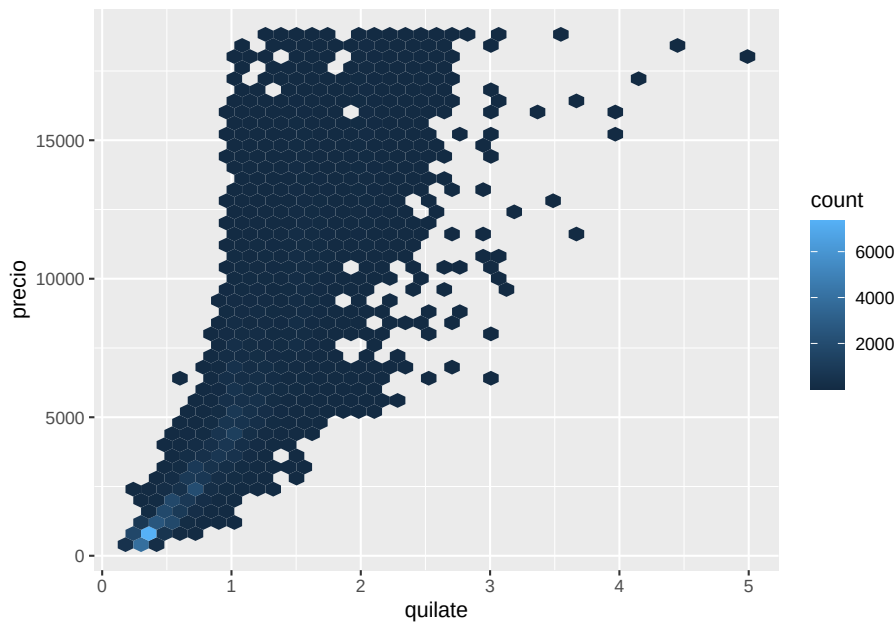
Este capítulo usa el enfoque de `modelr` (el del libro *R for Data Science*). Para un flujo más moderno con `tidymodels`, y para dar formato matemático a los ejes con LaTeX, mira los capítulos de *tidyverse avanzado* y de *ecuaciones matemáticas*.

```
library(tidyverse)
library(modelr)
library(datos)
options(na.action = na.warn)
```

## 28.2 Visualizar la relación

Usaremos los `diamantes`. A primera vista, los diamantes de peor calidad parecen costar más, porque el peso (`quilate`) influye fuertemente en el precio. Primero miramos la relación entre peso y precio:

```
ggplot(diamantes, aes(quilate, precio)) +
 geom_hex(bins = 40)
```

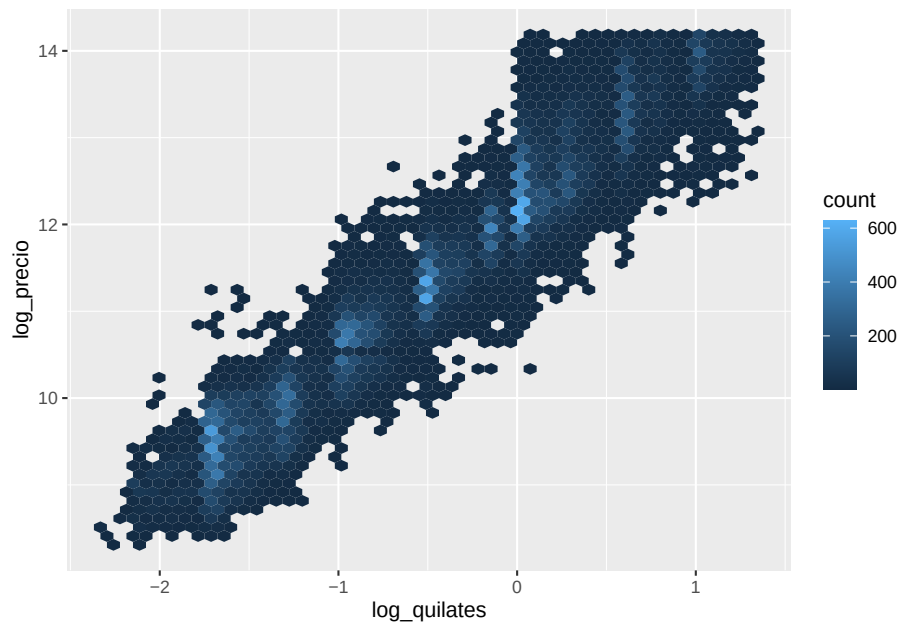


La relación se ve curva. Nos enfocamos en los diamantes de menos de 2.5 quilates (el 99.7% de los datos) y aplicamos una **transformación logarítmica**, que convierte la relación en una línea recta:

Cuando una relación se ve curva o los datos están muy sesgados, una transformación logarítmica a menudo la endereza y facilita ajustar un modelo lineal.

```
diamantes2 <- diamantes %>%
 filter(quilate <= 2.5) %>%
 mutate(log_precio = log2(precio),
 log_quilates = log2(quilate))

ggplot(diamantes2, aes(log_quilates, log_precio)) +
 geom_hex(bins = 50)
```



## 28.3 Ajustar un modelo lineal con `lm()`

`lm()`, `data_grid()`, `add_predictions()`, `add_residuals()` (`modelr`) — modelar y explorar.

- `lm(y ~ x, data)`: ajusta un modelo lineal.
- `data_grid()`: crea una rejilla de valores de las predictoras.
- `add_predictions()` / `add_residuals()`: añaden columnas con las predicciones y los residuales del modelo, cómodas para graficar.

La función `lm()` ajusta un modelo lineal. La fórmula `y ~ x` se lee “y explicada por x”.

```
mod_diamantes <- lm(log_precio ~ log_quilates, data = diamantes2)
mod_diamantes
```

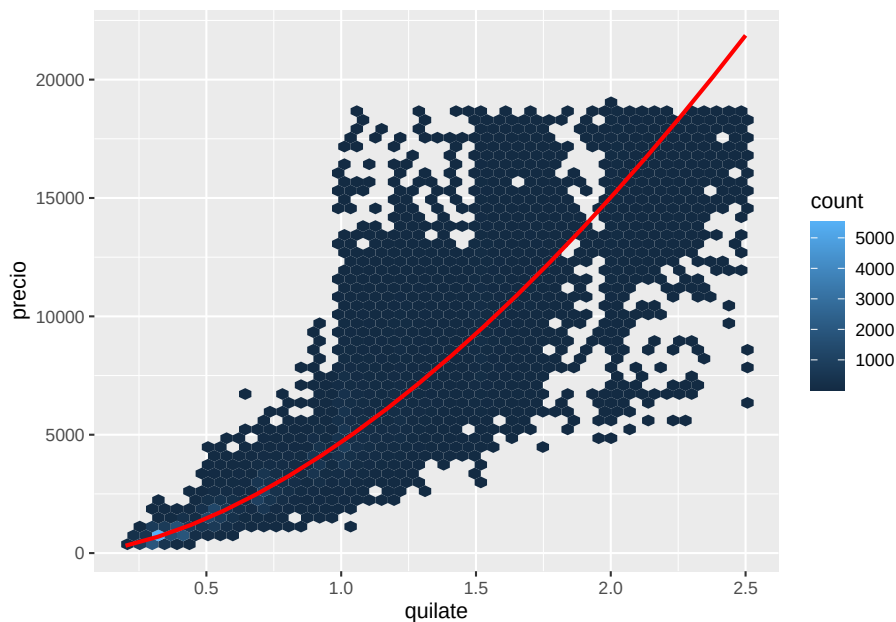
```
##
Call:
lm(formula = log_precio ~ log_quilates, data = diamantes2)
##
Coefficients:
(Intercept) log_quilates
12.194 1.681
```

## 28.4 Predicciones con `data_grid()` y `add_predictions()`

Para ver lo que el modelo “aprendió”, generamos una cuadrícula de valores de `quilate` con `data_grid()`, y le añadimos la predicción del modelo con `add_predictions()`. Deshacemos la transformación logarítmica para volver a la escala original de precio.

```
cuadrícula <- diamantes2 %>%
 data_grid(quilate = seq_range(quilate, 20)) %>%
 mutate(log_quilates = log2(quilate)) %>%
 add_predictions(mod_diamantes, "log_precio") %>%
 mutate(precio = 2 ^ log_precio)

ggplot(diamantes2, aes(quilate, precio)) +
 geom_hex(bins = 50) +
 geom_line(data = cuadrícula, colour = "red", size = 1)
```



La línea roja es la predicción del modelo.

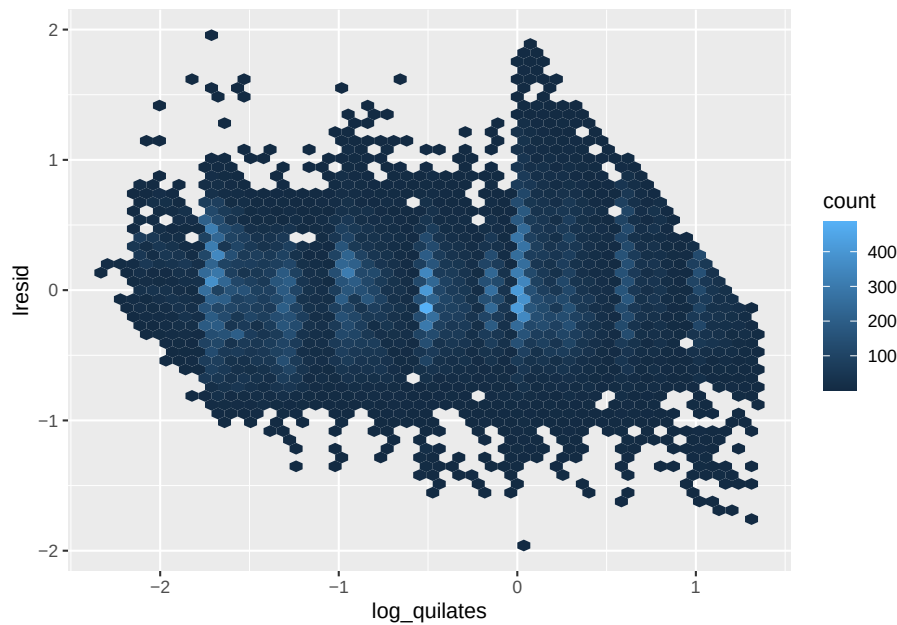
## 28.5 Residuales con `add_residuals()`

Los **residuales** son lo que el modelo *no* explica: la diferencia entre el valor observado y el predicho. Si el modelo es bueno, al graficar los residuales no debe quedar ningún patrón evidente.

Si en la gráfica de residuales todavía ves una tendencia o una forma clara, el modelo se está perdiendo algo: considera transformar las variables o añadir más predictores.

```
diamantes2 <- diamantes2 %>%
 add_residuals(mod_diamantes, "lresid")

ggplot(diamantes2, aes(log_quilates, lresid)) +
 geom_hex(bins = 50)
```



---

### 1. Ejercicios:

Hacer los ejercicios en la sección 23.2 del libro en español.

---

### 2. Ejercicios:

Ajusta un modelo lineal del atraso de llegada en función del atraso de salida con los datos `vuelos` (`lm(atraso_llegada ~ atraso_salida, data = vuelos)`). Añade los residuales con `add_residuals()` y gráficalos. ¿Queda algún patrón?





## Chapter 29

# Hex Stickers

```
[1] "2026-08-01"
library(hexSticker)
library(magick)
library(sysfonts)
library(tidyverse)
```

### 29.1 La función y los valores “default”

Los **hex stickers** (calcomanías hexagonales) se popularizaron en la comunidad de R alrededor de 2015, impulsados por desarrolladores de **RStudio/Posit** y el proyecto **rOpenSci**. El formato hexagonal permite unir las calcomanías sin dejar huecos —como un panal— por lo que se convirtió en la forma estándar de identificar paquetes de R en conferencias y computadoras portátiles. El paquete **hexSticker**, de **Guangchuang Yu**, automatiza su creación.

**sticker()** — crea una calcomanía hexagonal a partir de una imagen o gráfico.

- **subplot:** la imagen o gráfico (**ggplot**) que va en el centro; **s\_x**, **s\_y**, **s\_width**, **s\_height** controlan su posición y tamaño.
- **package:** el texto (normalmente el nombre del paquete); **p\_size**, **p\_color**, **p\_family** ajustan su tipografía.
- **h\_fill**, **h\_color**, **h\_size:** color de relleno, color y grosor del borde del hexágono.
- **filename:** dónde se guarda; la extensión (**.png**, **.svg**) decide el formato de salida.

```
sticker(
 subplot, # position and size of subplot
 s_x = 0.8,
```

```

s_y = 0.75,
s_width = 0.4,
s_height = 0.5,
package, # package name and characteristics
p_x = 1,
p_y = 1.4,
p_color = "#FFFFFF",
p_family = "Aller_Rg",
p_fontface = "plain",
p_size = 8,
h_size = 1.2, # hexagon border size and colors
h_fill = "#1881C2",
h_color = "#87B13F",
spotlight = FALSE,
l_x = 1, # spotlight x position
l_y = 0.5,
l_width = 3,
l_height = 3,
l_alpha = 0.4,
url = "", # url at lower border
u_x = 1,
u_y = 0.08,
u_color = "black",
u_family = "Aller_Rg",
u_size = 1.5,
u_angle = 30,
white_around_sticker = FALSE, # if TRUE, puts white triangles in the corners
...,
filename = paste0(package, ".png"), # filename to save sticker
asp = 1, # aspect ratio, only works if subplot is an image file
dpi = 300 # plot resolution
)

```

---

### Arguments

```

library(gt)
gt(argumentos)

```

Other Details, - The extension given in filename determines the graphics device that is used to render the sticker, e.g. filename = ‘sticker.png’ creates a png file and filename = ‘sticker.svg’ creates a svg file. For a list of supported graphics devices please see the documentation of ggplot2::ggsave()

```

sysfonts::font_add_google("Sixtyfour Convergence") sysfonts::font_add_google("Rubik")
sysfonts::font_add_google("Young Serif") ## No baja

```

## 29.2 Ejemplo de Crear un Hex con un gatito

Para usar una fuente de Google, cárgala con `font_add_google()` y activa `showtext_auto()` antes de llamar a `sticker()`. Si no, el texto no se dibujará con la fuente elegida.

```
#font_files() #a list of all fonts available

Loading Google fonts (https://fonts.google.com/)

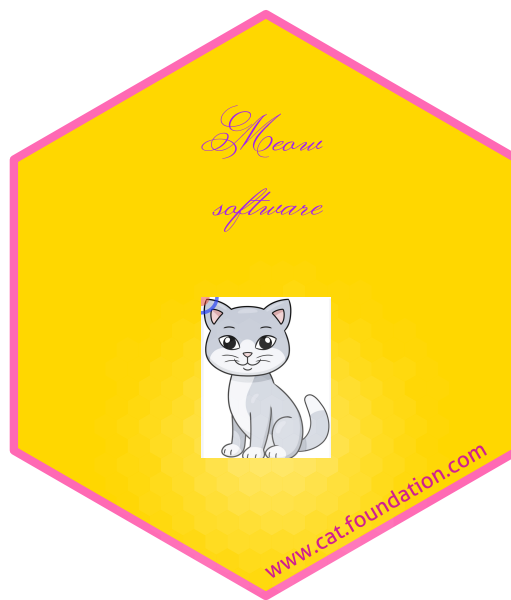
#sysfonts::font_add_google("Dancing Script") # selección de fonts de google
#sysfonts::font_add_google("Bebas Neue")
#sysfonts::font_add_google("Fuggles")
#sysfonts::font_add_google("Roboto")
#sysfonts::font_add_google("Montserrat")
#sysfonts::font_add_google("Lobster Two")
sysfonts::font_add_google("Monsieur La Doulaise")
#sysfonts::font_add_google("monsieur")

cat_img=image_read('img/cat.jpg')
analitica_img=image_read('img/Analitica3.png')

library(showtext)
font_add_google("Monsieur La Doulaise")
showtext_opts(dpi = 300)
showtext_auto()

sticker(
 subplot= cat_img,
 package="Meow \n software",
 filename = "img/hex_meow_software.png",
 s_width =.55,
 s_height = .55,
 s_x=1,
 s_y=.75,
 p_size=8, # tamaño del texto
 p_x = 1.0,
 p_y = 1.5,
 p_color= 'purple',
 p_family= "Monsieur La Doulaise",
```

```
h_fill = 'gold',
h_color = 'hotpink',
h_size = 2,
url = "www.cat.foundation.com",
u_size = 4,
u_color = "violetred",
spotlight = T
)
%>% print
```



```
save_sticker("img/hex_sticker_cat.jpg")
```

Función	Descripción
subplot	subplot
s_x	x position for subplot
s_y	y position for subplot
s_width	width for subplot
s_height	height for subplot
package	package name
p_x	x position for package name
p_y	y position for package name
p_color	color for package name
p_family	font family for package name
p_fontface	fontface for package name
p_size	font size for package name
h_size	size for hexagon border
h_fill	color to fill hexagon
h_color	color for hexagon border
spotlight	whether add spotlight
l_x	x position for spotlight
l_y	y position for spotlight
l_width	width for spotlight
l_height	height for spotlight
l_alpha	maximum alpha for spotlight
url	url at lower border
u_x	x position for url
u_y	y position for url
u_color	color for url
u_family	font family for url
u_size	text size for url
u_angle	angle for url
white_around_sticker	default to FALSE. If set to TRUE, it puts white triangles in the corners
...	additional parameter to geom_pkgname
filename	filename to save sticker
asp	aspect ratio, only works if subplot is an image file
dpi	plot resolution



## Chapter 30

# Nubes de palabra en R

```
[1] "2026-08-01"
```

### 30.1 Ejemplo #1

Para generar nubes de palabras con R. Se necesita los paquetes **wordcloud**, **RColorBrewer** y **wordcloud2**

La **minería de texto** (*text mining*) —analizar palabras con la computadora para descubrir patrones— tiene raíces en la lingüística computacional de los años 1950 y 60. Las **nubes de palabras** se popularizaron en la web alrededor de 2004 con sitios como *Flickr* y *Wordle*, mostrando el tamaño de cada palabra según su frecuencia. En R, el paquete **wordcloud** de **Ian Fellows** (2011) y el paquete **tm** (*text mining*) de **Ingo Feinerer** llevaron estas ideas al análisis de datos reproducible.

**wordcloud()** y **wordcloud2()** — dibujan una nube donde el tamaño de cada palabra es proporcional a su frecuencia.

- **words**, **freq**: el vector de palabras y su vector de frecuencias (normalmente calculados con el paquete **tm**).
- **min.freq**: frecuencia mínima para que una palabra aparezca; **max.words**: número máximo de palabras a mostrar.
- **colors**: la paleta de colores, a menudo de **brewer.pal()** del paquete **RColorBrewer**.
- **wordcloud2()** es una versión interactiva (HTML) que produce nubes más vistosas; recibe un **data.frame** con columnas de palabra y frecuencia.

Ese ejemplo es una copia de la siguiente página de web

<http://www.sthda.com/english/wiki/word-cloud-generator-in-r-one-killer-function-to-do-everything-you-need>

```

if (!require("pacman")) install.packages("pacman")
pacman::p_load(tm, SnowballC, wordcloud, RColorBrewer, wordcloud2)

library(wordcloud) # Un paquete para hacer word cloud
library(wordcloud2) # paquete más sencillo para hacer word cloud2
library(RColorBrewer) # paquete para cambiar los colores
library(tm) # paquete de text mining
library(SnowballC) # paquete para trabajar en otro idioma aparte de inglés

```

## 30.2 Usando los datos en el paquete wordcloud2 que se llama demoFreq

[https://raymondltrreblay.github.io/Ciencia\\_Datos\\_R/appendix-a.html](https://raymondltrreblay.github.io/Ciencia_Datos_R/appendix-a.html)

```

#ejercicio-de-transformaci%C3%B3n
library(wordcloud2)
head(demoFreq, n=10)
wordcloud2(data = demoFreq)

```

## 30.3 Paso 1

Importar los datos de la web

Este bloque **descarga un texto desde internet**, así que necesita conexión. Si el servidor no responde verás `cannot open the connection` o `cannot open URL`: no es un error de tu código, sino de la red o del servidor. Por eso lo envolvemos en `tryCatch()`, de modo que si la descarga falla se usa una copia local y el análisis continúa. Encontrarás más sobre estos errores en el capítulo *Errores comunes* (14).

```

filePath <- "http://www.sthda.com/sthda/RDoc/example-files/martin-luther-king-i-have-a-
Intenta leer el discurso desde la web; si no hay conexión o el servidor no
responde, usa un texto local de ejemplo para que el análisis no se detenga.
text <- tryCatch(
 suppressWarnings(readLines(filePath)),
 error = function(e) readLines("Datos/alice_original.txt")
)
head(text)

[1] "*** START OF THE PROJECT GUTENBERG EBOOK 11 ***"
[2] ""
[3] "[Illustration]"
[4] ""
[5] ""

```



```
[6] ""
```

Importar un texto de su computadora en formato `.txt` No va a funcionar el formato `.doc` de MSWord.

Guarda tu documento como texto plano (`.txt`) antes de leerlo con `readLines()`. Un archivo `.doc` o `.docx` guarda el formato en binario y obtendrás caracteres ilegibles en lugar del texto.

```
#mi_texto <- readLines(file.choose())
```

## 30.4 Subir el texto en formato Corpus

```
mi_texto=iconv(text,"WINDOWS-1252","UTF-8") # Use this for removing accents and non - english characters
Load the data as a corpus
docs <- Corpus(VectorSource(mi_texto))

head(docs)
```

```
<<SimpleCorpus>>
Metadata: corpus specific: 1, document level (indexed): 0
Content: documents: 6
```

## 30.5 Mirar el documento, para evaluar su contenido

```
#inspect(docs) # quita el hashtag para ver el documento
```

## 30.6 Transformar el texto para reemplazar algunos caracteres especiales, y remplazarlos por espacio en blanco

```
toSpace <- content_transformer(function (x , pattern) gsub(pattern, " ", x))
docs <- tm_map(docs, toSpace, "-")
docs <- tm_map(docs, toSpace, "@")
docs <- tm_map(docs, toSpace, "\\|")
```

## 30.7.1. Requesting the common text mining

```

Convert the text to lower case
docs <- tm_map(docs, content_transformer(tolower))

Remove numbers
docs <- tm_map(docs, removeNumbers)

Remove english common stopwords
stopwords("english") # list of common english stopwords that are often removed

```

```

[1] "i" "me" "my" "myself" "we"
[6] "our" "ours" "ourselves" "you" "your"
[11] "yours" "yourself" "yourselves" "he" "him"
[16] "his" "himself" "she" "her" "hers"
[21] "herself" "it" "its" "itself" "they"
[26] "them" "their" "theirs" "themselves" "what"
[31] "which" "who" "whom" "this" "that"
[36] "these" "those" "am" "is" "are"
[41] "was" "were" "be" "been" "being"
[46] "have" "has" "had" "having" "do"
[51] "does" "did" "doing" "would" "should"
[56] "could" "ought" "i'm" "you're" "he's"
[61] "she's" "it's" "we're" "they're" "i've"
[66] "you've" "we've" "they've" "i'd" "you'd"
[71] "he'd" "she'd" "we'd" "they'd" "i'll"
[76] "you'll" "he'll" "she'll" "we'll" "they'll"
[81] "isn't" "aren't" "wasn't" "weren't" "hasn't"
[86] "haven't" "hadn't" "doesn't" "don't" "didn't"
[91] "won't" "wouldn't" "shan't" "shouldn't" "can't"
[96] "cannot" "couldn't" "mustn't" "let's" "that's"
[101] "who's" "what's" "here's" "there's" "when's"
[106] "where's" "why's" "how's" "a" "an"
[111] "the" "and" "but" "if" "or"
[116] "because" "as" "until" "while" "of"
[121] "at" "by" "for" "with" "about"
[126] "against" "between" "into" "through" "during"
[131] "before" "after" "above" "below" "to"
[136] "from" "up" "down" "in" "out"
[141] "on" "off" "over" "under" "again"
[146] "further" "then" "once" "here" "there"
[151] "when" "where" "why" "how" "all"
[156] "any" "both" "each" "few" "more"
[161] "most" "other" "some" "such" "no"
[166] "nor" "not" "only" "own" "same"

```

## 30.8. CREAR UNA MATRIZ DE LAS PALABRAS DE MI DOCUMENTO 307

```
[171] "so" "than" "too" "very"

docs <- tm_map(docs, removeWords, stopwords("english"))

Remove your own stop word
specify your stopwords as a character vector
docs <- tm_map(docs, removeWords, c("blabla1", "blabla2"))

Remove punctuations
docs <- tm_map(docs, removePunctuation)

Eliminate extra white spaces
docs <- tm_map(docs, stripWhitespace)
Text stemming
docs <- tm_map(docs, stemDocument)

#stopwords() # Here are all the stopwords in the function **stopwords**
```

## 30.8 Crear una matriz de las palabras de mi documento

```
dtm <- TermDocumentMatrix(docs) # convertir el texto en una lista de palabras
m <- as.matrix(dtm) # convertir en una matriz
v <- sort(rowSums(m),decreasing=TRUE) # ordenar las palabras por frecuencia
d <- data.frame(word = names(v),freq=v) # crea un nuevo data frame de las palabras y su frecuencia
head(d, n=10) # las primeras 10 palabras más comunes
```

```
word freq
alice alice 145
little little 102
one one 65
said said 50
time time 49
went went 47
like like 44
quite quite 41
see see 41
way way 37
```

### 30.8.1 Check your words and remove unwanted words individually

“öö”

öorchid

pollinators

### 30.8.2 How would you remove from the data frame all words that have less or equal to 3 counts

Del paquete **wordcloud**

```
#head(d)
wordcloud(words = d$word, # las palabras
 freq = d$freq, # la frecuencia
 min.freq = 1, # la frecuencia mínima
 max.words=50, # el número máximo de palabras
 random.order=TRUE, # orden aleatorio
 rot.per=0.35, # rotación de las palabras
 colors=brewer.pal(8, "Dark2")) # colores
```



## 30.9 Ejemplo 2

Del paquete **wordcloud2**

```
wordcloud2(data = d)
```

## 30.10 Como remover palabras de otra idioma

### 30.10.1 Vea este enlace para los “stopwords” de muchos idiomas

<https://cran.r-project.org/web/packages/stopwords/readme/README.html>

## 30.11 En español

```
from CRAN
install.packages("stopwords")

Or get the development version from GitHub:
install.packages("devtools")
devtools::install_github("quanteda/stopwords")
```

### 30.11.1 Las 30 primeras palabras en la lista de stopword del paquete “stopwords” en español

```
head(stopwords::stopwords("es", source = "snowball"), 30)
```

```
[1] "de" "la" "que" "el" "en" "y" "a" "los"
[9] "del" "se" "las" "por" "un" "para" "con" "no"
[17] "una" "su" "al" "lo" "como" "más" "pero" "sus"
[25] "le" "ya" "o" "este" "sí" "porque"
```

---

Ejemplos #3

Aquí un tercer ejemplo

```
install.packages("pacman") # Si no tiene instalada la Biblioteca Pacman ejecutar esta línea de
library("pacman")

p_load("tm") # Biblioteca para realizar el preprocesado del texto,
p_load("tidyverse") # Biblioteca con funciones para manipular datos.
p_load("wordcloud") # Biblioteca para graficar nuestra nube de palabras.
p_load("RColorBrewer") # Biblioteca para seleccionar una paleta de colores de nuestra nube de pa
```

### 30.12 Un documento para hacer un word cloud “La inteligencia artificial”

```
library(readr)
articulo_IA <- "https://gist.github.com/EverVino/7bdbbe7ebdff5987970036f52f0e384f/raw/"
texto <- read_file(articulo_IA)
texto
```

```
[1] "La inteligencia artificial (IA) es, en informática, la inteligencia expresada p
Tratan de emular de forma racional el comportamiento humano; por ejemplo los agentes i
337 y el conocimiento sobre el conocimiento (lo que sabemos sobre lo que saben otras p
604\n\nEn los problemas clásicos de planificación, el agente puede asumir que es el ún
449 La planificación de múltiples agentes utiliza la cooperación y la competencia de m
455\nAprendizaje\n\nEl aprendizaje automático es un concepto fundamental de la investi
892 La visión artificial es la capacidad de analizar la información visual, que suele s
```

### 30.13 Convertir su documento en Corpus y identificar que es en español

```
texto2 <- VCorpus(VectorSource(texto),
 readerControl = list(reader = readPlain, language = "es", load = TRUE))
texto2
```

```
<<VCorpus>>
Metadata: corpus specific: 0, document level (indexed): 0
Content: documents: 1
```

```
#inspect(texto2)
```

### 30.14 Limpieza del documento

- remover los números
- remover las puntuaciones
- cambiar a letras minúsculas
- remover las palabras comunes en español
- usar solamente la base de las palabras (“stem”) por ejemplo remover las conjugaciones **espero**, **esperas**, **espera**, **esperamos**... se convierte en “esper”
- remover espacios blancos

Quita siempre los “stopwords” del idioma correcto. Si tu texto está en español usa `stopwords("es")` y no la lista en inglés, o palabras como “de”, “la” y “que” dominarán la nube.

```

texto2 <- tm_map(texto2, removeNumbers)
texto2 <- tm_map(texto2, removePunctuation)
texto2 <- tm_map(texto2, tolower)
texto2 <- tm_map(texto2, removeWords, stopwords::stopwords("es", source = "snowball"))
#texto2 <- tm_map(texto2, stemDocument, language="spanish")
texto2 <- tm_map(texto2, stripWhitespace)

```

### 30.15 Transformar el texto en un documento de texto sencillo (Plain Text)

```

texto2 <- tm_map(texto2, PlainTextDocument)

```

### 30.16 Transformar el texto para reemplazar algunos caracteres especiales

```

toSpace <- content_transformer(function (x , pattern) gsub(pattern, " ", x))
#texto2 <- tm_map(texto2, toSpace, "-")
texto2 <- tm_map(texto2, toSpace, "@")
texto2 <- tm_map(texto2, toSpace, "\\|")

```

### 30.17 Crear una matriz de las palabras

```

tabla_frecuencia <- DocumentTermMatrix(texto2)

```

### 30.18 Calcular la frecuencia de cada palabra

```

tabla_frecuencia <- cbind(palabras = tabla_frecuencia$dimnames$Terms,
 frecuencia = tabla_frecuencia$v)

Convertimos los valores enlazados con cbind a un objeto dataframe.
tabla_frecuencia<-as.data.frame(tabla_frecuencia)

Forzamos a que la columna de frecuencia contenga valores numéricos.
tabla_frecuencia$frecuencia<-as.numeric(tabla_frecuencia$frecuencia)

Ordenamos nuestra tabla de frecuencias de acuerdo a sus valores numéricos.
tabla_frecuencia<-tabla_frecuencia[order(tabla_frecuencia$frecuencia, decreasing=TRUE),]

```

```
palabras frecuencia
809 inteligencia 79
126 artificial 67
1378 sistemas 32
1347 ser 22
737 humano 19
1172 problemas 18

wordcloud(words = tabla_frecuencia$palabras,
 freq = tabla_frecuencia$frecuencia,
 min.freq = 5,
 max.words = 100,
 random.order = FALSE,
 colors = brewer.pal(8, "Paired"))
```



<<https://stackoverflow.com/questions/40570621/rstudio-how-to-show-plot-output-in-bottom-right-pane>>



## Chapter 31

# Temas Avanzados en Data Science con Tidyverse

### 31.1 Introducción

Este documento presenta temas avanzados en el ecosistema **tidyverse** con ejemplos prácticos en R.

#### 31.1.1 1. Manipulación avanzada con dplyr

**Evaluación *tidy*:** `{{ }}` y `:=` — funciones que reciben nombres de columnas.

- `{{ var }}` (“embrace”): permite pasar el nombre de una columna como argumento a una función que usa dplyr por dentro.
- `:=`: se usa en `mutate/summarise` cuando el nombre de la nueva columna también es una variable.
- **Para qué:** escribir tus propias funciones reutilizables sobre pipelines de dplyr.

Explicación: Uso de *tidy evaluation* para funciones personalizadas.

```
library(dplyr)

Función personalizada con tidy evaluation
mean_by_group <- function(data, group_var, value_var) {
 data %>%
 group_by({{ group_var }}) %>%
 summarise(mean_value = mean({{ value_var }}, na.rm = TRUE))
}

mean_by_group
```

```
function (data, group_var, value_var)
{
data %>% group_by({
{
group_var
}
}) %>% summarise(mean_value = mean({
{
value_var
}
}, na.rm = TRUE))
}
```

```
df <- tibble(grupo = c("A", "A", "B", "B"),
 valor = c(10, 20, 30, 40)) # los datos
```

```
mean_by_group(df, grupo, valor) # El uso de la función nueva
```

```
A tibble: 2 x 2
grupo mean_value
<chr> <dbl>
1 A 15
2 B 35
```

Otro ejemplo

```
dfnew= tibble(especie = c("setosa", "setosa", "versicolor", "versicolor"),
 longitud_sepalo = c(5.1, 4.9, 7.0, 6.4))
```

```
mean_by_group(dfnew, especie, longitud_sepalo)
```

```
A tibble: 2 x 2
especie mean_value
<chr> <dbl>
1 setosa 5
2 versicolor 6.7
```

```
mean_sd_by_group <- function(data, group_var, value_var) {
 data %>%
 group_by({{ group_var }}) %>%
 summarise(sd_value = sd({{ value_var }}, na.rm = TRUE),
 mean_value = mean({{ value_var }}, na.rm = TRUE))
}
```

```
mean_sd_by_group(dfnew, especie, longitud_sepalo)
```

```
A tibble: 2 x 3
especie sd_value mean_value
```

```
<chr> <dbl> <dbl>
1 setosa 0.141 5
2 versicolor 0.424 6.7
```

### 31.1.2 Componentes clave:

1. Definición de la función

```
mean_by_group <- function(data, group_var, value_var) { ... }
```

- data: El data frame o tibble que contiene los datos.
- group\_var: La variable por la que se agruparán los datos (por ejemplo, categoría).
- value\_var: La variable numérica de la que se calculará la media.

2. `{{ }}` (Curly-curly operator):

- Es parte del tidy evaluation en dplyr.
- Permite que los argumentos group\_var y value\_var se pasen como nombres de columna sin comillas.
- Sin esto, la función no podría interpretar correctamente los nombres de columna.

3. group\_by(`{{ group_var }}`):

- Agrupa el conjunto de datos por la variable indicada.
- Ejemplo: Si group\_var = especie, agrupa por especie.

4. summarise(mean\_value = mean(`{{ value_var }}`), na.rm = TRUE)):

- Calcula la media de la columna indicada (value\_var) dentro de cada grupo.
- na.rm = TRUE elimina valores faltantes (NA) para evitar errores.

Si olvidas `na.rm = TRUE`, funciones como `mean()` o `sd()` devuelven NA en cuanto la columna tiene un solo valor faltante, en vez de calcular el promedio.

```
datos=c(1, NA, 3:8, NA, 10:12)
mean(datos, na.rm=TRUE)
```

```
[1] 6.7
```

## 31.2 2. Modelado con tidymodels

Explicación: Flujo reproducible para regresión lineal.

```
library(tidymodels)
data(mtcars)

Convertir variables a factores
```

```
mtcars <- mtcars %>%
 mutate(cyl = factor(cyl),
 gear = factor(gear))

receta <- recipe(mpg ~ wt + hp + cyl + gear, data = mtcars)
modelo <- linear_reg() %>%
 set_engine("lm")

workflow() %>%
 add_recipe(receta) %>%
 add_model(modelo) %>%
 fit(data = mtcars)
```

```
== Workflow [trained] =====
Preprocessor: Recipe
Model: linear_reg()
##
-- Preprocessor -----
0 Recipe Steps
##
-- Model -----
##
Call:
stats::lm(formula = ..y ~ ., data = data)
##
Coefficients:
(Intercept) wt hp cyl6 cyl8 gear4
34.46173 -2.79186 -0.03424 -2.93920 -1.33080 1.42125
gear5
2.08577
```

Ver los resultados del modelo y hacer predicciones.

```
Ajustar el modelo
ajuste <- workflow() %>%
 add_recipe(receta) %>%
 add_model(modelo) %>%
 fit(data = mtcars)

Extraer el modelo ajustado
modelo_ajustado <- ajuste %>% extract_fit_parsnip()

Ver coeficientes
```

```

modelo_ajustado %>% tidy() # Coeficientes en formato tibble

A tibble: 7 x 5
term estimate std.error statistic p.value
<chr> <dbl> <dbl> <dbl> <dbl>
1 (Intercept) 34.5 2.46 14.0 2.52e-13
2 wt -2.79 0.856 -3.26 3.18e- 3
3 hp -0.0342 0.0177 -1.93 6.44e- 2
4 cyl6 -2.94 1.47 -1.99 5.71e- 2
5 cyl8 -1.33 2.78 -0.479 6.36e- 1
6 gear4 1.42 1.51 0.941 3.56e- 1
7 gear5 2.09 2.14 0.972 3.40e- 1

modelo_ajustado %>% glance() # Métricas globales (R², AIC, etc..)

A tibble: 1 x 12
r.squared adj.r.squared sigma statistic p.value df logLik AIC BIC
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 0.864 0.831 2.48 26.4 0.00000000113 6 -70.5 157. 169.
i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>

summary(modelo_ajustado$fit) # Resumen clásico del modelo lm

##
Call:
stats::lm(formula = ..y ~ ., data = data)
##
Residuals:
Min 1Q Median 3Q Max
-3.4215 -1.2428 -0.3401 0.8961 5.3657
##
Coefficients:
Estimate Std. Error t value Pr(>|t|)
(Intercept) 34.46173 2.46379 13.987 2.52e-13 ***
wt -2.79186 0.85567 -3.263 0.00318 **
hp -0.03424 0.01770 -1.935 0.06444 .
cyl6 -2.93920 1.47362 -1.995 0.05710 .
cyl8 -1.33080 2.77885 -0.479 0.63617
gear4 1.42125 1.51065 0.941 0.35580
gear5 2.08577 2.14490 0.972 0.34015

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
Residual standard error: 2.477 on 25 degrees of freedom
Multiple R-squared: 0.8638, Adjusted R-squared: 0.8311
F-statistic: 26.42 on 6 and 25 DF, p-value: 1.128e-09

```

```
Predicciones y métricas
predicciones <- predict(modelo_ajustado, mtcars)

bind_cols(mtcars, predicciones) %>%
 metrics(truth = mpg, estimate = .pred)
```

```
A tibble: 3 x 3
.metric .estimator .estimate
<chr> <chr> <dbl>
1 rmse standard 2.19
2 rsq standard 0.864
3 mae standard 1.68
```

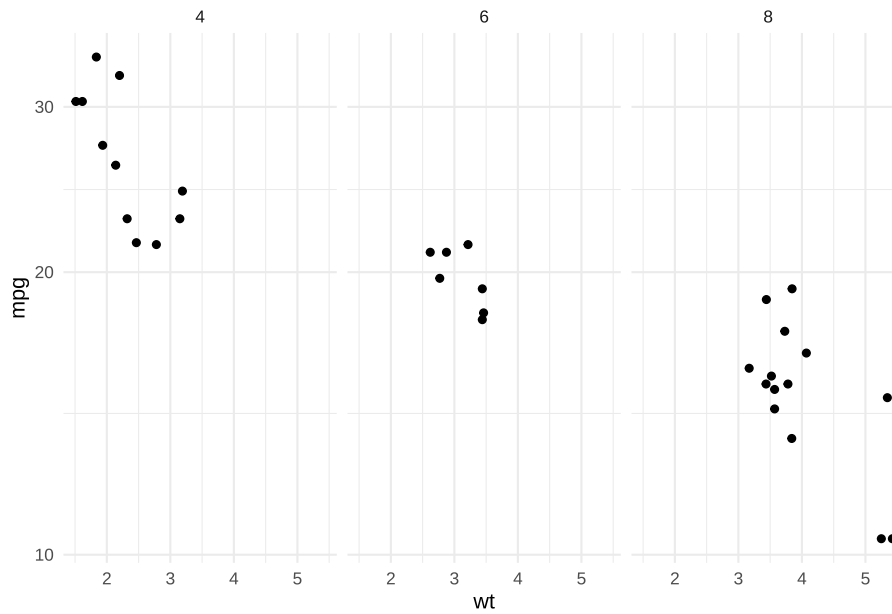
- `extract_fit_parsnip()` obtiene el modelo ajustado.
- `tidy()` muestra coeficientes y errores estándar.
- `metrics()` calcula métricas como RMSE,  $R^2$ , MAE.

### 31.3 3. Visualización avanzada con ggplot2

Explicación: Facetas y escalas personalizadas.

```
library(ggplot2)

ggplot(mtcars, aes(x = wt, y = mpg)) +
 geom_point() +
 facet_wrap(~ cyl) +
 scale_y_log10() +
 theme_minimal()
```



## 31.4 4. Datos anidados y listas con purrr

Explicación: Ajustar modelos por grupo y extraer coeficientes.

```
library(tidyr)
library(purrr)
library(broom)

nested <- mtcars %>%
 group_by(cyl) %>%
 nest()

nested <- nested %>%
 mutate(modelo = purrr::map(data, ~ lm(mpg ~ wt, data = .x)),
 coeficientes = purrr::map(modelo, tidy))

nested$data

[[1]]
A tibble: 7 x 10
mpg disp hp drat wt qsec vs am gear carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <fct> <dbl>
1 21 160 110 3.9 2.62 16.5 0 1 4 4
```

```
2 21 160 110 3.9 2.88 17.0 0 1 4 4
3 21.4 258 110 3.08 3.22 19.4 1 0 3 1
4 18.1 225 105 2.76 3.46 20.2 1 0 3 1
5 19.2 168. 123 3.92 3.44 18.3 1 0 4 4
6 17.8 168. 123 3.92 3.44 18.9 1 0 4 4
7 19.7 145 175 3.62 2.77 15.5 0 1 5 6
##
[[2]]
A tibble: 11 x 10
mpg disp hp drat wt qsec vs am gear carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <fct> <dbl>
1 22.8 108 93 3.85 2.32 18.6 1 1 4 1
2 24.4 147. 62 3.69 3.19 20 1 0 4 2
3 22.8 141. 95 3.92 3.15 22.9 1 0 4 2
4 32.4 78.7 66 4.08 2.2 19.5 1 1 4 1
5 30.4 75.7 52 4.93 1.62 18.5 1 1 4 2
6 33.9 71.1 65 4.22 1.84 19.9 1 1 4 1
7 21.5 120. 97 3.7 2.46 20.0 1 0 3 1
8 27.3 79 66 4.08 1.94 18.9 1 1 4 1
9 26 120. 91 4.43 2.14 16.7 0 1 5 2
10 30.4 95.1 113 3.77 1.51 16.9 1 1 5 2
11 21.4 121 109 4.11 2.78 18.6 1 1 4 2
##
[[3]]
A tibble: 14 x 10
mpg disp hp drat wt qsec vs am gear carb
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <fct> <dbl>
1 18.7 360 175 3.15 3.44 17.0 0 0 3 2
2 14.3 360 245 3.21 3.57 15.8 0 0 3 4
3 16.4 276. 180 3.07 4.07 17.4 0 0 3 3
4 17.3 276. 180 3.07 3.73 17.6 0 0 3 3
5 15.2 276. 180 3.07 3.78 18 0 0 3 3
6 10.4 472 205 2.93 5.25 18.0 0 0 3 4
7 10.4 460 215 3 5.42 17.8 0 0 3 4
8 14.7 440 230 3.23 5.34 17.4 0 0 3 4
9 15.5 318 150 2.76 3.52 16.9 0 0 3 2
10 15.2 304 150 3.15 3.44 17.3 0 0 3 2
11 13.3 350 245 3.73 3.84 15.4 0 0 3 4
12 19.2 400 175 3.08 3.84 17.0 0 0 3 2
13 15.8 351 264 4.22 3.17 14.5 0 1 5 4
14 15 301 335 3.54 3.57 14.6 0 1 5 8
nested$modelo

[[1]]
##
```



```
Call:
lm(formula = mpg ~ wt, data = .x)
##
Coefficients:
(Intercept) wt
28.41 -2.78
##
##
[[2]]
##
Call:
lm(formula = mpg ~ wt, data = .x)
##
Coefficients:
(Intercept) wt
39.571 -5.647
##
##
[[3]]
##
Call:
lm(formula = mpg ~ wt, data = .x)
##
Coefficients:
(Intercept) wt
23.868 -2.192
```

nested\$coeficientes

```
[[1]]
A tibble: 2 x 5
term estimate std.error statistic p.value
<chr> <dbl> <dbl> <dbl> <dbl>
1 (Intercept) 28.4 4.18 6.79 0.00105
2 wt -2.78 1.33 -2.08 0.0918
##
[[2]]
A tibble: 2 x 5
term estimate std.error statistic p.value
<chr> <dbl> <dbl> <dbl> <dbl>
1 (Intercept) 39.6 4.35 9.10 0.00000777
2 wt -5.65 1.85 -3.05 0.0137
##
[[3]]
A tibble: 2 x 5
term estimate std.error statistic p.value
```

```
<chr> <dbl> <dbl> <dbl> <dbl>
1 (Intercept) 23.9 3.01 7.94 0.00000405
2 wt -2.19 0.739 -2.97 0.0118
```

La alternativa es hacer cada factor individualmente

- Hacemos un modelo `lm(mpg ~ wt)` para cada subconjunto de datos agrupados por `cyl == 6`

```
modelo_cyl6 <- lm(mpg ~ wt, data = filter(mtcars, cyl == 6))
```

```
summary(modelo_cyl6)
```

```
##
Call:
lm(formula = mpg ~ wt, data = filter(mtcars, cyl == 6))
##
Residuals:
```

##	Mazda RX4	Mazda RX4 Wag	Hornet 4 Drive	Valiant	Merc 280
##	-0.1250	0.5840	1.9292	-0.6897	0.3547
##	Merc 280C	Ferrari Dino			
##	-1.0453	-1.0080			

```
##
Coefficients:
```

##		Estimate	Std. Error	t value	Pr(> t )
##	(Intercept)	28.409	4.184	6.789	0.00105 **
##	wt	-2.780	1.335	-2.083	0.09176 .

```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
Residual standard error: 1.165 on 5 degrees of freedom
Multiple R-squared: 0.4645, Adjusted R-squared: 0.3574
F-statistic: 4.337 on 1 and 5 DF, p-value: 0.09176
```

---

## Chapter 32

# Leaflet: Mapas interactivos

```
[1] "2026-08-01"
library(tidyverse)
```

### 32.1 Bajar los datos de la web del USGS

Website: <https://earthquake.usgs.gov/earthquakes/feed/v1.0/csv.php>

Este capítulo **descarga datos en vivo desde internet**. Si al ejecutar ves un error como `Error in download.file(...): cannot open URL` o un `HTTP error`, casi siempre significa que no hay conexión a internet o que el servidor no está respondiendo en ese momento. Verifica tu conexión y vuelve a intentar. En este libro el código usa una copia local guardada en `Datos/` cuando la descarga falla, para que el capítulo siempre se pueda generar. Encontrarás más errores frecuentes y cómo resolverlos en el capítulo *Errores comunes* (14).

```
library(data.table)
Intenta bajar los datos en vivo del USGS; si no hay conexión o el servidor
no responde, usa la copia local guardada en Datos/.
earthquakes <- tryCatch(
 fread("https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_month.csv"),
 error = function(e) readr::read_csv("Datos/Earthquake_all_month.csv")
)

head(earthquakes, n=10)
```

```
time latitude longitude depth mag magType nst gap
<POSc> <num> <num> <num> <num> <char> <int> <int>
1: 2026-08-02 03:16:45 10.53280 -68.6899 10.0000 4.50 mb 35 100
2: 2026-08-02 03:15:17 19.29800 -155.2372 0.7600 2.28 ml 42 174
```

```

3: 2026-08-02 03:04:56 57.79400 -153.9370 35.0000 1.50 ml 12 105
4: 2026-08-02 03:04:02 38.18360 -117.8322 6.3127 1.93 ml 20 97
5: 2026-08-02 02:37:37 34.84617 -119.0565 8.1900 1.27 ml 31 87
6: 2026-08-02 02:28:56 32.64183 -115.7475 5.7800 1.50 ml 34 61
7: 2026-08-02 02:23:07 32.64183 -115.7505 5.7200 2.03 ml 42 47
8: 2026-08-02 02:22:31 32.61783 -115.5773 3.1100 1.29 ml 26 76
9: 2026-08-02 01:55:05 39.24217 -123.2180 3.8200 1.17 md 7 107
10: 2026-08-02 01:54:27 58.16200 -155.3240 5.0000 0.70 ml 10 70
dmin rms net id updated
<num> <num> <char> <char> <POSc>
1: 0.75100 0.7300 us us6000thk4 2026-08-02 03:31:01
2: 0.02045 0.1700 hv hv75011652 2026-08-02 03:18:45
3: 0.40000 0.4000 ak aka2026pdqvke 2026-08-02 03:06:31
4: 0.49200 0.4722 nn nn00922699 2026-08-02 03:06:30
5: 0.08037 0.2000 ci ci40665010 2026-08-02 02:41:08
6: 0.01932 0.2100 ci ci40665002 2026-08-02 02:32:20
7: 0.02168 0.2300 ci ci40664994 2026-08-02 02:26:31
8: 0.07988 0.2000 ci ci40664986 2026-08-02 02:26:00
9: 0.06451 0.0500 nc nc75409947 2026-08-02 02:32:22
10: 0.00000 0.2000 ak aka2026pdombb 2026-08-02 01:55:28
place type horizontalError depthError
<char> <char> <num> <num>
1: 7 km SSW of Yumare, Venezuela earthquake 3.91 1.8410
2: 16 km S of Volcano, Hawaii earthquake 0.34 0.2900
3: 28 km N of Larsen Bay, Alaska earthquake 9.30 6.9384
4: 33 km SE of Mina, Nevada earthquake NA 0.6756
5: 8 km E of Pine Mountain Club, CA earthquake 0.31 0.5100
6: 17 km WNW of Progreso, B.C., MX earthquake 0.24 0.3400
7: 17 km WNW of Progreso, B.C., MX earthquake 0.25 0.4100
8: 4 km N of Progreso, B.C., MX earthquake 0.21 0.3200
9: 3 km SSW of Redwood Valley, CA earthquake 0.33 1.4400
10: 83 km NW of Karluk, Alaska earthquake 5.90 NA
magError magNst status locationSource magSource
<num> <int> <char> <char> <char>
1: 0.0610000 80 reviewed us us
2: 0.1600000 3 automatic hv hv
3: 0.2000000 4 automatic ak ak
4: 0.3700000 8 automatic nn nn
5: 0.3321156 29 automatic ci ci
6: 0.2226233 28 automatic ci ci
7: 0.3067757 26 automatic ci ci
8: 0.1111436 19 automatic ci ci
9: 0.1900000 3 automatic nc nc
10: 0.5000000 4 automatic ak ak

```

```
unique(earthquakes$type)

[1] "earthquake" "explosion" "quarry blast"
[4] "mining explosion" "ice quake" "experimental explosion"

earthquakes %>%
 filter(type=="sonic boom")

Empty data.table (0 rows and 22 cols): time,latitude,longitude,depth,mag,magType...

quarry_blasts=earthquakes %>%
 filter(type=="quarry blast")
```

## 32.2 Si bajo los datos a mi computadora

Use esta alternativa si bajo los datos a su computadora

```
#library(readr)
#Earthquake_all_month <- read_csv("Datos/Earthquake_all_month.csv")

#EQ=Earthquake_all_month
#head(EQ, n=10)

names(earthquakes)

[1] "time" "latitude" "longitude" "depth"
[5] "mag" "magType" "nst" "gap"
[9] "dmin" "rms" "net" "id"
[13] "updated" "place" "type" "horizontalError"
[17] "depthError" "magError" "magNst" "status"
[21] "locationSource" "magSource"

str(earthquakes)

Classes 'data.table' and 'data.frame': 10814 obs. of 22 variables:
$ time : POSIXct, format: "2026-08-02 03:16:45" "2026-08-02 03:15:17" ...
$ latitude : num 10.5 19.3 57.8 38.2 34.8 ...
$ longitude : num -68.7 -155.2 -153.9 -117.8 -119.1 ...
$ depth : num 10 0.76 35 6.31 8.19 ...
$ mag : num 4.5 2.28 1.5 1.93 1.27 1.5 2.03 1.29 1.17 0.7 ...
$ magType : chr "mb" "ml" "ml" "ml" ...
$ nst : int 35 42 12 20 31 34 42 26 7 10 ...
$ gap : int 100 174 105 97 87 61 47 76 107 70 ...
$ dmin : num 0.751 0.0204 0.4 0.492 0.0804 ...
$ rms : num 0.73 0.17 0.4 0.472 0.2 ...
$ net : chr "us" "hv" "ak" "nn" ...
$ id : chr "us6000thk4" "hv75011652" "aka2026pdqvke" "nn00922699" ...
```

```
$ updated : POSIXct, format: "2026-08-02 03:31:01" "2026-08-02 03:18:45" .
$ place : chr "7 km SSW of Yumare, Venezuela" "16 km S of Volcano, Hawai.
$ type : chr "earthquake" "earthquake" "earthquake" "earthquake" ...
$ horizontalError: num 3.91 0.34 9.3 NA 0.31 0.24 0.25 0.21 0.33 5.9 ...
$ depthError : num 1.841 0.29 6.938 0.676 0.51 ...
$ magError : num 0.061 0.16 0.2 0.37 0.332 ...
$ magNst : int 80 3 4 8 29 28 26 19 3 4 ...
$ status : chr "reviewed" "automatic" "automatic" "automatic" ...
$ locationSource : chr "us" "hv" "ak" "nn" ...
$ magSource : chr "us" "hv" "ak" "nn" ...
- attr(*, ".internal.selfref")=<pointer: 0x106368200>
```

### 32.3 Importación de Paquetes y Datos

```
library(tidyverse)
library(leaflet)
#library(rinat)
library(RColorBrewer)
library(flextable)
```

Seleccionar un subgrupo de los datos, solamente los primeros 100 datos

```
nrow(earthquakes)
```

```
[1] 10814
```

```
EQ_100 = earthquakes |> slice(1:100)
head(EQ_100)
```

```
time latitude longitude depth mag magType nst gap
<POSct> <num> <num> <num> <num> <char> <int> <int>
1: 2026-08-02 03:16:45 10.53280 -68.6899 10.0000 4.50 mb 35 100
2: 2026-08-02 03:15:17 19.29800 -155.2372 0.7600 2.28 ml 42 174
3: 2026-08-02 03:04:56 57.79400 -153.9370 35.0000 1.50 ml 12 105
4: 2026-08-02 03:04:02 38.18360 -117.8322 6.3127 1.93 ml 20 97
5: 2026-08-02 02:37:37 34.84617 -119.0565 8.1900 1.27 ml 31 87
6: 2026-08-02 02:28:56 32.64183 -115.7475 5.7800 1.50 ml 34 61
dmin rms net id updated
<num> <num> <char> <char> <POSct>
1: 0.75100 0.7300 us us6000thk4 2026-08-02 03:31:01
2: 0.02045 0.1700 hv hv75011652 2026-08-02 03:18:45
3: 0.40000 0.4000 ak aka2026pdqvke 2026-08-02 03:06:31
4: 0.49200 0.4722 nn nn00922699 2026-08-02 03:06:30
5: 0.08037 0.2000 ci ci40665010 2026-08-02 02:41:08
6: 0.01932 0.2100 ci ci40665002 2026-08-02 02:32:20
place type horizontalError depthError
```

```
<char> <char> <num> <num>
1: 7 km SSW of Yumare, Venezuela earthquake 3.91 1.8410
2: 16 km S of Volcano, Hawaii earthquake 0.34 0.2900
3: 28 km N of Larsen Bay, Alaska earthquake 9.30 6.9384
4: 33 km SE of Mina, Nevada earthquake NA 0.6756
5: 8 km E of Pine Mountain Club, CA earthquake 0.31 0.5100
6: 17 km WNW of Progreso, B.C., MX earthquake 0.24 0.3400
magError magNst status locationSource magSource
<num> <int> <char> <char> <char>
1: 0.0610000 80 reviewed us us
2: 0.1600000 3 automatic hv hv
3: 0.2000000 4 automatic ak ak
4: 0.3700000 8 automatic nn nn
5: 0.3321156 29 automatic ci ci
6: 0.2226233 28 automatic ci ci
```

## 32.4 Mapa básico con Leaflet

**Leaflet** es una biblioteca de **JavaScript** para mapas interactivos, creada por **Volodymyr Agafonkin** en 2011. Su ventaja es que es ligera y funciona en cualquier navegador. **RStudio/Posit** creó el paquete **leaflet** para R en 2015, permitiendo construir estos mapas sin escribir JavaScript. Los mapas base (“tiles”) provienen de **OpenStreetMap**, un proyecto colaborativo y abierto de cartografía mundial.

**leaflet()** — construye un mapa interactivo por capas, de forma parecida a **ggplot()**, pero uniendo las capas con el *pipe* (%>%).

- **leaflet(data)**: inicia el mapa con un conjunto de datos.
- **addTiles()**: añade el mapa base (por defecto, de OpenStreetMap).
- **addMarkers()** / **addCircleMarkers()**: colocan puntos; `~longitude` y `~latitude` (la tilde ~) indican qué columnas usar para las coordenadas, y **popup** el texto que aparece al hacer clic.

```
names(earthquakes)
```

```
[1] "time" "latitude" "longitude" "depth"
[5] "mag" "magType" "nst" "gap"
[9] "dmin" "rms" "net" "id"
[13] "updated" "place" "type" "horizontalError"
[17] "depthError" "magError" "magNst" "status"
[21] "locationSource" "magSource"
```

```
leaflet(EQ_100) %>%
 addTiles() %>%
 addCircleMarkers(lng = ~longitude,
 lat = ~latitude,
```

```
popup = ~mag,
color= ~(type))

#unique(earthquakes$type)
```

```
#color= ~(type),
```

## 32.5 Usando la paletas de color de viridis

- <https://cran.r-project.org/web/packages/viridis/vignettes/intro-to-viridis.html>
- Evaluar las siguientes paletas de colores:
  - viridis
  - inferno
  - plasma
  - magma
  - cividis
  - rocket
  - turbo

Pasos

1. Seleccionar los datos en las áreas de PR, latitud y longitud
2. Crear una paleta de colores/ o usar viridis
3. Crear un mapa con los datos de los terremotos

```
names(EQ_100)
unique(EQ_100$type)
pal = colorFactor(palette = "magma", domain = NULL)

leaflet(EQ_100) %>%
 addTiles() %>%
 addCircleMarkers(lng = ~longitude,
 lat = ~latitude,
 color= ~pal(mag),
 popup = ~paste("Magnitud", mag)
)
```

---



## 32.6 Trabajando con datos de iNaturalist

### 32.6.1 Ahora crea un mapa de los terremotos alrededor de Puerto Rico

**Obtener los datos de observaciones de una especie en iNaturalist desde R:**

Los datos a utilizar se obtendrán desde iNaturalist, una aplicación con base en la ciencia ciudadana que mantiene un registro de ocurrencias de especies.

La función `get_inat_obs()` permite acceder a los datos disponibles en iNaturalist, especificando los siguientes parámetros:

`get_inat_obs()` consulta la API de iNaturalist en tiempo real: requiere conexión a internet y el número de observaciones puede variar entre ejecuciones a medida que la comunidad añade registros.

- **taxon\_name:** La especie de interés, siempre escrita entre comillas (“). Puede utilizar nombre común o científico.
- **quality:** Siendo una aplicación basada en ciencia ciudadana, la calidad de algunos datos podría ser baja, debido a que les falta información - como nombre científico o ubicación - o que contienen información errónea - especies mal identificadas-. Colocando **'research'** en este parámetro se especifica que se desea obtener sólo aquellas observaciones con alta calidad - que tengan información completa y cuya identificación haya sido confirmada por varias personas de la comunidad -.
- **geo:** Al igual que el parámetro **quality**, este parámetro cuyo valor es lógico (**TRUE** o **FALSE**), es para especificar que se quiere obtener sólo los datos que están georeferenciados (cuando se coloca como **TRUE**).
- **maxresults:** A través del API (Interfaz de Programación de Aplicaciones) de iNaturalist se puede acceder a un máximo de 10,000 observaciones en una sola búsqueda. Con este parámetro, cuyo valor debe ser numérico, se especifica cuántas observaciones se desea obtener, y no se debe exceder de 10,000 por búsqueda.
- **bounds:** Un vector que contiene los límites geográficos de la búsqueda. Se puede insertar un cuadro delimitador ya especificado en un objeto, concatenar los valores del vector, o insertar un objeto de características simples (Simple Feature, sf). El argumento **bounds** es importante porque si obtenemos datos de iNaturalist, las únicas columnas que obtenemos con información acerca de la región o área donde se hicieron las observaciones son:
- **place\_guess:** Contiene el nombre de la región, pero su precisión y los nombres utilizados para cada región varía mucho.

- *latitude*: Posición en el eje vertical del sistema de coordenadas.
- *longitude*: Posición en el eje horizontal del sistema de coordenadas.  
Esto crea un problema a la hora de filtrar los datos por el nombre de una región, pues es difícil encontrar un solo valor que encompase a todas las variantes del nombre utilizado para esa región. iNaturalist se encuentra a nivel mundial, y usted puede acceder a todos esos datos. Sin embargo, hoy accederemos solamente a datos de Puerto Rico. Entonces, lo primero antes de acceder a los datos de iNaturalist, es crear un objeto que contenga los límites geográficos de interés crea. Para esto se crea un objeto que contenga un cuadro delimitador (*bounding box*):

```
cajapr <- c(
 17.75, #Latitud Sur
 -67.4, #Longitud Oeste
 18.75, #Latitud Norte
 -65.15 #Longitud Este
)

cajapr

[1] 17.75 -67.40 18.75 -65.15

library(rinat)
```

Ahora podemos acceder a datos de Puerto Rico utilizando el objeto `cajapr`. Para este ejercicio utilizaremos datos de orquídeas pertenecientes al género *Eulophia*. En Puerto Rico se han reportado 3 especies de este género *Eulophia alta*, *Eulophia maculata* y *Eulophia graminea*. De estos, sólo *E. alta* es considerada nativa de la isla. Entonces, utilizando la función `get_inat_obs` podemos acceder a los datos de *Eulophia* en Puerto Rico

```
Intenta consultar iNaturalist en vivo y guarda una copia local; si la
consulta falla (sin conexión o API no disponible), usa esa copia.
eulophia <- tryCatch(
 {
 d <- get_inat_obs(taxon_name = "Eulophia",
 quality = 'research',
 geo = TRUE,
 maxresults = 1000)
 readr::write_csv(d, "Datos/eulophia_inat.csv")
 d
 },
 error = function(e) readr::read_csv("Datos/eulophia_inat.csv")
)

eulophia$scientific_name <- as.factor(eulophia$scientific_name)
```

```
flectable(head(eulophia)) # Para ver las primeras 6 lineas en una tabla bonita
```

scientific_name	datetime	description	place_guess	latitude	longitude	tag_list	common_name	url
Eulophia picta	2026-08-02 09:38:07 +1000		Queensland, AU	-27.64107	153.13197		Shepherd's Crook Orchid	<a href="https://w">https://w</a>
Eulophia maculata	2026-07-17 08:10:00 -0400		Princeton, FL, USA	25.55715	-80.36470		Monk Orchid	<a href="https://w">https://w</a>
Eulophia maculata	2026-08-01 12:59:33 -0400		Parkland, FL, US	26.32145	-80.22162		Monk Orchid	<a href="https://w">https://w</a>
Eulophia maculata	2026-08-01 13:06:25 -0400		Parkland, FL, US	26.32139	-80.22096		Monk Orchid	<a href="https://w">https://w</a>
Eulophia maculata	2026-08-01 14:42:40 -0400		Parkland, FL, US	26.32147	-80.22150		Monk Orchid	<a href="https://w">https://w</a>
Eulophia maculata	2026-08-01 17:15:06 -0400		Batten Rd, Brooksville, FL, US	28.49245	-82.35167		Monk Orchid	<a href="https://w">https://w</a>

bounds = cajapr Con el siguiente código podemos ver de forma rápida cuántas observaciones de cada especie hemos obtenido de iNaturalist:

```
summary(as.factor(eulophia$scientific_name))
```

```
Eulophia alta Eulophia andamanensis Eulophia angolensis
14 5 6
Eulophia beravensis Eulophia bolusii Eulophia bouliawongo
1 1 1
Eulophia calcarata Eulophia caricifolia Eulophia citrina
1 4 4
Eulophia clitellifera Eulophia cochlearis Eulophia cordylinophylla
2 1 1
Eulophia cucullata Eulophia dentata Eulophia diffusiflora
```

##	2	1	5
##	Eulophia ensata	Eulophia eulophioides	Eulophia exaltata
##	5	1	1
##	Eulophia exigua	Eulophia explanata	Eulophia flabellata
##	1	3	1
##	Eulophia flavopurpurea	Eulophia graminea	Eulophia guineensis
##	1	323	6
##	Eulophia herbacea	Eulophia hians	Eulophia hians hians
##	1	4	1
##	Eulophia hians nutans	Eulophia horsfallii	Eulophia lamellata
##	1	18	1
##	Eulophia lonchophylla	Eulophia maculata	Eulophia micrantha
##	2	279	2
##	Eulophia nuda	Eulophia obtusa	Eulophia odontoglossa
##	19	1	1
##	Eulophia ovalis	Eulophia ovalis bainesii	Eulophia ovalis ovalis
##	10	3	3
##	Eulophia parviflora	Eulophia parvilabris	Eulophia pauciflora
##	10	2	1
##	Eulophia petersii	Eulophia petiolata	Eulophia picta
##	10	2	184
##	Eulophia pulchra	Eulophia recurva	Eulophia roseovariegata
##	2	1	2
##	Eulophia siamensis	Eulophia spathulifera	Eulophia speciosa
##	2	2	11
##	Eulophia streptopetala	Eulophia tristis	Eulophia welwitschii
##	5	1	1
##	Eulophia zeyheriana	Eulophia zollingeri	
##	2	25	

Ahora que hemos obtenido los datos, podemos observar su distribución en el mapa de Puerto Rico utilizando `leaflet`.

## 32.7 Utilizando Leaflet

La función base del paquete `leaflet` es `leaflet()`, con esta se activa unartilugio (*widget* en inglés) sobre el cuál se construye el mapa. Esta función debe estar unida a otra función como `addTiles()` que añade el mapa.

```
leaflet() %>%
 addTiles()
```

Una vez tenemos el artilugio y el mapa, añadimos una capa que incluye los datos que se desean visualizar con marcadores sobre el mapa con la función `addMarkers()` dentro del cual se deben especificar los parámetros `lng` (longitud) y `lat` (latitud). Note que también se especifica el `data.frame` que contiene

los datos bajo la función `leaflet()`. Cuando se trabaja en Leaflet, al especificar las variables a utilizar, es necesario colocar una virgulilla (`~`) antes de la variable. Esto le deja saber a R que lo que se especifica es una variable dentro del `data.frame` mencionado en la primera capa bajo la función `leaflet()`.

```
names(eulophia)
leaflet(eulophia) %>%
 addTiles() %>%
 addMarkers(lng = ~longitude,
 lat = ~latitude)
```

Una vez los marcadores están establecidos, es posible añadir etiquetas que contengan información acerca de las observaciones con el parámetro `popup` de la función `addMarkers()`. Por ejemplo, se pueden colocar etiquetas que contengan información del lugar de la observación especificando que la etiqueta contenga la información de la variable `place_guess`. Para ver la etiqueta debe hacer clic sobre el marcador de interés.

```
names(eulophia)
leaflet(eulophia) %>%
 addTiles() %>%
 addMarkers(lng = ~ longitude,
 lat = ~ latitude,
 popup = ~ place_guess) # cambiar a la variable de interés
```

La función `paste0` es una función base de R que concatena todos los elementos en una línea de caracteres. Los caracteres introducidos serán interpretados como UTF-8 o codificación de caracteres. Estos caracteres pueden ser mezclados con valores de una variable de caracteres en un `data.frame`. En el siguiente ejemplo se crea una etiqueta que contiene la ubicación y la especie observada. Se utiliza la función `paste0()` para combinar caracteres y las variables `scientific_name` y `place_guess` del `data.frame`.

```
leaflet(eulophia) %>%
 addTiles() %>%
 addMarkers(lng = ~ longitude,
 lat = ~ latitude,
 popup = paste0("Especie: ", "<I>", eulophia$scientific_name, "</I>",
 "
",
 "Colectado donde: ", eulophia$place_guess)
)
```

### 32.7.1 addAwesomeMarkers: Cambiando el diseño de los marcadores

La función `addAwesomeMarkers()` puede ser utilizada en lugar de `addMarkers()` y permite utilizar marcadores con diferentes diseños que se pueden obtener de

fontawesome. Es posible cambiar los colores de los marcadores utilizando la codificación de colores RGB en formato (#RRGGBB). La siguiente guía de colores en R creada por Melissa Clarkson (2010) explica como utilizar colores en R: A guide to using color in R. Adicional, esta guía creada por Derek H. Ogle (2022) es muy útil.

En `schemecolor` podemos encontrar las codificaciones de los colores y muchos funcionan para `leaflet`. Para hacer estos cambios utilizamos la función `awesomeIcons()` bajo el parámetro `icon` dentro de `addAwesomeMarkers()`.

Dentro de la función `awesomeIcons()` podemos especificar varios parámetros como:

- **icon:** El icono que desea utilizar para su marcador. Debe poner el nombre que aparece en la página web de iconos que esté utilizando (*i.e.*, fontawesome).
- **library:** Cuando se especifica un icono, es necesario especificar la biblioteca de donde se saca el icono. Si es de fontawesome sería 'fa'.
- **markerColor:** Para cambiar el color del marcador (parte que rodea el icono), utilizando la codificación de colores de `schemecolor`.
- **iconColor:** Para cambiar el color del icono del marcador, también con la codificación de `schemecolor`.

```
leaflet() %>%
 addTiles() %>%
 addAwesomeMarkers(lng = eulophia$longitude,
 lat = eulophia$latitude,
 popup = paste0("Especie: ", "<I>", eulophia$scientific_name, "</I>",
 "
",
 "Colectado por: ", eulophia$place_guess),
 icon = awesomeIcons(icon = "leaf", library = "fa",
 markerColor = "blue",
 iconColor = "#FFFFFF")
)
```

Cuando se visualiza una gran cantidad de datos, esto puede resultar en el solapamiento de los marcadores, lo cual no es muy agradable a la vista. Es posible crear agregados que ayuden a visualizar mejor y faciliten el movimiento a través del mapa con el parámetro `clusterOptions` bajo la función `addAwesomeMarkers()`. El valor de este parámetro debe ser la función `markerClusterOptions()`.

```
leaflet() %>%
 addTiles() %>%
 addAwesomeMarkers(lng = eulophia$longitude, lat = eulophia$latitude,
 popup = paste0("Especie: ", "<I>", eulophia$scientific_name, "</I>",
 "
",
```

```

 "Colectado por: ", eulophia$place_guess),
 icon = awesomeIcons(icon = "leaf", library = "fa",
 markerColor = "darkgreen",
 iconColor = "#FFFFFF"),
 clusterOptions = markerClusterOptions()
)

```

Y para culminar, es posible cambiar el diseño del mapa utilizando la función `addProviderTiles()` en lugar de `addTiles()`. Puede echar un vistazo a los diferentes diseños disponibles para Leaflet la página de proveedores de Leaflet en GitHub. Actualmente Leaflet en R admite diseños de los siguientes proveedores: OpenStreetMap, MapQuestOpen, Stamen, Esri and OpenWeatherMap.

```

icon1 <- awesomeIcons(icon = "smile-o",
 iconColor = "#FFFFFF",
 library = "fa",
 markerColor = "darkgreen")

leaflet() %>%
 addProviderTiles("OpenStreetMap.Mapnik") %>%
 addAwesomeMarkers(lng = eulophia$longitude, lat = eulophia$latitude,
 popup = paste0("Especie: ", "<I>", eulophia$scientific_name, "</I>",
 "
",
 "Colectado por: ", eulophia$place_guess),
 icon = icon1
)

```

## 32.8 Trabajando Variables Categóricas con Leaflet:

Es posible distinguir entre especies utilizando diferentes colores, pero debe utilizar la función `addCircleMarkers()` en lugar de `addAwesomeMarkers()`. Para colorear grupos en Leaflet es necesario generar una paleta de colores para las variables de interés de acuerdo al tipo de variable (*i.e.*, numérica, factorial). Para esto se utilizan las funciones `colorFactor()`, `colorNumeric()`, `colorBin()`, o `colorQuantile()`. `colorFactor()` y `colorNumeric()` generan paletas de colores para variables factoriales y numéricas, respectivamente; mientras que `colorBin()` y `colorQuantile()` generan paletas de colores para valores numéricos resumidos en grupos.

En este caso se está trabajando con una variable factorial que divide las observaciones entre 3 especies de orquídeas. Primero se crea una paleta de colores para una variable factorial con la función `colorFactor()`. Bajo esta función se especifican los parámetros `palette` y `levels`. `palette` para especificar la paleta de colores a utilizar y `levels` especifica los niveles de la variable factorial.

La paleta de colores que se utiliza ("Dark2") proviene del paquete RColorBrewer. Esta paleta de colores para los niveles de la variable factorial *scientific\_name* del data.frame *eulophia* se guardará en un objeto llamado *pal*.

```
mypal <- colorFactor(palette = 'Set3',
 levels = levels(eulophia$scientific_name))

eulophia$scientific_name=as.factor(eulophia$scientific_name)

mypal <- colorFactor(palette = 'Set3',
 levels = levels(eulophia$scientific_name))
```

Ahora que se ha especificado los colores para los valores factoriales, es posible utilizar el objeto *pal* para colorear los marcadores circulares del mapa de Leaflet de acuerdo a la especie observada especificando el parámetro *color*

```
leaflet(eulophia) %>%
 addTiles() %>%
 addCircleMarkers(lng = ~ longitude, lat = ~ latitude,
 popup = paste0("Especie: ", "<I>", eulophia$scientific_name,
 "
",
 "Colectado por: ", eulophia$place_guess),
 color = ~ mypal(scientific_name))
```

Para ayudar con la visualización de datos pertenecientes a diferentes grupos, en ocasiones es bueno presentar una leyenda que indique los colores que representan cada grupo. Es posible añadir una leyenda al mapa de Leaflet utilizando la función *addLegend()* y especificando los siguientes parámetros:

- **position:** Para determinar la posición de la leyenda, en este caso se colocará en la parte superior derecha, "topright", del mapa, pero podría colocarla en la parte inferior derecha ("bottomright"), la parte inferior izquierda ("bottomleft"), o la parte superior izquierda ("topleft").
- **pal:** La paleta de colores a utilizar generada a través de la función *colorFactor()* o sus equivalentes (*i.e.*, *colorNumeric()*)
- **values:** Los valores o la variable u objeto que contiene los valores utilizados.
- **title:** El título de la leyenda especificado como una línea de caracteres.

```
leaflet(eulophia) %>%
 addTiles() %>%
 addCircleMarkers(lng = ~ longitude, lat = ~ latitude,
 popup = paste0("Especie: ", "<I>", eulophia$scientific_name,
 "
",
 "Colectado por: ", eulophia$place_guess),
 color = ~ pal(scientific_name)) %>%
```



```

addLegend(position = "topright",
 pal = pal,
 values = ~ scientific_name,
 title = "Species"
)

iconcolor <- function(scientific_name)

leaflet() %>%
 addProviderTiles("Esri.WorldImagery") %>%
 addAwesomeMarkers(lng = eulophia$longitude, lat = eulophia$latitude,
 popup = paste0("Especie: ", "<I>", eulophia$scientific_name, "</I>",
 "
",
 "Colectado por: ", eulophia$place_guess),
 icon = icon1
)

```

L.circleMarker([place.lon, place.lat], { color: getColor(place.constructdate), //  
 you can call the getColor function fillColor: getColor(place.constructdate), fil-  
 lOpacity: 0.5 })



## Appendix A

# Appendix A

### A.1 Ejercicio de Transformación

---

#### Capítulo de Transformación de Datos

#### Ejercicios con destrezas de aplicar algunas funciones de transformaciones de datos

Para tener los puntos tiene que enseñar TODOS los scripts, los resultados y explicar en palabras los que encontró dentro de un documento bien organizado.

```
library(datos)
attach(bateadores)
library(tidyverse)
library(gt)
```

Use el data frame de “bateadores” en el paquete “datos”.

El paquete `datos` ofrece versiones en español de conjuntos de datos clásicos, con los nombres de las variables traducidos (por ejemplo, `carreras`, `al_bate` o `id_jugador`).

```
gt(head(bateadores, n=3))
```

---

id_jugador	id_anio	orden_equipos	id_equipo	id_liga	juegos	al_bate	carrera
aardsda01	2004	1	SFN	NL	11	0	
aardsda01	2006	1	CHN	NL	45	2	
aardsda01	2007	1	CHA	AL	25	0	

### 1. Busca el bateador que tuvo más carreras en cualquier año

```
max(bateadores$carreras)
```

```
[1] 198
```

```
bateadores |> dplyr::select(id_jugador, al_bate) |>
 arrange(desc(al_bate)) |>
 slice_max(al_bate)
```

```
id_jugador al_bate
99194 rollij01 716
```

### 2. ¿Cuál es el nombre del bateador (id\_jugador) que estuvo más veces “al\_bate”? Prepara una lista del bateador más frecuente al bate al menos

### 3. ¿Cuáles son las “ligas” de baseball (pelota) que están incluidas en este archivo?

### 4. Selecciona los años (1900, 1950, 2000 y 2020) y jonrones y hacer una tabla por año que demuestra el máximo de jonrones para cada año

¿Cuál fue el máximo de jonrones por año para los años seleccionados? Haga una lista.

**5. Haz una tabla de la los jugadores que jugaron más años**

- ¿Cuál es el jugador que ha jugado más años?
- 

**6. Selecciona solamente la liga “AL”, los años desde 2000 en adelante, y determina cuál es la suma de “carreras” anotadas por cada equipo.**

- a) ¿Cual es el equipo que tiene mayor carreras?
  - b) ¿Cual es el equipo que tiene menor carreras?
  - c) Como explica que ese equipo tiene tan pocas carreras. Piensa en un tipo de “Data wrangling”, que podría explicar esto (BONO), si lo resuelve tendrá un bono
- 
-



# Appendix B

```
[1] "2026-08-01"
```

---

## Ejercicios para las funciones de transformaciones 2

Seleccione los siguientes datos sobre COVID en PR

**LEEN** bien lo que estoy pidiendo. El nombre de las columnas pueden engañar!!!

El código lee los datos con `fread()` desde una URL de GitHub, por lo que necesitas conexión a internet para ejecutarlo.

```
library(tidyverse)
library(data.table)
url_COVID_PR=fread("https://raw.githubusercontent.com/rafalab/pr-covid/master/dashboard/data/Dato

library(gt)
Covid=url_COVID_PR
gt(tail(Covid, n=2))
```

Fecha	Muertes	IncrementoMuertes	CasosPCR_Salud	IncCasosPCR_Salud	C
10/18/21	3209	2	151033		6
10/19/21	3213	4	151089		56

1. Calcular el promedio de Muertes por día de nuevos muertos desde el comienzo hasta el último día del archivo

2. ¿Cuál es el máximo de nuevos casos en un día

3. ¿Calcular la cantidad cumulativa de número de camas CamasICU

- Enseñe solamente las últimas 4 filas

4. Haz una gráfica de la varianza y como cambia a través del tiempo.

- usa puntos para cada fecha
- añade una regresión LOESS
- cambia los nombres de los ejes para algo más claro
- los colores de los puntos
- cambia el color de la linea



# Appendix C

```
[1] "2026-08-01"
```

```
library(readr)
HIV <- read_csv("Datos/HIV.csv")
```

```
HIV
```

```
A tibble: 824 x 10
gender dob hiv_risk_factor_f Poverty_numeric hh_size ins_type_f
<chr> <chr> <chr> <dbl> <dbl> <chr>
1 Hombre 8/21/34 Heterosexual 0.5 2 Medicaid
2 Mujer 1/25/31 Heterosexual 0.5 3 Medicaid
3 Hombre 11/18/36 Heterosexual 0.5 1 Medicaid
4 Mujer 6/29/37 Heterosexual 0.5 1 Medicaid
5 Hombre 12/26/42 Heterosexual 0.5 2 Medicaid
6 Mujer 3/21/40 Heterosexual 0.5 2 Medicaid
7 Hombre 8/17/36 Heterosexual 0.5 1 Medicaid
8 Mujer 10/17/42 Heterosexual 0.5 1 Medicaid
9 Mujer 10/17/42 Heterosexual 0.5 1 Medicaid
10 Hombre 7/17/47 Heterosexual 0.5 2 Medicaid
i 814 more rows
i 4 more variables: vital_status <chr>, date_of_death <chr>, hiv_date <chr>,
lifespan <dbl>
```

```
library(tidyverse)
```

```
https://www.stat.berkeley.edu/~s133/dates.html
```

```
HIV$dob=as.Date(format(as.Date(HIV$dob,format="%m/%d/%y"), "19%y%m%d"), "%Y%m%d")
```

```
HIV
```

```
A tibble: 824 x 10
```

```
gender dob hiv_risk_factor_f Poverty_numeric hh_size ins_type_f
<chr> <date> <chr> <dbl> <dbl> <chr>
1 Hombre 1934-08-21 Heterosexual 0.5 2 Medicaid
2 Mujer 1931-01-25 Heterosexual 0.5 3 Medicaid
3 Hombre 1936-11-18 Heterosexual 0.5 1 Medicaid
4 Mujer 1937-06-29 Heterosexual 0.5 1 Medicaid
5 Hombre 1942-12-26 Heterosexual 0.5 2 Medicaid
6 Mujer 1940-03-21 Heterosexual 0.5 2 Medicaid
7 Hombre 1936-08-17 Heterosexual 0.5 1 Medicaid
8 Mujer 1942-10-17 Heterosexual 0.5 1 Medicaid
9 Mujer 1942-10-17 Heterosexual 0.5 1 Medicaid
10 Hombre 1947-07-17 Heterosexual 0.5 2 Medicaid
i 814 more rows
i 4 more variables: vital_status <chr>, date_of_death <chr>, hiv_date <chr>,
lifespan <dbl>
```

## Calculate the age of individuals when they were diagnosed with HIV

Al convertir años de dos dígitos con %y, R puede asignar el siglo equivocado (por ejemplo, “69” podría leerse como 2069). Por eso aquí se fuerza 19%y para las fechas de nacimiento.

## Calculate the time between the date of diagnosis of HIV and the date of death.

```
HIV = HIV |>
 mutate(date_d = mdy(date_of_death),
 hiv_d = mdy(hiv_date))
```

HIV

```
A tibble: 824 x 12
gender dob hiv_risk_factor_f Poverty_numeric hh_size ins_type_f
<chr> <date> <chr> <dbl> <dbl> <chr>
1 Hombre 1934-08-21 Heterosexual 0.5 2 Medicaid
2 Mujer 1931-01-25 Heterosexual 0.5 3 Medicaid
3 Hombre 1936-11-18 Heterosexual 0.5 1 Medicaid
4 Mujer 1937-06-29 Heterosexual 0.5 1 Medicaid
5 Hombre 1942-12-26 Heterosexual 0.5 2 Medicaid
6 Mujer 1940-03-21 Heterosexual 0.5 2 Medicaid
7 Hombre 1936-08-17 Heterosexual 0.5 1 Medicaid
8 Mujer 1942-10-17 Heterosexual 0.5 1 Medicaid
```

```
9 Mujer 1942-10-17 Heterosexual 0.5 1 Medicaid
10 Hombre 1947-07-17 Heterosexual 0.5 2 Medicaid
i 814 more rows
i 6 more variables: vital_status <chr>, date_of_death <chr>, hiv_date <chr>,
lifespan <dbl>, date_d <date>, hiv_d <date>
```

```
HIV1 <- HIV %>%
 mutate(hiv_date = mdy(hiv_date))
```

```
HIV1
```

```
A tibble: 824 x 12
gender dob hiv_risk_factor_f Poverty_numeric hh_size ins_type_f
<chr> <date> <chr> <dbl> <dbl> <chr>
1 Hombre 1934-08-21 Heterosexual 0.5 2 Medicaid
2 Mujer 1931-01-25 Heterosexual 0.5 3 Medicaid
3 Hombre 1936-11-18 Heterosexual 0.5 1 Medicaid
4 Mujer 1937-06-29 Heterosexual 0.5 1 Medicaid
5 Hombre 1942-12-26 Heterosexual 0.5 2 Medicaid
6 Mujer 1940-03-21 Heterosexual 0.5 2 Medicaid
7 Hombre 1936-08-17 Heterosexual 0.5 1 Medicaid
8 Mujer 1942-10-17 Heterosexual 0.5 1 Medicaid
9 Mujer 1942-10-17 Heterosexual 0.5 1 Medicaid
10 Hombre 1947-07-17 Heterosexual 0.5 2 Medicaid
i 814 more rows
i 6 more variables: vital_status <chr>, date_of_death <chr>, hiv_date <date>,
lifespan <dbl>, date_d <date>, hiv_d <date>
```

```
HIV2 <- HIV1 %>%
 mutate(edad = dob %--% hiv_date / years())
```

```
HIV2
```

```
A tibble: 824 x 13
gender dob hiv_risk_factor_f Poverty_numeric hh_size ins_type_f
<chr> <date> <chr> <dbl> <dbl> <chr>
1 Hombre 1934-08-21 Heterosexual 0.5 2 Medicaid
2 Mujer 1931-01-25 Heterosexual 0.5 3 Medicaid
3 Hombre 1936-11-18 Heterosexual 0.5 1 Medicaid
4 Mujer 1937-06-29 Heterosexual 0.5 1 Medicaid
5 Hombre 1942-12-26 Heterosexual 0.5 2 Medicaid
6 Mujer 1940-03-21 Heterosexual 0.5 2 Medicaid
7 Hombre 1936-08-17 Heterosexual 0.5 1 Medicaid
8 Mujer 1942-10-17 Heterosexual 0.5 1 Medicaid
9 Mujer 1942-10-17 Heterosexual 0.5 1 Medicaid
10 Hombre 1947-07-17 Heterosexual 0.5 2 Medicaid
i 814 more rows
```

```
i 7 more variables: vital_status <chr>, date_of_death <chr>, hiv_date <date>,
lifespan <dbl>, date_d <date>, hiv_d <date>, edad <dbl>
```

```
HIV3 <- HIV2 %>%
 mutate(date_of_death = mdy(date_of_death))
```

```
HIV3
```

```
A tibble: 824 x 13
```

```
gender dob hiv_risk_factor_f Poverty_numeric hh_size ins_type_f
<chr> <date> <chr> <dbl> <dbl> <chr>
1 Hombre 1934-08-21 Heterosexual 0.5 2 Medicaid
2 Mujer 1931-01-25 Heterosexual 0.5 3 Medicaid
3 Hombre 1936-11-18 Heterosexual 0.5 1 Medicaid
4 Mujer 1937-06-29 Heterosexual 0.5 1 Medicaid
5 Hombre 1942-12-26 Heterosexual 0.5 2 Medicaid
6 Mujer 1940-03-21 Heterosexual 0.5 2 Medicaid
7 Hombre 1936-08-17 Heterosexual 0.5 1 Medicaid
8 Mujer 1942-10-17 Heterosexual 0.5 1 Medicaid
9 Mujer 1942-10-17 Heterosexual 0.5 1 Medicaid
10 Hombre 1947-07-17 Heterosexual 0.5 2 Medicaid
```

```
i 814 more rows
```

```
i 7 more variables: vital_status <chr>, date_of_death <date>,
hiv_date <date>, lifespan <dbl>, date_d <date>, hiv_d <date>, edad <dbl>
```

```
HIV4 <- HIV3 %>%
 mutate(diagnosed_death = hiv_date %--% date_of_death / years())
```

```
HIV4
```

```
A tibble: 824 x 14
```

```
gender dob hiv_risk_factor_f Poverty_numeric hh_size ins_type_f
<chr> <date> <chr> <dbl> <dbl> <chr>
1 Hombre 1934-08-21 Heterosexual 0.5 2 Medicaid
2 Mujer 1931-01-25 Heterosexual 0.5 3 Medicaid
3 Hombre 1936-11-18 Heterosexual 0.5 1 Medicaid
4 Mujer 1937-06-29 Heterosexual 0.5 1 Medicaid
5 Hombre 1942-12-26 Heterosexual 0.5 2 Medicaid
6 Mujer 1940-03-21 Heterosexual 0.5 2 Medicaid
7 Hombre 1936-08-17 Heterosexual 0.5 1 Medicaid
8 Mujer 1942-10-17 Heterosexual 0.5 1 Medicaid
9 Mujer 1942-10-17 Heterosexual 0.5 1 Medicaid
10 Hombre 1947-07-17 Heterosexual 0.5 2 Medicaid
```

```
i 814 more rows
```

```
i 8 more variables: vital_status <chr>, date_of_death <date>,
hiv_date <date>, lifespan <dbl>, date_d <date>, hiv_d <date>, edad <dbl>,
diagnosed_death <dbl>
```

## Calculate the difference in life between the date of diagnosis and death

```
HIV$months= floor(time_length(difftime(HIV$date_d, HIV$hiv_d), "months"))
HIV$months
```

```
[1] 182 17 3 11 159 46 143 190 188 265 196 272 97 332 307 282 179 368
[19] 363 363 228 192 195 80 55 299 299 17 1 273 216 86 201 201 154 110
[37] 55 32 182 123 74 324 223 213 127 72 264 116 100 95 57 4 220 204
[55] 102 10 4 339 306 294 208 205 167 102 86 1 320 320 240 181 176 115
[73] 33 2 314 197 203 181 183 137 47 47 46 27 373 239 239 239 231 223
[91] 201 195 200 174 160 158 81 27 17 326 332 312 312 280 226 226 211 171
[109] 163 129 105 80 76 58 11 10 309 275 243 234 117 56 329 302 271 266
[127] 196 161 124 118 72 71 313 209 198 157 158 155 44 267 273 254 188 134
[145] 110 77 48 6 261 218 110 90 25 252 154 65 18 14 276 227 157 161
[163] 29 273 243 118 79 51 42 1 262 229 87 246 158 3 256 144 143 61
[181] 31 19 2 141 173 23 15 5 102 102 179 67 36 177 132 0 5 197
[199] 121 2 225 225 164 166 2 132 132 106 66 87 67 295 236 346 211 204
[217] 286 334 150 166 90 215 24 69 4 132 2 230 222 169 112 27 233 65
[235] 2 279 271 238 232 21 228 223 191 207 164 151 162 65 65 67 17 286
[253] 97 212 19 324 86 110 68 208 203 183 284 62 52 30 137 10 193 22
[271] 101 101 97 96 90 0 0 266 298 227 97 128 10 312 323 4 0 4
[289] 274 241 209 168 168 316 207 84 287 100 461 237 237 235 139 214 98 355
[307] 323 280 283 315 298 298 231 180 212 224 221 221 221 337 174 142 423 371
[325] 166 282 265 165 164 391 394 348 352 352 338 242 196 162 128 291 263 235
[343] 235 269 204 214 201 175 160 146 154 81 339 291 277 259 219 213 207 213
[361] 180 132 140 119 111 102 392 283 281 229 189 38 274 168 168 170 164 157
[379] 150 150 135 87 79 322 322 322 274 259 216 167 138 317 289 275 237 186
[397] 150 144 125 80 72 49 56 315 292 233 221 219 195 195 198 177 152 144
[415] 103 67 384 381 379 345 332 210 170 159 143 142 132 99 90 95 95 87
[433] 33 33 33 26 377 340 307 301 225 179 166 115 97 93 54 416 335 249
[451] 222 213 201 189 141 52 42 318 300 300 300 244 238 218 186 176 136 142
[469] 131 131 127 124 109 76 54 264 214 185 181 151 154 154 73 70 403 336
[487] 281 228 216 187 190 179 144 131 100 47 43 347 289 277 259 259 259 218
[505] 218 211 157 141 53 4 281 257 229 224 217 218 134 134 90 95 56 34
[523] 35 21 22 386 284 248 235 235 207 199 146 144 122 110 66 273 197 153
[541] 107 100 67 254 209 173 122 122 114 56 2 247 127 106 85 76 67 3
[559] 193 198 199 164 108 65 127 112 103 179 34 14 212 191 150 144 80 171
[577] 133 66 121 95 182 166 21 6 1 31 23 6 388 388 62 20 26 158
[595] 332 370 274 242 228 98 15 277 222 306 52 187 186 269 295 195 172 214
[613] 184 123 6 55 3 142 148 93 139 316 270 271 262 179 182 240 189 73
[631] 32 106 134 83 281 236 137 137 144 249 304 192 34 259 197 3 155 223
[649] 165 152 184 329 329 253 107 135 50 134 141 227 142 124 57 223 175 171
[667] 241 215 71 1 269 230 292 252 25 248 63 248 261 188 91 190 175 69
[685] 25 192 162 211 248 261 278 199 233 456 455 229 156 111 124 22 233 323
```

```
[703] 344 340 114 193 118 246 267 254 195 168 229 236 219 280 332 313 244 244
[721] 295 187 137 138 182 86 125 17 57 361 384 128 358 212 374 310 157 99
[739] 99 84 126 214 125 321 129 111 58 69 190 190 52 24 226 226 321 296
[757] 211 118 101 276 258 222 115 182 137 251 208 32 8 2 0 206 210 186
[775] 112 475 57 316 231 351 143 252 159 327 336 65 143 254 254 3 15 90
[793] 209 183 183 271 81 235 163 283 248 190 59 119 230 288 410 216 24 120
[811] 54 1 290 279 268 293 260 81 269 272 309 361 336 315
```

```
HIV %>%
```

```
 mutate(Period_HIV_death = as.duration(hiv_d %--% date_d)) |>
 summarise(mean_time=mean(Period_HIV_death))
```

```
A tibble: 1 x 1
mean_time
<dbl>
1 462043713.
```

---

# Appendix D

```
[1] "2026-08-01"
```

---

## A.2 COVID 19 Cumulativo

Usa los datos de COVID-19 en Puerto Rico (PR) del Departamento de Salud de PR.

```
url_COVID_PR <- read_csv("Datos/url_COVID_PR.csv")
```

```
head(url_COVID_PR)
```

```
A tibble: 6 x 18
...1 Fecha Muertes IncrementoMuertes CasosPCR_Salud IncCasosPCR_Salud
<dbl> <chr> <dbl> <dbl> <dbl> <dbl>
1 1 3/12/20 0 0 NA NA
2 2 3/13/20 0 0 NA NA
3 3 3/14/20 0 0 NA NA
4 4 3/15/20 0 0 NA NA
5 5 3/16/20 0 0 NA NA
6 6 3/17/20 0 0 NA NA
i 12 more variables: CasosSaludNuevo <dbl>, IncCasosSaludNuevo <dbl>,
HospitCOV19 <dbl>, CamasICU <dbl>, CamasICU_disp <dbl>, Ventiladores <dbl>,
MuertesSalud <dbl>, IncMueSalud <dbl>, VacDoses <dbl>, VacAdm <dbl>,
N1MoreDoses <dbl>, N2Doses <dbl>
```

---

1. Calcula los promedios en periodos de 7 (días; promedio corriendo) y añádele a tu dataframe

Un promedio *cumulative* (`cummean`) usa todos los datos desde el inicio hasta cada fecha, mientras que un promedio *corriendo* o móvil (`rollmean` con ventana de

7 días) solo promedia los días más cercanos.

`cummean()` (paquete **dplyr**) y `rollmean()` (paquete **zoo**) — resumen una serie de valores a lo largo del tiempo.

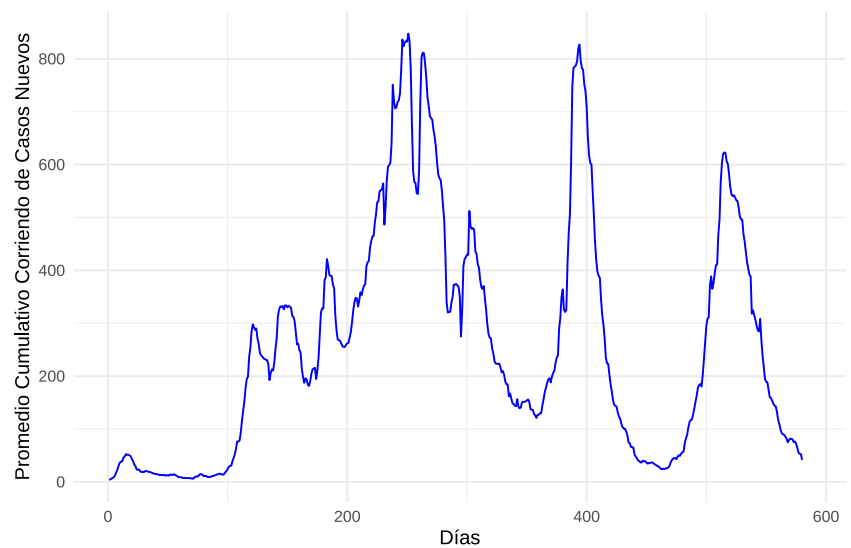
- `cummean(x)`: devuelve el promedio *acumulado* — cada valor es el promedio de todos los datos hasta esa fila.
- `rollmean(x, k = 7, fill = NA, align = "left")`: el promedio *móvil* de una ventana de `k` días; `fill = NA` rellena los extremos donde no hay ventana completa y `align` fija dónde se ancla la ventana.
- `rollapply(x, width, FUN)`: versión general que aplica *cualquier* función (aquí `var`) sobre una ventana móvil.

— Usa la columna de `IncCasosSaludNuevo` para todos los cálculos

- Enseñe las primeras 20 filas de tu dataframe con las nuevas columnas añadidas

2. Haz una figura que muestra el cambio en el promedio cumulativo corriendo

Promedio Cumulativo Corriendo de Casos Nuevos de COVID-19 en PR



de casos nuevos.